

IKHEDU FOUNDATION • C++ BẢNG B • LEVEL 1

GIÁO TRÌNH LÝ THUYẾT & THUẬT TOÁN

TOÀN TẬP

Cẩm nang 21 chuyên đề trọng tâm thi Học sinh giỏi & Tin học trẻ



HỆ THỐNG ĐÀO TẠO TIN HỌC iKHEDU

Phiên bản Trendy • Xuất bản số 2026 • Khóa học C++ Cơ bản (Bảng B)

Mục lục

Ikhedu c++ bảng b — tổng hợp nội dung 21 chuyên đề

Mục lục tổng quan

Module 01: Thuật toán sắp xếp & kỹ thuật mảng

Module 02: Mảng tiền tố & tìm kiếm nhị phân

Module 03: Số học & đại số modular

Module 04: Đề quy, chia để trị & quay lui

Module 05: Quy hoạch động (dynamic programming)

Module 06: Cấu trúc dữ liệu nâng cao

Module 07: Đồ thị & cây truy vấn đoạn

Module 01: Thuật toán sắp xếp & kỹ thuật mảng

Chuyên đề 01: Thuật toán sắp xếp

1. Khái niệm & bản chất của sắp xếp trong tối ưu thuật toán

2. Tính chất lân cận & chứng minh toán học

2.1. Định lý về cặp phần tử có khoảng cách nhỏ nhất

2.2. Chứng minh toán học

Ví dụ minh họa 1: Tìm khoảng cách nhỏ nhất giữa hai phần tử

3. Các ứng dụng thuật toán kinh điển của sắp xếp

4. Hàm sort & nguyên lý strict weak ordering

4.1. Cú pháp chuẩn trong c++

4.2. Nguyên lý strict weak ordering (toán tử so sánh nghiêm ngặt)

Cảnh báo quan trọng:

5. Các kỹ thuật custom comparator nâng cao

5.1. Sắp xếp đa tiêu chí với vector lồng nhau (multi-criteria sorting)

Ví dụ minh họa 2: Sắp xếp danh sách 4 đoạn thẳng

5.2. Sắp xếp lưu chỉ số ban đầu (index tracking)

5.3. Comparator hàm mục tiêu (objective comparison)

6. Mẫu cài đặt chuẩn thi đấu (competitive template)

7. Ranh giới áp dụng: Khi nào được & không được sắp xếp?

Câu hỏi trắc nghiệm củng cố khái niệmCâu 1 (nhận diện — recognize):Câu 2 (dự đoán — predict):Câu 3 (bản chất — explain):Câu 4 (chuyển giao — transfer):Câu 5 (cú pháp & ứng dụng — syntax):Câu 6 (phân biệt cấu trúc — compare):Câu 7 (thuật toán lân cận — technique):Câu 8 (comparator hàm mục tiêu — logic):Câu 9 (sắp xếp đa tiêu chí — multi-criteria):Câu 10 (xử lý kiểu dữ liệu & tràn số — robustness):Ma trận bài tập thực hành (p0 → p5)**Chuyên đề 02: Kỹ thuật hai con trỏ**1. Khái niệm & nguyên lý hoạt động2. Chứng minh bất biến lặp (loop invariant) & tính đúng đắn2.1. Phát biểu bất biến lặp2.2. Chứng minh quy nạp toán học3. Các mô hình bài toán đặc trưng3.1. Mô hình 1: Tìm cặp số có tổng đúng bằng S (two sum)Ví dụ minh họa 1: Tìm cặp số có tổng $S = 14$ 3.2. Mô hình 2: Đếm số cặp có tổng thỏa mãn bất đẳng thức $A_i + A_j \leq S$ Ví dụ minh họa 2: Đếm số cặp có tổng ≤ 10 trên mảng $A = [1, 2, 4, 7, 9]$ 3.3. Mô hình 3: Ghép cặp cực trị tham lam (bài toán thuyền cứu hộ / xe chờ hàng)3.4. Mô hình 4: Khử chiều đa biến (bài toán 3-sum và 4-sum)4. Phân tích độ phức tạp thời gian & không gian5. Mẫu cài đặt chuẩn thi đấu (competitive template)6. Các bẫy lỗi kỹ thuật thường gặp (bug traps)Câu hỏi trắc nghiệm củng cố khái niệmCâu 1 (nhận diện — recognize):Câu 2 (dự đoán — predict):Câu 3 (bản chất — explain):Câu 4 (chuyển giao — transfer):

Câu 5 (bẫy điều kiện dừng — bug traps):

Câu 6 (chiến lược tham lam — greedy pairing):

Câu 7 (khử chiều đa biến — dimensionality reduction):

Câu 8 (xử lý trùng lặp — duplicates handling):

Câu 9 (hai mảng độc lập — multi-array pointers):

Câu 10 (phòng thủ kiểu dữ liệu — data overflow):

Ma trận bài tập thực hành (p0 → p5)

Chuyên đề 03: Kỹ thuật cửa sổ trượt

1. Khái niệm & bản chất của kỹ thuật cửa sổ trượt

2. Cơ chế chuyển dịch & phân tích độ phức tạp $O(N)$

2.1. Cơ chế hai con trỏ cùng chiều ($L \rightarrow R$)

2.2. Phân tích chi phí khấu hao (amortized complexity analysis)

3. Phân loại hai dạng cửa sổ trượt chuẩn mực

3.1. Dạng 1: Cửa sổ cố định độ dài K (fixed-size window)

Ví dụ minh họa 1: Tìm tổng đoạn con $K = 3$ lớn nhất trên dãy $A = [2, 1, 5, 1, 3, 2]$

3.2. Dạng 2: Cửa sổ biến thiên (variable-size window)

Ví dụ minh họa 2: Tìm đoạn con ngắn nhất có tổng $\geq S = 7$ trên $A = [2, 3, 1, 2, 4, 3]$

4. Điều kiện áp dụng & giới hạn thất bại khi mảng có số âm

4.1. Điều kiện tiên quyết: Tính đơn điệu của hàm trạng thái

4.2. Giới hạn: Vì sao sliding window thất bại khi có số âm?

5. Mẫu cài đặt chuẩn thi đấu (competitive template)

Mẫu: Đoạn con liên tiếp ngắn nhất có tổng $\geq S$ ($A_i \geq 0$)

6. Kỹ thuật cửa sổ trượt với bảng đếm ký tự / trạng thái

7. Các bẫy lỗi thường gặp (bug traps)

Câu hỏi trắc nghiệm củng cố khái niệm

Câu 1 (nhận diện — recognize):

Câu 2 (dự đoán — predict):

Câu 3 (bản chất — explain):

Câu 4 (chuyển giao — transfer):

Câu 5 (chiến lược điều khiển — control flow):

Câu 6 (đếm tổ hợp đoạn con — combinatorial counting):

Câu 7 (cửa sổ chuỗi ký tự — frequency map):

Câu 8 (cửa sổ bao phủ tối thiểu — minimum window):

Câu 9 (xử lý giới hạn dữ liệu lớn — large constraints):

Câu 10 (kỹ thuật hiệu đếm đoạn con — interval counting trick):

Ma trận bài tập thực hành (p0 → p5)

Module 02: Mảng tiền tố & tìm kiếm nhị phân

Chuyên đề 04: Mảng tiền tố & mảng hiệu

1. Khái niệm & bản chất của mảng tiền tố (prefix sum 1d)

1.1. Công thức xây dựng mảng tiền tố

1.2. Công thức truy vấn tổng đoạn con trong $O(1)$

1.3. Chứng minh toán học

Ví dụ minh họa 1: Xây dựng và truy vấn prefix sum 1d

2. Kỹ thuật mảng tiền tố hai chiều (prefix sum 2d)

2.1. Bản chất nguyên lý bao hàm - loại trừ (inclusion-exclusion principle)

2.2. Công thức xây dựng bảng tiền tố 2d trong $O(N \times M)$

2.3. Công thức truy vấn tổng hình chữ nhật $(x_1, y_1) \rightarrow (x_2, y_2)$ trong $O(1)$

Ví dụ minh họa 2: Truy vấn hình chữ nhật trên ma trận 3×3

3. Kỹ thuật mảng hiệu (difference array 1d)

3.1. Bài toán đặt ra

3.2. Cơ chế hoạt động của mảng hiệu

3.3. Khôi phục mảng kết quả sau Q thao tác

Ví dụ minh họa 3: Mảng hiệu trên dãy $N = 5$ phần tử

4. Kỹ thuật mảng hiệu hai chiều (difference array 2d)

5. Mẫu cài đặt chuẩn thi đấu (competitive template)

Mẫu 1: Prefix sum 1d (truy vấn tổng đoạn)

Mẫu 2: Difference array 1d (cập nhật đoạn)

6. Các bẫy lỗi lập trình thường gặp (bug traps)

7. Ranh giới áp dụng: Khi nào nên & không nên dùng?

Câu hỏi trắc nghiệm củng cố khái niệm

Câu 1 (nhận diện — recognize):

Câu 2 (dự đoán — predict):

Câu 3 (bản chất — explain):

Câu 4 (chuyển giao — transfer):

Câu 5 (prefix sum 2d — geometry):

Câu 6 (mảng hiệu 2d — technique):

Câu 7 (đoạn con tổng bằng 0 — logic):

Câu 8 (đồng dư tiền tố — prefix modulo):

Câu 9 (bẫy chỉ số mảng hiệu — bug traps):

Câu 10 (tràn số dữ liệu lớn — data types):

Câu 11 (nén chiều ma trận — 2d submatrix compression):

Câu 12 (cân bằng đa trạng thái — multidimensional balance):

Ma trận bài tập thực hành (p0 → p5)

Chuyên đề 05: Thuật toán tìm kiếm nhị phân

1. Khái niệm & bản chất của tìm kiếm nhị phân (binary search)

1.1. Điều kiện tiên quyết (prerequisite condition)

1.2. Phân tích độ phức tạp thời gian $O(\log N)$

2. Tìm kiếm nhị phân trên mảng đã sắp xếp

2.1. Tìm chính xác giá trị X (exact search)

Ví dụ minh họa 1: Tìm kiếm giá trị $X = 23$

2.2. Tìm kiếm phần tử biên: lower_bound Và upper_bound

Ví dụ minh họa 2: Mảng có phần tử lặp lại

3. Kỹ thuật chặt nhị phân trên tập kết quả (binary search on answer)

3.1. Nhận diện tính chất đơn điệu & phân loại 2 hướng

Ví dụ minh họa 3: Bài toán cắt gỗ lấy tối thiểu $M = 7$ mét gỗ

4. Chặt nhị phân trên tập số thực (real-number binary search)

5. Mẫu cài đặt chuẩn thi đấu (competitive templates)

Mẫu 1a: Tìm giá trị lớn nhất thỏa mãn (dạng True -> False)

Mẫu 1b: Tìm giá trị nhỏ nhất thỏa mãn (dạng False -> True)

Mẫu 2: Binary search số thực (100 vòng lặp robust)

6. Các bẫy lỗi lập trình kinh điển (bug traps)

7. Ranh giới áp dụng: Khi nào nên & không nên dùng?

Câu hỏi trắc nghiệm củng cố khái niệm

Câu 1 (bản chất — complexity):

Câu 2 (điều kiện tiên quyết — monotonicity):

Câu 3 (cú pháp chuẩn — bug trap):

Câu 4 (hàm stl — lower bound):

Câu 5 (hàm stl — upper bound):

Câu 6 (đếm số lần xuất hiện — counting):

Câu 7 (binary search on answer — logic):

Câu 8 (binary search on answer — search space):

Câu 9 (chất nhị phân số thực — real numbers):

Câu 10 (ranh giới thất bại — failure boundary):

Câu 11 (mảng xoay vòng — rotated array):

Câu 12 (ma trận 2d đã sắp xếp — 2d matrix binary search):

Câu 13 (đỉnh dãy núi — mountain array peak):

Câu 14 (trung vị hai mảng đã sắp xếp — advanced partition):

Ma trận bài tập thực hành (p0 → p5)

Chuyên đề 06: Phép toán bit & biểu diễn trạng thái

1. Khái niệm & 6 phép toán bit cơ bản

1.1. Bảng chân trị của 6 phép toán bit trong c++

1.2. Các tính chất đại số quan trọng của phép xor (oplus)

2. 4 Thao tác thao tác bit chuẩn thi đấu

2.1. Kiểm tra bit thứ k có đang bật (bằng 1) hay không:

2.2. Bật bit thứ k (gán thành 1):

2.3. Tắt bit thứ k (gán thành 0):

2.4. Đảo bit thứ k ($0 \rightarrow 1, 1 \rightarrow 0$):

Ví dụ minh họa 1: Thao tác trên số $N = 13 = 1101_2$

3. Các tuyệt kỹ bit & hàm nội tại cpu (builtin functions)

3.1. Kiểm tra một số nguyên dương có phải là lũy thừa của 2

3.2. Lấy bit 1 nhỏ nhất (lowest set bit / lowbit)

3.3. Các hàm nội tại tối ưu hóa phần cứng trong gcc/clang:

4. Kỹ thuật mặt nạ bit (bitmask & subset enumeration)

4.1. Duyệt toàn bộ 2^N tập con (vết cạn nhị phân):

4.2. Duyệt tất cả tập con (submasks) của một mask trong $O(3^N)$:

Ví dụ minh họa 2: Biểu diễn tập con của tập 3 phần tử $S = \{A_0, A_1, A_2\}$

5. Mẫu cài đặt chuẩn thi đấu (competitive template)

[Mẫu 1: Vết cạn tập con bằng mặt nạ bit \(subset sum\)](#)

[6. Các bẫy lỗi lập trình kinh điển \(bug traps\)](#)

[7. Ranh giới áp dụng: Khi nào nên & không nên dùng?](#)

[Câu hỏi trắc nghiệm củng cố khái niệm](#)

[Câu 1 \(bản chất xor — identity\):](#)

[Câu 2 \(bẫy độ ưu tiên toán tử — precedence\):](#)

[Câu 3 \(kỹ thuật lật bit — manipulation\):](#)

[Câu 4 \(kỹ thuật tắt bit — manipulation\):](#)

[Câu 5 \(lũy thừa của 2 — bit trick\):](#)

[Câu 6 \(đếm bit 1 — builtin\):](#)

[Câu 7 \(không gian tập con — complexity\):](#)

[Câu 8 \(tìm phần tử đơn lẻ — xor application\):](#)

[Câu 9 \(bẫy dịch bit 64-bit — 64-bit shift\):](#)

[Câu 10 \(duyet submask — advanced technique\):](#)

[Câu 11 \(cặp tổng lũy thừa của 2 — power of 2 pairs\):](#)

[Câu 12 \(tập độc lập về bit — bit independence\):](#)

[Ma trận bài tập thực hành \(p0 → p5\)](#)

[Module 03: Số học & đại số modular](#)

[Chuyên đề 07: Lý thuyết số & số nguyên tố](#)

[1. Bản chất vấn đề & trực giác thuật toán \(the core problem & intuition\)](#)

[Vấn đề 1: Tìm ước chung lớn nhất \(gcd\)](#)

[Vấn đề 2: Kiểm tra số nguyên tố & phân tích thừa số nguyên tố](#)

[2. Mô phỏng từng bước \(visual step-by-step simulation\)](#)

[Ví dụ 1: Mô phỏng thuật toán euclid tìm gcd\(252, 105\)](#)

[Ví dụ 2: Mô phỏng sàng eratosthenes tìm các số nguyên tố \$\leq 20\$](#)

[3. Lý thuyết cốt lõi & bất biến toán học \(core invariants\)](#)

[3.1. Mối quan hệ giữa gcd và lcm](#)

[Cảnh báo quan trọng:](#)

[3.2. Định lý cơ bản của số học & công thức nhân tính](#)

[3.3. Sàng ước nguyên tố nhỏ nhất \(spf - smallest prime factor\)](#)

[3.4. Phi hàm euler \(euler's totient function \$\phi\(N\)\$ \)](#)

4. Các bẫy lỗi lập trình kinh điển (bug traps)

5. Mẫu cài đặt chuẩn thi đấu (competitive templates)

Mẫu 1: Ước chung lớn nhất & bội chung nhỏ nhất

Mẫu 2: Sàng eratosthenes & sàng spf (tối ưu phân tích thừa số)

Câu hỏi trắc nghiệm củng cố khái niệm

Câu 1 (độ phức tạp):

Câu 2 (bản chất toán học):

Câu 3 (cú pháp & bẫy lỗi):

Câu 4 (thuật toán kiểm tra số nguyên tố):

Câu 5 (ứng dụng sàng nguyên tố):

Câu 6 (sàng spf):

Câu 7 (đếm số lượng ước):

Câu 8 (đặc điểm số chính phương):

Câu 9 (công thức legendre):

Câu 10 (chữ số 0 tận cùng):

Câu 11 (sàng phân đoạn - segmented sieve):

Câu 12 (số nguyên tố cùng nhau):

Ma trận bài tập thực hành (p0 → p5)

Chuyên đề 08: Đồng dư thức, lũy thừa nhị phân & nghịch đảo modulo

1. Bản chất vấn đề & trực giác thuật toán (the core problem & intuition)

Vấn đề 1: Phép tính lũy thừa $A^B \bmod M$

Vấn đề 2: Phép chia trên vành modulo $\left(\frac{A}{B} \bmod M\right)$

2. Mô phỏng từng bước (visual step-by-step simulation)

Ví dụ 1: Mô phỏng tính $3^{13} \bmod 1000$ bằng lũy thừa nhị phân

Ví dụ 2: Mô phỏng tìm nghịch đảo modulo của $3 \bmod 7$

3. Lý thuyết cốt lõi & bất biến toán học (core invariants)

3.1. Các quy tắc đồng dư cơ bản (+, -, *)

Cảnh báo quan trọng:

3.2. Định lý fermat nhỏ & nghịch đảo modulo

Chú ý:

3.3. Thuật toán euclid mở rộng (extended euclidean algorithm)

3.4. Tính tổ hợp $C(N, K) \bmod M$ trong $O(1)$ mỗi truy vấn

3.5. Tính tổng cấp số nhân modulo bằng chia để trị4. Các bẫy lỗi lập trình kinh điển (bug traps)5. Mẫu cài đặt chuẩn thi đấu (competitive templates)Mẫu 1: Lũy thừa nhị phân & nghịch đảo modulo chuẩnMẫu 2: Tiền xử lý tổ hợp $C(N, K) \bmod M$ trong $O(1)$ mỗi truy vấnCâu hỏi trắc nghiệm củng cố khái niệmCâu 1 (độ phức tạp):Câu 2 (xử lý số âm):Câu 3 (định lý fermat nhỏ):Câu 4 (phép chia modulo):Câu 5 (điều kiện tồn tại nghịch đảo):Câu 6 (tổ hợp modulo $O(1)$):Câu 7 (tối ưu tính nghịch đảo giai thừa):Câu 8 (rút gọn số mũ lớn):Câu 9 (nhân an toàn chống tràn số 64-bit):Câu 10 (phương trình diophantine & euclid mở rộng):Ma trận bài tập thực hành ($p0 \rightarrow p5$)Ghi chú:**Chuyên đề 09: Xử lý số nguyên lớn (bigint)**1. Bản chất vấn đề & trực giác thuật toán (the core problem & intuition)Big integer hay modular arithmetic: Chọn vũ khí nào?2. Mô phỏng từng bước (visual step-by-step simulation)Ví dụ 1: Mô phỏng phép cộng số lớn $A = 9876$ và $B = 543$ Ví dụ 2: Mô phỏng phép nhân số lớn $A = 48$ với số nhỏ $b = 7$ 3. Lý thuyết cốt lõi & bất biến thuật toán (core invariants)3.1. Mô hình biểu diễn số lớn & little-endian3.2. Bảng tổng hợp các phép toán số nguyên lớn ($O(L^2)$)3.3. Thuật toán chia số lớn cho số nhỏ & bất biến hornerGhi chú:3.4. Tối ưu hóa base 10^9 (chunking optimization)4. Các bẫy lỗi lập trình kinh điển (bug traps)

5. Mẫu cài đặt chuẩn thi đấu (competitive templates)

Câu hỏi trắc nghiệm củng cố khái niệm

Câu 1 (lưu trữ dữ liệu little-endian):

Câu 2 (độ phức tạp phép nhân):

Câu 3 (độ dài tối đa kết quả phép nhân):

Câu 4 (xử lý số 0 vô nghĩa):

Câu 5 (phép chia số lớn cho số nhỏ):

Câu 6 (trường hợp phép trừ số âm):

Câu 7 (tối ưu base 10^9):

Câu 8 (giai thừa số lớn $1000!$):

Câu 9 (lũy thừa số lớn A^B):

Câu 10 (so sánh hai số lớn dạng chuỗi):

Ma trận bài tập thực hành (p0 → p5)

Ghi chú:

Module 04: Đệ quy, chia để trị & quay lui

Chuyên đề 10: Thuật toán đệ quy & cây gọi hàm

1. Bản chất vấn đề & trực giác thuật toán (the core problem & intuition)

Khái niệm đệ quy (recursion):

2. Mô phỏng từng bước hoạt động của call stack (visual step-by-step simulation)

Ví dụ 1: Mô phỏng hàm tính giai thừa $\text{fact}(4)$

Bảng mô phỏng từng bước ngăn xếp (call stack trace):

Ví dụ 2: So sánh vị trí lệnh in (winding vs unwinding)

Quy luật vàng (winding vs unwinding):

3. Lý thuyết cốt lõi & bất biến thuật toán (core invariants)

3.1. Khái niệm stack frame & phân tích an toàn bộ nhớ (stack safety)

Lưu ý kỹ thuật về tail recursion trong C++:

3.2. Hệ thống phân loại thuật ngữ đệ quy (recursion taxonomy)

3.3. Độ phức tạp toán học của fibonacci đệ quy & cầu nối sang quy hoạch động

4. Các bẫy lỗi lập trình kinh điển (bug traps)

5. Mẫu cài đặt chuẩn thi đấu (competitive templates)

Câu hỏi trắc nghiệm củng cố khái niệm

Câu 1 (bản chất base case):

Câu 2 (winding vs unwinding trace prediction):

Câu 3 (cấu trúc bộ nhớ stack frame):

Câu 4 (độ phức tạp chính xác của fibonacci đệ quy):

Câu 5 (bẫy tràn stack overflow):

Câu 6 (bẫy gọi đệ quy lặp lại):

Câu 7 (đặc điểm tail recursion trong c++):

Câu 8 (số bước di chuyển tháp hà nội):

Câu 9 (bản chất đệ quy chia đôi tìm min/max):

Câu 10 (hiện tượng overlapping subproblems):

Ma trận bài tập thực hành (p0 → p5)

Phân tầng lộ trình học tập lesson 10:

Chuyên đề 11: Kỹ thuật chia để trị

1. Cầu nối tư duy: Recurrence → recursion tree → complexity

2. Phân biệt cốt lõi: Đệ quy (recursion) vs chia để trị (divide & conquer)

3. Bản chất vấn đề & hai kiểu phân rã: Divide vs partition

Phân biệt hai kiểu phân rã dữ liệu:

4. Khung tư duy d&c (the d&c mental model)

5. Mô phỏng từng bước thuật toán sắp xếp trộn (merge sort simulation)

Sơ đồ cây phân rã & gộp mảng (divide & merge tree):

6. Combine step & loop invariant — trái tim của divide & conquer

Bất biến vòng lặp (loop invariant) trong hàm merge():

7. Bảng trực giác recurrence & định lý thợ (master theorem)

Công thức tổng quát (master theorem):

8. Bảng nhận diện dấu hiệu thuật toán (d&c pattern recognition)

9. Chuỗi chuyển giao tri thức: Merge → merge sort → inversion counting

Ứng dụng đỉnh cao: Đếm cặp nghịch thế (inversion counting) $O(N \log N)$

10. Các bẫy lỗi lập trình kinh điển (bug traps)

11. Mẫu cài đặt chuẩn thi đấu (competitive templates)

Câu hỏi trắc nghiệm củng cố khái niệm

Câu 1 (bản chất 3 giai đoạn chia để trị):

Câu 2 (suy luận hệ thức truy hồi từ cấu trúc code):

Câu 3 (so sánh bản chất: Cùng chia đôi nhưng khác biệt độ phức tạp):

Câu 4 (kiểu dữ liệu cho đếm cặp nghịch thế):

Câu 5 (cơ chế đếm cặp nghịch thế khi merge):

Câu 6 (bẫy maximum subarray d&c):

Câu 7 (tối ưu bộ nhớ trong merge sort):

Câu 8 (tournament tree tìm phần tử lớn thứ hai với $N = 2^k$):

Câu 9 (đặc tính stable sort của merge sort):

Câu 10 (lũy thừa ma trận chia để trị):

Ma trận bài tập thực hành (p0 → p5)

Phân tầng lộ trình học tập lesson 11:

Chuyên đề 12: Thuật toán quay lui & nhánh cận

1. Cầu nối kiến trúc: Recursion → divide & conquer → state-space search → dynamic programming

2. Bản chất trạng thái (state definition & state identity)

Khái niệm state (trạng thái) & state identity:

3. Khung phương pháp luận: Design-time framework vs runtime pattern

1. Khung thiết kế thuật toán (design-time framework):

2. Khung thực thi mã nguồn (runtime pattern):

4. Khung tư duy mental model: Hai sơ đồ cốt lõi của lesson 12

Quy trình 1: Luồng ra quyết định quay lui thuần túy (backtracking)

Quy trình 2: Luồng ra quyết định nhánh cận (branch & bound)

5. Bất biến trung tâm: State restoration invariant

6. Phân biệt cắt tỉa ràng buộc (feasibility) vs cắt tỉa tối ưu (branch & bound)

Định nghĩa chuẩn xác: OPT(state) Vs best hiện tại:

Nguyên tắc thiết lập hàm bound chuẩn xác:

7. Phân loại 4 cấp độ kỹ thuật trong tìm kiếm toàn vẹn

8. Cầu nối sâu sang dp: Từ cây tìm kiếm (search tree) đến đồ thị trạng thái (state dag)

9. Các bẫy lỗi lập trình kinh điển (bug traps)

10. Mẫu cài đặt chuẩn thi đấu (competitive templates)

Câu hỏi trắc nghiệm củng cố khái niệm

Câu 1 (bản chất 3 bước quay lui):

Câu 2 (phân biệt feasibility vs optimality pruning):

Câu 3 (hậu quả của việc quên unchoose):

Câu 4 (đánh dấu các họ đường chéo n-queens):

Câu 5 (bản chất quy tắc warnsdorff trong mã đi tuần):

Câu 6 (tình huống thực tế đánh giá hàm bound trong bài toán cực tiểu):

Câu 7 (độ phức tạp không gian: Exponential tree ne exponential stack):

Câu 8 (cắt tia kết hợp sắp xếp trong subset sum):

Câu 9 (độ phức tạp tổng thể khi in toàn bộ xâu nhị phân):

Câu 10 (cầu nối từ backtracking sang dynamic programming):

Câu 11 (bản chất state identity trong bài toán subset sum):

Câu 12 (nguyên tắc an toàn của hàm bound trong branch & bound):

Ma trận bài tập thực hành (p0 → p5)

Lộ trình phân tầng học tập chuẩn mực:

Module 05: Quy hoạch động (dynamic programming)

Chuyên đề 13: Quy hoạch động 1d & dãy con tăng dài nhất

1. Bản chất vấn đề & cầu nối từ đệ quy sang quy hoạch động

2. Khung phương pháp luận: 7 Bước dp state invariant

3. Các mô hình quy hoạch động 1d cốt lõi (core patterns)

3.1. Mô hình bậc thang & bước nhảy (staircase / frog jump)

3.2. Mô hình đổi tiền ít xu nhất (coin change 1d)

3.3. Dãy con tăng dài nhất (longest increasing subsequence — lis $O(N^2)$)

3.4. Kỹ thuật khôi phục vết nghiệm 1d (traceback / reconstruction)

3.5. Mở rộng nâng cao: Lis $O(N \log N)$ bằng binary search

4. Các biến thể thiết kế trạng thái từ mô hình 1d (state design variations)

4.1. Pattern a: Quyết định nhị phân (binary choice — chọn / bỏ qua)

4.2. Pattern b: Mở rộng trạng thái hữu hạn (state dimension expansion)

4.3. Pattern c: Thay đổi đại lượng tối ưu (maximum sum increasing subsequence — msis)

4.4. Pattern d: Phân hoạch đoạn tối ưu (pattern mở rộng / preview — optimal array partitioning & rod cutting)

5. Các bẫy lỗi lập trình kinh điển (bug traps)

6. Mẫu cài đặt chuẩn thi đấu (competitive templates)

[Mẫu 1: Dãy con tăng dài nhất lis \$O\(N^2\)\$ kèm truy vết nghiệm](#)

[Mẫu 2: Đổi tiền ít xu nhất \(coin change 1d\)](#)

7. Hệ thống câu hỏi kiểm tra khái niệm (concept quiz)

[Câu 1 \(bản chất quy hoạch động\):](#)

[Câu 2 \(bất biến trạng thái lis\):](#)

[Câu 3 \(trường hợp cơ sở bài toán bậc thang\):](#)

[Câu 4 \(thứ tự tính toán evaluation order\):](#)

[Câu 5 \(bẫy số nguyên vô cùng\):](#)

[Câu 6 \(khôi phục vết nghiệm traceback\):](#)

[Câu 7 \(độ phức tạp bài toán đổi tiền\):](#)

[Câu 8 \(bài toán house robber 1d\):](#)

[Câu 9 \(mô hình cắt thanh gỗ rod cutting\):](#)

[Câu 10 \(ranh giới lis \$O\(N^2\)\$ vs \$O\(N \log n\)\$ \):](#)

[Câu 11 \(state transformation — lis sang msis\):](#)

[Câu 12 \(state dimension — alternating subsequence\):](#)

[Câu 13 \(segmentation dp — phân hoạch đoạn\):](#)

[Câu 14 \(coin change — phân biệt thứ tự duyệt\):](#)

[Câu 15 \(ý nghĩa mảng tails trong lis \$O\(N \log n\)\$ \):](#)

8. Ma trận 15 bài tập thực hành theo mức độ (p0 → p5)

Chuyên đề 14: Quy hoạch động 2d & bài toán cái túi (knapsack)

1. Bản chất không gian trạng thái 2d

2. Quy hoạch động trên lưới tọa độ (grid dp)

[2.1. Đếm số đường đi trên lưới & xử lý vật cản](#)

[2.2. Tìm đường đi có tổng giá trị lớn nhất / nhỏ nhất](#)

3. Bài toán cái túi 0/1 (0/1 knapsack problem)

[3.1. Bảng phương án 2d chuẩn mực \(\$DP\[i\]\[w\]\$ \)](#)

[3.2. Tuyệt kỹ nén mảng 1d \(space optimization & backward traversal\)](#)

4. Bài toán cái túi không giới hạn (unbounded knapsack)

5. Kỹ thuật đổi trục dp khi sức chứa W quá lớn ($DP[v] = \text{Min Weight}$)

6. Các bẫy lỗi lập trình kinh điển (bug traps)

7. Mẫu cài đặt chuẩn thi đấu (competitive templates)

[Mẫu 1: Bài toán cái túi 0/1 nén mảng 1d](#)

Mẫu 2: Đường đi có tổng lớn nhất trên lưới 2d**8. Hệ thống câu hỏi kiểm tra khái niệm (concept quiz)**Câu 1 (bản chất không gian trạng thái 2d):Câu 2 (chiều duyệt vòng lặp nén mảng 0/1 knapsack):Câu 3 (unbounded knapsack vs 0/1 knapsack):Câu 4 (xử lý vật cản trên lưới 2d):Câu 5 (kỹ thuật đổi trục dp theo value):Câu 6 (bài toán subset sum):Câu 7 (khôi phục danh sách đồ trong cái túi 0/1):Câu 8 (độ phức tạp lưới tam giác triangle dp):Câu 9 (bẫy bộ nhớ 2d mle):Câu 10 (bài toán phân chia tập con bằng nhau):Câu 11 (quy hoạch động từ dưới lên trên triangle dp):Câu 12 (knapsack 2 chiều ràng buộc):Câu 13 (tối ưu hóa bộ nhớ lưới 2d bằng 2 dòng):Câu 14 (knapsack phân chia chênh lệch nhỏ nhất):Câu 15 (truy vết đường đi trên lưới 2d):**9. Ma trận 15 bài tập thực hành theo mức độ (p0 → p5)****Chuyên đề 15: Quy hoạch động chuỗi: Lcs & edit distance****1. Không gian trạng thái hai tiền tố (2-prefix state)****2. Dãy con chung dài nhất (longest common subsequence — lcs)**2.1. Bản chất toán học & hệ thức truy hồi2.2. Kỹ thuật khôi phục chuỗi lcs tối ưu (traceback)**3. Khoảng cách biến đổi chuỗi (edit distance / levenshtein distance)**3.1. Bản chất 3 phép biến đổi3.2. Hệ thức chuyển trạng thái**4. Phân biệt rạch ròi: Chuỗi con đối xứng (substring) vs dãy con đối xứng (subsequence)**4.1. Chuỗi con liên tiếp đối xứng dài nhất (longest palindromic substring)4.2. Dãy con không liên tiếp đối xứng dài nhất (longest palindromic subsequence)**5. Các bẫy lỗi lập trình kinh điển (bug traps)****6. Mẫu cài đặt chuẩn thi đấu (competitive templates)**Mẫu 1: Dãy con chung dài nhất (lcs) kèm khôi phục chuỗi

Mẫu 2: Khoảng cách biến đổi xâu (edit distance)7. Hệ thống câu hỏi kiểm tra khái niệm (concept quiz)Câu 1 (không gian trạng thái 2-prefix):Câu 2 (trường hợp ký tự trùng khớp trong lcs):Câu 3 (cơ sở của edit distance):Câu 4 (phân biệt substring vs subsequence):Câu 5 (độ phức tạp của lcs và edit distance):Câu 6 (mối liên hệ giữa lcs và xâu con chung ngắn nhất scs):Câu 7 (tìm dãy con đối xứng dài nhất bằng lcs):Câu 8 (thứ tự duyệt trong dp đoạn con palindrome substring):Câu 9 (bẫy truy cập ký tự 0-based trong c++):Câu 10 (số phép chèn tối thiểu để tạo xâu đối xứng):Câu 11 (truy vết chuỗi thao tác edit distance):Câu 12 (so khớp chuỗi mẫu đại diện wildcard dp):Câu 13 (nén bộ nhớ lcs còn 2 dòng):Câu 14 (bài toán xâu đan xen interleaving string):Câu 15 (đếm số lần xuất hiện xâu con distinct subsequences):8. Ma trận 15 bài tập thực hành theo mức độ (p0 → p5)**Module 06: Cấu trúc dữ liệu nâng cao****Chuyên đề 16: Cấu trúc dữ liệu stl: Set, map & heap**1. Bản chất các cấu trúc dữ liệu nâng cao trong thư viện chuẩn stl2. Kỹ thuật nén tọa độ (coordinate compression)2.1. Bản chất bài toán & khi nào cần nén tọa độ?2.2. Quy trình 4 bước chuẩn mực trong c++3. Hàng đợi ưu tiên (priority queue / heap)4. Các bẫy lỗi lập trình kinh điển (bug traps)5. Mẫu cài đặt chuẩn thi đấu (competitive templates)Mẫu 1: Kỹ thuật nén tọa độ chuẩn mựcMẫu 2: Duy trì trung vị động bằng 2 heap (median of stream)Câu hỏi trắc nghiệm củng cố khái niệmCâu 1 (bản chất std::set):

[*Câu 2 \(xóa 1 phần tử trong std: : Multiset\):*](#)

[*Câu 3 \(độ phức tạp nén tọa độ\):*](#)

[*Câu 4 \(mục đích cốt lõi của nén tọa độ\):*](#)

[*Câu 5 \(cấu trúc min-heap trong c++\):*](#)

[*Câu 6 \(độ phức tạp các thao tác priority_queue\):*](#)

[*Câu 7 \(bẫy truy cập std: : Map\):*](#)

[*Câu 8 \(khác biệt giữa set và unordered_set\):*](#)

[*Câu 9 \(tìm kiếm lower_bound trên std: : Set\):*](#)

[*Câu 10 \(duy trì trung vị động\):*](#)

[*Câu 11 \(cấu trúc std: : Map lồng nhau\):*](#)

[*Câu 12 \(đếm số phần tử phân biệt trong cửa sổ\):*](#)

[*Câu 13 \(hàm std: : Unique trong c++\):*](#)

[*Câu 14 \(hàng đợi ưu tiên lưu struct / comparator\):*](#)

[*Câu 15 \(duyệt toàn bộ phần tử trong std: : Map\):*](#)

Ma trận bài tập thực hành (p0 → p5)

Chuyên đề 17: Cấu trúc dữ liệu ngăn xếp (stack) & monotonic stack

[1. Bản chất cấu trúc dữ liệu ngăn xếp \(stack\)](#)

[2. Kỹ thuật ngăn xếp đơn điệu \(monotonic stack\)](#)

[2.1. Bản chất bài toán & khi nào cần monotonic stack?](#)

[2.2. Phân tích độ phức tạp khấu hao \(amortized analysis O\(N\)\)](#)

[3. Bài toán kinh điển: Hình chữ nhật lớn nhất trên biểu đồ cột \(largest rectangle in histogram\)](#)

[4. Các bẫy lỗi lập trình kinh điển \(bug traps\)](#)

[5. Mẫu cài đặt chuẩn thi đấu \(competitive templates\)](#)

[Mẫu 1: Tìm phần tử lớn hơn tiếp theo \(next greater element\)](#)

[Mẫu 2: Hình chữ nhật lớn nhất trên biểu đồ cột \(histogram\)](#)

Câu hỏi trắc nghiệm củng cố khái niệm

[*Câu 1 \(bản chất lifo của stack\):*](#)

[*Câu 2 \(độ phức tạp monotonic stack\):*](#)

[*Câu 3 \(điều kiện dãy ngoặc đúng\):*](#)

[*Câu 4 \(lưu trữ trong monotonic stack\):*](#)

[*Câu 5 \(hình chữ nhật lớn nhất trong ma trận 0-1\):*](#)

[*Câu 6 \(biểu thức hậu tố rpn\):*](#)

[*Câu 7 \(bẫy runtime error với stack\):*](#)

[*Câu 8 \(phần tử nhỏ hơn gần nhất bên trái\):*](#)

[*Câu 9 \(mục đích của phần tử lính canh trong histogram\):*](#)

[*Câu 10 \(dãy con có tổng nhỏ nhất / min subarray\):*](#)

[*Câu 11 \(xóa k chữ số để được số nhỏ nhất\):*](#)

[*Câu 12 \(kiểm tra dãy ngoặc nhiều loại\):*](#)

[*Câu 13 \(thuật toán shunting-yard\):*](#)

[*Câu 14 \(hứng nước mưa — trapping rain water\):*](#)

[*Câu 15 \(stack dùng mảng tự tạo vs std:: : Stack\):*](#)

Ma trận bài tập thực hành (p0 → p5)

Chuyên đề 18: Cấu trúc dữ liệu hàng đợi (queue, deque) & monotonic deque

1. Bản chất cấu trúc dữ liệu hàng đợi (queue & deque)

[*1.1. Hàng đợi chuẩn \(queue — fifo\)*](#)

[*1.2. Hàng đợi hai đầu \(double-ended queue — deque\)*](#)

2. Kỹ thuật deque cửa sổ trượt min/max $O(N)$ (sliding window monotonic deque)

[*2.1. Bản chất bài toán*](#)

[*2.2. Bất biến 3 bước duy trì min cửa sổ*](#)

3. Ứng dụng nền tảng: Tìm đường đi ngắn nhất bằng queue (bfs nhập môn)

4. Các bẫy lỗi lập trình kinh điển (bug traps)

5. Mẫu cài đặt chuẩn thi đấu (competitive templates)

[*Mẫu 1: Min trên mọi cửa sổ trượt độ dài k bằng monotonic deque \$O\(N\)\$*](#)

[*Mẫu 2: Bfs tìm bước đi ngắn nhất từ 1 đến n*](#)

Câu hỏi trắc nghiệm củng cố khái niệm

[*Câu 1 \(bản chất fifo của queue\):*](#)

[*Câu 2 \(độ phức tạp monotonic deque\):*](#)

[*Câu 3 \(khi nào cần deque thay vì queue\):*](#)

[*Câu 4 \(bẫy đánh dấu visited trong bfs\):*](#)

[*Câu 5 \(duy trì max cửa sổ bằng deque\):*](#)

[*Câu 6 \(thuật toán bfs 0-1\):*](#)

[*Câu 7 \(bài toán đổi tiền ít xu nhất bằng bfs\):*](#)

[*Câu 8 \(đoạn con có tổng lớn nhất độ dài tối đa k\):*](#)

[*Câu 9 \(trạng thái rỗng của deque\):*](#)

Câu 10 (sinh các số nhị phân từ 1 đến n):

Câu 11 (truy vết đường đi trong bfs):

Câu 12 (queue vòng tròn circular queue):

Câu 13 (kiểm tra đồ thị hai phía bipartite graph):

Câu 14 (hàng đợi hai đầu trong sliding window median):

Câu 15 (số bước biến đổi từ a sang b nhỏ nhất):

Ma trận bài tập thực hành (p0 → p5)

Module 07: Đồ thị & cây truy vấn đoạn

Chuyên đề 19: Lý thuyết đồ thị cơ bản: Duyệt bfs & dfs

1. Bản chất đồ thị & các phương pháp biểu diễn

1.1. Ma trận kề (adjacency matrix)

1.2. Danh sách kề (adjacency list — chuẩn mực thi đấu)

2. Hai chiến lược duyệt đồ thị kinh điển: Bfs vs dfs

2.1. Tìm kiếm theo chiều rộng (breadth-first search — bfs)

2.2. Tìm kiếm theo chiều sâu (depth-first search — dfs)

3. Ứng dụng: Đếm số thành phần liên thông & kiểm tra chu trình

4. Các bẫy lỗi lập trình kinh điển (bug traps)

5. Mẫu cài đặt chuẩn thi đấu (competitive templates)

Mẫu 1: Dfs đếm số thành phần liên thông và tìm kích thước từng thành phần

Mẫu 2: Phát hiện chu trình trên đồ thị vô hướng bằng dfs

Câu hỏi trắc nghiệm củng cố khái niệm

Câu 1 (bộ nhớ của danh sách kề):

Câu 2 (độ phức tạp thời gian duyệt toàn bộ đồ thị):

Câu 3 (điều kiện phát hiện chu trình đồ thị vô hướng):

Câu 4 (tìm đường đi ngắn nhất không trọng số):

Câu 5 (bậc của đỉnh trong đồ thị vô hướng):

Câu 6 (kiểm tra đồ thị liên thông):

Câu 7 (thứ tự duyệt topo trên dag):

Câu 8 (đồ thị đầy đủ K N):

Câu 9 (độ sâu của cây dfs):

Câu 10 (cạnh cầu trong đồ thị bridge):

Câu 11 (đỉnh khớp trong đồ thị articulation point):

Câu 12 (cây khung của đồ thị spanning tree):

Câu 13 (đồ thị euler):

Câu 14 (bfs đa nguồn multi-source bfs):

Câu 15 (đồ thị có hướng liên thông mạnh scc):

Ma trận bài tập thực hành (p0 → p5)

Chuyên đề 20: Đồ thị lưới 2d, kỹ thuật flood fill & tính chất cây

1. Bản chất mô hình hóa lưới 2d thành đồ thị

2. Thuật toán loang (flood fill)

3. Lý thuyết cây trên đồ thị (tree properties & invariants)

4. Các bẫy lỗi lập trình kinh điển (bug traps)

5. Mẫu cài đặt chuẩn thi đấu (competitive templates)

Mẫu 1: Đếm số lượng hòn đảo và diện tích lớn nhất (flood fill dfs)

Mẫu 2: Tìm đường đi ngắn nhất trong mê cung (grid bfs)

Câu hỏi trắc nghiệm củng cố khái niệm

Câu 1 (số cạnh tối đa trên lưới 2d 4 hướng):

Câu 2 (mảng hướng 8 hướng bao gồm cả đường chéo):

Câu 3 (độ phức tạp thuật toán loang flood fill):

Câu 4 (đường đi của quân mã trên bàn cờ knight moves):

Câu 5 (đặc tính bất biến của cây n đỉnh):

Câu 6 (đường kính của cây — tree diameter):

Câu 7 (độ cao của cây khi chọn gốc):

Câu 8 (tô màu vùng kín enclosed regions):

Câu 9 (số lượng lá tối thiểu của một cây $N \geq 2$):

Câu 10 (truy vết đường đi trong mê cung 2d):

Câu 11 (thuật toán loang đa nguồn trên lưới):

Câu 12 (bậc của đỉnh trong cây):

Câu 13 (cây con subtree size):

Câu 14 (chu trình trong đồ thị lưới):

Câu 15 (số thành phần liên thông của tập ô đất liền):

Ma trận bài tập thực hành (p0 → p5)

Chuyên đề 21: Cấu trúc cây phân đoạn (segment tree) & fenwick tree (bit)

1. Bản chất bài toán truy vấn đoạn động (dynamic range queries)

2. Cây fenwick (binary indexed tree — bit)

2.1. Phép toán ma thuật: $\text{lowbit}(x) = x \& (-x)$

2.2. Hai thao tác cốt lõi siêu tinh gọn (chỉ 5 dòng code)

3. Cây phân đoạn (segment tree)

3.1. Cấu trúc cây nhị phân hoàn hảo

3.2. Ưu thế vượt trội của segment tree

4. Các bẫy lỗi lập trình kinh điển (bug traps)

5. Mẫu cài đặt chuẩn thi đấu (competitive templates)

Mẫu 1: Cây fenwick (bit) point update & range sum query

Mẫu 2: Cây phân đoạn (segment tree) range minimum query (rmq)

Câu hỏi trắc nghiệm củng cố khái niệm

Câu 1 (khi nào dùng segment tree thay vì fenwick tree):

Câu 2 (kích thước mảng segment tree):

Câu 3 (phép toán bit lowbit trong fenwick tree):

Câu 4 (bẫy số 0 trong fenwick tree):

Câu 5 (độ phức tạp khởi tạo build segment tree):

Câu 6 (đếm số cặp nghịch thế bằng fenwick tree):

Câu 7 (độ phức tạp mỗi truy vấn segment tree):

Câu 8 (cập nhật đoạn range update & điểm point query bằng fenwick):

Câu 9 (tìm kiếm k-th phần tử trên fenwick tree):

Câu 10 (nút cha và nút con trong segment tree dạng mảng):

Câu 11 (truy vấn ước chung lớn nhất range gcd):

Câu 12 (điều kiện dừng đệ quy trong query segment tree):

Câu 13 (segment tree không đệ quy — iterative segment tree):

Câu 14 (đếm số lượng số 0 trong đoạn):

Câu 15 (kỹ thuật lazy propagation — giới thiệu mở rộng):

Ma trận bài tập thực hành (p0 → p5)

Mục lục tự động theo Heading 1–4 • Nhấn vào tên mục để nhảy nhanh trong PDF

Ikhedu c++ bảng b — tổng hợp nội dung 21 chuyên đề

*File tổng hợp tự động — Nối liền toàn bộ nội dung Production Content của 21 Chuyên đề thuộc 7 Module trong khóa **C++ Bảng B Level 1**.*

Tổng số Module: 7 | Tổng số Chuyên đề: 21

Mục lục tổng quan

Module 01: Thuật toán sắp xếp & kỹ thuật mảng

- Chuyên đề 01: Sắp Xếp
- Chuyên đề 02: Hai Con Trỏ
- Chuyên đề 03: Cửa Sổ Trượt

Module 02: Mảng tiền tố & tìm kiếm nhị phân

- Chuyên đề 04: Mảng Tiền Tố
- Chuyên đề 05: Tìm Kiếm Nhị Phân
- Chuyên đề 06: Phép Toán Bit

Module 03: Số học & đại số modular

- Chuyên đề 07: Ước, Bội & Số Nguyên Tố
- Chuyên đề 08: Đồng Dư, Lũy Thừa & Nghịch Đảo
- Chuyên đề 09: Số Nguyên Lớn (BigInt)

Module 04: Đề quy, chia để trị & quay lui

- Chuyên đề 10: Đề Quy Cơ Bản
- Chuyên đề 11: Chia Để Trị
- Chuyên đề 12: Quay Lui & Nhánh Cận

Module 05: Quy hoạch động (dynamic programming)

- Chuyên đề 13: Quy Hoạch Động 1D & LIS
- Chuyên đề 14: Quy Hoạch Động 2D & Knapsack
- Chuyên đề 15: Quy Hoạch Động Chuỗi & LCS

Module 06: Cấu trúc dữ liệu nâng cao

- Chuyên đề 16: STL: set, map & Nén Tọa Độ
- Chuyên đề 17: Stack & Monotonic Stack
- Chuyên đề 18: Queue & Deque

Module 07: Đồ thị & cây truy vấn đoạn

- Chuyên đề 19: Lý Thuyết Đồ Thị, BFS & DFS
- Chuyên đề 20: Đồ Thị Lưới 2D & Flood Fill
- Chuyên đề 21: Cây Phân Đoạn & Fenwick Tree

```
<!-- ===== --
>  <!--  BẮT  ĐẦU  NỘI  DUNG  CHI  TIẾT  21  CHUYÊN  ĐỀ  -->  <!--
===== -->
```

```
=====
```

Module 01: Thuật toán sắp xếp & kỹ thuật mảng

<!-- Chuyên đề 01: Sắp Xếp | 333 dòng | 20,775 bytes -->

Chuyên đề 01: Thuật toán sắp xếp

Tối ưu hóa dữ liệu với sort & Custom Comparator trong C++

1. Khái niệm & bản chất của sắp xếp trong tối ưu thuật toán

Sắp xếp (Sorting) là quá trình tái sắp đặt các phần tử trong một tập dữ liệu theo một trật tự xác định (thường là tăng dần hoặc giảm dần theo một hoặc nhiều tiêu chí).

Trong lập trình thi đấu và khoa học máy tính, sắp xếp không đơn thuần là định dạng lại dữ liệu hiển thị, mà là một **phép biến đổi cấu trúc dữ liệu** nhằm:

- **Tạo tính đơn điệu (Monotonicity):** Đưa dãy số về trạng thái có trật tự để áp dụng các kỹ thuật tối ưu như *Hai con trỏ (Two Pointers)*, *Tìm kiếm nhị phân (Binary Search)* hoặc *Tham lam (Greedy)*.
- **Khai thác tính chất lân cận (Adjacency Property):** Gom các phần tử có giá trị bằng nhau hoặc gần nhau nhất về các vị trí liên kề, giúp giảm không gian tìm kiếm từ $O(N^2)$ xuống $O(N)$.

2. Tính chất lân cận & chứng minh toán học

2.1. Định lý về cặp phần tử có khoảng cách nhỏ nhất

Định lý: Trong một tập hợp các số thực $A = \{A_1, A_2, \dots, A_N\}$, sau khi sắp xếp tăng dần $A_1 \leq A_2 \leq \dots \leq A_N$, giá trị nhỏ nhất của $|A_i - A_j|$ (với $i \neq j$) luôn đạt được tại ít nhất một cặp phần tử kề nhau (A_k, A_{k+1}) .

2.2. Chứng minh toán học

Xét hai chỉ số bất kỳ $i < j$. Nếu $j > i + 1$ (hai phần tử không kề nhau), tồn tại phần tử trung gian A_{i+1} thỏa mãn:

$$A_i \leq A_{i+1} \leq A_j$$

Hiệu khoảng cách giữa A_i và A_j :

$$A_j - A_i = (A_j - A_{i+1}) + (A_{i+1} - A_i)$$

Vì $A_j - A_{i+1} \geq 0$, ta luôn có:

$$A_j - A_i \geq A_{i+1} - A_i$$

Hệ quả: Mọi cặp phần tử không kề nhau đều có khoảng cách lớn hơn hoặc bằng khoảng cách của cặp kề nhau (A_i, A_{i+1}) . Do đó, để tìm khoảng cách nhỏ nhất, ta chỉ cần duyệt qua $N - 1$ cặp kề nhau sau khi sắp xếp.

Ví dụ minh họa 1: Tìm khoảng cách nhỏ nhất giữa hai phần tử

Cho mảng gồm 6 phần tử chưa sắp xếp: $A = [15, 3, 9, 22, 4, 11]$

1. **Bước 1:** Sắp xếp tăng dần $O(N \log N)$:

$$A = [3, 4, 9, 11, 15, 22]$$

1. **Bước 2:** Quét $N - 1 = 5$ cặp kề nhau $O(N)$:

Cặp Kề Nhau (A_i, A_{i+1})	Tính Hiệu Số $A_{i+1} - A_i$	Hiệu Nhỏ Nhất Tạm Thời (<i>min</i>)
$(3, 4)$	$4 - 3 = 1$	1
$(4, 9)$	$9 - 4 = 5$	1
$(9, 11)$	$11 - 9 = 2$	1
$(11, 15)$	$15 - 11 = 4$	1
$(15, 22)$	$22 - 15 = 7$	1

⇒ **Kết quả:** Khoảng cách nhỏ nhất là 1 (giữa cặp 3 và 4), tìm ra trong đúng 5 phép trừ!

3. Các ứng dụng thuật toán kinh điển của sắp xếp

Dạng Bài Toán	Cách Xử Lý Chưa Sắp Xếp	Sau Khi Sắp Xếp $O(N \log N)$	Độ Phức Tạp Tối Ưu
Tìm cặp có hiệu nhỏ nhất	Duyệt mọi cặp (i, j)	So sánh $N-1$ cặp kề (A_i, A_{i+1})	$O(N \log N)$
Đếm số giá trị phân biệt	Quét trùng lặp từng phần tử	Đếm khi $A_i \neq A_{i-1}$	$O(N \log N)$
Tìm phần tử có tần suất cực đại	Bảng đếm / Quét lặp $O(N^2)$	Đếm độ dài khối bằng nhau liên tiếp	$O(N \log N)$
**Gom cụm chênh lệch $\leq K$ **	Tìm kiếm nhánh cận	Duyệt tuyến tính gom đoạn kề nhau	$O(N \log N)$

4. Hàm `sort` & nguyên lý strict weak ordering

4.1. Cú pháp chuẩn trong c++

C++ cung cấp hai hàm sắp xếp có sẵn:

- `sort(first, last)` : Sử dụng thuật toán **IntroSort** (kết hợp giữa QuickSort, HeapSort và InsertionSort), đạt độ phức tạp thời gian $O(N \log N)$ trong mọi trường hợp (trung bình và xấu nhất). Không bảo toàn thứ tự ban đầu của các phần tử bằng nhau.
- `stable_sort(first, last)` : Sử dụng thuật toán **MergeSort**, độ phức tạp $O(N \log N)$, đảm bảo bảo toàn nguyên vẹn thứ tự xuất hiện ban đầu của các phần tử có giá trị bằng nhau.

4.2. Nguyên lý strict weak ordering (toán tử so sánh nghiêm ngặt)

Một hàm so sánh `cmp(a, b)` truyền vào `sort` **bắt buộc** phải thỏa mãn 3 tiên đề toán học:

1. **Tính bất phản xạ (Irreflexivity):** `cmp(a, a)` luôn trả về `false`.
2. **Tính bất đối xứng (Asymmetry):** Nếu `cmp(a, b)` là `true` thì `cmp(b, a)` bắt buộc phải là `false`.
3. **Tính bắc cầu (Transitivity):** Nếu `cmp(a, b)` là `true` và `cmp(b, c)` là `true` thì `cmp(a, c)` phải là `true`.

Cảnh báo quan trọng:

Cảnh báo bẫy lỗi: **BẤY DẤU $\leq$ TRONG COMPARATOR**

Nếu viết `return a ≤ b;`, khi $a = b$ thì cả `cmp(a, b)` và `cmp(b, a)` đều trả về `true`
 $\Rightarrow$ Vi phạm tiên đề Bất phân xạ và Bất đối xứng $\Rightarrow$ `sort` sẽ tiếp tục truy cập vùng nhớ ngoài biên của mảng $\Rightarrow$ **RUNTIME ERROR / CRASH CHƯƠNG TRÌNH. Lưu ý quan trọng:**
 Luôn dùng toán tử so sánh nghiêm ngặt (`<` hoặc `>`). Khi hai phần tử bằng nhau ($a = b$), hàm so sánh bắt buộc phải trả về `false` !

5. Các kỹ thuật custom comparator nâng cao

5.1. Sắp xếp đa tiêu chí với vector lồng nhau (multi-criteria sorting)

Khi mỗi phần tử gồm nhiều thuộc tính số (ví dụ: điểm bắt đầu $L = a[0]$ và điểm kết thúc $R = a[1]$ của một đoạn thẳng), ta sử dụng **vector lồng nhau** `vector<vector<int>>` để tận dụng mảng sẵn có:

```
// Ví dụ: Sắp xếp các đoạn thẳng theo điểm kết thúc a[1] tăng dần,
// nếu trùng điểm kết thúc thì theo điểm bắt đầu a[0] giảm dần
bool cmpInterval(const vector<int> &a, const vector<int> &b) {
    if (a[1] != b[1]) {
        return a[1] < b[1]; // Ưu tiên kết thúc sớm hơn đứng trước
    }
    return a[0] > b[0]; // Cùng điểm kết thúc: Bắt đầu muộn hơn đứng trước
}
```

Ví dụ minh họa 2: Sắp xếp danh sách 4 đoạn thẳng

Cho 4 đoạn thẳng: $\{ [1, 5], [2, 3], [3, 6], [1, 3] \}$

- **Trước khi sắp xếp:** $[1, 5], [2, 3], [3, 6], [1, 3]$
- **Tiêu chí 1 (Điểm kết thúc tăng dần):** Các đoạn kết thúc tại `3` đứng trước, sau đó đến `5`, rồi đến `6`.
- ****Tiêu chí 2 (Cùng điểm kết thúc $\Rightarrow$ bắt đầu giảm dần):**** Giữa $[2, 3]$ và $[1, 3]$, đoạn $[2, 3]$ có điểm bắt đầu $2 > 1$ nên được xếp trước.

Kết quả sau sắp xếp: $[[2, 3], [1, 3], [1, 5], [3, 6]]$

5.2. Sắp xếp lưu chỉ số ban đầu (index tracking)

Khi bài toán yêu cầu in ra vị trí gốc của các phần tử sau khi sắp xếp, sử dụng **vector 2 chiều** `vector<vector<long long>>` lưu cặp `{giá_trị, chỉ_số_gốc}`:

```
// Khởi tạo vector 2 chiều n hàng, 2 cột: a[i][0] là giá trị, a[i][1] là chỉ số gốc
vector<vector<long long>> a(n, vector<long long>(2));

for (int i = 0; i < n; ++i) {
    cin >> a[i][0]; // Giá trị phần tử

    a[i][1] = i + 1; // Chỉ số ban đầu (1-based)
}

// sort mặc định so sánh cột 0 (giá trị), nếu bằng nhau so sánh tiếp cột 1 (chỉ số gốc)
sort(a.begin(), a.end());
```

5.3. Comparator hàm mục tiêu (objective comparison)

Bài toán ghép N chuỗi số để tạo thành số lớn nhất:

```
bool cmpConcat(const string &a, const string &b) {
    // Sắp xếp sao cho chuỗi ghép a + b lớn hơn chuỗi ghép b + a
    return a + b > b + a;
}
```

6. Mẫu cài đặt chuẩn thi đấu (competitive template)

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    // Tối ưu hóa tốc độ I/O
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;

    vector<long long> a(n);

    for (int i = 0; i < n; ++i) {
        cin >> a[i];
    }

    // Bước 1: Sắp xếp mảng  $O(N \log N)$ 
    sort(a.begin(), a.end());

    // Bước 2: Khai thác trật tự tuyến tính  $O(N)$ 
    long long min_diff = a[1] - a[0];
    for (int i = 1; i < n - 1; ++i) {
        min_diff = min(min_diff, a[i + 1] - a[i]);
    }

    cout << min_diff << "\n";
    return 0;
}
```

7. Ranh giới áp dụng: Khi nào được & không được sắp xếp?

- **ĐƯỢC PHÉP SẮP XẾP:** Khi bài toán khảo sát tính chất trên **toàn bộ tập hợp** mà không phụ thuộc vào vị trí ban đầu của phần tử (như tìm *min/max*, đếm giá trị phân biệt, tìm cặp thỏa mãn điều kiện đại số).
- **KHÔNG ĐƯỢC PHÉP SẮP XẾP:** Khi bài toán có ràng buộc gắn liền với **dòng thời gian** hoặc **vị trí liên kề nguyên thủy** (như tìm đoạn con liên tiếp, chuỗi con tăng dài nhất bảo toàn thứ tự ban đầu).

Câu hỏi trắc nghiệm củng cố khái niệm

Câu 1 (nhận diện — recognize):

Một bài toán ghi nhận biến động giá vàng trong N ngày liên tiếp: `[28, 31, 30, 27, 29]`. Đề bài yêu cầu tìm **hai ngày liên tiếp nhau có mức chênh lệch giá nhỏ nhất**. Bạn có được phép dùng hàm `sort()` để sắp xếp lại mảng này trước khi xử lý không?

- A. Được phép, vì sắp xếp luôn giúp tìm hiệu số nhỏ nhất nhanh hơn.
- B. **[Đáp án đúng]** Không được phép, vì yêu cầu "hai ngày liên tiếp" gắn chặt với trục thời gian gốc; sắp xếp lại sẽ làm đảo lộn thứ tự thời gian và dẫn đến kết quả sai hoàn toàn.
- C. Được phép, nhưng phải sắp xếp theo thứ tự giảm dần.
- D. Được phép, nếu ta lưu lại giá trị trung bình của mảng.

Giải thích: Yêu cầu đề bài là "hai thời điểm liên tiếp nhau", nghĩa là các vị trí kề nhau theo trật tự thời gian gốc. Sắp xếp lại sẽ làm đảo lộn thứ tự thời gian và dẫn đến kết quả sai hoàn toàn.

Câu 2 (dự đoán — predict):

Cho dãy tọa độ chưa sắp xếp `A = [21, 5, 13, 8, 30, 14]`. Sau khi sắp xếp tăng dần thành `[5, 8, 13, 14, 21, 30]`, để tìm khoảng cách nhỏ nhất giữa 2 điểm bất kỳ, ta chỉ cần kiểm tra những cặp số nào?

- A. `(5, 30), (8, 21), (13, 14)`
- B. **[Đáp án đúng]** `(5, 8), (8, 13), (13, 14), (14, 21), (21, 30)`
- C. Bắt buộc phải kiểm tra tất cả 15 cặp có thể tạo ra từ 6 số.
- D. Chỉ cần kiểm tra cặp đầu tiên `(5, 8)` và cặp cuối cùng `(21, 30)`.

Giải thích: Nhờ tính chất lân cận của dãy số tăng dần, khoảng cách nhỏ nhất toàn cục luôn nằm ở một trong $N - 1 = 5$ cặp kề nhau.

Câu 3 (bản chất — explain):

Trong hàm so sánh Custom Comparator `bool cmp(int a, int b)`, nếu lập trình viên viết `return a ≤ b;` thì điều gì sẽ xảy ra khi mảng có các phần tử bằng nhau và N lớn?

- A. Chương trình vẫn chạy đúng và sắp xếp ổn định.
- B. Mảng sẽ được sắp xếp theo thứ tự giảm dần.
- C. **[Đáp án đúng]** Chương trình có thể bị dừng đột ngột (Runtime Error) do vi phạm nguyên lý Strict Weak Ordering khi $a = b$.
- D. Hàm `sort` tự động chuyển sang `stable_sort` để xử lý.

Giải thích: Khi $a = b$, `cmp(a, b)` và `cmp(b, a)` đều trả về `true`, vi phạm tính bất đối xứng nghiêm ngặt khiến thuật toán `sort` truy cập vùng nhớ ngoài biên dẫn đến Crash.

Câu 4 (chuyển giao — transfer):

Khi $N = 10^5$, vì sao phương pháp sắp xếp rồi duyệt kề nhau $O(N \log N)$ lại vượt trội hơn hẳn phương pháp duyệt mọi cặp $O(N^2)$?

- A. Vì hàm `sort()` làm giảm bớt số lượng phần tử cần lưu trữ trong bộ nhớ.
- B. **[Đáp án đúng]** Vì việc sắp xếp tạo ra cấu trúc trật tự, giúp loại bỏ hàng tỷ cặp không có khả năng tối ưu mà chỉ cần xét $N-1$ cặp kề nhau.
- C. Vì `sort()` được biên dịch sang mã máy đa luồng của CPU.
- D. Vì số phép tính của 2 cách là như nhau nhưng `sort()` tốn ít bộ nhớ RAM hơn.

Giải thích: Bản chất của việc sắp xếp là biến đổi bài toán để giảm thiểu không gian tìm kiếm, giúp ta không phải kiểm tra những trường hợp chắc chắn không tối ưu.

Câu 5 (cú pháp & ứng dụng — syntax):

Cách nào sau đây là chuẩn mực và an toàn nhất trong C++ để sắp xếp một `vector<int> a` theo thứ tự giảm dần?

- A. `sort(a.begin(), a.end(), less<int>());`
- B. **[Đáp án đúng]** `sort(a.begin(), a.end(), greater<int>());` hoặc `sort(a.rbegin(), a.rend());`
- C. `sort(a.end(), a.begin());`
- D. `sort(a.begin(), a.end()); reverse(a.begin() + 1, a.end());`

Giải thích: `greater<int>()` là functor so sánh lớn hơn chuẩn của C++, hoặc dùng cặp iterator đảo ngược `rbegin()` và `rend()` để sắp xếp mảng giảm dần.

Câu 6 (phân biệt cấu trúc — compare):

Sự khác biệt cốt lõi giữa `sort` và `stable_sort` trong thư viện chuẩn C++ là gì?

- A. `sort` có độ phức tạp $O(N^2)$, còn `stable_sort` là $O(N \log N)$.
- B. **[Đáp án đúng]** `stable_sort` đảm bảo giữ nguyên thứ tự xuất hiện ban đầu của các phần tử có giá trị tương đương nhau, còn `sort` thì không đảm bảo điều này.
- C. `sort` chỉ sắp xếp được số nguyên, còn `stable_sort` sắp xếp được chuỗi.
- D. `stable_sort` không tốn thêm bất kỳ bộ nhớ phụ trợ nào ($O(1)$).

Giải thích: Tính ổn định (Stability) nghĩa là nếu $A_i = A_j$ và $i < j$, sau khi sort thì A_i vẫn đứng trước A_j . `stable_sort` đảm bảo tính chất này (dùng MergeSort).

Câu 7 (thuật toán lân cận — technique):

Sau khi sắp xếp một mảng N phần tử tăng dần, thuật toán đếm số lượng giá trị phân biệt (Distinct values) hoạt động trong thời gian bao lâu?

- A. $O(N^2)$ vì phải so sánh từng cặp.
- B. $O(N \log N)$ vì phải dùng thêm cây nhị phân tìm kiếm.
- C. **[Đáp án đúng]** $O(N)$ vì các giá trị bằng nhau đã gom thành các khối liên tiếp, chỉ cần duyệt 1 vòng và đếm khi $A[i] \neq A[i-1]$.
- D. $O(1)$ bằng công thức toán học.

Giải thích: Sau khi sort, toàn bộ các phần tử trùng lặp đều nằm liền kề. Duyệt qua mảng và tăng biến đếm mỗi khi gặp một giá trị khác với phần tử đứng trước nó chỉ tốn $O(N)$.

Câu 8 (comparator hàm mục tiêu — logic):

Trong bài toán ghép N chuỗi số $S_1, S_2, \dots, S_N$ để tạo ra số nguyên lớn nhất, tại sao hàm so sánh `bool cmp(string a, string b)` lại được định nghĩa là `return a + b > b + a; ?`

- A. Vì chuỗi có độ dài dài hơn luôn tạo ra số lớn hơn.
- B. **[Đáp án đúng]** Vì thứ tự ghép trực tiếp `a + b` so với `b + a` phản ánh chính xác đóng góp vị trí chữ số của `a` và `b` vào số ghép tổng thể, đồng thời thỏa mãn tính chất bắc cầu.
- C. Vì đây là quy ước bắt buộc của chuẩn ANSI C++.
- D. Vì phép cộng chuỗi tự động ép kiểu về số nguyên 64-bit.

Giải thích: Nếu ghép a trước b tạo ra chuỗi lớn hơn ghép b trước a ($a + b > b + a$), thì việc đặt a đứng trước b trong mảng sẽ tối ưu hóa toàn cục chuỗi kết quả.

Câu 9 (sắp xếp đa tiêu chí — multi-criteria):

Khi sắp xếp danh sách các đoạn thẳng $[L_i, R_i]$ theo tiêu chí: Điểm bắt đầu L tăng dần; nếu trùng L thì điểm kết thúc R giảm dần, comparator nào sau đây viết đúng chuẩn Strict Weak Ordering?

- A. `return (a[0] ≤ b[0]) && (a[1] ≥ b[1]);`
- B. `if (a[0] < b[0]) return true; else return a[1] > b[1];`
- C. **[Đáp án đúng]** `if (a[0] ≠ b[0]) return a[0] < b[0]; return a[1] > b[1];`
- D. `return a[0] < b[0] || a[1] > b[1];`

Giải thích: Phải kiểm tra sự khác biệt của tiêu chí chính trước ($a[0] \neq b[0]$). Chỉ khi tiêu chí chính bằng nhau mới so sánh tiêu chí phụ. Cả hai nhánh đều phải dùng toán tử nghiêm ngặt $<$ hoặc $>$.

Câu 10 (xử lý kiểu dữ liệu & tràn số — robustness):

Cho bài toán tìm khoảng cách nhỏ nhất giữa 2 điểm trong N điểm trên trục tọa độ, với tọa độ $X_i \in [-10^{18}, 10^{18}]$. Sai lầm nguy hiểm nhất khi duyệt cặp kề nhau (X_i, X_{i+1}) là gì?

- A. Sử dụng `sort` thay vì tự viết QuickSort.
- B. **[Đáp án đúng]** Lưu biến kết quả bằng kiểu `int` hoặc `long` (32-bit), gây tràn số âm khi tính hiệu $X_{i+1} - X_i$.
- C. Đọc dữ liệu bằng `cin` có Fast I/O.
- D. Duyệt vòng lặp từ `i = 0` đến $N - 2$.

Giải thích: Với X_i lên tới 10^{18} , khoảng cách giữa 2 điểm có thể đạt tới 2×10^{18} , vượt xa giới hạn 2×10^9 của kiểu `int`. Bắt buộc phải dùng kiểu `long long` (64-bit) cho toàn bộ mảng và biến tính khoảng cách.

Ma trận bài tập thực hành (p0 → p5)

STT	Mã Bài	Tên Bài Toán	Cấp Độ	Ràng Buộc Dữ Liệu	Mục Tiêu Rèn Luyện
01	CPPB-SX-01	Xếp Hàng Điểm Danh	P0	$N \leq 1000, A_i \leq 10^6$	Cú pháp <code>sort</code> cơ bản
02	CPPB-SX-02	Khoảng Cách Nhỏ Nhất	P1	$N \leq 10^5, A_i \leq 10^9$	Sắp xếp duyệt cặp kề
03	CPPB-SX-03	Sắp Xếp Theo Trị Tuyệt Đối	P1	$N \leq 10^5, \text{vert } A_i \text{ vert } \leq 10^9$	Custom comparator cơ bản
04	CPPB-SX-04	Đếm Giá Trị Phân Biệt	P2	$N \leq 2 \times 10^5, \text{vert } A_i \text{ vert } \leq 10^9$	Gom nhóm sau sắp xếp
05	CPPB-SX-05	Hai Trạm Kiểm Soát Gần Nhau Nhất	P2	$N \leq 10^5, X_i \leq 10^{12}$	Xử lý dữ liệu lớn <code>long long</code>
06	CPPB-SX-06	Khoảng Trống Lớn Nhất Trên Trục Tọa Độ	P3	$N \leq 10^5, \text{vert } A_i \text{ vert } \leq 10^{18}$	Khai thác trật tự tuyến tính
07	CPPB-SX-07	Sắp Xếp Theo Tổng Chữ Số	P1	$N \leq 10^5, A_i \leq 10^9$	Hàm biến đổi phụ trong comparator
08	CPPB-SX-08	Gom Cụm Chênh Lệch Không Quá K	P2	$N \leq 2 \times 10^5, A_i \leq 10^9$	Tham lam tuyến tính trên mảng sắp xếp
09	CPPB-SX-09	Tìm Phần Tử Xuất Hiện Nhiều Nhất	P2	$N \leq 2 \times 10^5, \text{vert } A_i \text{ vert } \leq 10^9$	Đếm tần suất khối liên tiếp
10	CPPB-SX-10	Sắp Xếp Lưu Vị Trí Ban Đầu	P3	$N \leq 10^5, \text{vert } A_i \text{ vert } \leq 10^9$	Theo dõi chỉ số gốc (Index Tracking)
11	CPPB-SX-11	Ghép Chuỗi Tạo Số Lớn Nhất	P3	$N \leq 10^4$	So sánh chuỗi bắc cầu (<code>a+b > b+a</code>)

12	CPPB-SX-12	Bảng Điểm Học Sinh Đa Trường	P4	$N \leq 10^5$	Sắp xếp đa khóa ưu tiên
13	CPPB-SX-13	Bảng Xếp Hạng Giải Đấu Thể Thao	P4	$N \leq 10^5$	Sắp xếp tổ hợp 4 tiêu chí
14	CPPB-SX-14	Sắp Xếp Đoạn Thẳng Không Giao Lỗi	P5	$N \leq 2 \times 10^5$	Strict Weak Ordering & <code>stable_sort</code>

<!-- Chuyên đề 02: Hai Con Trỏ | 321 dòng | 21,143 bytes -->

Chuyên đề 02: Kỹ thuật hai con trỏ

Tối ưu hóa tìm kiếm tuyến tính $O(N)$ trong C++

1. Khái niệm & nguyên lý hoạt động

Kỹ thuật Hai con trỏ (Two Pointers Technique) là phương pháp sử dụng hai biến chỉ số (thường ký hiệu là L và R) duyệt trên cấu trúc dữ liệu tuyến tính (mảng hoặc chuỗi) nhằm thu hẹp không gian tìm kiếm từ $O(N^2)$ xuống $O(N)$.

Trong mô hình **Hai con trỏ đối đầu (Opposite-direction Two Pointers)**:

- Con trỏ trái L khởi tạo tại đầu mảng ($L = 0$).
- Con trỏ phải R khởi tạo tại cuối mảng ($R = N - 1$).
- Dãy số bắt buộc phải có **tính đơn điệu** (thường là mảng đã sắp xếp tăng dần $A_0 \leq A_1 \leq \dots \leq A_{N-1}$).

Tại mỗi bước, thuật toán tính toán một hàm mục tiêu trên cặp phần tử (A_L, A_R) (ví dụ: $Sum = A_L + A_R$) và so sánh với giá trị đích S :

- **Nếu $Sum == S$:** Tìm thấy nghiệm hợp lệ.
- **Nếu $Sum < S$:** Tổng hiện tại nhỏ hơn mục tiêu $\Rightarrow$ Tăng con trỏ trái ($++L$) để tìm kiếm tổng lớn hơn.
- **Nếu $Sum > S$:** Tổng hiện tại lớn hơn mục tiêu $\Rightarrow$ Giảm con trỏ phải ($--R$) để tìm kiếm tổng nhỏ hơn.

2. Chứng minh bất biến lặp (loop invariant) & tính đúng đắn

2.1. Phát biểu bất biến lặp

Bất biến lặp: Tại bất kỳ thời điểm nào trong quá trình thực thi, nếu tồn tại một cặp nghiệm (i, j) thỏa mãn $A_i + A_j = S$ ($i < j$), thì cặp nghiệm đó chắc chắn nằm trọn vẹn trong đoạn chỉ số đang xét: $L \leq i < j \leq R$.

2.2. Chứng minh quy nạp toán học

1. **Khởi tạo:** Ban đầu $L = 0, R = N - 1$, đoạn $[L, R]$ bao phủ toàn bộ mảng, bất biến lặp hiển nhiên đúng.
2. **Duy trì:** Giả sử bất biến lặp đúng tại bước hiện tại $[L, R]$.
 - **Trường hợp 1: $A_L + A_R > S$ (Thao tác $--R$):**

Vì mảng tăng dần, với mọi chỉ số $k \in [L, R-1]$, ta luôn có $A_k \geq A_L \Rightarrow A_k + A_R \geq A_L + A_R > S$.

Do đó, phần tử A_R không thể tạo thành tổng S với bất kỳ phần tử nào trong đoạn $[L, R-1]$. Việc loại bỏ R bằng cách giảm $R \rightarrow R - 1$ không làm mất bất kỳ nghiệm hợp lệ nào.

• ****Trường hợp 2:** $A_L + A_R < S$ (Thao tác **++L**):**

Vì mảng tăng dần, với mọi chỉ số $k \in [L+1, R]$, ta luôn có $A_k \leq A_R \Rightarrow A_L + A_k \leq A_L + A_R < S$.

Do đó, phần tử A_L không thể tạo thành tổng S với bất kỳ phần tử nào trong đoạn $[L+1, R]$. Việc loại bỏ L bằng cách tăng $L \rightarrow L + 1$ là an toàn **100%**.

1. **Kết thúc:** Vòng lặp dừng khi $L \geq R$. Nếu không tìm thấy nghiệm, chứng tỏ không tồn tại cặp (i, j) nào thỏa mãn $A_i + A_j = S$.

3. Các mô hình bài toán đặc trưng

3.1. Mô hình 1: Tìm cặp số có tổng đúng bằng S (two sum)

- **Mục tiêu:** Tìm $i < j$ sao cho $A_i + A_j = S$.
- **Quy tắc di chuyển:**
 - Nếu $A_L + A_R < S \Rightarrow L \rightarrow L + 1$ (Tổng nhỏ hơn mục tiêu, tăng cận dưới).
 - Nếu $A_L + A_R > S \Rightarrow R \rightarrow R - 1$ (Tổng lớn hơn mục tiêu, giảm cận trên).
 - Nếu $A_L + A_R = S \Rightarrow$ Ghi nhận nghiệm và dừng thuật toán.

Ví dụ minh họa 1: Tìm cặp số có tổng $S = 14$

Cho mảng $N = 6$ phần tử đã sắp xếp: $A = [2, 3, 5, 8, 11, 15]$ (0-based indexing).

Phần tử	$A[0]$	$A[1]$	$A[2]$	$A[3]$	$A[4]$	$A[5]$
Giá trị	2	3	5	8	11	15

Bảng mô phỏng từng bước lặp Hai con trỏ:

Bước	Con trỏ (L, R)	Giá trị (A_L, A_R)	Tổng $A_L + A_R$	Đánh giá & Hành động
1	(0, 5)	(2, 15)	$2 + 15 = 17$	$17 > 14 \Rightarrow$ Giảm con trỏ phải: $R \rightarrow 4$
2	(0, 4)	(2, 11)	$2 + 11 = 13$	$13 < 14 \Rightarrow$ Tăng con trỏ trái: $L \rightarrow 1$
3	(1, 4)	(3, 11)	$3 + 11 = 14$	$14 = 14 \Rightarrow$ Khớp mục tiêu! Cặp nghiệm $(A_1, A_4) = (3, 11)$

3.2. Mô hình 2: Đếm số cặp có tổng thỏa mãn bất đẳng thức $A_i + A_j \leq S$

- **Mục tiêu:** Đếm số lượng cặp (i, j) với $i < j$ thỏa mãn $A_i + A_j \leq S$.
- **Khai thác tổ hợp:**

Nếu tại bước (L, R) ta có $A_L + A_R \leq S$, thì do mảng tăng dần, mọi phần tử A_k với $L < k \leq R$ khi ghép với A_L đều thỏa mãn:

$$A_L + A_k \leq A_L + A_R \leq S$$

Do đó, có đúng $R - L$ cặp hợp lệ** xuất phát từ L : $(L, L+1), (L, L+2), \dots, (L, R)$.

- **Thao tác:** Cộng $(R - L)$ vào kết quả đếm, sau đó tăng $L \leftarrow L + 1$. Ngược lại, nếu $A_L + A_R > S$, giảm $R \leftarrow R - 1$.

Ví dụ minh họa 2: Đếm số cặp có tổng ≤ 10 trên mảng $A = [1, 2, 4, 7, 9]$

Bước	Con trỏ (L, R)	Tổng $A_L + A_R$	Điều kiện ≤ 10	Số cặp đếm được $(R - L)$	Hành động kế tiếp
1	$(0, 4) \rightarrow (1, 9)$	$1 + 9 = 10$	≤ 10 (Hợp lệ)	$+ (4 - 0) = 4$ cặp: $(1,2), (1,4), (1,7), (1,9)$	Tăng $L \leftarrow 1$
2	$(1, 4) \rightarrow (2, 9)$	$2 + 9 = 11$	> 10 (Vi phạm)	$+ 0$ cặp	Giảm $R \leftarrow 3$
3	$(1, 3) \rightarrow (2, 7)$	$2 + 7 = 9$	≤ 10 (Hợp lệ)	$+ (3 - 1) = 2$ cặp: $(2,4), (2,7)$	Tăng $L \leftarrow 2$
4	$(2, 3) \rightarrow (4, 7)$	$4 + 7 = 11$	> 10 (Vi phạm)	$+ 0$ cặp	Giảm $R \leftarrow 2$
Dừng	$(2, 2)$	—	$L \geq R$	**Tổng số cặp thỏa mãn = $4 + 2 = 6$ cặp**	Kết thúc thuật toán

3.3. Mô hình 3: Ghép cặp cực trị tham lam (bài toán thuyền cứu hộ / xe chờ hàng)

- **Bài toán:** Mỗi xe chờ tối đa 2 kiện hàng có tổng trọng lượng $\leq C$. Tìm số xe ít nhất để chở hết N kiện hàng.
- **Chiến lược:** Sắp xếp mảng trọng lượng tăng dần. Đặt $L = 0, R = N - 1$.
- Thử ghép kiện nặng nhất A_R với kiện nhẹ nhất A_L .
- Nếu $A_L + A_R \leq C$: Cả hai kiện đi chung xe $\Rightarrow L \leftarrow L + 1, R \leftarrow R - 1$.
- Nếu $A_L + A_R > C$: Kiện A_R buộc phải đi xe riêng $\Rightarrow R \leftarrow R - 1$.
- Mỗi lần lặp tốn 1 xe (++ans).

3.4. Mô hình 4: Khử chiều đa biến (bài toán 3-sum và 4-sum)

- **Bài toán 3-Sum:** Tìm bộ ba (i, j, k) có tổng $A_i + A_j + A_k = S$.
- **Chiến lược:** Sắp xếp mảng. Cố định phần tử thứ nhất i từ 0 đến $N - 3$, chuyển bài toán về tìm 2 số trong đoạn $[i+1 \dots N-1]$ có tổng bằng $S - A_i$ bằng Two Pointers.
- **Độ phức tạp:** Giảm từ $O(N^3)$ xuống $O(N^2)$.

4. Phân tích độ phức tạp thời gian & không gian

- **Thời gian (Time Complexity):**
 - Bước sắp xếp: $O(N \log N)$.
 - Bước duyệt Hai con trỏ: $O(N)$ (do tại mỗi phép so sánh, ít nhất một trong hai con trỏ di chuyển 1 bước, tổng số bước di chuyển tối đa là N).
 - Tổng thời gian: $O(N \log N + N) = O(N \log N)$.
- **Không gian bộ nhớ (Space Complexity):**
 - $O(1)$ bộ nhớ phụ trợ khi xử lý trực tiếp trên mảng (in-place).

5. Mẫu cài đặt chuẩn thi đấu (competitive template)

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    long long s;
    if (!(cin >> n >> s)) return 0;

    vector<long long> a(n);

    for (int i = 0; i < n; ++i) {
        cin >> a[i];
    }

    // Bước 1: Sắp xếp mảng tạo tính đơn điệu  $O(N \log N)$ 
    sort(a.begin(), a.end());

    // Bước 2: Khởi tạo Hai con trỏ đối đầu  $O(N)$ 
    int l = 0, r = n - 1;
    bool found = false;

    while (l < r) {
        long long current_sum = a[l] + a[r];
        if (current_sum == s) {
            cout << a[l] << " " << a[r] << "\n";
            found = true;
            break;
        } else if (current_sum < s) {
            ++l; // Tổng nhỏ hơn mục tiêu → tăng giá trị cận dưới
        } else {
            --r; // Tổng lớn hơn mục tiêu → giảm giá trị cận trên
        }
    }

    if (!found) {
        cout << -1 << "\n";
    }

    return 0;
}
```

6. Các bẫy lỗi kỹ thuật thường gặp (bug traps)

1. **Bẫy điều kiện dừng** $l \leq r$ thay vì $l < r$: Khi $L = R$, phần tử A_L tự cộng với chính nó ($2 \cdot A_L$), vi phạm yêu cầu chọn hai vị trí phân biệt ($i < j$).
2. **Bẫy tràn số nguyên 32-bit**: Khi các phần tử $A_i \approx 10^9$, tổng $A_L + A_R$ có thể đạt $2 \cdot 10^9$, vượt soát giới hạn kiểu `int` ($2^{31}-1$). Bắt buộc sử dụng `long long` cho biến tính tổng.
3. **Bẫy mảng chưa sắp xếp**: Áp dụng Hai con trỏ trên mảng chưa có trật tự đơn điệu sẽ dẫn đến sai lệch logic hoàn toàn.

Câu hỏi trắc nghiệm củng cố khái niệm

Câu 1 (nhận diện — recognize):

Kỹ thuật hai con trỏ đối đầu ($L = 0, R = N - 1$) có thể áp dụng trực tiếp trên dãy số nào sau đây?

- A. Dãy số ngẫu nhiên ban đầu chưa qua xử lý.
- B. **[Đáp án đúng]** Dãy số đã được sắp xếp tăng dần hoặc giảm dần đơn điệu.
- C. Dãy số có tổng các phần tử bằng 0.
- D. Dãy số chỉ gồm toàn các số nguyên dương lẻ.

Giải thích: Tính chất đơn điệu là điều kiện tiên quyết để việc dịch chuyển con trỏ không bỏ sót nghiệm. Do đó mảng bắt buộc phải có thứ tự đơn điệu.

Câu 2 (dự đoán — predict):

Cho mảng đã sắp xếp $A = [3, 7, 11, 15, 20]$ và mục tiêu $S = 22$. Tại bước khởi đầu với $L = 0$ ($A[0]=3$) và $R = 4$ ($A[4]=20$), tổng là $3 + 20 = 23 > 22$. Hành động đúng tiếp theo là gì?

- A. Tăng con trỏ trái $L = L + 1$.
- B. **[Đáp án đúng]** Giảm con trỏ phải $R = R - 1$ (đưa R về vị trí 3 có giá trị 15).
- C. Dừng thuật toán và kết luận không có nghiệm.
- D. Hoán đổi giá trị của $A[L]$ và $A[R]$.

Giải thích: Do tổng hiện tại lớn hơn mục tiêu S , phần tử lớn nhất $A[R]=20$ cộng với phần tử nhỏ nhất $A[L]=3$ đã vượt quá 22, nên 20 không thể ghép với bất kỳ số nào khác để tạo ra 22. Ta phải loại bỏ 20 bằng cách giảm R .

Câu 3 (bản chất — explain):

Trong bài toán đếm số cặp có $A_i + A_j \leq S$ trên mảng tăng dần, khi $A[L] + A[R] \leq S$, tại sao ta có thể khẳng định ngay có đúng $R - L$ cặp hợp lệ kết thúc tại R ?

- A. Vì $R - L$ là độ dài của mảng ban đầu.
- B. **[Đáp án đúng]** Vì mảng tăng dần nên với mọi k thỏa mãn $L < k \leq R$, ta luôn có $A_L + A_k \leq A_L + A_R \leq S$.
- C. Vì hàm `sort` tự động nhóm các cặp này lại với nhau.
- D. Vì số lượng cặp luôn bằng hiệu hai con trỏ trong mọi bài toán.

Giải thích: Do $A[L]$ đã thỏa mãn khi cộng với $A[R]$, tất cả các phần tử từ $L+1$ đến R khi ghép với $A[L]$ đều có tổng $\leq S$. Có đúng $R - L$ cặp như vậy xuất phát từ L .

Câu 4 (chuyển giao — transfer):

Độ phức tạp thời gian tổng thể của bài toán Two Sum gồm 2 bước: Sắp xếp mảng N phần tử bằng `sort` rồi duyệt bằng Two Pointers là bao nhiêu?

- A. $O(N^2)$
- B. $O(N)$
- C. **[Đáp án đúng]** $O(N \log N)$ (trong đó sắp xếp mất $O(N \log N)$ và duyệt Two Pointers mất $O(N)$).
- D. $O(\log N)$

Giải thích: Bước sắp xếp tốn $O(N \log N)$, bước duyệt 2 con trỏ tốn $O(N)$. Tổng thời gian bị chi phối bởi bước sắp xếp là $O(N \log N)$, nhanh hơn vượt bậc so với vét cạn $O(N^2)$.

Câu 5 (bẫy điều kiện dừng — bug traps):

Tại sao trong vòng lặp Two Pointers tìm cặp phần tử phân biệt ($i < j$), ta bắt buộc phải dùng điều kiện `while (l < r)` thay vì `while (l ≤ r)`?

- A. Vì nếu dùng $\leq$ thì chương trình sẽ bị lỗi tràn bộ nhớ (Out of Memory).
- B. **[Đáp án đúng]** Vì khi $L = R$, phần tử $A[L]$ sẽ tự cộng với chính nó ($2 * A[L]$), vi phạm yêu cầu chọn 2 vị trí phân biệt của đề bài.
- C. Vì trình biên dịch C++ không hỗ trợ toán tử $\leq$ trong vòng lặp `while`.
- D. Vì khi $L = R$ con trỏ sẽ nhảy về vị trí 0.

Giải thích: Cặp nghiệm đòi hỏi 2 chỉ số khác nhau $i < j$. Khi $L = R$, hai con trỏ trỏ vào cùng 1 phần tử, không thể tạo thành một cặp 2 phần tử phân biệt.

Câu 6 (chiến lược tham lam — greedy pairing):

Trong bài toán **Ghép thuyền cứu hộ** (mỗi thuyền chở tối đa 2 người có tổng cân nặng $\leq C$), tại sao khi $W[L] + W[R] > C$, ta lại để người nặng nhất $W[R]$ đi thuyền riêng một mình?

- A. Vì người nặng nhất luôn có quyền ưu tiên đi một mình.
- B. **[Đáp án đúng]** Vì người nặng nhất $W[R]$ ghép với người nhẹ nhất hiện tại $W[L]$ mà vẫn bị quá tải, thì $W[R]$ không thể ghép được với bất kỳ ai khác *implies* Bắt buộc phải đi riêng.
- C. Vì ta muốn dành người nhẹ nhất $W[L]$ cho một người khác nặng hơn.
- D. Vì thuật toán muốn giảm số lượng thuyền xuống mức tối thiểu.

Giải thích: Nếu người nhẹ nhất trong tập hợp còn lại mà không thể đi chung với $W[R]$, thì bất kỳ ai khác (đều có cân nặng $\geq W[L]$) khi đi cùng $W[R]$ cũng sẽ làm quá tải thuyền.

Câu 7 (khử chiều đa biến — dimensionality reduction):

Đối với bài toán **3-Sum** (tìm 3 số $A_i + A_j + A_k = S$ với $i < j < k$), kỹ thuật Two Pointers giúp tối ưu hóa thuật toán như thế nào?

- A. Giảm từ $O(N^2)$ xuống $O(N \log N)$.
- B. **[Đáp án đúng]** Cố định chỉ số i bằng 1 vòng for ($O(N)$), sau đó dùng Two Pointers trên đoạn $[i+1 \dots N-1]$ ($O(N)$) để tìm $A_j + A_k = S - A_i$, giảm tổng thời gian từ $O(N^3)$ xuống $O(N^2)$.
- C. Chạy 3 con trỏ cùng lúc từ 3 đầu mảng trong $O(N)$.
- D. Tính tổng tiền tố của 3 mảng con trong $O(1)$.

Giải thích: Bằng cách cố định 1 biến, bài toán 3 biến quy về bài toán Two Sum 2 biến trên đoạn còn lại, giúp giảm đúng 1 bậc lũy thừa của độ phức tạp thời gian.

Câu 8 (xử lý trùng lặp — duplicates handling):

Khi mảng có nhiều phần tử bằng nhau (ví dụ: `[2, 2, 2, 2]` và `S = 4`), để đếm chính xác số lượng cặp có tổng bằng `S` mà không bị chạy $O(N^2)$, ta xử lý như thế nào?

- **A.** Xóa bỏ tất cả các phần tử trùng lặp trước khi chạy.
- **B. [Đáp án đúng]** Đếm số lượng phần tử bằng nhau liên tiếp ở 2 đầu `L` và `R` (ví dụ có `cnt_L` số bằng `A[L]` và `cnt_R` số bằng `A[R]`), sau đó cộng `cnt_L * cnt_R` vào kết quả (hoặc `(cnt_L * (cnt_L - 1)) / (2)` nếu `A[L] = A[R]`).
- **C.** Chỉ duyệt một lần và bỏ qua các số giống nhau.
- **D.** Dùng vòng lặp lồng nhau duyệt lại đoạn trùng.

Giải thích: Nhân trực tiếp số lượng tần suất ở 2 đầu cho phép nhảy qua toàn bộ khối phần tử trùng lặp trong $O(1)$, giữ nguyên độ phức tạp tuyến tính $O(N)$.

Câu 9 (hai mảng độc lập — multi-array pointers):

Cho 2 mảng đã sắp xếp tăng dần `A` kích thước `N` và `B` kích thước `M`. Để tìm giá trị nhỏ nhất của `|Ai - Bj|`, thuật toán Hai con trỏ điều khiển con trỏ `i` (trên `A`) và `j` (trên `B`) như thế nào?

- **A.** Luôn tăng `i` trước, sau đó tăng `j`.
- **B. [Đáp án đúng]** So sánh `A[i]` và `B[j]`: Nếu `A[i] < B[j]` thì tăng `++i`; nếu `A[i] > B[j]` thì tăng `++j`; nếu bằng nhau thì khoảng cách bằng 0 (dừng lại).
- **C.** Đặt `i = 0` và `j = M - 1` rồi thu hẹp vào giữa.
- **D.** Tăng cả hai con trỏ `++i` và `++j` đồng thời tại mỗi bước.

Giải thích: Muốn thu hẹp khoảng cách giữa `A[i]` và `B[j]`, ta phải tăng phần tử có giá trị nhỏ hơn để nó tiến gần hơn đến giá trị của phần tử lớn hơn.

Câu 10 (phòng thủ kiểu dữ liệu — data overflow):

Trong bài toán Two Sum với các phần tử mảng `Ai ∈ [1, 109]` và `S = 2 * 109`, phát biểu nào sau đây về kiểu dữ liệu là chính xác?

- **A.** Dùng kiểu `int` cho biến `sum = a[l] + a[r]` là hoàn toàn an toàn vì `2 * 109 < 231 - 1`.
- **B. [Đáp án đúng]** Biến `current_sum` và biến đếm số lượng cặp bắt buộc phải khai báo `long long` để phòng ngừa tràn số 32-bit (số lượng cặp có thể lên tới $(N(N-1))/(2) \approx 5 * 10^9$).
- **C.** Chỉ cần dùng kiểu `double` là giải quyết được mọi trường hợp.
- **D.** Không cần quan tâm kiểu dữ liệu vì compiler tự động ép kiểu 64-bit.

*Giải thích: Giá trị tổng $A[L] + A[R]$ có thể vượt ngưỡng $2^{31}-1$ khi các số lớn hơn 10^9 , và số lượng cặp đếm được với $N = 2 \cdot 10^5$ có thể đạt tới $2 \cdot 10^{10}$, bắt buộc phải dùng **long long** cho biến đếm.*

Ma trận bài tập thực hành (p0 → p5)

STT	Mã Bài	Tên Bài Toán	Cấp Độ	Ràng Buộc Dữ Liệu	Mục Tiêu Rèn Luyện
01	CPPB- HCT-01	Mô Phòng Hai Con Trò Đối Đầu	P0	$N \leq 10^5, A_i \leq 10^9$	Cơ chế co hẹp $L \rightarrow R$
02	CPPB- HCT-02	Cặp Số Có Tổng Bằng S (Two Sum)	P1	$N \leq 10^5, A_i \leq 10^9$	Sắp xếp + Hai con trỏ
03	CPPB- HCT-03	Đếm Cặp Có Tổng Không Quá S	P2	$N \leq 2 \times 10^5, A_i \leq 10^9$	Cộng dồn tổ hợp đoạn ($R - L$)
04	CPPB- HCT-04	Đếm Cặp Có Tổng Lớn Hơn Hoặc Bằng S	P2	$N \leq 2 \times 10^5, A_i \leq 10^9$	Biến thể chặn dưới tổ hợp
05	CPPB- HCT-05	Ghép Thuyền Cứu Hộ Tối Ưu	P3	$N \leq 10^5, C \leq 10^9$	Tham lam ghép cực trị
06	CPPB- HCT-06	Vận Chuyển Thùng Hàng Cực Đại	P4	$N \leq 10^5, W_i \leq 10^{12}$	Ghép cặp với dữ liệu cực lớn
07	CPPB- HCT-07	Tìm Cặp Có Tổng Gần S Nhất	P1	$N \leq 10^5, A_i \leq 10^9$	Tối ưu sai số tuyệt đối
08	CPPB- HCT-08	Tìm Cặp Có Hiệu Đúng Bằng K	P2	$N \leq 10^5, K \leq 10^{18}$	Hai con trỏ truy vết hiệu
09	CPPB- HCT-09	Bộ Ba Số Có Tổng Bằng S (3-Sum)	P3	$N \leq 3000, A_i \leq 10^9$	Cố định 1 phần tử + Two Pointers
10	CPPB- HCT-10	Đếm Số Tam Giác Có Thể Tạo Thành	P3	$N \leq 3000, A_i \leq 10^9$	Cố định cạnh lớn nhất + Two Pointers
11	CPPB- HCT-11	Đếm Cặp Tổng S Trên Mảng Trùng Lặp	P4	$N \leq 2 \times 10^5$	Xử lý tần suất giá trị trùng nhau
12	CPPB- HCT-12	Ghép Cặp Trẻ Em Và Bánh Quy	P4	$N, M \leq 10^5$	Hai con trỏ trên 2 mảng khác nhau
13	CPPB- HCT-13	Bộ Bốn Số Có Tổng Bằng S (4-Sum)	P5	$N \leq 1000$	Cố định 2 phần tử + Two Pointers
14	CPPB- HCT-14	Cặp Số Tối Ưu Với Chênh Lệch Cực Hạn	P5	$N, M \leq 2 * 10^5$	Tìm $\min A_i - B_j $ tuyến tính

<!-- Chuyên đề 03: Cửa Sổ Trượt | 316 dòng | 20,832 bytes -->

Chuyên đề 03: Kỹ thuật cửa sổ trượt

Tối ưu hóa đoạn con tuyến tính $O(N)$ trong C++

1. Khái niệm & bản chất của kỹ thuật cửa sổ trượt

Kỹ thuật Cửa sổ trượt (Sliding Window Technique) là phương pháp tối ưu hóa trên cấu trúc dữ liệu mảng hoặc chuỗi nhằm giải quyết các bài toán liên quan đến **đoạn con liên tiếp (Contiguous Subarray / Substring)**.

Thay vì phải tính toán lại từ đầu hàm mục tiêu trên từng đoạn con $[i \dots j]$ với độ phức tạp $O(K)$ hoặc $O(N)$, kỹ thuật này duy trì một "khung cửa sổ" $[L \dots R]$ và cập nhật trạng thái mục tiêu trong **** $O(1)$ thời gian**** bằng cách:

$$State_mới = State_cũ + Phần tử nạp vào A_R - Phần tử nhả ra A_{L-1}$$

2. Cơ chế chuyển dịch & phân tích độ phức tạp $O(N)$

2.1. Cơ chế hai con trỏ cùng chiều ($L \longrightarrow R$)

- ****Con trỏ phải R (Right / Lead pointer):**** Mở rộng biên phải để nạp thêm phần tử A_R vào cửa sổ nhằm thỏa mãn điều kiện bài toán.
- ****Con trỏ trái L (Left / Trail pointer):**** Co hẹp biên trái để loại bỏ phần tử A_L ra khỏi cửa sổ nhằm tối ưu hóa kích thước hoặc khôi phục tính hợp lệ của cửa sổ.

2.2. Phân tích chi phí khấu hao (amortized complexity analysis)

Mặc dù thuật toán thường được cài đặt dưới dạng một vòng lặp **while** lồng bên trong một vòng lặp **for**:

- Con trỏ R duyệt từ 0 đến $N - 1$ (thực hiện đúng N bước tăng).
- Con trỏ L duyệt từ 0 đến N (thực hiện tối đa N bước tăng).
- **Mỗi phần tử trong mảng chỉ đi vào cửa sổ đúng 1 lần và ra khỏi cửa sổ tối đa 1 lần.**

Do đó, tổng số thao tác thêm/bớt phần tử trong toàn bộ chương trình không bao giờ vượt quá $2N$. Độ phức tạp thời gian đạt **** $O(N)$ tuyến tính tuyệt đối****.

3. Phân loại hai dạng cửa sổ trượt chuẩn mực

3.1. Dạng 1: Cửa sổ cố định độ dài K (fixed-size window)

Áp dụng cho các bài toán yêu cầu khảo sát mọi đoạn con liên tiếp có độ dài đúng bằng K .

- Công thức trượt $O(1)$:
- Khởi tạo: $Current_Sum = \sum_{i=0}^{K-1} A_i$.
- Trượt từ vị trí $i = K$ đến $N - 1$:

$$Current_Sum \leftarrow Current_Sum + A_i - A_{i-K}$$

- Cập nhật giá trị cực trị: $Ans = \max(Ans, Current_Sum)$.

Ví dụ minh họa 1: Tìm tổng đoạn con $K=3$ lớn nhất trên dãy $A = [2, 1, 5, 1, 3, 2]$

Phần tử	$A[0]$	$A[1]$	$A[2]$	$A[3]$	$A[4]$	$A[5]$
Giá trị	2	1	5	1	3	2

Bảng mô phỏng quá trình trượt cửa sổ:

Vị trí i	Đoạn cửa sổ	Thao tác nạp / loại	Tổng mới (Sum)	Max_Sum
Khởi tạo ($i=2$)	$[2, 1, 5]$ (đoạn $0..2$)	Tính tổng 3 phần tử đầu	$2 + 1 + 5 = 8$	8
** $i = 3$ **	$[1, 5, 1]$ (đoạn $1..3$)	$+ A_3(1) - A_0(2)$	$8 + 1 - 2 = 7$	8
** $i = 4$ **	$[5, 1, 3]$ (đoạn $2..4$)	$+ A_4(3) - A_1(1)$	$7 + 3 - 1 = 9$	9
** $i = 5$ **	$[1, 3, 2]$ (đoạn $3..5$)	$+ A_5(2) - A_2(5)$	$9 + 2 - 5 = 6$	9

⇒ Kết quả: Tổng lớn nhất của đoạn dài 3 là 9 (đoạn $[5, 1, 3]$), trượt trong đúng $O(1)$ mỗi bước!

3.2. Dạng 2: Cửa sổ biến thiên (variable-size window)

Áp dụng cho các bài toán tìm đoạn con liên tiếp dài nhất/ngắn nhất hoặc đếm số lượng đoạn con thỏa mãn điều kiện $f[L \dots R]$.

Dạng Bài Toán	Chiến Lược Điều Khiển Con Trỏ	Công Thức Cập Nhật Kết Quả
**Đoạn con ngắn nhất có tổng $\geq S$ **	Mở R cho đến khi $Sum \geq S$, sau đó co L tối đa để tìm $\min(R - L + 1)$	$Min_Len = \min(Min_Len, R - L + 1)$
**Đoạn con dài nhất có tổng $\leq S$ **	Mở R , nếu $Sum > S$ thì co L cho đến khi $Sum \leq S$	$Max_Len = \max(Max_Len, R - L + 1)$
**Đếm số lượng đoạn con có tổng $\leq S$ **	Mở R , co L cho đến khi $Sum \leq S$. Mọi đoạn con kết thúc tại R bắt đầu từ $[L \dots R]$ đều thỏa mãn	$Total \leftarrow Total + (R - L + 1)$

Ví dụ minh họa 2: Tìm đoạn con ngắn nhất có tổng $\geq S = 7$ trên $A = [2, 3, 1, 2, 4, 3]$

Bước (R)	Nạp A_R	Tổng cửa số	Đánh giá $\geq$ 7	Thao tác co L & Độ dài tìm được	Min_Len
$R = 0$	$A_0 = 2$	2	Chưa đủ	—	∞
$R = 1$	$A_1 = 3$	5	Chưa đủ	—	∞
$R = 2$	$A_2 = 1$	6	Chưa đủ	—	∞
$R = 3$	$A_3 = 2$	8	≥ 7 (Thỏa mãn)	Co $L=0 \rightarrow 1$ (bỏ $A_0=2$, tổng còn $6 < 7$) $\Rightarrow$ Đoạn $[3, 1, 2]$ dài 3	** 3 **
$R = 4$	$A_4 = 4$	10	≥ 7 (Thỏa mãn)	Co $L=1 \rightarrow 3$ (bỏ A_1, A_2 , tổng còn $6 < 7$) $\Rightarrow$ Đoạn $[2, 4]$ dài 2	** 2 **
$R = 5$	$A_5 = 3$	9	≥ 7 (Thỏa mãn)	Co $L=3 \rightarrow 5$ (bỏ A_3, A_4 , tổng còn $3 < 7$) $\Rightarrow$ Đoạn $[4, 3]$ dài 2	** 2 **

$\Rightarrow$ Kết quả: Độ dài ngắn nhất là 2 (đoạn $[2, 4]$ hoặc $[4, 3]$)!

4. Điều kiện áp dụng & giới hạn thất bại khi mảng có số âm

4.1. Điều kiện tiên quyết: Tính đơn điệu của hàm trạng thái

Cửa sổ trượt biến thiên **bắt buộc yêu cầu hàm mục tiêu phải có tính đơn điệu**:

- Khi mở rộng R ($R \rightarrow R + 1$): Trạng thái phải tăng dần (hoặc không giảm).
- Khi co hẹp L ($L \rightarrow L + 1$): Trạng thái phải giảm dần (hoặc không tăng).

Đối với bài toán tổng đoạn con, điều này tương đương với điều kiện: **Tất cả các phần tử trong mảng phải là số không âm ($A_i \geq 0$)**.

4.2. Giới hạn: Vì sao sliding window thất bại khi có số âm?

Xét mảng $A = [2, -5, 10, -2, 8]$ với mục tiêu tìm đoạn con ngắn nhất có tổng ≥ 8 .

- Khi R nạp thêm số âm -5 , tổng cửa sổ bị giảm.
- Khi L dịch qua số âm -5 , tổng cửa sổ lại tăng lên.
- Tính chất đơn điệu bị phá vỡ $\Rightarrow$ Con trỏ L không thể đưa ra quyết định di chuyển một chiều chắc chắn $\Rightarrow$ Bỏ sót nghiệm tối ưu.
- **Giải pháp chuẩn:** Chuyển sang sử dụng **Mảng cộng dồn (Prefix Sum)** kết hợp **Hàng đợi hai đầu (Deque)** / **Cây chỉ số Fenwick / Segment Tree**.

5. Mẫu cài đặt chuẩn thi đấu (competitive template)

Mẫu: Đoạn con liên tiếp ngắn nhất có tổng $\geq S$ ($A_i \geq 0$)

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    long long s;
    if (!(cin >> n >> s)) return 0;

    vector<long long> a(n);

    for (int i = 0; i < n; ++i) {
        cin >> a[i];
    }

    int l = 0;
    long long current_sum = 0;
    int min_len = n + 1; // Khởi tạo vô cực

    // Duyệt con trỏ R tuyến tính O(N)
    for (int r = 0; r < n; ++r) {
        current_sum += a[r]; // Nạp a[r] vào cửa sổ

        // Co hẹp con trỏ L khi cửa sổ đã thỏa mãn điều kiện
        while (current_sum ≥ s) {
            min_len = min(min_len, r - l + 1); // Cập nhật độ dài nhỏ nhất
            current_sum -= a[l];               // Nhả a[l] ra khỏi cửa sổ
            ++l;                               // Dịch chuyển biên trái
        }
    }

    if (min_len > n) {
        cout << 0 << "\n"; // Không tồn tại đoạn thỏa mãn
    } else {
        cout << min_len << "\n";
    }

    return 0;
}
```

6. Kỹ thuật cửa sổ trượt với bảng đếm ký tự / trạng thái

Khi xử lý bài toán chuỗi ký tự (như Đoạn con dài nhất chứa tối đa K ký tự khác nhau):

- Sử dụng mảng đếm tần suất `int count[256]` hoặc `int count[26]` và biến `distinct_count` lưu số ký tự khác nhau hiện có trong cửa sổ.
- Khi nạp ký tự `S[R]`: nếu `count[S[R]] == 0`, tăng `distinct_count`. Tăng `count[S[R]]++`.
- Khi `distinct_count > K`: co con trỏ `L`, giảm `count[S[L]]--`; nếu `count[S[L]] == 0`, giảm `distinct_count`. Tăng `++L`.

7. Các bẫy lỗi thường gặp (bug traps)

1. **Bẫy tràn số nguyên khi tính tổng cửa sổ:** Tổng đoạn con của mảng $N = 10^5$ phần tử với $A_i = 10^9$ có thể lên tới 10^{14} . Khai báo biến `current_sum` kiểu `long long`.
2. **Bẫy điều kiện khởi tạo kết quả cực trị:** Khi tìm `min`, khởi tạo `ans = n + 1` (hoặc `∞`); khi không tìm thấy nghiệm phải in ra `0` hoặc `-1` theo đúng quy cách đề bài.
3. **Bẫy chỉ số âm khi trượt cửa sổ cố định:** Luôn đảm bảo chỉ thực hiện phép trừ `a[i - k]` khi chỉ số $i \geq K$.

Câu hỏi trắc nghiệm củng cố khái niệm

Câu 1 (nhận diện — recognize):

Kỹ thuật Cửa sổ trượt biến thiên (mở R , co L) áp dụng an toàn nhất trên tập dữ liệu nào sau đây?

- A. Mảng số nguyên có cả số dương lớn và số âm nhỏ.
- B. **[Đáp án đúng]** Mảng các số nguyên không âm ($A_i \geq 0$).
- C. Mảng các chuỗi ký tự ngẫu nhiên đã được đảo ngược.
- D. Mảng 2 chiều kích thước $N * N$.

Giải thích: Tính không âm đảm bảo rằng khi mở rộng R thì tổng luôn tăng hoặc giữ nguyên, và khi co L thì tổng luôn giảm hoặc giữ nguyên (tính chất đơn điệu).

Câu 2 (dự đoán — predict):

Cho mảng $A = [1, 4, 2, 10, 2, 3, 1, 0, 20]$ và cửa sổ cố định kích thước $K = 4$. Tổng của cửa sổ đầu tiên $[1, 4, 2, 10]$ là 17 . Khi trượt cửa sổ sang phải để xét đoạn $[4, 2, 10, 2]$, tổng mới được tính nhanh nhất bằng phép toán nào?

- A. Cộng lại từ đầu: $4 + 2 + 10 + 2 = 18$.
- B. **[Đáp án đúng]** Lấy tổng cũ trừ phần tử rời đi và cộng phần tử mới: $17 - 1 + 2 = 18$.
- C. Nhân đôi tổng cũ rồi chia cho 4.
- D. Lấy 17 cộng thêm 4 .

Giải thích: Quy tắc trượt cửa sổ cố định: $sum = sum - A[i-K] + A[i]$ chỉ mất $O(1)$ thời gian.

Câu 3 (bản chất — explain):

Mặc dù có vòng lặp `while` lồng bên trong vòng lặp `for`, tại sao thuật toán Cửa sổ trượt trên mảng N phần tử vẫn đạt độ phức tạp thời gian $O(N)$?

- A. Vì vòng lặp `while` chỉ chạy đúng 1 lần duy nhất trong toàn bộ chương trình.
- B. Vì trình biên dịch C++ tự động tối ưu hóa vòng lặp `while` thành câu lệnh `if`.
- C. **[Đáp án đúng]** Vì con trỏ L chỉ dịch chuyển sang phải và mỗi phần tử chỉ bị loại bỏ khỏi cửa sổ tối đa đúng 1 lần.
- D. Vì số phép tính của vòng `while` luôn bị giới hạn bởi hằng số 10.

Giải thích: R duyệt từ 0 to $N-1$ (N bước) và L duyệt từ 0 to N (tối đa N bước). Tổng số bước di chuyển của cả 2 con trỏ không bao giờ vượt quá $2N$.

Câu 4 (chuyển giao — transfer):

Nếu đề bài yêu cầu tìm đoạn con ngắn nhất có tổng $\geq S$ nhưng trong mảng có xuất hiện các số âm, tại sao ta **không được dùng** kỹ thuật Cửa sổ trượt đơn thuần?

- A. Vì số âm làm tràn bộ nhớ của mảng.
- B. **[Đáp án đúng]** Vì số âm phá vỡ tính chất đơn điệu của tổng, khiến việc co con trỏ L có thể làm tăng tổng và bỏ sót nghiệm tối ưu.
- C. Vì hàm `min()` trong C++ không so sánh được số âm.
- D. Vì vòng lặp `for` sẽ bị lặp vô tận.

Giải thích: Khi có số âm, việc mở rộng R chưa chắc làm tăng tổng và việc co L chưa chắc làm giảm tổng, khiến 2 con trỏ không thể đưa ra quyết định di chuyển một chiều chắc chắn.

Câu 5 (chiến lược điều khiển — control flow):

Trong bài toán tìm **đoạn con dài nhất có tổng $\leq S$ ** ($A_i \geq 0$), vòng lặp `while` co con trỏ L được kích hoạt khi nào?

- A. Khi tổng cửa sổ `current_sum  $\leq S$` .
- B. **[Đáp án đúng]** Khi tổng cửa sổ `current_sum  $> S$` (cửa sổ vi phạm điều kiện, cần co L cho đến khi tổng $\leq S$ trở lại).
- C. Khi con trỏ R chạm đến cuối mảng.
- D. Sau mỗi lần tăng con trỏ R .

Giải thích: Với bài toán tìm `max` độ dài thỏa mãn $sum \leq S$, ta chỉ co L khi tổng đang bị vượt quá ngưỡng cho phép ($> S$) để đưa cửa sổ về trạng thái hợp lệ.

Câu 6 (đếm tổ hợp đoạn con — combinatorial counting):

Trong bài toán **Đếm số lượng đoạn con liên tiếp có tổng $\leq S$ ** ($A_i \geq 0$), sau khi co L để đảm bảo tổng đoạn $[L \dots R] \leq S$, số lượng đoạn con hợp lệ kết thúc tại R được tính bằng công thức nào?

- A. 1
- B. $R - L$
- C. **[Đáp án đúng]** $R - L + 1$ (gồm các đoạn $[R \dots R], [R-1 \dots R], \dots, [L \dots R]$).
- D. $((R - L + 1) * (R - L + 2)) / (2)$

Giải thích: Vì đoạn dài nhất $[L \dots R]$ có tổng $\leq S$ và mảng không âm, nên mọi đoạn con kết thúc tại R bắt đầu từ bất kỳ vị trí nào từ L đến R đều có tổng $\leq S$. Có đúng $R - L + 1$ đoạn như vậy.

Câu 7 (cửa sổ chuỗi ký tự — frequency map):

Để tìm đoạn con dài nhất chứa tối đa K ký tự phân biệt trên chuỗi chỉ gồm chữ cái thường tiếng Anh, ta nên quản lý trạng thái cửa sổ như thế nào tối ưu nhất?

- A. Quét lại toàn bộ cửa sổ để đếm số ký tự khác nhau trong mỗi bước ($O(K)$).
- B. **[Đáp án đúng]** Sử dụng một mảng đếm tần suất `int count[26] = {0}` và một biến đếm `distinct_chars` ($O(1)$) thời gian cho mỗi thao tác nạp/nhả).
- C. Khởi tạo mảng mới tại mỗi bước lặp.
- D. Sắp xếp lại chuỗi ký tự trước khi chạy.

Giải thích: Bảng đếm tần suất kích thước cố định 26 cho phép cập nhật số lượng ký tự phân biệt trong $O(1)$, đảm bảo toàn bộ thuật toán chạy trong $O(N)$ thời gian và $O(1)$ bộ nhớ phụ trợ.

Câu 8 (cửa sổ bao phủ tối thiểu — minimum window):

Trong bài toán tìm **đoạn con ngắn nhất chứa đầy đủ tất cả các ký tự của một tập hợp T **, điều kiện để bắt đầu co con trỏ L là gì?

- A. Khi độ dài cửa sổ đạt tới độ dài của T .
- B. **[Đáp án đúng]** Khi cửa sổ hiện tại `[L ... R]` đã chứa đủ tần suất của mọi ký tự trong tập T .
- C. Khi con trỏ R duyệt đến cuối chuỗi.
- D. Khi gặp một ký tự không thuộc tập T .

Giải thích: Khi cửa sổ đã bao phủ đủ các ký tự yêu cầu (đạt trạng thái hợp lệ), ta tiến hành co L để tìm kiếm độ dài ngắn nhất có thể mà vẫn duy trì tính bao phủ đầy đủ.

Câu 9 (xử lý giới hạn dữ liệu lớn — large constraints):

Một bài toán yêu cầu tìm đoạn con có tổng lớn nhất trong mảng $N = 10^5$ phần tử với $A_i \leq 10^9$. Biến tính tổng cửa sổ `current_sum` có thể đạt giá trị tối đa là bao nhiêu và cần kiểu dữ liệu gì?

- A. 10^9 , dùng kiểu `int`.
- B. 2×10^9 , dùng kiểu `int`.
- C. **[Đáp án đúng]** 10^{14} , bắt buộc dùng kiểu `long long` (64-bit).
- D. 10^{18} , bắt buộc dùng kiểu `__int128`.

Giải thích: Tổng của 10^5 phần tử có giá trị 10^9 là $10^5 \cdot 10^9 = 10^{14}$, vượt xa giới hạn khoảng $2.14 \cdot 10^9$ của kiểu `int` 32-bit.

Câu 10 (kỹ thuật hiệu đếm đoạn con — interval counting trick):

Để đếm số lượng đoạn con liên tiếp có tổng nằm trong khoảng `[A, B]` (tức $A \leq \text{sum} \leq B$) trên mảng số nguyên dương, kỹ thuật chuẩn mực là gì?

- **A.** Chạy 2 vòng lặp lồng nhau duyệt mọi đoạn con.
- **B. [Đáp án đúng]** Gọi `F(X)` là số lượng đoạn con có tổng $\leq X$. Kết quả cần tìm chính là $F(B) - F(A - 1)$, trong đó hàm `F(X)` được tính bằng Sliding Window trong $O(N)$.
- **C.** Sử dụng cây Segment Tree với độ phức tạp $O(N \log^2 N)$.
- **D.** Nhân đôi mảng và áp dụng Two Pointers đối đầu.

Giải thích: Quy bài toán đếm đoạn trong khoảng `[A, B]` về hiệu của hai bài toán đếm tiền tố $\leq X$ giúp tận dụng trọn vẹn thuật toán Sliding Window tuyến tính $O(N)$ mà không cần cấu trúc dữ liệu phức tạp.

Ma trận bài tập thực hành (p0 → p5)

STT	Mã Bài	Tên Bài Toán	Cấp Độ	Ràng Buộc Dữ Liệu	Mục Tiêu Rèn Luyện
01	CPPB-CST-01	Tổng Cửa Sổ Cố Định K	P0	$N \leq 10^5, K \leq N$	Trượt cố định $O(1)$ mỗi bước
02	CPPB-CST-02	Giá Trị Trung Bình Lớn Nhất Của Đoạn K	P1	$N \leq 10^5, K \leq N$	Cửa sổ cố định với số thực
03	CPPB-CST-03	Đoạn Con Ngắn Nhất Có Tổng Đạt S	P1	$N \leq 10^5, S \leq 10^{14}$	Cửa sổ co giãn tìm min length
04	CPPB-CST-04	Đoạn Con Dài Nhất Có Tổng Không Quá S	P2	$N \leq 2 * 10^5, S \leq 10^{14}$	Cửa sổ co giãn tìm max length
05	CPPB-CST-05	Đoạn Con Chứa Tối Đa K Số 0 (Lật Bit)	P3	$N \leq 10^5, K \leq N$	Cửa sổ đếm trạng thái nhị phân
06	CPPB-CST-06	Giám Sát Camera Giao Thông Thông Minh	P4	$N \leq 10^5, K \leq N$	Tối ưu hóa cửa sổ thực tế
07	CPPB-CST-07	Tìm Min Trong Mọi Cửa Sổ Độ Dài K	P1	$N \leq 10^4, K \leq N$	Kiểm tra cửa sổ liên tiếp
08	CPPB-CST-08	Đếm Số Lượng Đoạn Con Có Tổng Không Quá S	P2	$N \leq 2 * 10^5, S \leq 10^{14}$	Cộng dồn $(R - L + 1)$ đoạn con
09	CPPB-CST-09	Đếm Số Lượng Đoạn Con Có Tổng Đúng Bằng S	P2	$N \leq 2 * 10^5, A_i > 0$	Đếm đoạn trên mảng đơn điệu

10	CPPB- CST-10	Đoạn Con Dài Nhất Chứa Tối Đa K Ký Tự Khác Nhau	P3	$N \leq 10^5, K \leq 26$	Cửa sổ ký tự với mảng đếm tần suất
11	CPPB- CST-11	Đoạn Con Ngắn Nhất Chứa Đủ Mọi Ký Tự Của Tập Hợp	P3	$N \leq 10^5, M \leq 26$	Bài toán Minimum Window Substring
12	CPPB- CST-12	Phủ Sóng Trạm Phát Sóng Wifi Đô Thị	P4	$N \leq 10^5, X_i \leq 10^{14}$	Hai con trỏ + Tham lam vị trí
13	CPPB- CST-13	Đoạn Con Có Độ Chênh Lệch Max - Min Không Quá K	P4	$N \leq 5000$	Khống chế biên độ trong cửa sổ
14	CPPB- CST-14	Tối Ưu Cửa Sổ Trượt Tuyến Tính Khi $N = 2 \cdot 10^5$	P5	$N \leq 2 \cdot 10^5, A_i > 0$	Kỹ thuật hiệu $F(B) - F(A - 1)$

Module 02: Mảng tiền tố & tìm kiếm nhị phân

<!-- Chuyên đề 04: Mảng Tiền Tố | 401 dòng | 23,187 bytes -->

Chuyên đề 04: Mảng tiền tố & mảng hiệu

Tối ưu hóa truy vấn tổng & cập nhật đoạn $O(1)$ trong C++

1. Khái niệm & bản chất của mảng tiền tố (prefix sum 1d)

Mảng tiền tố (Prefix Sum) là một kỹ thuật tiền xử lý dữ liệu mảng ban đầu thành một mảng cộng dồn tích lũy, cho phép tính toán tổng của bất kỳ đoạn con liên tiếp $[L \dots R]$ nào chỉ trong **$O(1)$ thời gian**, thay vì phải duyệt vòng lặp $O(N)$.

1.1. Công thức xây dựng mảng tiền tố

Cho mảng số nguyên A gồm N phần tử. Quy ước đánh số chỉ số từ 1 đến N (**1-based indexing**):

- Khởi tạo: $P_0 = 0$.
- Công thức truy hồi với i từ 1 đến N :

$$P_i = P_{i-1} + A_i \Longleftrightarrow P_i = \sum_{k=1}^i A_k$$

1.2. Công thức truy vấn tổng đoạn con trong $O(1)$

Tổng các phần tử trong đoạn từ chỉ số L đến chỉ số R ($1 \leq L \leq R \leq N$) được tính bằng hiệu của hai giá trị tiền tố:

$$Sum(L, R) = \sum_{k=L}^R A_k = P_R - P_{L-1}$$

1.3. Chứng minh toán học

Theo định nghĩa:

$$P_R = A_1 + A_2 + \dots + A_{L-1} + A_L + \dots + A_R$$

$$P_{L-1} = A_1 + A_2 + \dots + A_{L-1}$$

Lấy hiệu hai vế:

$$P_R - P_{L-1} = (A_1 + \dots + A_{L-1} + A_L + \dots + A_R) - (A_1 + \dots + A_{L-1}) = A_L + A_{L+1} + \dots + A_R = \text{Sum}(L, R)$$

Bất biến toán học: Phép trừ $P_R - P_{L-1}$ đã loại bỏ chính xác đoạn tiền tố thừa từ 1 đến $L-1$, chỉ giữ lại trọn vẹn đoạn con $[L \dots R]$ cần tính.

Ví dụ minh họa 1: Xây dựng và truy vấn prefix sum 1d

Cho mảng $N = 6$ phần tử: $A = [3, 1, 4, 1, 5, 9]$ (1-based indexing).

Chỉ số (i)	0	1	2	3	4	5	6
Mảng ban đầu (A_i)	—	3	1	4	1	5	9
Mảng tiền tố (P_i)	0	3	4	8	9	14	23
Công thức truy hồi	$P_0 = 0$	$P_0 + A_1 = 3$	$P_1 + A_2 = 4$	$P_2 + A_3 = 8$	$P_3 + A_4 = 9$	$P_4 + A_5 = 14$	$P_5 + A_6 = 23$

• **Truy vấn 1:** Tính tổng đoạn từ $L = 2$ đến $R = 5$ (đoạn $[1, 4, 1, 5]$):

$$\text{Sum}(2, 5) = P[5] - P[2 - 1] = P[5] - P[1] = 14 - 3 = 11$$

(Kiểm tra trực tiếp: $1 + 4 + 1 + 5 = 11$ — Hoàn toàn chính xác trong $O(1)$).

• **Truy vấn 2:** Tính tổng toàn bộ mảng từ $L = 1$ đến $R = 6$:

$$\text{Sum}(1, 6) = P[6] - P[0] = 23 - 0 = 23$$

2. Kỹ thuật mảng tiền tố hai chiều (prefix sum 2d)

2.1. Bản chất nguyên lý bao hàm - loại trừ (inclusion-exclusion principle)

Trên ma trận 2 chiều kích thước $N \times M$, gọi $P[i][j]$ là tổng của tất cả các phần tử trong hình chữ nhật có góc trái trên tại $(1, 1)$ và góc phải dưới tại (i, j) :

$$P[i][j] = \sum_{r=1}^i \sum_{c=1}^j A[r][c]$$

2.2. Công thức xây dựng bảng tiền tố 2d trong $O(N \times M)$

Tại mỗi ô (i, j) :

$$P[i][j] = P[i-1][j] + P[i][j-1] - P[i-1][j-1] + A[i][j]$$

(Giải thích: Cộng vùng phía trên và vùng bên trái, trừ đi phần giao nhau bị cộng lặp $P[i-1][j-1]$, rồi cộng thêm giá trị ô hiện tại $A[i][j]$).

2.3. Công thức truy vấn tổng hình chữ nhật $(x_1, y_1) \rightarrow (x_2, y_2)$ trong $O(1)$

$$\text{Sum}((x_1, y_1), (x_2, y_2)) = P[x_2][y_2] - P[x_1-1][y_2] - P[x_2][y_1-1] + P[x_1-1][y_1-1]$$

Ví dụ minh họa 2: Truy vấn hình chữ nhật trên ma trận 3×3

Cho ma trận A :

$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} \rightsquigarrow \text{Xây dựng } P$
 $P = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 3 \\ 0 & 5 & 12 \end{bmatrix}$

Cần tính tổng hình chữ nhật từ $(x_1=2, y_1=2)$ đến $(x_2=3, y_2=3)$ (vùng các ô $\begin{bmatrix} 5 & 6 \\ 8 & 9 \end{bmatrix}$):

$$\text{Sum} = P[3][3] - P[1][3] - P[3][1] + P[1][1] = 12 - 3 - 5 + 1 = 5$$

(Kiểm tra trực tiếp: $5 + 6 + 8 + 9 = 28$ — Tính toán trong đúng 4 phép toán $O(1)$).

3. Kỹ thuật mảng hiệu (difference array 1d)

3.1. Bài toán đặt ra

Cho mảng ban đầu gồm N phần tử (toàn số 0 hoặc có giá trị sẵn). Thực hiện Q thao tác, mỗi thao tác yêu cầu: **Cộng thêm một giá trị V vào tất cả các phần tử từ chỉ số L đến R **. Sau Q thao tác, in ra mảng kết quả cuối cùng.

- **Cách tiếp cận ngây thơ:** Với mỗi thao tác, dùng vòng lặp chạy từ L đến R để cộng. Tổng thời gian: $O(Q \times N) \approx 10^5 \times 10^5 = 10^{10}$ phép tính $\Rightarrow$ **Time Limit Exceeded (TLE)**.
- **Tối ưu bằng Mảng hiệu:** Thực hiện mỗi thao tác cộng đoạn trong $O(1)$ thời gian**.

3.2. Cơ chế hoạt động của mảng hiệu

Xây dựng mảng hiệu D thỏa mãn: $A_i = \sum_{k=1}^i D_k$ (Mảng ban đầu chính là mảng tiền tố của mảng hiệu). Để cộng giá trị V vào mọi phần tử trong đoạn $[L \dots R]$, ta chỉ cần thực hiện 2 thao tác điểm:

- **Tại điểm bắt đầu đoạn L : $** D[L] \mathrel{+}= V$
- **Tại điểm sau kết thúc đoạn $R + 1$: $** D[R + 1] \mathrel{-}= V$

3.3. Khôi phục mảng kết quả sau Q thao tác

Sau khi hoàn thành tất cả Q thao tác cập nhật $O(1)$, ta khôi phục lại mảng kết quả A bằng một lần chạy tiền tố duy nhất trong $**O(N)$ thời gian**:

$$A_i = A_{i-1} + D_i \quad (i = 1 \dots N)$$

Ví dụ minh họa 3: Mảng hiệu trên dãy $N = 5$ phần tử

Ban đầu dãy toàn số 0: $A = [0, 0, 0, 0, 0]$, mảng hiệu $D = [0, 0, 0, 0, 0, 0]$ (kích thước $N+2$).

1. **Thao tác 1:** Cộng $V = 3$ vào đoạn $[1 \dots 3] \Rightarrow D[1] += 3, D[4] -= 3$.

$$D = [0, +3, 0, 0, -3, 0]$$

1. **Thao tác 2:** Cộng $V = 2$ vào đoạn $[2 \dots 5] \Rightarrow D[2] += 2, D[6] -= 2$.

$$D = [0, +3, +2, 0, -3, 0, -2]$$

Bước khôi phục mảng kết quả A bằng Prefix Sum trên D : $$

- $A_1 = D_1 = 3$
- $A_2 = A_1 + D_2 = 3 + 2 = 5$
- $A_3 = A_2 + D_3 = 5 + 0 = 5$
- $A_4 = A_3 + D_4 = 5 + (-3) = 2$
- $A_5 = A_4 + D_5 = 2 + 0 = 2$

Kết quả cuối cùng: $A = [3, 5, 5, 2, 2]$

4. Kỹ thuật mảng hiệu hai chiều (difference array 2d)

Để cộng thêm giá trị V vào tất cả các ô trong hình chữ nhật từ (x_1, y_1) đến (x_2, y_2) trên ma trận $N \times M$, ta chỉ cần tác động lên **4 điểm góc** của mảng hiệu $2D$ trong $O(1)$:

Điểm Góc Tác Động	Tọa Độ Ô Mảng Hiệu	Thao Tác Cập Nhật $O(1)$
Góc trên - trái	$(x1, y1)$	$D[x1][y1] += V$
Góc trên - phải	$(x1, y2 + 1)$	$D[x1][y2 + 1] -= V$
Góc dưới - trái	$(x2 + 1, y1)$	$D[x2 + 1][y1] -= V$
Góc dưới - phải	$(x2 + 1, y2 + 1)$	$D[x2 + 1][y2 + 1] += V$

Sau khi thực hiện xong Q thao tác, khôi phục ma trận gốc bằng công thức Prefix Sum 2D trong $O(N \times M)$.

5. Mẫu cài đặt chuẩn thi đấu (competitive template)

Mẫu 1: Prefix sum 1d (truy vấn tổng đoạn)

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q;
    if (!(cin >> n >> q)) return 0;

    vector<long long> a(n + 1);

    vector<long long> p(n + 1, 0);

    // Bước 1: Đọc dữ liệu và xây dựng mảng tiền tố O(N)
    for (int i = 1; i ≤ n; ++i) {
        cin >> a[i];

        p[i] = p[i - 1] + a[i];
    }

    // Bước 2: Trả lời từng truy vấn trong O(1)
    while (q--) {
        int l, r;
        cin >> l >> r;

        cout << p[r] - p[l - 1] << "\n";
    }

    return 0;
}
```


Mẫu 2: Difference array 1d (cập nhật đoạn)

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q;
    if (!(cin >> n >> q)) return 0;

    // Khởi tạo mảng hiệu kích thước n + 2 để an toàn khi truy cập r + 1
    vector<long long> d(n + 2, 0);

    // Bước 1: Tiếp nhận Q thao tác cập nhật O(1)
    while (q--) {
        int l, r;
        long long v;
        cin >> l >> r >> v;

        d[l] += v;
        d[r + 1] -= v;
    }

    // Bước 2: Khôi phục mảng kết quả bằng tiền tố O(N)
    vector<long long> a(n + 1, 0);

    for (int i = 1; i ≤ n; ++i) {
        a[i] = a[i - 1] + d[i];
        cout << a[i] << (i == n ? "" : " ");
    }
    cout << "\n";

    return 0;
}
```

6. Các bẫy lỗi lập trình thường gặp (bug traps)

- Bẫy chỉ số 0-based vs 1-based:** Khi dùng chỉ số 0-based, truy vấn đoạn bắt đầu từ $L=0$ sẽ phải tính $P[R] - P[-1]$ dẫn đến lỗi truy cập vùng nhớ ngoài biên. **Khuyến nghị chuẩn:** Luôn chuyển toàn bộ mảng tiền tố và mảng hiệu sang **1-based indexing** với $P[0] = 0$.
- Bẫy tràn số nguyên 32-bit khi cộng dồn:** Mảng $N = 2 \cdot 10^5$ phần tử với $A_i = 10^9$ sẽ có tổng tiền tố lên tới $2 \cdot 10^{14}$, vượt ngưỡng $2 \cdot 10^9$ của `int`. Khai báo toàn bộ mảng P và D kiểu `long long`.

3. ****Bẫy tràn biên $R + 1$ trong mảng hiệu:**** Khi đoạn cập nhật có $R = N$, thao tác $D[R+1] -= V$ sẽ ghi vào vị trí $N + 1$. Bắt buộc phải cấp phát mảng hiệu có kích thước tối thiểu là $N + 2$.

7. Ranh giới áp dụng: Khi nào nên & không nên dùng?

• KHI NÀO ÁP DỤNG TỐI ƯU:

- **Mảng tĩnh (Static Queries):** Toàn bộ dữ liệu mảng cố định, chỉ nhận các truy vấn tính tổng đoạn liên tiếp $\Rightarrow$ ****Prefix Sum đạt $O(1)$ tuyệt đối****.
- **Cập nhật Offline (Batch Updates):** Nhận toàn bộ Q thao tác cộng đoạn $[L, R]$ trước, sau đó mới cần in kết quả một lần ở cuối $\Rightarrow$ ****Difference Array đạt $O(Q + N)$ ****.

• KHI NÀO KHÔNG ÁP DỤNG ĐƯỢC (Bẫy Lỗi KỸ THUẬT):

- **Cập nhật và truy vấn xen kẽ Online:** Nếu chương trình vừa yêu cầu cập nhật giá trị một phần tử/đoạn, vừa yêu cầu truy vấn tổng đoạn ngay lập tức lặp đi lặp lại Q lần:
- Dùng Prefix Sum sẽ tốn $O(N)$ để cập nhật lại mảng $P \Rightarrow$ Tổng thời gian $O(Q \times N)$ (TLE).
- Dùng Difference Array sẽ tốn $O(N)$ để khôi phục mỗi khi có truy vấn $\Rightarrow$ Tổng thời gian $O(Q \times N)$ (TLE).
- **Giải pháp chuẩn thi đấu:** Khi có cập nhật và truy vấn xen kẽ liên tục, bắt buộc phải sử dụng các cấu trúc dữ liệu cây động như **Cây chỉ số nhị phân (Fenwick Tree)** hoặc **Cây phân đoạn (Segment Tree)** (thuộc Module 08).

Câu hỏi trắc nghiệm củng cố khái niệm

Câu 1 (nhận diện — recognize):

Mảng tiền tố P của mảng $A = [4, 1, 7, 3, 2]$ (đánh số từ 1 đến 5) là dãy số nào sau đây?

- A. $P = [0, 4, 5, 12, 15, 17]$
- B. **[Đáp án đúng]** $P = [0, 4, 5, 12, 15, 17]$ với $P_0 = 0, P_1 = 4, P_2 = 5, P_3 = 12, P_4 = 15, P_5 = 17$.
- C. $P = [4, 5, 12, 15, 17, 0]$.
- D. $P = [17, 13, 12, 5, 2, 0]$.

Giải thích: $P_0 = 0, P_1 = 4, P_2 = 4+1=5, P_3 = 5+7=12, P_4 = 12+3=15, P_5 = 15+2=17$.

Câu 2 (dự đoán — predict):

Cho mảng tiền tố $P = [0, 3, 8, 14, 20, 25]$. Tổng của đoạn từ vị trí $L = 2$ đến $R = 4$ được tính bằng biểu thức nào?

- A. $P[4] - P[2] = 20 - 8 = 12$.
- B. **[Đáp án đúng]** $P[4] - P[1] = 20 - 3 = 17$.
- C. $P[4] + P[2] = 20 + 8 = 28$.
- D. $P[5] - P[2] = 25 - 8 = 17$.

Giải thích: Công thức tính tổng đoạn $[L \dots R]$ là $P[R] - P[L-1]$. Với $L=2, R=4$, ta có $Sum = P[4] - P[2-1] = P[4] - P[1] = 20 - 3 = 17$.

Câu 3 (bản chất — explain):

Tại sao khi thao tác trên mảng hiệu D để cộng giá trị V vào đoạn $[L \dots R]$, ta lại phải thực hiện $D[R+1] \mathrel{+}= V$?

- A. Để giảm bớt giá trị của phần tử đứng ngay sau R .
- B. **[Đáp án đúng]** Để triệt tiêu lượng tăng V khi lấy tổng tiền tố từ vị trí $R+1$ trở đi, đảm bảo các phần tử ngoài đoạn $[L \dots R]$ không bị tăng thêm giá trị.
- C. Để tránh tràn số nguyên khi tính toán.
- D. Vì trình biên dịch C++ yêu cầu các thao tác mảng phải đối xứng.

Giải thích: Khi lấy tổng tiền tố, thao tác $+V$ tại L sẽ lan truyền tới tất cả các vị trí từ L tới N . Do đó ta phải đặt $-V$ tại $R+1$ để chặn sự lan truyền này từ vị trí $R+1$ trở đi.

Câu 4 (chuyển giao — transfer):

Nếu có $Q = 10^5$ thao tác cập nhật cộng đoạn trên mảng $N = 10^5$ phần tử, việc sử dụng Mảng hiệu giúp giảm độ phức tạp thời gian từ bao nhiêu xuống bao nhiêu?

- A. Từ $O(N \log N)$ xuống $O(N)$.
- B. **[Đáp án đúng]** Từ $O(Q * N) \approx 10^{10}$ phép tính xuống $O(Q + N) \approx 2 * 10^5$ phép tính.
- C. Từ $O(N^2)$ xuống $O(N \log N)$.
- D. Từ $O(1)$ xuống $O(Q)$.

Giải thích: Mỗi thao tác cập nhật mất $O(1)$ (tổng Q thao tác mất $O(Q)$), bước khôi phục mảng mất $O(N)$. Tổng thời gian là $O(Q + N)$, chạy dưới 0.05 giây.

Câu 5 (prefix sum 2d — geometry):

Trong công thức tính tổng hình chữ nhật 2D: $Sum = P[x2][y2] - P[x1-1][y2] - P[x2][y1-1] + P[x1-1][y1-1]$, tại sao lại có dấu cộng $+ P[x1-1][y1-1]$ ở cuối?

- A. Vì đây là công thức tính đường chéo hình chữ nhật.
- B. **[Đáp án đúng]** Vì vùng hình chữ nhật góc $(1, 1)$ to $(x1-1, y1-1)$ đã bị trừ 2 lần ở hai số hạng phía trước, nên cần cộng bù lại 1 lần theo nguyên lý Bao hàm - Loại trừ.
- C. Vì ô $(x1-1, y1-1)$ mang giá trị âm.
- D. Vì góc trên bên trái luôn phải có trọng số gấp đôi.

Giải thích: Cả hai vùng bị trừ là $P[x1-1][y2]$ và $P[x2][y1-1]$ đều cùng chứa vùng giao nhau $(1, 1)$ to $(x1-1, y1-1)$. Việc trừ cả hai vùng đã trừ vùng giao 2 lần, bắt buộc phải cộng bù lại 1 lần.

Câu 6 (mảng hiệu 2d — technique):

Để cộng giá trị V vào tất cả các ô trong hình chữ nhật $(x1, y1)$ to $(x2, y2)$ trên ma trận bằng mảng hiệu 2D, cần cập nhật bao nhiêu ô và dấu như thế nào?

- A. Cập nhật 2 ô: $+V$ tại $(x1, y1)$ và $-V$ tại $(x2, y2)$.
- B. **[Đáp án đúng]** Cập nhật 4 ô: $+V$ tại $(x1, y1)$ và $(x2+1, y2+1)$; $-V$ tại $(x1, y2+1)$ và $(x2+1, y1)$.
- C. Cập nhật tất cả các ô nằm trên biên của hình chữ nhật.
- D. Cập nhật 4 ô với dấu $+V$ ở tất cả các góc.

Giải thích: Đây là công thức mảng hiệu 2 chiều chuẩn mực để khi lấy Prefix Sum 2D khôi phục ma trận, chỉ có các ô bên trong hình chữ nhật nhận giá trị $+V$.

Câu 7 (đoạn con tổng bằng 0 — logic):

Nếu tồn tại hai chỉ số $i < j$ trong mảng tiền tố thỏa mãn $P[i] = P[j]$, ta có thể rút ra kết luận gì về mảng ban đầu?

- A. Tất cả các phần tử từ i đến j đều bằng 0.
- B. **[Đáp án đúng]** Đoạn con liên tiếp từ vị trí $i+1$ đến j có tổng đúng bằng 0 ($Sum(i+1, j) = P[j] - P[i] = 0$).
- C. Mảng ban đầu đối xứng qua tâm.
- D. Toàn bộ mảng có tổng bằng 0.

Giải thích: $Sum(i+1, j) = P[j] - P[i]$. Nếu $P[j] = P[i]$ thì hiệu này bằng 0, nghĩa là tổng các phần tử trong đoạn $[i+1 \dots j]$ bằng 0.

Câu 8 (đồng dư tiền tố — prefix modulo):

Để đếm số lượng đoạn con có tổng chia hết cho K , ta tính mảng tiền tố lấy dư $M[i] = P[i] \bmod K$. Đoạn con $[L \dots R]$ có tổng chia hết cho K khi và chỉ khi điều kiện nào thỏa mãn?

- A. $M[R] + M[L-1] = K$.
- B. **[Đáp án đúng]** $M[R] = M[L-1]$ (hai vị trí có cùng số dư khi chia cho K).
- C. $M[R] - M[L-1] = 1$.
- D. $M[R] * M[L-1] = 0$.

Giải thích: $(P[R] - P[L-1]) \equiv 0 \bmod K \iff P[R] \equiv P[L-1] \bmod K \iff M[R] = M[L-1]$.

Câu 9 (bẫy chỉ số mảng hiệu — bug traps):

Khi làm việc với mảng hiệu 1D cho dãy có N phần tử, tại sao mảng D bắt buộc phải được khai báo với kích thước tối thiểu là $N + 2$?

- A. Để lưu trữ giá trị trung bình ở cuối mảng.
- B. **[Đáp án đúng]** Vì khi đoạn cập nhật kết thúc tại $R = N$, câu lệnh $D[R+1] \mathrel{:=} V$ sẽ ghi vào vị trí $N + 1$; nếu mảng chỉ có kích thước $N + 1$ sẽ gây lỗi tràn bộ nhớ (Out of Bounds).
- C. Vì mảng hiệu luôn cần 2 ô nhớ trống ở đầu và cuối để chạy đa luồng.
- D. Vì số lượng thao tác Q có thể lớn hơn N .

Giải thích: R có thể đạt giá trị cực đại là N , khi đó $R + 1 = N + 1$. Với 1-based indexing, mảng cần các chỉ số từ $0 \dots N + 1$, tức kích thước tối thiểu phải là $N + 2$.

Câu 10 (tràn số dữ liệu lớn — data types):

Cho bài toán gồm $Q = 10^5$ truy vấn tổng đoạn trên ma trận $N * M = 1000 * 1000$, mỗi phần tử $A[i][j] \leq 10^9$. Bảng tiền tố $P[i][j]$ có thể đạt giá trị tối đa là bao nhiêu và cần kiểu dữ liệu gì?

- A. 10^9 , dùng kiểu `int`.
- B. 2×10^9 , dùng kiểu `int`.
- C. **[Đáp án đúng]** $10^6 * 10^9 = 10^{15}$, bắt buộc phải khai báo mảng P bằng kiểu `long long`.
- D. 10^{18} , bắt buộc dùng kiểu `__int128`.

Giải thích: Tổng của toàn bộ $1000\ 1000 = 10^6$ ô, mỗi ô có giá trị 10^9 , là 10^{15} . Giá trị này vượt xa giới hạn $2.14\ 10^9$ của kiểu `int` 32-bit, bắt buộc phải dùng `long long` 64-bit.

Câu 11 (nén chiều ma trận — 2d submatrix compression):

Để tìm ma trận con hình chữ nhật có tổng lớn nhất trên ma trận $N \times M$, kỹ thuật tối ưu kết hợp Mảng tiền tố và Thuật toán Kadane giảm độ phức tạp từ $O(N^2 M^2)$ xuống bao nhiêu?

- A. $O(N \times M)$
- B. **[Đáp án đúng]** $O(N^2 * M)$ (Cố định 2 hàng `r1`, `r2`, dùng tiền tố cột nén thành mảng 1D rồi chạy Kadane).
- C. $O(N^3 * M^3)$
- D. $O((N + M) \log(NM))$

Giải thích: Cố định 2 hàng `r1`, `r2` mất $O(N^2)$, tổng các cột giữa 2 hàng này được tính trong $O(1)$ bằng tiền tố cột, sau đó chạy Kadane 1D mất $O(M)$ implies Tổng thời gian $O(N^2 M)$.

Câu 12 (cân bằng đa trạng thái — multidimensional balance):

Để tìm đoạn con dài nhất chứa số lượng 3 loại ký tự 'A', 'B', 'C' bằng nhau, ta cần lưu trữ và so khớp giá trị nào tại mỗi vị trí tiền tố i ?

- A. Tổng số lượng $cntA + cntB + cntC$.
- B. **[Đáp án đúng]** Cặp hiệu hai chiều $(cntA[i] - cntB[i], cntB[i] - cntC[i])$.
- C. Tích $cntA[i] * cntB[i] * cntC[i]$.
- D. Chỉ số $i \bmod 3$.

Giải thích: Ba đại lượng bằng nhau $X = Y = Z$ iff $X - Y = 0$ và $Y - Z = 0$. Khi lấy hiệu giữa hai mốc R và $L-1$, điều này tương đương với $(cntA - cntB)$ và $(cntB - cntC)$ tại R và $L-1$ phải bằng nhau.

Ma trận bài tập thực hành (p0 → p5)

STT	Mã Bài	Tên Bài Toán	Cấp Độ	Ràng Buộc Dữ Liệu	Mục Tiêu Rèn Luyện
01	CPPB-PT-01	Truy Vấn Tổng Đoạn Con 1D	P0	$N, Q \leq 10^5$	Prefix sum cơ bản $P[R] - P[L-1]$
02	CPPB-PT-02	Đếm Số Lượng Số Chẵn Trong Đoạn	P1	$N, Q \leq 10^5$	Tiền tố trên mảng điều kiện logic
03	CPPB-PT-03	Tìm Vị Trí Cân Bằng Của Mảng	P1	$N \leq 2 \times 10^5$	Tổng trái bằng tổng phải
04	CPPB-PT-04	Đoạn Con Có Tổng Bằng 0	P2	$N \leq 10^5$	Nhận diện $P[i] = P[j]$
05	CPPB-PT-05	Cập Nhật Cộng Đoạn Tuyến Tính (Mảng Hiệu)	P2	$N, Q \leq 2 * 10^5$	Difference Array 1D cơ bản
06	CPPB-PT-06	Trồng Cây Phù Đoạn Tối Ưu	P3	$N \leq 10^5, Q \leq 10^5$	Mảng hiệu kết hợp quét mảng
07	CPPB-PT-07	Truy Vấn Tổng Hình Chữ Nhật 2D	P1	$N, M \leq 1000, Q \leq 10^5$	Prefix sum 2D nguyên bản
08	CPPB-PT-08	Tìm Hình Vuông $K * K$ Có Tổng Lớn Nhất	P2	$N, M \leq 1000, K \leq \min(N, M)$	Quét cửa sổ 2D kết hợp Prefix 2D
09	CPPB-PT-09	Cập Nhật Cộng Hình Chữ Nhật (Mảng Hiệu 2D)	P3	$N, M \leq 1000, Q \leq 10^5$	Difference Array 2D (4 góc)
10	CPPB-PT-10	Đoạn Con Có Tổng Chia Hết Cho K	P3	$N \leq 2 * 10^5, K \leq 10^5$	Mảng tiền tố kết hợp đồng dư
11	CPPB-PT-11	Mảng Tiền Tố XOR Đoạn Con	P3	$N, Q \leq 2 * 10^5$	Tính chất $A \text{ XOR } A = 0$ trên Prefix XOR
12	CPPB-PT-12	Đoạn Con Cân Bằng Số Lượng 0 và 1	P4	$N \leq 2 \times 10^5$	Biến đổi 0 to -1 đưa về bài toán tổng 0
13	CPPB-PT-13	Truy Vấn Ma Trận Đa Vùng Cực Đại	P4	$N, M \leq 1500, Q \leq 10^5$	Tối ưu hóa bộ nhớ và truy vấn 2D
14	CPPB-PT-14	Phân Phối Tài Nguyên Không Gian Tuyến Tính	P5	$N, Q \leq 2 * 10^5$	Mảng hiệu 2 tầng (Arithmetic Progression Update)
15			P4	$N, M \leq 400$	Nén 2D về 1D + Thuật toán Kadane kết hợp Prefix Sum

STT	Mã Bài	Tên Bài Toán	Cấp Độ	Ràng Buộc Dữ Liệu	Mục Tiêu Rèn Luyện
	CPPB-PT-15	Tìm Ma Trận Con Có Tổng Lớn Nhất (Max Submatrix)			
16	CPPB-PT-16	Cân Bằng Tiền Tố Đa Chiều	P5	$N \leq 10^5$	Cân bằng 3 trạng thái đồng thời

<!-- Chuyên đề 05: Tìm Kiếm Nhị Phân | 502 dòng | 29,801 bytes -->

Chuyên đề 05: Thuật toán tìm kiếm nhị phân

Tối ưu hóa không gian tìm kiếm từ $O(N)$ xuống $O(\log N)$ trong C++

1. Khái niệm & bản chất của tìm kiếm nhị phân (binary search)

Tìm kiếm nhị phân (Binary Search) là thuật toán tìm kiếm dựa trên nguyên lý **chia để trị (Divide and Conquer)**. Bằng cách so sánh giá trị cần tìm với phần tử ở chính giữa không gian tìm kiếm, thuật toán loại bỏ chính xác **một nửa không gian tìm kiếm** sau mỗi bước lặp.

1.1. Điều kiện tiên quyết (prerequisite condition)

Thuật toán tìm kiếm nhị phân **CHỈ HOẠT ĐỘNG ĐƯỢC** khi không gian tìm kiếm hoặc mảng dữ liệu có **tính chất đơn điệu (Monotonicity)**:

- Mảng đã được sắp xếp tăng dần hoặc giảm dần.
- Hoặc một hàm mệnh đề logic $f(x) \in \{True, False\}$ thỏa mãn: nếu $f(x_0) = True$ thì $\text{forall } x \geq x_0, f(x) = True$ (hoặc ngược lại).

1.2. Phân tích độ phức tạp thời gian $O(\log N)$

Giả sử không gian tìm kiếm ban đầu có kích thước N :

- Bước 1: Thu hẹp còn $\frac{N}{2}$.
- Bước 2: Thu hẹp còn $\frac{N}{4} = \frac{N}{2^2}$.
- Bước k : Thu hẹp còn $\frac{N}{2^k}$.

Quá trình dừng lại khi kích thước không gian tìm kiếm bằng $1 \Rightarrow \frac{N}{2^k} = 1 \Leftrightarrow 2^k = N \Leftrightarrow k = \log_2 N$.

- Với $N = 10^5 \Rightarrow \log_2(10^5) \approx 17$ lần thu hẹp không gian.
- Với $N = 10^9 \Rightarrow \log_2(10^9) \approx 30$ lần thu hẹp không gian.
- Với $N = 10^{18} \Rightarrow \log_2(10^{18}) \approx 60$ lần thu hẹp không gian.

Bản chất hiệu năng: Trên không gian nghiệm lên tới 10^{18} , thuật toán chỉ cần khoảng ** 60 lần thu hẹp không gian**. Tổng thời gian thực tế của chương trình sẽ bằng:

$Total\ Time = O(\log(Range) \times Complexity(check))$ Nếu hàm kiểm tra $check(mid)$ chạy trong $O(N)$ với $N = 10^5$, chương trình chỉ mất khoảng $60 \times 10^5 = 6 \cdot 10^6$ phép tính (thực thi trong khoảng 0.02 giây).

2. Tìm kiếm nhị phân trên mảng đã sắp xếp

2.1. Tìm chính xác giá trị X (exact search)

Khởi tạo hai con trỏ biên: $low = 0, high = N - 1$.

- Tính trung điểm an toàn: $mid = low + \frac{high - low}{2}$.
- Nếu $A[mid] == X \Rightarrow$ Tìm thấy tại vị trí mid .
- Nếu $A[mid] < X \Rightarrow$ Giá trị X chỉ có thể nằm ở nửa phải $\Rightarrow low = mid + 1$.
- Nếu $A[mid] > X \Rightarrow$ Giá trị X chỉ có thể nằm ở nửa trái $\Rightarrow high = mid - 1$.

Ví dụ minh họa 1: Tìm kiếm giá trị $X = 23$

Cho mảng đã sắp xếp gồm 10 phần tử: $A = [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]$

Phần tử	$A[0]$	$A[1]$	$A[2]$	$A[3]$	$A[4]$	$A[5]$	$A[6]$	$A[7]$	$A[8]$	$A[9]$
Giá trị	2	5	8	12	16	23	38	56	72	91

Bảng mô phỏng từng bước thu hẹp không gian tìm kiếm:

Bước	Đoạn $[low, high]$	Vị trí mid	Giá trị $A[mid]$	So sánh với $X = 23$	Quyết định thu hẹp
1	$[0, 9]$	$mid = 4$	$A[4] = 16$	$16 < 23$	X nằm bên phải $\Rightarrow low \leftarrow 5$
2	$[5, 9]$	$mid = 7$	$A[7] = 56$	$56 > 23$	X nằm bên trái $\Rightarrow high \leftarrow 6$
3	$[5, 6]$	$mid = 5$	$A[5] = 23$	$23 = 23$	Khớp chính xác! Tìm thấy tại chỉ số $index = 5$

2.2. Tìm kiếm phần tử biên: **lower_bound** Và **upper_bound**

Trong lập trình thi đấu, dạng toán tìm vị trí biên quan trọng hơn nhiều so với tìm chính xác:

1. **** lower_bound** (Tìm phần tử nhỏ nhất $\geq X$):**
 - Tìm vị trí đầu tiên mà giá trị tại đó $\geq X$.
 - Nếu tất cả các phần tử đều $< X$, trả về vị trí sau phần tử cuối cùng (N).
1. **** upper_bound** (Tìm phần tử nhỏ nhất $> X$):**
 - Tìm vị trí đầu tiên mà giá trị tại đó $> X$.
 - Vị trí phần tử lớn nhất $\leq X$ chính là **upper_bound - 1**.

Ví dụ minh họa 2: Mảng có phần tử lặp lại

Cho mảng: $A = [1, 3, 5, 5, 5, 8, 12]$, tìm các mốc biên với $X = 5$:

Chỉ số (0-based)	0	1	2	3	4	5	6
**Giá trị mảng <i>A</i> **	1	3	5	5	5	8	12
Vị trí con trỏ STL	—	—	lower_bound(5) (chỉ số 2)	—	—	upper_bound(5) (chỉ số 5)	—

- `lower_bound(A.begin(), A.end(), 5) - A.begin()` $\Rightarrow$ Trả về **chỉ số 2** (số 5 đầu tiên).
- `upper_bound(A.begin(), A.end(), 5) - A.begin()` $\Rightarrow$ Trả về **chỉ số 5** (phần tử đầu tiên > 5).
- Số lần xuất hiện của số 5: $Count(5) = upper - lower = 5 - 2 = 3$ phần tử.
- Vị trí xuất hiện cuối cùng của số 5: $upper - 1 = 5 - 1 = 4$.

3. Kỹ thuật chặt nhị phân trên tập kết quả (binary search on answer)

Đây là kỹ thuật cốt lõi trong các kỳ thi học sinh giỏi và Olympic tin học.

3.1. Nhận diện tính chất đơn điệu & phân loại 2 hướng

Trước khi cài đặt, **bắt buộc phải xác định hướng biến thiên đơn điệu** của hàm kiểm tra `check(x)`:

- ****Hướng 1: Dạng `True  $\rightarrow$  False` (Tìm giá trị X LỚN NHẤT thỏa mãn):****
- Đồ thị nghiệm: `[True, True, ..., True, Truemax, False, ..., False]`.
- Nếu `check(mid) == true` $\Rightarrow$ `mid` thỏa mãn, ghi nhận `ans = mid` và tìm nghiệm lớn hơn ở bên phải: `low = mid + 1`.
- Nếu `check(mid) == false` $\Rightarrow$ `mid` quá lớn, thu hẹp về bên trái: `high = mid - 1`.
- *Ví dụ điển hình:* Cắt gỗ lấy tối thiểu M mét (độ cao cưa càng thấp càng nhiều gỗ $\Rightarrow$ tìm độ cao Max).
- ****Hướng 2: Dạng `False  $\rightarrow$  True` (Tìm giá trị X NHỎ NHẤT thỏa mãn):****
- Đồ thị nghiệm: `[False, False, ..., False, Truemini, True, ..., True]`.
- Nếu `check(mid) == true` $\Rightarrow$ `mid` thỏa mãn, ghi nhận `ans = mid` và tìm nghiệm nhỏ hơn ở bên trái: `high = mid - 1`.
- Nếu `check(mid) == false` $\Rightarrow$ `mid` chưa đủ lớn, tăng giá trị lên: `low = mid + 1`.
- *Ví dụ điển hình:* Vận chuyển hàng trong D ngày (tải trọng thuyền càng lớn càng dễ chở $\Rightarrow$ tìm tải trọng Min).

Ví dụ minh họa 3: Bài toán cắt gỗ lấy tối thiểu $M = 7$ mét gỗ

Cho $N = 4$ cây có chiều cao: $A = [20, 15, 10, 17]$. Cần tìm độ cao máy cưa H **lớn nhất** sao cho tổng lượng gỗ thu được ≥ 7 .

- Không gian tìm kiếm: $low = 0, high = \max(A) = 20$.
- Hàm **check(H)**: Tính tổng $\sum \max(0, A_i - H)$. Nếu $\geq 7 \Rightarrow$ **True**, ngược lại **False**.

Bảng mô phỏng từng bước chặt nhị phân:

Bước	low	high	mid (H)	Lượng gỗ cắt được từ từng cây	Tổng gỗ thu được	check(H) ≥ 7	Quyết định cập nhật
1	0	20	** 10 **	$(20-10) + (15-10) + (0) + (17-10) = 10 + 5 + 0 + 7$	** 22m **	True (≥ 7)	Lưu ans = 10 , thử tăng độ cao: $low = 11$
2	11	20	** 15 **	$(20-15) + (0) + (0) + (17-15) = 5 + 0 + 0 + 2$	** 7m **	True (≥ 7)	Lưu ans = 15 , thử tăng độ cao: $low = 16$
3	16	20	** 18 **	$(20-18) + (0) + (0) + (0) = 2 + 0 + 0 + 0$	** 2m **	False (< 7)	Thiếu gỗ! Phải hạ cưa xuống: $high = 17$
4	16	17	** 16 **	$(20-16) + (0) + (0) + (17-16) = 4 + 0 + 0 + 1$	** 5m **	False (< 7)	Thiếu gỗ! Phải hạ cưa xuống: $high = 15$
Dừng	16	15	—	$low > high \Rightarrow$ Thuật toán kết thúc	—	—	**Đáp án tối ưu: $H = 15$ **

4. Chặt nhị phân trên tập số thực (real-number binary search)

Khi đề bài yêu cầu tìm nghiệm thực với độ chính xác sai số tuyệt đối $\leq 10^{-6}$:

- **Vấn đề của điều kiện $while (high - low > 1e-7)$** : Khi khoảng cách giữa low và $high$ đạt tới giới hạn độ phân giải của kiểu **double** (bit mantissa), phép tính trung điểm $mid = (low + high) / 2.0$ có thể bị làm tròn thành đúng low hoặc $high$, khiến hiệu số $high - low$ không thể thu hẹp thêm, dẫn đến nguy cơ vòng lặp không tiến triển hoặc chạy vô hạn.
- **Giải pháp chuẩn thi đấu**: Sử dụng vòng lặp với **số lần lặp cố định** (60 ... 100 lần):

$$\text{Độ thu hẹp} = \frac{high - low}{2^{100}} \approx \frac{10^9}{1.26 \times 10^{30}} \approx 10^{-21} \ll 10^{-6}$$

Đảm bảo thuật toán luôn dừng đúng số bước, an toàn tuyệt đối và đạt độ chính xác tối đa của phần cứng.

5. Mẫu cài đặt chuẩn thi đấu (competitive templates)

Mẫu 1a: Tìm giá trị lớn nhất thỏa mãn (dạng `True → False`)

```

#include <bits/stdc++.h>
using namespace std;

// Hàm kiểm tra: Lượng gỗ thu được khi cưa ở độ cao mid có  $\geq$  M hay không?
bool check(long long mid, const vector<long long>& a, long long m) {
    long long wood = 0;
    for (long long x : a) {
        if (x > mid) wood += (x - mid);
    }
    return wood  $\geq$  m;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    long long m;
    if (!(cin >> n >> m)) return 0;

    vector<long long> a(n);

    long long max_val = 0;
    for (int i = 0; i < n; ++i) {
        cin >> a[i];

        max_val = max(max_val, a[i]);
    }

    long long low = 0, high = max_val;
    long long ans = 0;

    while (low  $\leq$  high) {
        long long mid = low + (high - low) / 2;
        if (check(mid, a, m)) {
            ans = mid;      // Ghi nhận nghiệm hợp lệ
            low = mid + 1;  // Tìm giá trị lớn hơn ở bên phải
        } else {
            high = mid - 1; // Không thỏa mãn, thu hẹp về bên trái
        }
    }

    cout << ans << "\n";
    return 0;
}

```


Mẫu 1b: Tìm giá trị nhỏ nhất thỏa mãn (dạng **False** → **True**)

```

#include <bits/stdc++.h>
using namespace std;

// Hàm kiểm tra: Với tải trọng phà là mid, có chở hết hàng trong ≤ D ngày hay không?
bool check(long long mid, const vector<long long>& w, int d) {
    int days = 1;
    long long current_load = 0;
    for (long long x : w) {
        if (current_load + x > mid) {

            days++;
            current_load = x;
        } else {
            current_load += x;
        }
    }
    return days ≤ d;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, d;
    if (!(cin >> n >> d)) return 0;

    vector<long long> w(n);

    long long max_w = 0, sum_w = 0;
    for (int i = 0; i < n; ++i) {
        cin >> w[i];

        max_w = max(max_w, w[i]);
        sum_w += w[i];
    }

    // Không gian tìm kiếm: Tải trọng tối thiểu phải chở được kiện nặng nhất
    long long low = max_w, high = sum_w;
    long long ans = sum_w;

    while (low ≤ high) {
        long long mid = low + (high - low) / 2;
        if (check(mid, w, d)) {
            ans = mid;        // Ghi nhận nghiệm hợp lệ
            high = mid - 1;   // Tìm giá trị nhỏ hơn ở bên trái
        } else {
            low = mid + 1;    // Tải trọng chưa đủ, phải tăng lên
        }
    }

    cout << ans << "\n";
}

```

```
    return 0;
}
```

Mẫu 2: Binary search số thực (100 vòng lặp robust)

```
#include <bits/stdc++.h>
using namespace std;

bool check_real(double mid) {
    return (mid * mid * mid + 2.0 * mid * mid + 10.0 * mid ≥ 100.0);
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    double low = 0.0, high = 1e9;

    // Lặp cố định 100 lần để đạt sai số < 10^-15
    for (int iter = 0; iter < 100; ++iter) {
        double mid = low + (high - low) / 2.0;
        if (check_real(mid)) {
            high = mid;
        } else {
            low = mid;
        }
    }

    cout << fixed << setprecision(7) << low << "\n";
    return 0;
}
```

6. Các bẫy lỗi lập trình kinh điển (bug traps)

- Bẫy tràn số khi tính `mid`** : Biểu thức $mid = (low + high)/2$ sẽ bị tràn số kiểu `int` 32-bit nếu $low + high \geq 2 \cdot 10^9$. **Quy tắc bắt buộc:** Luôn viết $mid = low + (high - low)/2$.
- Bẫy vòng lặp vô tận (Infinite Loop)**: Khi không gian tìm kiếm chỉ còn 2 phần tử ($low = high - 1$), nếu cập nhật $low = mid$ trong khi mid bị làm tròn xuống sẽ khiến low không bao giờ tăng, gây TLE. Cần cập nhật $low = mid + 1$ hoặc $high = mid - 1$.
- **Bẫy biên không gian tìm kiếm $[low, high]$ **** Đặt $high$ quá nhỏ dẫn đến bỏ sót nghiệm đúng, hoặc đặt $low = 0$ dẫn đến lỗi chia cho 0 ($mid = 0$) trong hàm `check`.
- Bẫy phần tử trùng lặp trong mảng xoay vòng**: Nếu mảng xoay vòng có các phần tử trùng lặp thỏa mãn $A[low] == A[mid] == A[high]$, ta không thể xác định nửa nào được sắp xếp đơn điệu $\Rightarrow$ Trường hợp xấu nhất phải co cả hai đầu $low++$ và $high--$, làm độ phức tạp suy biến về $O(N)$.

7. Ranh giới áp dụng: Khi nào nên & không nên dùng?

• KHI NÀO ÁP DỤNG:

- Không gian tìm kiếm có tính chất **đơn điệu (Monotonic)**: Đồ thị hàm kiểm tra có dạng dài phân cách rõ ràng: `[True, ..., True, False, ..., False]`.
- Cần tối ưu nghiệm trên miền cực lớn ($1 \dots 10^{18}$) mà không thể duyệt tuần tự.

• KHI NÀO THẤT BẠI:

- Không gian tìm kiếm **không đơn điệu** (hàm dao động, có nhiều cực trị cục bộ). Lúc này chặt nhị phân sẽ bỏ sót nghiệm tối ưu toàn cục. Bắt buộc phải dùng **Ternary Search (Tìm kiếm Tam phân)** nếu hàm lồi/lõm, hoặc Quy hoạch động / Duyệt đồ thị.

Câu hỏi trắc nghiệm củng cố khái niệm

Câu 1 (bản chất — complexity):

Tại sao thuật toán tìm kiếm nhị phân trên không gian kích thước $N = 10^9$ chỉ cần tối đa khoảng 30 bước lặp?

- A. Vì mỗi bước chia không gian thành 10 phần.
- B. **[Đáp án đúng]** Vì mỗi bước loại bỏ chính xác 50% không gian tìm kiếm, và $2^{30} \approx 1.07 \times 10^9 > 10^9$.
- C. Vì mảng số nguyên trong C++ chỉ chứa tối đa 30 phần tử âm.
- D. Do trình biên dịch C++ tối ưu hóa vòng lặp thành lệnh SIMD.

Giải thích: Sau k bước lặp, không gian còn lại là $N / 2^k$. Với $N = 10^9$, $2^{30} > 10^9 \Rightarrow k \approx 30$ bước là không gian thu hẹp về 1 phần tử.

Câu 2 (điều kiện tiên quyết — monotonicity):

Yêu cầu bắt buộc để có thể áp dụng thuật toán Tìm kiếm nhị phân là gì?

- A. Mảng phải chứa toàn số dương.
- B. Kích thước mảng phải là một lũy thừa của 2.
- C. **[Đáp án đúng]** Không gian tìm kiếm hoặc hàm kiểm tra phải có tính chất đơn điệu (Monotonicity).
- D. Tất cả các phần tử trong mảng phải đôi một khác nhau.

Giải thích: Tính đơn điệu đảm bảo khi so sánh với phần tử trung điểm mid , ta chắc chắn biết nửa nào chứa nghiệm và nửa nào có thể loại bỏ an toàn.

Câu 3 (cú pháp chuẩn — bug trap):

Biểu thức nào sau đây tính trung điểm `mid` an toàn nhất để chống tràn số trong C++?

- A. `mid = (low + high) / 2;`
- B. **[Đáp án đúng]** `mid = low + (high - low) / 2;`
- C. `mid = (low + high) >> 1;`
- D. `mid = low + high / 2;`

*Giải thích: Nếu $low = 1.5 * 10^9$ và $high = 1.8 * 10^9$, tổng $low + high = 3.3 * 10^9$ vượt giới hạn $2.14 * 10^9$ của `int`. Dùng $low + (high - low) / 2$ phép trừ $(high - low)$ luôn không âm và nhỏ hơn $high$, không bao giờ tràn số.*

Câu 4 (hàm stl — lower_bound):

Cho mảng đã sắp xếp `A = [2, 4, 4, 4, 7, 9]`. Giá trị trả về của `lower_bound(A.begin(), A.end(), 4) - A.begin()` là gì?

- A. 0
- B. **[Đáp án đúng]** 1 (chỉ số của số 4 đầu tiên).
- C. 3 (chỉ số của số 4 cuối cùng).
- D. 4 (chỉ số của số 7).

Giải thích: `lower_bound(..., X)` trả về con trỏ tới phần tử đầu tiên có giá trị $\geq X$. Số 4 đầu tiên nằm tại chỉ số 1 (0-based).

Câu 5 (hàm stl — upper_bound):

Cho mảng đã sắp xếp `A = [2, 4, 4, 4, 7, 9]`. Giá trị trả về của `upper_bound(A.begin(), A.end(), 4) - A.begin()` là gì?

- A. 1
- B. 3
- C. **[Đáp án đúng]** 4 (chỉ số của số 7, phần tử đầu tiên > 4).
- D. 5

Giải thích: `upper_bound(..., X)` trả về con trỏ tới phần tử đầu tiên có giá trị nghiêm ngặt $> X$. Phần tử đầu tiên > 4 là số 7 tại chỉ số 4.

Câu 6 (đếm số lần xuất hiện — counting):

Để đếm số lần xuất hiện của giá trị X trong một vector A gồm N phần tử đã sắp xếp tăng dần trong thời gian $O(\log N)$, ta dùng biểu thức nào?

- A. `upper_bound(A.begin(), A.end(), X) - A.begin()`
- B. `count(A.begin(), A.end(), X)`
- C. **[Đáp án đúng]** `upper_bound(A.begin(), A.end(), X) - lower_bound(A.begin(), A.end(), X)`
- D. `lower_bound(A.begin(), A.end(), X) - A.begin()`

Giải thích: Hiệu vị trí của phần tử đầu tiên $> X$ và phần tử đầu tiên $\geq X$ chính là số lượng phần tử có giá trị đúng bằng X . Hàm `count` duyệt tuần tự $O(N)$ sẽ bị TLE.

Câu 7 (binary search on answer — logic):

Trong bài toán "Tìm chiều cao cắt H lớn nhất sao cho tổng lượng gỗ thu được $\geq M$ ", tính chất đơn điệu của hàm kiểm tra `check(H)` thể hiện như thế nào?

- A. Chiều cao H càng tăng thì lượng gỗ thu được càng tăng.
- B. **[Đáp án đúng]** Chiều cao H càng tăng thì lượng gỗ thu được càng giảm (hàm giảm đơn điệu).
- C. Lượng gỗ thu được luôn không đổi với mọi chiều cao H .
- D. Hàm lượng gỗ biến thiên ngẫu nhiên theo H .

Giải thích: Khi nâng máy cắt lên cao (H tăng), phần ngọn cây bị cắt sẽ ngắn đi, do đó tổng lượng gỗ thu được chắc chắn giảm dần. Đây là hàm đơn điệu giảm.

Câu 8 (binary search on answer — search space):

Nếu bài toán yêu cầu tìm giá trị X nhỏ nhất thỏa mãn `check(X) = true`, sau khi kiểm tra tại `mid` thấy `check(mid) = true`, ta cần cập nhật bước tiếp theo như thế nào?

- A. `low = mid + 1;`
- B. **[Đáp án đúng]** `ans = mid; high = mid - 1;` (Ghi nhận `mid` là một đáp án hợp lệ và tiếp tục tìm giá trị nhỏ hơn ở nửa trái).
- C. `ans = mid; return ans;`
- D. `high = mid + 1;`

*Giải thích: Vì đề bài yêu cầu tìm X **nhỏ nhất**, một giá trị mid thỏa mãn có thể chưa phải là nhỏ nhất implies ghi nhận $ans = mid$ rồi thu hẹp không gian tìm kiếm sang bên trái $high = mid - 1$.*

Câu 9 (chặt nhị phân số thực — real numbers):

Tại sao khi chặt nhị phân trên tập số thực, ta nên dùng vòng lặp `for (int iter = 0; iter < 100; ++iter)` thay vì `while (high - low > 1e-7)` ?

- A. Để chương trình chạy nhanh hơn gấp 100 lần.
- B. **[Đáp án đúng]** Để tránh nguy cơ lặp vô tận do sai số làm tròn số thực (Floating-point round-off error) khiến hiệu `high - low` không bao giờ nhỏ hơn epsilon.
- C. Vì số thực trong C++ chỉ biểu diễn được tối đa 100 chữ số thập phân.
- D. Do tiêu chuẩn thi đấu Olympic cấm dùng vòng lặp `while`.

Giải thích: Với kiểu `double`, khi $high$ và low rất gần nhau, phép trừ $high - low$ có thể bị kẹt do giới hạn bit mantissa. Lặp 100 lần đảm bảo chia đôi khoảng cách 2^{100} lần, đạt độ chính xác cực cao mà không bao giờ bị kẹt vòng lặp.

Câu 10 (ranh giới thất bại — failure boundary):

Trường hợp nào sau đây **KHÔNG THỂ** giải bằng thuật toán Tìm kiếm nhị phân một cách trực tiếp?

- A. Tìm căn bậc hai của số nguyên lớn $N \leq 10^{18}$.
- B. Tìm phần tử nhỏ nhất lớn hơn X trong mảng đã sắp xếp.
- C. **[Đáp án đúng]** Tìm giá trị X để hàm số đa thức bậc 4 có 3 điểm cực trị `f(x)` đạt giá trị lớn nhất trên đoạn $[-1000, 1000]$.
- D. Chia mảng thành K đoạn con liên tiếp sao cho tổng đoạn lớn nhất là nhỏ nhất.

*Giải thích: Hàm đa thức bậc 4 có 3 điểm cực trị không có tính chất đơn điệu trên toàn đoạn $[-1000, 1000]$ (đổi chiều tăng/giảm nhiều lần), do đó Binary Search không thể loại bỏ an toàn một nửa không gian. Lưu ý: Thuật toán Tìm kiếm Tam phân (Ternary Search) cũng chỉ áp dụng được cho hàm **đơn đỉnh (unimodal)** có đúng 1 cực trị duy nhất, không áp dụng trực tiếp cho hàm đa cực trị như đa thức bậc 4 này.*

Câu 11 (mảng xoay vòng — rotated array):

Cho mảng gồm các phần tử đôi một phân biệt đã sắp xếp nhưng bị xoay vòng tại một vị trí P (ví dụ: $[4, 5, 6, 7, 0, 1, 2]$). Khi xét phần tử trung điểm $A[mid]$, tính chất cốt lõi nào cho phép ta tiếp tục tìm kiếm nhị phân?

- A. Cả hai nửa trái và phải đều đã được sắp xếp tăng dần.
- B. **[Đáp án đúng]** Ít nhất một trong hai nửa $[low \dots mid]$ hoặc $[mid \dots high]$ chắc chắn là một dãy tăng dần đơn điệu bình thường.
- C. Phần tử nhỏ nhất luôn nằm ở chính giữa mảng.
- D. Mảng luôn có số lượng phần tử là số lẻ.

Giải thích: Điểm gãy (Pivot) chỉ nằm ở 1 trong 2 nửa. Do đó, nửa còn lại luôn là một mảng tăng dần hoàn hảo, ta có thể kiểm tra xem X có thuộc khoảng giá trị của nửa đó không để thu hẹp không gian. (Lưu ý: Nếu mảng chứa các phần tử trùng lặp thỏa $A[low] = A[mid] = A[high]$, ta không thể xác định nửa nào được sắp xếp, thuật toán buộc phải co $low++$, $high--$ và có thể suy biến về $O(N)$).

Câu 12 (ma trận 2d đã sắp xếp — 2d matrix binary search):

Cho ma trận $N \times M$ gồm các số nguyên tăng dần từ trái sang phải trên từng hàng và phần tử đầu mỗi hàng luôn lớn hơn phần tử cuối hàng trước. Để tìm kiếm phần tử X trong $O(\log(N \times M))$, ta ánh xạ chỉ số 1D mid sang tọa độ ô (r, c) bằng công thức nào?

- A. $r = mid \bmod M, c = mid / M$
- B. **[Đáp án đúng]** $r = mid / M, c = mid \bmod M$ (với chỉ số 0-based).
- C. $r = mid / N, c = mid \bmod N$
- D. $r = mid * M, c = mid + M$

Giải thích: Coi ma trận $N \times M$ như một mảng 1D độ dài $N \times M$. Chỉ số dòng là $r = \text{floor}(mid / M)$ và chỉ số cột là $c = mid \bmod M$.

Câu 13 (đỉnh dãy núi — mountain array peak):

Trong một mảng dạng đỉnh núi (tăng dần rồi giảm dần: $A_0 < A_1 < \dots < A_p > A_{p+1} > \dots > A_{N-1}$), điều kiện nào tại vị trí `mid` cho biết đỉnh núi nằm ở bên phải `mid`?

- A. $A[mid] > A[mid + 1]$
- B. **[Đáp án đúng]** $A[mid] < A[mid + 1]$ (đang ở sườn dốc đi lên, đỉnh núi chắc chắn nằm bên phải $\text{implies } low = mid + 1$).
- C. $A[mid] = A[mid + 1]$
- D. $A[mid] < A[mid - 1]$

Giải thích: Nếu $A[mid] < A[mid+1]$, dãy đang có xu hướng tăng tại `mid`, do đó đỉnh núi chưa đạt được và nằm về phía bên phải.

Câu 14 (trung vị hai mảng đã sắp xếp — advanced partition):

Thuật toán tìm phần tử trung vị của hai mảng đã sắp xếp `A` (kích thước `N`) và `B` (kích thước `M`) trong thời gian tối ưu $O(\log(\min(N, M)))$ dựa trên việc chặt nhị phân đối tượng nào?

- A. Chặt nhị phân giá trị của phần tử trung vị từ $-10^9 \dots 10^9$.
- B. **[Đáp án đúng]** Chặt nhị phân vị trí vách ngăn (cut partition) trên mảng có kích thước nhỏ hơn để chia tổng hai mảng thành 2 nửa bằng nhau.
- C. Sắp xếp lại toàn bộ mảng gộp trong $O((N+M)\log(N+M))$.
- D. Duyệt tuần tự 2 con trỏ qua cả 2 mảng.

Giải thích: Bằng cách chặt nhị phân số lượng phần tử lấy từ mảng nhỏ hơn $i \in [0, N]$, số lượng phần tử lấy từ mảng lớn hơn được cố định $j = (N + M + 1)/2 - i$. Ta kiểm tra điều kiện vách ngăn hợp lệ trong $O(1)$ implies Tổng thời gian $O(\log(\min(N, M)))$.

Ma trận bài tập thực hành (p0 → p5)

STT	Mã Bài	Tên Bài Toán	Cấp Độ	Ràng Buộc Dữ Liệu	Mục Tiêu Rèn Luyện
01	CPPB-BS-01	Tìm Kiếm Phần Tử Trên Mảng Đã Sắp Xếp	P0	$N, Q \leq 10^5$	Cài đặt Binary Search cơ bản
02	CPPB-BS-02	Tìm Vị Trí Xuất Hiện Đầu Tiên & Cuối Cùng	P1	$N, Q \leq 10^5$	Bản chất <code>lower_bound</code> / <code>upper_bound</code>
03	CPPB-BS-03	**Đếm Số Phần Tử Trong Đoạn $[L, R]$ **	P1	$N, Q \leq 10^5$	Hiệu hai con trỏ nhị phân <code>upper - lower</code>
04	CPPB-BS-04	Tìm Căn Bậc Hai Số Nguyên Lớn	P2	$N \leq 10^{18}$	Binary Search trên tập số nguyên 64-bit
05	CPPB-BS-05	Tìm Phần Tử Nhỏ Nhất Lớn Hơn X	P2	$N, Q \leq 10^5$	Chặn trên nghiêm ngặt
06	CPPB-BS-06	Chia Kẹo Cho Học Sinh Đạt Chuẩn	P3	$N \leq 10^5, K \leq 10^{14}$	Chặt nhị phân kết quả (Check chia đều)
07	CPPB-BS-07	Cắt Gỗ Xây Dựng (Woodcutting / EKO)	P2	$N \leq 10^5, M \leq 10^{14}$	Bài toán kinh điển tìm độ cao máy cắt
08	CPPB-BS-08	Đặt Trạm Phát Sóng Cách Nhau Xa Nhất (Aggressive Cows)	P3	$N \leq 10^5, C \leq N$	Tối đại hóa khoảng cách nhỏ nhất
09	CPPB-BS-09	Chia Mảng Thành K Đoạn Có Tổng Max Nhỏ Nhất	P3	$N \leq 10^5, K \leq N$	Tối thiểu hóa tổng đoạn con lớn nhất
10	CPPB-BS-10	Vận Chuyển Hàng Hóa Qua Phà Trong D Ngày	P3	$N \leq 10^5, D \leq 10^5$	Chặt nhị phân tải trọng thuyền
11	CPPB-BS-11	Tìm Nghiệm Thực Của Phương Trình Đơn Điện	P4	Sai số 10^{-7}	Chặt nhị phân số thực với số lần lặp cố định
12	CPPB-BS-12	Phần Tử Thứ K Của Hai Mảng Đã Sắp Xếp	P4	$N, M \leq 10^5$	Chặt nhị phân số lượng phần tử $\leq X$
13	CPPB-BS-13	Tìm Đoạn Con Có Trung Bình Lớn Nhất Độ Dài $\geq K$	P5	$N \leq 10^5, K \leq N$	Chặt nhị phân trung bình + Mảng tiền tố
14	CPPB-BS-14	Tối Ưu Hóa Tuyến Đường Vận Tải Đa Điểm	P5	$N \leq 2 \times 10^5$	Chặt nhị phân kết hợp cấu trúc đơn điện
15			P3	$N \leq 10^5$	

STT	Mã Bài	Tên Bài Toán	Cấp Độ	Ràng Buộc Dữ Liệu	Mục Tiêu Rèn Luyện
	CPPB-BS-15	Tìm Kiếm Trên Mảng Sắp Xếp Bị Xoay Vòng (Rotated Array)			Phân đoạn đơn điệu trong mảng xoay
16	CPPB-BS-16	Tìm Kiếm Trên Ma Trận 2D Đã Sắp Xếp (Matrix Search)	P2	$N, M \leq 1000$	Chuyển tọa độ 1D $\rightarrow$ 2D trong nhị phân
17	CPPB-BS-17	Tìm Đỉnh Của Dãy Núi (Peak in Mountain Array)	P3	$N \leq 10^5$	Chặt nhị phân theo đạo hàm / độ dốc
18	CPPB-BS-18	Trung Vị Của Hai Mảng Đã Sắp Xếp (Median of Two Sorted)	P5	$N, M \leq 10^5$	Phân chia vách ngăn nhị phân tối ưu $O(\log(\min(N, M)))$

<!-- Chuyên đề 06: Phép Toán Bit | 389 dòng | 18,532 bytes -->

Chuyên đề 06: Phép toán bit & biểu diễn trạng thái

Tối ưu hóa cấp độ thanh ghi & kỹ thuật mặt nạ bit (Bitmask) trong C++

1. Khái niệm & 6 phép toán bit cơ bản

Máy tính biểu diễn tất cả dữ liệu dưới dạng chuỗi nhị phân (gồm các bit `0` và `1`). **Phép toán bit (Bitwise Operations)** là các thao tác tác động trực tiếp lên từng bit của thanh ghi CPU, đạt tốc độ thực thi nhanh nhất trong mọi câu lệnh phần mềm.

1.1. Bảng chân trị của 6 phép toán bit trong c++

Toán Từ C++ +	Tên Phép Toán	Ký Hiệu Toán	Quy Tắc Bit	Ví dụ ($a = 5 = 101_2, b = 3 = 011_2$)
<code>&</code>	AND (Và)	<i>wedge</i>	Ra <code>1</code> khi và chỉ khi cả 2 bit đều là <code>1</code>	$5 \& 3 = 101_2 \& 011_2 = 001_2 = 1$
<code> </code>	OR (Hoặc)	<i>vee</i>	Ra <code>1</code> khi có ít nhất một bit là <code>1</code>	$5 3 = 101_2 011_2 = 111_2 = 7$
<code>^</code>	XOR (Hoặc loại trừ)	<i>oplus</i>	Ra <code>1</code> khi 2 bit khác nhau, ra <code>0</code> khi 2 bit giống nhau	$5 \oplus 3 = 101_2 \oplus 011_2 = 110_2 = 6$
<code>~</code>	NOT (Đảo bit)	<i>neg</i>	Đổi $0 \rightarrow 1$ và $1 \rightarrow 0$	$\sim 5 = \sim(00...0101_2) = -6$
<code><<</code>	Dịch trái (Left Shift)	<i>ll</i>	Dịch các bit sang trái k vị trí (nhân 2^k)	$5 \ll 2 = 10100_2 = 20$
<code>>></code>	Dịch phải (Right Shift)	<i>gg</i>	Dịch các bit sang phải k vị trí (chia nguyên 2^k)	$5 \gg 1 = 10_2 = 2$

1.2. Các tính chất đại số quan trọng của phép xor (oplus)

- Tính tự triệt tiêu: $A \oplus A = 0$.
- Phần tử trung hòa: $A \oplus 0 = A$.
- Giao hoán & Kết hợp: $A \oplus B = B \oplus A$ và $(A \oplus B) \oplus C = A \oplus (B \oplus C)$.

- Đổi giá trị 2 biến không cần biến phụ: `a ^= b; b ^= a; a ^= b;`.

2. 4 Thao tác thao tác bit chuẩn thi đấu

Quy ước đánh số các bit từ phải sang trái, bắt đầu từ bit 0 (bit có trọng số nhỏ nhất 2^0).

2.1. Kiểm tra bit thứ k có đang bật (bằng 1) hay không:

```
bool is_set = (mask >> k) & 1;

// Hoặc: bool is_set = (mask & (1LL << k)) != 0;
```

2.2. Bật bit thứ k (gán thành 1):

```
mask = mask | (1LL << k);
// Viết gọn: mask |= (1LL << k);
```

2.3. Tắt bit thứ k (gán thành 0):

```
mask = mask & ~(1LL << k);
// Viết gọn: mask &= ~(1LL << k);
```

2.4. Đảo bit thứ k ($0 \rightarrow 1, 1 \rightarrow 0$):

```
mask = mask ^ (1LL << k);
// Viết gọn: mask ^= (1LL << k);
```

Ví dụ minh họa 1: Thao tác trên số $N = 13 = 1101_2$

Trọng số nhị phân	$2^4 = 16$	$2^3 = 8$	$2^2 = 4$	$2^1 = 2$	$2^0 = 1$	Giá trị thập phân
Vị trí bit (k)	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	—
**Giá trị bit của N **	0	1	1	0	1	** 13 **

Bảng mô phỏng 4 thao tác bit:

Thao Tác Cần Thực Hiện	Mã Lệnh C++	Phép Toán Nhị Phân	Kết Quả Nhị Phân	Giá Trị Thập Phân Mới
1. Kiểm tra bit 2	<code>(n >> 2) & 1</code>	<code>(1101 >> 2) & 0001 = 0011 & 0001</code>	1	Bit 2 đang bật (true)
2. Bật bit 1	<code>n 1</code>	<code>= (1 << 1)</code>	<code>1101 0010`</code>	<code>1111</code> <code>13 → 15</code>
3. Tắt bit 3	<code>n &= ~(1 << 3)</code>	<code>1101 & ~(1000) = 1101 & 0111</code>	<code>0101</code>	<code>13 → 5</code>
4. Đảo bit 0	<code>n ^= (1 << 0)</code>	<code>1101 ^ 0001</code>	<code>1100</code>	<code>13 → 12</code>

3. Các tuyệt kỹ bit & hàm nội tại cpu (builtin functions)

3.1. Kiểm tra một số nguyên dương có phải là lũy thừa của 2

Một số $N > 0$ là lũy thừa của 2 (2^k) khi và chỉ khi trong biểu diễn nhị phân của nó có **đúng duy nhất một bit 1**:

```
bool is_power_of_two = (n > 0) && ((n & (n - 1)) == 0);
```

3.2. Lấy bit 1 nhỏ nhất (lowest set bit / lowbit)

Dùng trong cấu trúc Fenwick Tree và giải thuật bit:

```
long long lowbit = x & (-x);
```

3.3. Các hàm nội tại tối ưu hóa phần cứng trong gcc/clang:

- `__builtin_popcount(unsigned int x)` / `__builtin_popcountll(unsigned long long x)` : Đếm số lượng bit 1 trong $O(1)$ chu kỳ CPU.
- `__builtin_clz(x)` / `__builtin_clzll(x)` : Đếm số lượng bit 0 liên tiếp ở đầu (Count Leading Zeros).
- `__builtin_ctz(x)` / `__builtin_ctzll(x)` : Đếm số lượng bit 0 liên tiếp ở cuối (Count Trailing Zeros).

4. Kỹ thuật mặt nạ bit (bitmask & subset enumeration)

Mặt nạ bit (**Bitmask**) là kỹ thuật dùng một số nguyên N bit để biểu diễn một tập hợp con gồm các phần tử được chọn từ tập N phần tử:

- Bit thứ $i = 1 \Rightarrow$ Phần tử thứ i được chọn.
- Bit thứ $i = 0 \Rightarrow$ Phần tử thứ i không được chọn.

4.1. Duyệt toàn bộ 2^N tập con (vét cạn nhị phân):

```
int n = 4;
for (int mask = 0; mask < (1 << n); ++mask) {
    for (int i = 0; i < n; ++i) {
        if ((mask >> i) & 1) {
            // Phần tử i thuộc tập con hiện tại
        }
    }
}
```

4.2. Duyệt tất cả tập con (submasks) của một mask trong $O(3^N)$:

```
for (int sub = mask; sub > 0; sub = (sub - 1) & mask) {
    // sub là một tập con hợp lệ của mask
}
```

Ví dụ minh họa 2: Biểu diễn tập con của tập 3 phần tử $S = \{A_0, A_1, A_2\}$

Với $N = 3$, có $2^3 = 8$ mặt nạ bit từ 0 đến 7:

Giá Trị Mask (Thập phân)	Biểu Diễn Nhị Phân ($b_2 b_1 b_0$)	Bit 2 (A_2)	Bit 1 (A_1)	Bit 0 (A_0)	Tập Con Tương Ứng
0	000	0	0	0	<i>emptyset</i> (Tập rỗng)
1	001	0	0	1	$\{A_0\}$
2	010	0	1	0	$\{A_1\}$
3	011	0	1	1	$\{A_0, A_1\}$
4	100	1	0	0	$\{A_2\}$
5	101	1	0	1	$\{A_0, A_2\}$
6	110	1	1	0	$\{A_1, A_2\}$
7	111	1	1	1	$\{A_0, A_1, A_2\}$ (Tập đầy đủ)

5. Mẫu cài đặt chuẩn thi đấu (competitive template)

Mẫu 1: Vết cạn tập con bằng mặt nạ bit (subset sum)

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    long long target_s;
    if (!(cin >> n >> target_s)) return 0;

    vector<long long> a(n);

    for (int i = 0; i < n; ++i) {
        cin >> a[i];
    }

    bool found = false;
    int total_masks = (1 << n);

    for (int mask = 0; mask < total_masks; ++mask) {
        long long current_sum = 0;
        for (int i = 0; i < n; ++i) {
            if ((mask >> i) & 1) {
                current_sum += a[i];
            }
        }
        if (current_sum == target_s) {
            found = true;
            break;
        }
    }

    cout << (found ? "YES\n" : "NO\n");
    return 0;
}
```

6. Các bẫy lỗi lập trình kinh điển (bug traps)

1. **Bẫy thứ tự ưu tiên toán tử (Operator Precedence Bug):** Trong C++, các phép toán bit `&`, `|`, `^` có độ ưu tiên **thấp hơn** các phép toán so sánh `=`, `≠`, `<`, `>`.
 - **Lỗi sai:** `if (mask & (1 << k) ≠ 0)` sẽ bị hiểu thành `if (mask & ((1 << k) ≠ 0))` ⇒ Sai kết quả!
 - **Cú pháp chuẩn:** `if ((mask & (1 << k)) ≠ 0)` hoặc `if ((mask >> k) & 1)`.
1. **Bẫy tràn số khi dịch bit quá 31:** Hằng số `1` mặc định là số nguyên 32-bit có dấu. Biểu thức `1 << 40` sẽ gây tràn số và lỗi hành vi không xác định (Undefined Behavior).
 - **Quy tắc bắt buộc:** Luôn viết `1LL << k` khi $k \geq 31$.

7. Ranh giới áp dụng: Khi nào nên & không nên dùng?

- **KHI NÀO ÁP DỤNG:**
 - Kích thước tập hợp nhỏ: $N \leq 20$ ($2^{20} \approx 10^6$ phép tính) hoặc $N \leq 24$ ($2^{24} \approx 1.6 \cdot 10^7$ phép tính).
 - Cần tối ưu bộ nhớ trạng thái và tốc độ truy vấn tập hợp $O(1)$.
- **KHI NÀO THẤT BẠI:**
 - Khi $N \geq 30$ ($2^{30} \approx 10^9$ phép tính ⇒ TLE). Lúc này bắt buộc phải dùng:
 - **Chia đôi tập hợp (Meet-in-the-middle)** khi $N \leq 40$ ($O(2N/2) = 2^{20} \approx 10^6$).
 - Quy hoạch động hoặc Thuật toán Tham lam nếu bài toán có cấu trúc con tối ưu.

Câu hỏi trắc nghiệm củng cố khái niệm

Câu 1 (bản chất xor — identity):

Giá trị của biểu thức `A XOR B XOR A` trong C++ luôn bằng gì?

- A. 0
- B. `A`
- C. [Đáp án đúng] `B` (vì `A XOR A = 0` và `0 XOR B = B`).
- D. `2A + B`

Giải thích: Tính chất giao hoán và tự triệt tiêu của phép XOR: `A XOR B XOR A = (A XOR A) XOR B = 0 XOR B = B`.

Câu 2 (bấđ độ ưu tiên toán tử — precedence):

Đoạn mã C++ `if ((mask >> 3) & 1)` có ý nghĩa là gì?

- A. Dịch biến `mask` sang phải 4 vị trí.
- B. **[Đáp án đúng]** Kiểm tra xem bit thứ 3 của `mask` có đang được bật (bằng 1) hay không.
- C. Bật bit thứ 3 của `mask` lên 1.
- D. Tắt bit thứ 3 của `mask`.

Giải thích: Dịch phải 3 vị trí đưa bit thứ 3 về vị trí số 0, sau đó `& 1` sẽ trích xuất đúng giá trị của bit này (0 hoặc 1).

Câu 3 (kỹ thuật bật bit — manipulation):

Để bật bit thứ `k` của biến số nguyên `mask` lên 1 mà không làm thay đổi các bit khác, ta dùng câu lệnh nào?

- A. `mask = mask & (1LL << k);`
- B. **[Đáp án đúng]** `mask = mask | (1LL << k);`
- C. `mask = mask ^ (1LL << k);`
- D. `mask = mask + (1LL << k);`

Giải thích: Phép OR với số có bit thứ `k` bằng 1 và các bit khác bằng 0 sẽ biến bit thứ `k` thành 1 mà giữ nguyên các bit còn lại.

Câu 4 (kỹ thuật tắt bit — manipulation):

Để tắt bit thứ `k` của biến số nguyên `mask` về 0, ta dùng câu lệnh nào?

- A. `mask = mask | ~(1LL << k);`
- B. `mask = mask - (1LL << k);`
- C. **[Đáp án đúng]** `mask = mask & ~(1LL << k);`
- D. `mask = mask ^ (1LL << k);`

Giải thích: `~(1LL << k)` tạo ra một mặt nạ chứa toàn bit 1 ngoại trừ bit `k` bằng 0. Khi `&` với `mask`, bit thứ `k` chắc chắn về 0.

Câu 5 (lũy thừa của 2 — bit trick):

Biểu thức $n > 0 \ \&\& \ (n \ \& \ (n - 1)) = 0$ trả về `true` khi và chỉ khi:

- A. n là một số nguyên chẵn.
- B. **[Đáp án đúng]** n là một lũy thừa của 2 ($n = 2^k$ với $k \geq 0$).
- C. n là một số nguyên tố.
- D. n chia hết cho 4.

Giải thích: Một lũy thừa của 2 có dạng $100 \dots 0_2$, khi trừ 1 sẽ thành $011 \dots 1_2$. Phép AND giữa hai số này bằng đúng 0.

Câu 6 (đếm bit 1 — builtin):

Để đếm số lượng bit 1 của một số nguyên 64-bit `long long x` trong thời gian $O(1)$, hàm nào sau đây là chuẩn xác nhất?

- A. `__builtin_popcount(x)`
- B. **[Đáp án đúng]** `__builtin_popcountll(x)`
- C. `__builtin_ctzll(x)`
- D. `__builtin_clzll(x)`

Giải thích: Với kiểu `long long` 64-bit, bắt buộc phải dùng phiên bản có hậu tố `ll` là `__builtin_popcountll`. Phiên bản không có `ll` chỉ đếm 32 bit thấp.

Câu 7 (không gian tập con — complexity):

Một tập hợp có $N = 20$ phần tử. Số lượng tập con được sinh ra bởi mặt nạ bit là bao nhiêu và thời gian duyệt vét cạn có chạy kịp 1 giây không?

- A. $20^2 = 400$ tập con, chạy kịp.
- B. **[Đáp án đúng]** $2^{20} = 1,048,576$ tập con, chạy mất khoảng `0.01` giây, hoàn toàn kịp thời gian 1 giây.
- C. $20! \approx 2.4 \times 10^{18}$ tập con, bị quá thời gian.
- D. $2^{20} \approx 10^9$ tập con, bị quá thời gian.

Giải thích: Mỗi phần tử có 2 lựa chọn (chọn hoặc không) implies $2^{20} \approx 1.05 \times 10^6$ trạng thái. Vòng lặp 10^6 chạy dưới `0.02` giây trong C++.

Câu 8 (tìm phần tử đơn lẻ — xor application):

Cho mảng gồm $2N + 1$ số nguyên, trong đó có đúng một số xuất hiện 1 lần, tất cả các số còn lại đều xuất hiện đúng 2 lần. Thuật toán tìm số xuất hiện 1 lần tối ưu nhất là gì?

- A. Dùng 2 vòng lặp lồng nhau $O(N^2)$.
- B. Sắp xếp mảng mất $O(N \log N)$.
- C. **[Đáp án đúng]** Tính XOR tất cả các phần tử trong mảng trong $O(N)$ thời gian và $O(1)$ bộ nhớ.
- D. Dùng bảng băm đếm tần suất.

Giải thích: Các cặp số giống nhau khi XOR với nhau sẽ triệt tiêu về 0 ($x \text{ XOR } x = 0$). Kết quả XOR của toàn bộ mảng chính là số xuất hiện 1 lần duy nhất.

Câu 9 (bẫy dịch bit 64-bit — 64-bit shift):

Đoạn code `long long mask = 1 << 40;` sẽ gây ra lỗi gì trong C++?

- A. Lỗi biên dịch không thể dịch bit.
- B. **[Đáp án đúng]** Tràn số nguyên 32-bit (vì số `1` mặc định là `int`), dẫn đến kết quả sai hoặc hành vi không xác định (Undefined Behavior).
- C. Lỗi tràn bộ nhớ RAM.
- D. Tự động ép kiểu thành 64-bit mà không có lỗi gì.

Giải thích: Hằng số `1` mang kiểu `int` 32-bit, không thể dịch 40 vị trí. Bắt buộc phải viết `1LL << 40`.

Câu 10 (duyet submask — advanced technique):

Vòng lặp `for (int sub = mask; sub > 0; sub = (sub - 1) & mask)` dùng để làm gì?

- A. Duyệt tất cả các số từ `mask` về 1.
- B. **[Đáp án đúng]** Duyệt chính xác và đầy đủ tất cả các tập con thực sự (Submasks) của `mask` mà không duyệt thừa bất kỳ trạng thái nào khác.
- C. Xóa tất cả các bit 1 của `mask`.
- D. Đếm số lượng bit 0 của `mask`.

Giải thích: Đây là kỹ thuật kinh điển trong quy hoạch động Bitmask để sinh tất cả các tập con của một mặt nạ bit trong $O(3^N)$ tổng thời gian cho toàn bộ các mask.

Câu 11 (cặp tổng lũy thừa của 2 — power of 2 pairs):

Cho $A_i \leq 10^9$. Để đếm số cặp $A_i + A_j = 2^k$, tại sao ta chỉ cần lặp tối đa k từ 1 đến 30?

- A. Vì kiểu `long long` trong C++ chỉ biểu diễn được 30 bit.
- B. **[Đáp án đúng]** Vì giá trị tổng lớn nhất của hai số là $10^9 + 10^9 = 2 * 10^9 < 2^{31}$, do đó chỉ có tối đa 30 lũy thừa của 2 khả dĩ.
- C. Vì số 30 là số nguyên tố.
- D. Do thuật toán chỉ kiểm tra các số chẵn.

Giải thích: $A_i + A_j \leq 2 * 10^9 < 2^{31} \approx 2.147 * 10^9$. Do đó k chỉ có thể nhận các giá trị từ 1 ... 30.

Câu 12 (tập độc lập về bit — bit independence):

Hai số nguyên dương X và Y được gọi là độc lập về bit khi biểu thức nào sau đây bằng 0?

- A. $X \text{ XOR } Y = 0$.
- B. **[Đáp án đúng]** $X \& Y = 0$ (hai số không có bất kỳ bit 1 nào nằm ở cùng vị trí).
- C. $X | Y = 0$.
- D. $X + Y = 0$.

Giải thích: Phép AND kiểm tra các bit trùng nhau. $X \& Y = 0$ iff không có vị trí bit nào mà cả X và Y cùng bằng 1.

Ma trận bài tập thực hành (p0 → p5)

STT	Mã Bài	Tên Bài Toán	Cấp Độ	Ràng Buộc Dữ Liệu	Mục Tiêu Rèn Luyện
01	CPPB-BIT-01	Bật, Tắt Và Kiểm Tra Bit Thứ K	P0	$N \leq 10^{18}, K \leq 60$	Thao tác <code>(1LL << k)</code> , <code>&</code> , <code>^</code>
02	CPPB-BIT-02	Đếm Số Lượng Bit 1 (Popcount)	P1	$N \leq 10^{18}$	<code>__builtin_popcountll</code> và thuật toán bit
03	CPPB-BIT-03	Kiểm Tra Số Có Phải Lũy Thừa Của 2	P1	$N \leq 10^{18}$	Kỹ thuật <code>n > 0 && (n & (n - 1)) == 0</code>

04	CPPB- BIT-04	Tìm Phần Tử Xuất Hiện 1 Lần Duy Nhất	P2	$N \leq 2 \times 10^5$	Tính chất tự triệt tiêu $A \text{ XOR } A = 0$
05	CPPB- BIT-05	Tìm Hai Số Xuất Hiện 1 Lần Duy Nhất	P2	$N \leq 2 \times 10^5$	Phân tách nhóm bằng bit khác biệt đầu tiên
06	CPPB- BIT-06	Đảo Bit Và Giá Trị Bù 1	P2	$N \leq 10^9$	Phép toán NOT kết hợp mặt nạ
07	CPPB- BIT-07	**Duyệt Toàn Bộ 2^N Tập Con Bằng Mặt Nạ Bit**	P2	$N \leq 20$	<pre>for (int mask = 0; mask < (1 << n); + +mask)</pre>
08	CPPB- BIT-08	Bài Toán Tổng Tập Con Bằng S (Subset Sum)	P3	$N \leq 20, S \leq 10^9$	Duyệt nhị phân vét cạn 2^N
09	CPPB- BIT-09	Chia Tập Hợp Thành 2 Phần Có Tổng Chênh Lệch Nhỏ Nhất	P3	$N \leq 20$	Vét cạn bitmask tối ưu hiệu
10	CPPB- BIT-10	Đếm Cặp Có Tích Bit AND Bằng 0	P3	$N \leq 10^5, A_i < 2^{16}$	Tần suất bit và kiểm tra tương thích
11	CPPB- BIT-11	Tìm Cặp Có XOR Lớn Nhất Trong Mảng	P4	$N \leq 10^5, A_i \leq 10^9$	Duyệt từng bit từ cao xuống thấp (Greedy Bit)
12	CPPB- BIT-12	Duyệt Tất Cả Các Tập Con Của Một Mặt Nạ Bit	P4	$N \leq 18$	Kỹ thuật $submask = (submask - 1) \& mask$
13	CPPB- BIT-13	Tìm Dãy Con Có Tổng XOR Bằng K	P4	$N \leq 22$	Vét cạn nâng cao kết hợp bit
14	CPPB- BIT-14	Tối Ưu Hóa Gán Việc Cho N Người ($N \leq 20$)	P5	$N \leq 20$	Bitmask trạng thái và tối ưu hóa tổ hợp
15	CPPB- BIT-15	Đếm Số Cặp Có Tổng Bằng Lũy Thừa Của 2	P3	$N \leq 10^5, A_i \leq 10^9$	Kết hợp bitmask và hai con trò / chặt nhị phân
16	CPPB- BIT-16	Tập Hợp Độc Lập Về Bit Lớn Nhất	P4	$N \leq 24$	Bitmask đồ thị độc lập cực đại

Module 03: Số học & đại số modular

<!-- Chuyên đề 07: Ước, Bội & Số Nguyên Tố | 378 dòng | 19,246 bytes -->

Chuyên đề 07: Lý thuyết số & số nguyên tố

1. Bản chất vấn đề & trực giác thuật toán (the core problem & intuition)

Trong khoa học máy tính và lập trình thi đấu, các bài toán xoay quanh **ước số, bội số và số nguyên tố** là nền tảng của mật mã học (như thuật toán mã hóa khóa công khai RSA), phân tích độ phức tạp thuật toán và tối ưu hóa tài nguyên.

Vấn đề 1: Tìm ước chung lớn nhất (gcd)

Cho hai số nguyên dương A và B . Ước chung lớn nhất $\text{gcd}(A, B)$ là số nguyên dương lớn nhất đồng thời chia hết cả A và B .

- **Cách ngây thơ:** Thử tất cả các số từ $\min(A, B)$ giảm dần về $1 \Rightarrow O(\min(A, B))$. Khi $A, B \approx 10^{18}$, cách này hoàn toàn bất khả thi.
- **Định lý Euclid:** $\text{gcd}(A, B) = \text{gcd}(B, A \bmod B)$.
- Mỗi bước lấy dư $A \bmod B$, giá trị giảm ít nhất một nửa sau mỗi 2 bước lặp $\Rightarrow$ Thuật toán dừng lại sau tối đa $O(\log(\min(A, B)))$ bước (khoảng ≤ 60 phép tính với số 10^{18}).

Vấn đề 2: Kiểm tra số nguyên tố & phân tích thừa số nguyên tố

Một số nguyên $N > 1$ là số nguyên tố nếu nó chỉ có đúng 2 ước là 1 và chính nó.

- **Tính chất đối xứng của ước số:** Nếu d là ước của N thì $\frac{N}{d}$ cũng là ước của N .
- ****Bất biến $\sqrt{N}$:**** Nếu N là hợp số, nó **bắt buộc** phải có ít nhất một ước nguyên tố $p \leq \sqrt{N}$. Do đó, ta chỉ cần duyệt kiểm tra các số từ 2 đến $\lfloor \sqrt{N} \rfloor$ trong $O(\sqrt{N})$ thay vì $O(N)$.

2. Mô phỏng từng bước (visual step-by-step simulation)

Ví dụ 1: Mô phỏng thuật toán euclid tìm $\text{gcd}(252, 105)$

Bước lặp	A	B	Phép chia lấy dư $A \bmod B$	Trạng thái tiếp theo $(A', B') = (B, A \bmod B)$
1	252	105	$252 \bmod 105 = 42$	$(105, 42)$
2	105	42	$105 \bmod 42 = 21$	$(42, 21)$
3	42	21	$42 \bmod 21 = 0$	$(21, 0)$
Kết thúc	21	0	$B = 0 \Rightarrow$ Dừng	** $\text{gcd}(252, 105) = 21$ **

Ví dụ 2: Mô phỏng sàng eratosthenes tìm các số nguyên tố ≤ 20

1. Khởi tạo mảng đánh dấu `isPrime` từ 2 ... 20 đều là `true`.
2. Xét $i = 2$ (nguyên tố) $\Rightarrow$ Gạch bỏ các bội 4, 6, 8, 10, 12, 14, 16, 18, 20.
3. Xét $i = 3$ (nguyên tố) $\Rightarrow$ Gạch bỏ các bội 9, 12, 15, 18 (bắt đầu gạch từ $i^2 = 9$).
4. Xét $i = 4$ (đã bị gạch) $\Rightarrow$ Bỏ qua.
5. Vì $i^2 = 5^2 = 25 > 20$, vòng lặp dừng lại.

N	2	3	4	5	6	7	8	9	10	11	12
isPrime	true	true	false	true	false	true	false	false	false	true	false

****Danh sách số nguyên tố ≤ 20 :**** {2, 3, 5, 7, 11, 13, 17, 19} (gồm 8 số).

3. Lý thuyết cốt lõi & bất biến toán học (core invariants)

3.1. Mối quan hệ giữa gcd và lcm

$$\gcd(A, B) \times \text{lcm}(A, B) = A \times B \Rightarrow \text{lcm}(A, B) = \frac{A \times B}{\gcd(A, B)}$$

Cảnh báo quan trọng:

Bẫy Lỗi TRÀN SỐ KHI TÍNH BỘI CHUNG NHỎ NHẤT (LCM):

* Không viết $(A * B) / \gcd(A, B)$ vì tích $A \times B$ có thể lên tới 10^{36} gây tràn số `long long`. * Luôn viết: `long long lcm = (a / gcd(a, b)) * b;`

* **Lưu ý chuyên sâu:** Việc chia trước giúp triệt tiêu nguy cơ tràn số ở bước trung gian; tuy nhiên, nếu bản thân giá trị $\text{lcm}(A, B)$ thực tế vượt quá $9 \cdot 10^{18}$ (giới hạn của `long long`), ta bắt buộc phải sử dụng `__int128` hoặc kiểu dữ liệu số lớn (Big Integer).

3.2. Định lý cơ bản của số học & công thức nhân tính

Mọi số nguyên $N > 1$ đều phân tích duy nhất thành tích các thừa số nguyên tố:

$$N = p_1^{a_1} \times p_2^{a_2} \times \dots \times p_k^{a_k}$$

• ****Số lượng ước số của N ($\sigma_0(N)$):****

$$d(N) = (a_1 + 1)(a_2 + 1) \dots (a_k + 1)$$

- ****Tổng các ước số của N ($\sigma(N)$):****

$$\sigma(N) = \frac{p_1^{a_1+1} - 1}{p_1 - 1} \times \frac{p_2^{a_2+1} - 1}{p_2 - 1} \times \dots \times \frac{p_k^{a_k+1} - 1}{p_k - 1}$$

3.3. Sàng ước nguyên tố nhỏ nhất (spf - smallest prime factor)

Thay vì chỉ lưu mảng `bool`, ta lưu mảng `spf[x]` là ****ước số nguyên tố nhỏ nhất của x ****.

- Phân tích thừa số nguyên tố bằng SPF cần tối đa $O(\log X)$ lần chia liên tiếp, giúp trả lời cực nhanh cho hàng trăm nghìn truy vấn độc lập.

3.4. Phi hàm euler (euler's totient function $\phi(N)$)

Phi hàm Euler $\phi(N)$ đếm số lượng số nguyên dương trong đoạn $[1, N]$ nguyên tố cùng nhau với N ($\gcd(k, N) = 1$):

$$\phi(N) = N \times \prod_{p|N} \left(1 - \frac{1}{p}\right) = N \times \left(1 - \frac{1}{p_1}\right) \times \left(1 - \frac{1}{p_2}\right) \times \dots \times \left(1 - \frac{1}{p_k}\right)$$

- **Tính chất bất biến:** $\sum_{d|N} \phi(d) = N$.
- ****Sàng Phi hàm Euler trong $O(N \log \log N)$: **** Cho phép tính $\phi(1) \dots \phi(N)$ đồng thời trên mảng, dùng để đếm tổng số cặp số $(x, y) \leq N$ thỏa mãn $\gcd(x, y) = 1$ qua công thức $2 \sum_{i=1}^N \phi(i) - 1$.

4. Các bẫy lỗi lập trình kinh điển (bug traps)

1. Tràn số khi so sánh vòng lặp căn bậc hai:

- Viết `for (int i = 2; i * i ≤ n; ++i)` sẽ bị tràn số số nguyên 32-bit nếu $i \approx 46341 \Rightarrow i^2 < 0$ dẫn đến vòng lặp vô tận (TLE).
- **Cách sửa:** Dùng `1LL * i * i ≤ n` hoặc `i ≤ n / i`.

1. ****Quên xử lý phần dư cuối cùng sau khi phân tích $O(\sqrt{N})$: ****

- Sau khi chia triệt để cho các ước nguyên tố $p \leq \sqrt{N}$, nếu $N > 1$ thì giá trị còn lại của N ****chắc chắn là một số nguyên tố lớn hơn $\sqrt{N}$ ****. Nếu bỏ qua bước này sẽ thiếu thừa số cuối cùng.

1. ****Số 0 và số 1 không phải là số nguyên tố:****

- Hàm kiểm tra số nguyên tố bắt buộc phải kiểm tra `if (n < 2) return false;`.

5. Mẫu cài đặt chuẩn thi đấu (competitive templates)

Mẫu 1: Ước chung lớn nhất & bội chung nhỏ nhất

```
#include <bits/stdc++.h>
using namespace std;

// GCD bằng thuật toán Euclid lặp  $O(\log(\min(A, B)))$ 
long long getGcd(long long a, long long b) {
    while (b  $\neq$  0) {
        long long r = a % b;
        a = b;
        b = r;
    }
    return a;
}

// LCM an toàn chống tràn số
long long getLcm(long long a, long long b) {
    if (a == 0 || b == 0) return 0;
    return (a / getGcd(a, b)) * b;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    long long a, b;
    if (!(cin >> a >> b)) return 0;

    cout << getGcd(a, b) << " " << getLcm(a, b) << "\n";
    return 0;
}
```

Mẫu 2: Sàng eratosthenes & sàng spf (tối ưu phân tích thừa số)

```

#include <bits/stdc++.h>
using namespace std;

const int MAXN = 1000000;
vector<int> spf(MAXN + 1);

// Tiền xử lý Sàng SPF trong  $O(N \log \log N)$ 
void sieveSPF() {
    for (int i = 1; i ≤ MAXN; ++i) spf[i] = i;
    for (int i = 2; 1LL * i * i ≤ MAXN; ++i) {
        if (spf[i] == i) { // i là số nguyên tố
            for (int j = i * i; j ≤ MAXN; j += i) {
                if (spf[j] == j) {
                    spf[j] = i;
                }
            }
        }
    }
}

// Phân tích thừa số nguyên tố  $O(\log N)$  cho mỗi truy vấn
vector<pair<int, int>> factorize(int n) {
    vector<pair<int, int>> factors;

    while (n > 1) {
        int p = spf[n];
        int count = 0;
        while (n % p == 0) {
            count++;
            n /= p;
        }
        factors.push_back({p, count});
    }
    return factors;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    sieveSPF();

    int q;
    if (!(cin >> q)) return 0;

    while (q--) {
        int n;
        cin >> n;

        auto factors = factorize(n);
    }
}

```

```

        for (int i = 0; i < (int)factors.size(); ++i) {
            cout << factors[i].first << "^" << factors[i].second << (i + 1 ==
(int)factors.size() ? "" : " * ");
        }
        cout << "\n";
    }
    return 0;
}

```

Câu hỏi trắc nghiệm củng cố khái niệm

Câu 1 (độ phức tạp):

Thuật toán Euclid tìm $\gcd(A, B)$ có độ phức tạp thời gian trong trường hợp xấu nhất là bao nhiêu?

- A. $O(\min(A, B))$
- B. **[Đáp án đúng]** $O(\log(\min(A, B)))$
- C. $O(\sqrt{\min(A, B)})$
- D. $O(1)$

Giải thích: Sau mỗi hai bước lặp của phép lấy dư Euclid, số nhỏ hơn sẽ giảm ít nhất một nửa, do đó số bước lặp tối đa không vượt quá $2 \log_2(\min(A, B))$.

Câu 2 (bản chất toán học):

Trường hợp xấu nhất khiến thuật toán Euclid phải thực hiện số bước lặp nhiều nhất xảy ra khi A và B là hai số nào sau đây?

- A. Hai lũy thừa của 2: $A = 2^x, B = 2^y$.
- B. **[Đáp án đúng]** Hai số Fibonacci liên tiếp: $A = F_{k+1}, B = F_k$.
- C. Hai số nguyên tố rất lớn: $A = p, B = q$.
- D. Một số chẵn và một số lẻ.

Giải thích: Định lý Lamé chứng minh rằng hai số Fibonacci liên tiếp luôn tạo ra các thương số bằng 1 ở mọi bước lặp, khiến phép chia lấy dư thu hẹp chậm nhất.

Câu 3 (cú pháp & bẫy lỗi):

Trong template C++ chuẩn thi đấu, công thức nào sau đây được sử dụng để tính Bội chung nhỏ nhất `lcm(A, B)` nhằm triệt tiêu nguy cơ tràn số ở bước nhân trung gian?

- A. `(a * b) / getGcd(a, b)`
- B. **[Đáp án đúng]** `(a / getGcd(a, b)) * b`
- C. `a * b * getGcd(a, b)`
- D. `(a + b) / getGcd(a, b)`

Giải thích: Vì A luôn chia hết cho $\text{gcd}(A, B)$, ta thực hiện phép chia trước $(a / \text{gcd}(a, b))$ để thu nhỏ giá trị trung gian trước khi nhân với B , giúp chống tràn số 64-bit hiệu quả.

Câu 4 (thuật toán kiểm tra số nguyên tố):

Tại sao để kiểm tra số N có phải là số nguyên tố hay không, ta chỉ cần kiểm tra các ước nguyên từ 2 đến `floor(sqrt(N))` ?

- A. Vì các số lớn hơn `sqrt(N)` luôn là số lẻ.
- B. **[Đáp án đúng]** Vì nếu $N = a * b$, không thể xảy ra trường hợp cả a và b đều đồng thời lớn hơn `sqrt(N)`.
- C. Vì hàm `sqrt(N)` trong C++ chạy trong $O(1)$.
- D. Vì số lượng ước của N không bao giờ vượt quá `sqrt(N)`.

*Giải thích: Nếu $a > \text{sqrt}(N)$ và $b > \text{sqrt}(N)$ thì $a * b > N$ (vô lý). Do đó, nếu N là hợp số, ước nhỏ hơn bắt buộc phải nằm trong khoảng $[2, \text{sqrt}(N)]$.*

Câu 5 (ứng dụng sàng nguyên tố):

Độ phức tạp thời gian chuẩn của thuật toán Sàng Eratosthenes để tìm tất cả các số nguyên tố $\leq N$ là:

- A. $O(N \sqrt{N})$
- B. $O(N \log N)$
- C. **[Đáp án đúng]** $O(N \log \log N)$
- D. $O(N)$

Giải thích: Tổng số thao tác gạch bỏ bằng $\sum_{p \leq N} (1/p)$. Theo định lý Mertens, chuỗi nghịch đảo các số nguyên tố có tổng tiệm cận $\ln(\ln N)$, do đó độ phức tạp là $O(N \log \log N)$, gần như tuyến tính tuyệt đối.

Câu 6 (sàng spf):

Trong kỹ thuật Sàng Ước Nguyên Tố Nhỏ Nhất (SPF), mảng `spf[x]` lưu thông tin gì?

- A. Số lượng ước nguyên tố của `x`.
- B. Tổng các chữ số của `x`.
- C. **[Đáp án đúng]** Ước số nguyên tố nhỏ nhất của số nguyên `x`.
- D. Số nguyên tố lớn nhất nhỏ hơn hoặc bằng `x`.

Giải thích: `spf[x]` lưu Smallest Prime Factor của `x`, giúp phân tích thừa số nguyên tố của `x` trong $O(\log x)$ bước chia liên tiếp.

Câu 7 (đếm số lượng ước):

Một số nguyên `N` có dạng phân tích thừa số nguyên tố $N = p_1^3 \cdot p_2^4 \cdot p_3^1$ (với `p1`, `p2`, `p3` là các số nguyên tố phân biệt). Số `N` có tất cả bao nhiêu ước số nguyên dương?

- A. $3 * 4 * 1 = 12$
- B. $3 + 4 + 1 = 8$
- C. **[Đáp án đúng]** $(3+1) * (4+1) * (1+1) = 4 * 5 * 2 = 40$ ước
- D. $40 - 1 = 39$ ước

Giải thích: Theo công thức nhân tính, số lượng ước số của $N = \prod p_i^{a_i}$ là $\prod (a_i + 1)$.

Câu 8 (đặc điểm số chính phương):

Một số nguyên dương `N` là số chính phương ($N = k^2$) khi và chỉ khi điều kiện nào sau đây được thỏa mãn?

- A. `N` có số lượng thừa số nguyên tố phân biệt là một số chẵn.
- B. Tổng các chữ số của `N` chia hết cho 9.
- C. **[Đáp án đúng]** Số lượng ước số nguyên dương của `N` là một số lẻ (tương đương số mũ của mọi thừa số nguyên tố đều là số chẵn).
- D. `N` có chữ số tận cùng thuộc tập `2, 3, 7, 8`.

Giải thích: Các ước số luôn đi thành từng cặp đối xứng $(d, N/d)$. Chỉ khi $N = k^2$ thì cặp ước tại $k = \sqrt{N}$ mới trùng nhau, tạo ra số lượng ước số lẻ. Về mặt thừa số nguyên tố, $N = \prod p_i^{a_i}$ nên số lượng ước $(2a_1 + 1)(2a_2 + 1) \dots$ luôn là tích các số lẻ (kết quả là số lẻ).

Câu 9 (công thức Legendre):

Công thức Legendre $E_p(N!) = \sum_{k=1}^{\infty} \lfloor N/p^k \rfloor$ dùng để tính đại lượng nào?

- A. Số lượng số nguyên tố nhỏ hơn N .
- B. **[Đáp án đúng]** Số mũ của thừa số nguyên tố p trong phân tích thừa số nguyên tố của $N!$.
- C. Ước chung lớn nhất của $N!$ và p .
- D. Số chữ số của $N!$.

Giải thích: Công thức Legendre đếm số lượng bội của $p, p^2, p^3, \dots$ đóng góp vào tích $N! = 1 \times 2 \times \dots \times N$.

Câu 10 (chữ số 0 tận cùng):

Số lượng chữ số 0 liên tiếp tận cùng của $100!$ là bao nhiêu?

- A. 10
- B. 20
- C. **[Đáp án đúng]** $\lfloor 100/5 \rfloor + \lfloor 100/25 \rfloor = 20 + 4 = 24$
- D. 25

Giải thích: Mỗi chữ số 0 tận cùng được tạo bởi tích 2×5 . Trong $N!$, số lượng thừa số 2 luôn nhiều hơn số lượng thừa số 5, do đó số chữ số 0 bằng số mũ của 5 trong $100!$.

Câu 11 (sàng phân đoạn - segmented sieve):

Kỹ thuật Sàng phân đoạn (Segmented Sieve) được sử dụng tối ưu nhất trong tình huống nào?

- A. Khi cần tìm số nguyên tố trong khoảng $[1, 10^7]$.
- B. **[Đáp án đúng]** Khi cần tìm số nguyên tố trong đoạn $[L, R]$ với $R \leq 10^{12}$ nhưng độ dài đoạn $R - L \leq 10^6$.
- C. Khi L và R đều là số chẵn.
- D. Khi bộ nhớ RAM máy tính có dung lượng trên 16GB.

Giải thích: Ta không thể tạo mảng kích thước 10^{12} , nhưng có thể sàng trên mảng kích thước $R - L + 1 \leq 10^6$ bằng cách chỉ dùng các số nguyên tố $\leq \sqrt{R} \leq 10^6$.

Câu 12 (số nguyên tố cùng nhau):

Hai số nguyên dương A và B được gọi là nguyên tố cùng nhau (Coprime) khi và chỉ khi:

- A. Cả A và B đều là số nguyên tố.
- B. $A + B$ là số nguyên tố.
- C. [Đáp án đúng] $\gcd(A, B) = 1$.
- D. $\text{lcm}(A, B) = A \times B + 1$.

Giải thích: Hai số nguyên tố cùng nhau là hai số không có ước chung nào khác ngoài 1, tức $\gcd(A, B) = 1$.

Ma trận bài tập thực hành (p0 → p5)

STT	Mã Bài	Tên Bài Toán	Cấp Độ	Dạng Thuật Toán & Kỹ Năng Cốt Lõi
01	CPPB-NT-01	Ước Chung Lớn Nhất & Bội Chung Nhỏ Nhất	P0	Thuật toán Euclid tối ưu và chia trước nhân sau
02	CPPB-NT-02	Kiểm Tra Số Nguyên Tố Cơ Bản	P0	Kiểm tra nguyên tố trong $O(\sqrt{N})$ bước nhảy 6k ± 1
03	CPPB-NT-03	Phân Tích Thừa Số Nguyên Tố	P1	Phân tích $N = \prod p_i a_i$ trong $O(\sqrt{N})$
04	CPPB-NT-04	Đếm Số Lượng & Tính Tổng Các Ước	P1	Ứng dụng công thức nhân tính $\sigma(N)$ và $\sigma_1(N)$
05	CPPB-NT-05	Số Chính Phương & Số Lập Phương	P1	Kiểm tra số có số lượng ước lẻ bằng căn bậc hai nguyên
06	CPPB-NT-06	Sàng Nguyên Tố Eratosthenes Kinh Điển	P2	Cài đặt sàng nguyên tố với mảng <code>vector<bool></code> $N \leq 10^7$
07	CPPB-NT-07	Đếm Số Nguyên Tố Trong Đoạn [L, R]	P2	Sàng Eratosthenes kết hợp Mảng Tiền Tố $O(1)$ mỗi truy vấn
08	CPPB-NT-08	Sàng Ước Nguyên Tố Nhỏ Nhất (SPF)	P2	Phân tích thừa số nguyên tố $O(\log N)$ cho 10^5 truy vấn
09	CPPB-NT-09	Sàng Phân Đoạn (Segmented Sieve)	P3	Sàng trên khoảng $[L, R]$ với $R \leq 10^{12}$, $R - L \leq 10^6$
10	CPPB-NT-10	Cặp Số Nguyên Tố Sinh Đôi (Twin Primes)	P3	Tìm cặp $(p, p+2)$ bằng Sàng Eratosthenes
11	CPPB-NT-11	Số Hoàn Hảo & Định Lý Euclid-Euler	P3	Kiểm tra số hoàn hảo dạng $2^{p-1}(2^p - 1)$
12	CPPB-NT-12	Số Có Đúng 3 Ước Số	P3	Nhận diện số có dạng p^2 với p là số nguyên tố
13	CPPB-NT-13	Số Gần Nguyên Tố (Almost Prime)	P4	Sàng đếm số lượng ước nguyên tố phân biệt của mọi số $\leq N$
14	CPPB-NT-14	Phân Tích Giai Thừa Ra Thừa Số (Legendre)	P4	Công thức Legendre $E_p(N!) = \sum \lfloor \frac{N}{p^k} \rfloor$
15	CPPB-NT-15	Đếm Số Số Không Tận Cùng Của N!	P4	Đếm số mũ của 5 trong phân tích $N!$

STT	Mã Bài	Tên Bài Toán	Cấp Độ	Dạng Thuật Toán & Kỹ Năng Cốt Lõi
16	CPPB-NT-16	Số Học Cực Hạn: Cặp Nguyên Tố Cùng Nhau Cực Đại	P5	Ứng dụng Hàm Phi Euler $\phi(N)$ và sàng nguyên tố đa năng

<!-- Chuyên đề 08: Đồng Dư, Lũy Thừa & Nghịch Đảo | 382 dòng | 19,663 bytes -->

Chuyên đề 08: Đồng dư thức, lũy thừa nhị phân & nghịch đảo modulo

1. Bản chất vấn đề & trực giác thuật toán (the core problem & intuition)

Trong nhiều bài toán lập trình và thi đấu thuật toán, kết quả tính toán hoặc số cách đếm tổ hợp thường tăng rất nhanh và vượt quá giới hạn lưu trữ của kiểu số nguyên 64-bit (`long long`). Để tránh việc phải xử lý số lớn phức tạp, đề bài thường yêu cầu: ***In ra kết quả sau khi chia lấy dư cho M *** (thông thường $M = 10^9 + 7$ hoặc 998244353 — là các số nguyên tố lớn).

Từ yêu cầu thực tế này, bộ ba kỹ thuật nền tảng được hình thành: *Đồng Dư Cơ Bản* (+, -, *) $\rightarrow$ *Lũy Thừa Nhị Phân* $O(\log B)$ $\rightarrow$ *Nghịch Đảo Modulo* (B^{-1})

Vấn đề 1: Phép tính lũy thừa $A^B \bmod M$

- **Cách ngây thơ:** Nhân B lần liên tiếp: $A \times A \times \dots \times A \Rightarrow O(B)$. Khi $B = 10^{18}$, cách này hoàn toàn bất khả thi.
- **Trực giác Chia để trị (Binary Exponentiation):**
- Nếu B chẵn: $A^B = (A^2)^{B/2} = (A^{B/2})^2$.
- Nếu B lẻ: $A^B = A \times A^{B-1}$.
- Sau mỗi bước, số mũ B giảm đi một nửa $\Rightarrow$ Số phép nhân chỉ còn $O(\log_2 B)$ (chưa tới 60 phép tính với $B = 10^{18}$).

Vấn đề 2: Phép chia trên vành modulo $\frac{A}{B} \bmod M$ (right)

- Trong số học đồng dư, **không thể thực hiện phép chia bằng phép chia số nguyên thông thường** (tức $\frac{A}{B} \bmod M \neq \frac{A \bmod M}{B \bmod M}$).
- ****Nghịch đảo Modulo (B^{-1}):**** Muốn tính $\frac{A}{B} \bmod M$, ta chuyển phép chia thành phép nhân với nghịch đảo modulo B^{-1} (nếu nghịch đảo tồn tại):

$$\frac{A}{B} \bmod M \equiv (A \times B^{-1}) \bmod M$$

với B^{-1} là số nguyên thỏa mãn: $(B \times B^{-1}) \equiv 1 \bmod M$.

2. Mô phỏng từng bước (visual step-by-step simulation)

Ví dụ 1: Mô phỏng tính $3^{13} \bmod 1000$ bằng lũy thừa nhị phân

Biểu diễn nhị phân của số mũ $13 = 1101_2 = 8 + 4 + 1$. Do đó: $3^{13} = 3^8 \times 3^4 \times 3^1$.

Bước lặp	Số mũ B	Trạng thái (B chẵn hay lẻ)	Cơ số A ($A \geq 3$, $A \pmod M$)	Kết quả tích lũy ans ($ans \geq 3$, $ans \times A \pmod M$)
Khởi tạo	13	Lẻ (bit $0 = 1$)	$A = 3$	$ans = 1 \times 3 = 3$
1	6	Chẵn (bit $1 = 0$)	$A \geq 3^2 = 9$	$ans = 3$ (không nhân)
2	3	Lẻ (bit $2 = 1$)	$A \geq 3^2 = 9$	$ans \geq 3 (3 \times 81) = 243$
3	1	Lẻ (bit $3 = 1$)	$A \geq 3^2 = 9$ $81^2 = 6561 \text{ equiv } 561$	$ans \geq 3 (243 \times 561) \pmod{1000} = 323$
Kết thúc	0	Dừng	—	Đáp án: $3^{13} \pmod{1000} = 323$ (vì $3^{13} = 1594323$)

Ví dụ 2: Mô phỏng tìm nghịch đảo modulo của $3 \pmod 7$

Ta cần tìm số nguyên $X \in \{1, \dots, 6\}$ sao cho $(3 \times X) \pmod 7 = 1$.

Thử giá trị X	Phép nhân $3 \times X$	Lấy dư $(3 \times X) \pmod 7$	Kết luận
$X = 1$	$3 \times 1 = 3$	3	Không thỏa mãn
$X = 2$	$3 \times 2 = 6$	6	Không thỏa mãn
$X = 3$	$3 \times 3 = 9$	2	Không thỏa mãn
$X = 4$	$3 \times 4 = 12$	5	Không thỏa mãn
** $X = 5$ **	$3 \times 5 = 15$	** 1 **	** $3^{-1} \text{ equiv } 5 \pmod 7$ (Thỏa mãn) **

Kiểm chứng bằng Định lý Fermat nhỏ: $3^{7-2} = 3^5 = 243 \text{ equiv } 5 \pmod 7$.

3. Lý thuyết cốt lõi & bất biến toán học (core invariants)

3.1. Các quy tắc đồng dư cơ bản (+, -, *)

- Phép Cộng:** $(A + B) \pmod M = ((A \pmod M) + (B \pmod M)) \pmod M$.
- Phép Trừ (Tránh số âm):** $(A - B) \pmod M = ((A \pmod M) - (B \pmod M) + M) \pmod M$.
- Phép Nhân:** $(A \times B) \pmod M = ((A \pmod M) \times (B \pmod M)) \pmod M$.

Cảnh báo quan trọng:

2 Bẫy Lỗi KHI THỰC HIỆN PHÉP TOÁN ĐỒNG DƯ:

1. Số dư âm trong C++: Trong C++, phép toán $-7 \% 5$ trả về -2 (không phải 3). Để luôn nhận kết quả không âm, bắt buộc phải viết: $(a \% m + m) \% m$. **2. Tràn số 32-bit khi nhân:** Nếu $A, B \approx 10^9$, tích $A \times B \approx 10^{18}$ vượt giới hạn kiểu `int`. Bắt buộc phải ép kiểu 64-bit trước khi nhân: $(1LL * a * b) \% m$.

3.2. Định lý fermat nhỏ & nghịch đảo modulo

Nếu M là một **số nguyên tố** và A không chia hết cho M ($\gcd(A, M) = 1$), thì:

$$AM^{-1} \equiv 1 \pmod{M} \Rightarrow A \times AM^{-2} \equiv 1 \pmod{M}$$

$$\Rightarrow A^{-1} \equiv A^{M-2} \pmod{M}$$

Ta có thể tính $A^{-1} \pmod{M}$ chỉ bằng một hàm Lũy thừa nhị phân: `power(A, M - 2, M)` trong $O(\log M)$.

Chú ý:

ĐIỀU KIỆN TIÊN QUYẾT CỦA ĐỊNH LÝ FERMAT NHỎ:

** Quy tắc $AM^{-1} \equiv 1 \pmod{M}$ và việc rút gọn số mũ $B \geq M$ $B \pmod{M-1}$ **CHỈ ĐÚNG KHI M LÀ SỐ NGUYÊN TỐ VÀ $\gcd(A, M) = 1$ *. * Tuyệt đối không tùy tiện áp dụng nếu A chia hết cho M hoặc M là hợp số.*

3.3. Thuật toán euclid mở rộng (extended euclidean algorithm)

Khi M không phải là **số nguyên tố** (nhưng $\gcd(A, M) = 1$), định lý Fermat nhỏ không áp dụng được. Ta dùng thuật toán Euclid mở rộng để giải phương trình nghiệm nguyên:

$$A \times x + M \times y = \gcd(A, M) = 1$$

Khi đó, $x \pmod{M}$ chính là nghịch đảo modulo A^{-1} .

3.4. Tính tổ hợp $C(N, K) \pmod{M}$ trong $O(1)$ mỗi truy vấn

Công thức số tổ hợp chập K của N :

$$C(N, K) = \frac{N!}{K! \times (N - K)!} \equiv N! \times (K!)^{-1} \times ((N - K)!)^{-1} \pmod{M}$$

- **Tiền xử lý trong $O(N)$:**
1. Tính mảng giai thừa: `fact[i] = (fact[i-1] * i) % M`.
 2. Tính mảng nghịch đảo giai thừa: `invFact[N] = power(fact[N], M - 2, M)`, sau đó đi ngược về 0: `invFact[i - 1] = (invFact[i] * i) % M`.
- **Trả lời mỗi truy vấn trong $O(1)$:**

$$C(N, K) = \text{fact}[N] \times \text{invFact}[K] \pmod{M} \times \text{invFact}[N - K] \pmod{M}$$

3.5. Tính tổng cấp số nhân modulo bằng chia để trị

Cần tính tổng:

$$S_N = 1 + A + A^2 + \dots + A^N \pmod{M}$$

- **Trường hợp cơ sở:** Nếu $N = 0 \Rightarrow S_0 = 1$.
- **Nếu N lẻ (Tổng có $N + 1$ số hạng chẵn):**

$$S_N = (1 + A + \dots + A^{(N-1)/2}) + A^{(N+1)/2} (1 + A + \dots + A^{(N-1)/2})$$

$$S_N = S_{(N-1)/2} \times (1 + A^{(N+1)/2}) \pmod{M}$$

- **Nếu N chẵn:** Tách riêng số hạng cuối cùng:

$$S_N = 1 + A \times S_{N-1} \pmod{M}$$

- **Độ phức tạp:** $O(\log^2 N)$ hoặc $O(\log N)$, mở đường cho kỹ thuật nhân lũy thừa ma trận và quy hoạch động cấu trúc đại số.

4. Các bẫy lỗi lập trình kinh điển (bug traps)

1. Chia trực tiếp trên Modulo:

- Viết $((A \% M) / (B \% M)) \% M$ là **HOÀN TOÀN SAI BẢN CHẤT TOÁN HỌC**. Phép chia bắt buộc phải chuyển thành nhân với nghịch đảo: $(A * \text{inverse}(B)) \% M$.

1. **Quên xử lý trường hợp $K > N$ hoặc $K < 0$ khi tính tổ hợp:**

- $C(N, K) = 0$ khi $K < 0$ hoặc $K > N$. Nếu không kiểm tra sẽ bị truy cập ô nhớ âm hoặc rác.

1. **Trường hợp $M = 1$:**

- $A^B \bmod 1$ luôn bằng 0 . Hàm lũy thừa cần trả về 0 khi $M = 1$.

5. Mẫu cài đặt chuẩn thi đấu (competitive templates)

Mẫu 1: Lũy thừa nhị phân & nghịch đảo modulo chuẩn

```
#include <bits/stdc++.h>
using namespace std;

// Tính  $(a^b) \% m$  trong  $O(\log b)$ 
long long powerMod(long long a, long long b, long long m) {
    if (m == 1) return 0;
    long long ans = 1 % m;
    a %= m;
    while (b > 0) {
        if (b & 1) ans = (ans * a) % m;
        a = (a * a) % m;
        b >>= 1;
    }
    return ans;
}

// Nghịch đảo Modulo bằng Định lý Fermat nhỏ (khi m là số nguyên tố)
long long modInversePrime(long long a, long long m) {
    return powerMod(a, m - 2, m);
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    long long a, b, m;
    if (!(cin >> a >> b >> m)) return 0;

    cout << powerMod(a, b, m) << "\n";
    return 0;
}
```

Mẫu 2: Tiền xử lý tổ hợp $C(N, K) \bmod M$ trong $O(1)$ mỗi truy vấn

```

#include <bits/stdc++.h>
using namespace std;

const int MAXN = 1000000;
const long long MOD = 1000000007;

vector<long long> fact(MAXN + 1);

vector<long long> invFact(MAXN + 1);

long long powerMod(long long a, long long b, long long m) {
    long long ans = 1;
    a %= m;
    while (b > 0) {
        if (b & 1) ans = (ans * a) % m;
        a = (a * a) % m;
        b >>= 1;
    }
    return ans;
}

void precomputeCombinatorics() {
    fact[0] = 1;
    for (int i = 1; i ≤ MAXN; ++i) {
        fact[i] = (fact[i - 1] * i) % MOD;
    }
    invFact[MAXN] = powerMod(fact[MAXN], MOD - 2, MOD);
    for (int i = MAXN; i ≥ 1; --i) {
        invFact[i - 1] = (invFact[i] * i) % MOD;
    }
}

long long nCr(int n, int r) {
    if (r < 0 || r > n) return 0;

    return fact[n] * invFact[r] % MOD * invFact[n - r] % MOD;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    precomputeCombinatorics();

    int q;
    if (!(cin >> q)) return 0;

    while (q--) {
        int n, r;
        cin >> n >> r;
    }
}

```

```

        cout << nCr(n, r) << "\n";
    }
    return 0;
}

```

Câu hỏi trắc nghiệm củng cố khái niệm

Câu 1 (độ phức tạp):

Thuật toán Lũy thừa nhị phân (Binary Exponentiation) tính $A^B \bmod M$ có độ phức tạp thời gian là:

- A. $O(B)$
- B. $O(\sqrt{B})$
- C. [Đáp án đúng] $O(\log_2 B)$
- D. $O(1)$

Giải thích: Sau mỗi vòng lặp, số mũ B giảm đi một nửa ($B \geq 1 \Rightarrow B \leftarrow \lfloor B/2 \rfloor$). Do đó số lần lặp tối đa là $\lfloor \log_2 B \rfloor + 1$.

Câu 2 (xử lý số âm):

Trong C++, biểu thức $(-8) \bmod 5$ trả về kết quả là -3 . Cách viết chuẩn mực nào để luôn nhận được số dư không âm trong khoảng $[0, M-1]$?

- A. $\text{abs}((-8) \% 5)$
- B. [Đáp án đúng] $((-8) \% 5 + 5) \% 5$
- C. $(-8) \% 5 + 5$
- D. $5 - ((-8) \% 5)$

Giải thích: Cộng thêm M rồi lấy dư lại lần nữa đảm bảo nếu số dư ban đầu là âm (thuộc $(-M, 0)$), nó sẽ được đưa về miền dương $[0, M-1]$, còn nếu ban đầu đã dương thì không đổi.

Câu 3 (định lý fermat nhỏ):

Định lý Fermat nhỏ phát biểu rằng: Nếu M là số nguyên tố và $\gcd(A, M) = 1$, thì $AM^{-1} \equiv 1 \pmod M$. Từ đó suy ra nghịch đảo modulo $A^{-1} \pmod M$ bằng biểu thức nào?

- A. $A^M \bmod M$
- B. $AM^{+1} \bmod M$
- C. [Đáp án đúng] $AM^{-2} \bmod M$
- D. $AM^{-1} - 1 \bmod M$

Giải thích: Nhân cả 2 vế của $AM^{-1} \equiv 1 \pmod{M}$ với A^{-1} , ta được $A^{-1} \equiv AM^{-2} \pmod{M}$.

Câu 4 (phép chia modulo):

Khi cần tính giá trị biểu thức $\frac{A}{B} \pmod{M}$ với $M = 10^9 + 7$ (số nguyên tố) và $B \not\equiv 0 \pmod{M}$, ta thực hiện phép toán nào sau đây?

- A. $(A / B) \% M$
- B. $(A \% M) / (B \% M)$
- C. **[Đáp án đúng]** $(A \% M) * \text{powerMod}(B, M - 2, M) \% M$
- D. $(A \% M) * \text{powerMod}(B, M - 1, M) \% M$

Giải thích: Phép chia trên vành modulo bắt buộc phải nhân với nghịch đảo của mẫu số: $A \cdot B^{-1} \pmod{M}$.

Câu 5 (điều kiện tồn tại nghịch đảo):

Nghịch đảo modulo của số nguyên A theo modulo M (tức số X sao cho $A \cdot X \equiv 1 \pmod{M}$) chắc chắn tồn tại khi và chỉ khi:

- A. A và M đều là số lẻ.
- B. $A < M$.
- C. **[Đáp án đúng]** $\text{gcd}(A, M) = 1$ (A và M nguyên tố cùng nhau).
- D. M phải là số chẵn.

Giải thích: Theo định lý Bézout, phương trình $Ax + My = 1$ chỉ có nghiệm nguyên khi và chỉ khi $\text{gcd}(A, M) = 1$.

Câu 6 (tổ hợp modulo $O(1)$):

Để trả lời 10^5 truy vấn tính số tổ hợp $\text{binom}\{N\}\{K\} \pmod{10^9 + 7}$ với $N, K \leq 10^6$ trong tổng thời gian dưới **0.1s**, phương pháp tối ưu nhất là gì?

- A. Tính trực tiếp $C(N, K)$ bằng tam giác Pascal tại mỗi truy vấn.
- B. Tính $N!$, $K!$, $(N-K)!$ từ đầu tại mỗi truy vấn.
- C. **[Đáp án đúng]** Tiền xử lý mảng Giai thừa `fact[]` và Nghịch đảo giai thừa `invFact[]` trong $O(N)$, sau đó trả lời mỗi truy vấn trong $O(1)$.
- D. Dùng đệ quy quay lui có nhớ.

Giải thích: Tiền xử lý $O(N)$ cho phép tính $C(N, K) = fact[N] \cdot invFact[K] \cdot invFact[N-K] \bmod M$ trong đúng $O(1)$ phép nhân.

Câu 7 (tối ưu tính nghịch đảo giai thừa):

Thay vì gọi hàm lũy thừa N lần để tính $invFact[i]$, ta có thể tính toàn bộ mảng $invFact$ từ $1 \dots N$ chỉ với **1 lần gọi hàm lũy thừa duy nhất** bằng công thức quy nạp lùi nào?

- A. $invFact[i - 1] = invFact[i] / i$
- B. **[Đáp án đúng]** $invFact[i - 1] = (invFact[i] * i) \% MOD$
- C. $invFact[i - 1] = (invFact[i] * (MOD - i)) \% MOD$
- D. $invFact[i] = invFact[i - 1] * (i + 1)$

*Giải thích: Vì $\frac{1}{(i-1)!} = \frac{1}{i!} \cdot i$, do đó $invFact[i - 1] = (invFact[i] * i) \% MOD$. Ta chỉ cần tính $invFact[N] = power(fact[N], MOD - 2)$ rồi đi lùi về 0.*

Câu 8 (rút gọn số mũ lớn):

Theo định lý Fermat nhỏ, với $M = 10^9 + 7$ (số nguyên tố) và $\gcd(A, M) = 1$, nếu số mũ B là một số khổng lồ gồm hàng chục nghìn chữ số, ta có thể rút gọn số mũ B trước khi tính lũy thừa bằng cách nào?

- A. $B \geq B \bmod M$
- B. **[Đáp án đúng]** $B \geq B \bmod (M - 1)$
- C. $B \geq B \bmod (M + 1)$
- D. $B \geq B \bmod \sqrt{M}$

Giải thích: Vì M là số nguyên tố và $\gcd(A, M) = 1$, theo Fermat nhỏ $A^{M-1} \equiv 1 \pmod M$. Do đó $A^B = A^q \cdot (A^{M-1})^r = (A^{M-1})^q \cdot A^r \equiv 1^q \cdot A^r \equiv A^r \pmod M$ với $r = B \bmod (M - 1)$.

Câu 9 (nhân an toàn chống tràn số 64-bit):

Khi nào phép nhân trực tiếp $(a * b) \% m$ có nguy cơ gây tràn số và bắt buộc phải áp dụng kỹ thuật nhân modulo an toàn (như Nhân Ẩn Độ $O(\log B)$ hoặc kiểu dữ liệu `__int128`)?

- A. Khi $A, B \leq 10^9$ và $M = 10^9 + 7$.
- B. **[Đáp án đúng]** Khi $A, B \leq 10^{18}$ và $M \leq 10^{18}$ (tích $A \times B$ có thể lên tới 10^{36} , vượt quá giới hạn 64-bit của `unsigned long long`).
- C. Khi M là số chẵn.
- D. Khi B là số âm.

Giải thích: Khi $A, B \approx 10^{18}$, tích $A \times B \approx 10^{36}$ vượt xa ngưỡng $2^{64}-1 \approx 1.8 \times 10^{19}$. Ta cần phân rã phép nhân thành các phép cộng có lấy dư (Nhân Ẩn Độ) hoặc dùng kiểu số nguyên 128-bit.

Câu 10 (phương trình diophantine & euclid mở rộng):

Thuật toán Euclid mở rộng tìm cặp nghiệm nguyên (x, y) cho phương trình $Ax + My = \gcd(A, M)$. Nếu $\gcd(A, M) = 1$, giá trị $x \bmod M$ đại diện cho đại lượng nào?

- A. Ước chung lớn nhất của A và M .
- B. Phần dư của A chia cho M .
- C. **[Đáp án đúng]** Nghịch đảo modulo của A theo modulo M ($A^{-1} \bmod M$).
- D. Bội chung nhỏ nhất của A và M .

Giải thích: Phương trình $Ax + My = 1 \Leftrightarrow Ax \equiv 1 \pmod M$, nghĩa là x chính là nghịch đảo modulo của A .

Ma trận bài tập thực hành (p0 → p5)

Ghi chú:

Phân tầng lộ trình học tập:

** **Nhóm Cốt Lõi (Core Foundations - Bắt buộc CPPB-MOD-01 → 09):** Nắm vững các phép toán đồng dư, lũy thừa nhị phân, nghịch đảo Fermat/Euclid và tổ hợp $C(N, K)$. **
***Nhóm Thử Thách Mở Rộng (Advanced / Challenge CPPB-MOD-10 → 16):** Dành cho học sinh giỏi nâng cao tiếp cận các mô hình toán học chuyên sâu.*

STT	Mã Bài	Tên Bài Toán	Cấp Độ	Phân Loại	Dạng Thuật Toán & Kỹ Năng Cốt Lõi
01	CPPB-MOD-01	**Phép Tính Đồng Dư Cơ Bản (+, -, *)**	P0	Core	Quy tắc cộng trừ nhân đồng dư và xử lý số dư âm
02	CPPB-MOD-02	**Lũy Thừa Nhị Phân Cơ Bản ($A^B \bmod M$)**	P0	Core	Thuật toán Lũy thừa nhị phân lặp $O(\log B)$
03	CPPB-MOD-03	**Lũy Thừa Chuỗi Số Lớn ($A^B \bmod M$)**	P1	Core	Định lý Fermat nhỏ và rút gọn số mũ $B \bmod \{M - 1\}$
04	CPPB-MOD-04	**Nhân Ấn Độ Chống Tràn Số ($A \times B \bmod M$)**	P1	Core	Nhân nhân đôi nhị phân $O(\log B)$ hoặc <code>__int128</code>
05	CPPB-MOD-05	Tính Tổng Cấp Số Nhân Đồng Dư	P2	Core	Chia để trị tính $S = 1 + A + \dots + A^N \bmod M$
06	CPPB-MOD-06	Nghịch Đảo Modulo Bằng Fermat Nhỏ	P2	Core	Tính $A^{-1} \equiv A^{M-2} \bmod M$ với M nguyên tố
07	CPPB-MOD-07	Nghịch Đảo Modulo Bằng Euclid Mở Rộng	P2	Core	Giải phương trình $Ax + My = 1$ khi $\gcd(A, M) = 1$
08	CPPB-MOD-08	**Phép Chia Đồng Dư $\frac{A}{B} \bmod M$ **	P2	Core	Thực hiện phép nhân với nghịch đảo modulo $A \times B^{-1}$
09	CPPB-MOD-09	**Tính Số Tổ Hợp $C(N, K) \bmod M$ **	P3	Core	Tiền xử lý Giai thừa và Nghịch đảo trong $O(N)$
10	CPPB-MOD-10	**Tính Số Chỉnh Hợp $A(N, K) \bmod M$ **	P3	Advanced	Tính $A(N, K) = N! \times ((N-K)!)^{-1} \bmod M$
11	CPPB-MOD-11	Dãy Fibonacci Đồng Dư Lớn	P3	Advanced	Nhân ma trận nhị phân $O(\log N)$ tính $F_N \bmod M$
12	CPPB-MOD-12	**Số Catalan Đồng Dư $C_N \bmod M$ **	P3	Advanced	Công thức $C_N = \frac{1}{N+1} C(2N, N) \bmod M$
13	CPPB-MOD-13	Lũy Thừa Tầng (Tower of Powers)	P4	Advanced	Tính $A^{B^C} \bmod M$ bằng định lý Euler / Fermat nhỏ
14	CPPB-MOD-14	**Nghịch Đảo Tuyến Tính $1 \dots N$ Trong $O(N)$ **	P4	Advanced	Công thức hồi quy tính nghịch đảo toàn bộ mảng
15			P4	Advanced	

STT	Mã Bài	Tên Bài Toán	Cấp Độ	Phân Loại	Dạng Thuật Toán & Kỹ Năng Cốt Lõi
	CPPB-MOD-15	**Giải Phương Trình Đồng Dư Tuyến Tính $Ax \equiv B \pmod M$ **			Thuật toán Euclid mở rộng tổng quát
16	CPPB-MOD-16	Đồng Dư Cực Hạn: Căn Bậc Hai Modulo (Tonelli-Shanks)	P5	Advanced	Giải phương trình $x^2 \equiv A \pmod P$

<!-- Chuyên đề 09: Số Nguyên Lớn (BigInt) | 385 dòng | 21,333 bytes -->

Chuyên đề 09: Xử lý số nguyên lớn (bigint)

1. Bản chất vấn đề & trực giác thuật toán (the core problem & intuition)

Trong ngôn ngữ lập trình C++, kiểu dữ liệu số nguyên có kích thước lớn nhất được hỗ trợ phần cứng là `unsigned long long` (64-bit, tối đa xấp xỉ 1.84×10^{19}) hoặc phần mở rộng GCC `__int128` (128-bit, tối đa xấp xỉ 3.4×10^{38}).

Tuy nhiên, trong các bài toán thực tế và đề thi học sinh giỏi (như tính $100!$, tính số Fibonacci thứ 1000 , hoặc tính 2^{10000} **mà không lấy dư modulo**), kết quả có thể dài hàng nghìn đến hàng chục nghìn chữ số. Vì C++ không có sẵn kiểu dữ liệu BigInteger như Python hay Java, lập trình viên thi đấu C++ bắt buộc phải **tự mô phỏng các phép tính số học đặt tính rồi tính như toán tiểu học** trên mảng ký tự (`string`) hoặc mảng số nguyên (`vector<int>`).

Big integer hay modular arithmetic: Chọn vũ khí nào?

! [Phân định lựa chọn giải thuật: Modulo vs Big Integer] (/Users/vu/Developer/ikhEdu_lessons/courses/cpp-bang-b/lessons/lesson-09-so-nguyen-lon-bigint/assets/bigint_vs_modulo_vi.svg)

Đề bài yêu cầu	Quy mô kết quả	Vũ khí tối ưu	Kỹ thuật cốt lõi
Tính $A^B \bmod M$ ($B \leq 10^{18}$)	$\leq M$	Modulo	Lũy thừa nhị phân $O(\log B)$
Tính $\text{frac}\{A\}\{B\} \bmod M$	$\leq M$	Modulo	Nghịch đảo Modulo $A \times B^{-1}$
Tính $F_{10^6} \bmod M$	$\leq M$	Modulo	Nhân ma trận nhị phân $O(\log N)$
Tính chính xác 2^{10000}	<i>approx 3011</i> chữ số	Big Integer	Lũy thừa nhị phân trên BigInt
Tính chính xác $1000!$	<i>2568</i> chữ số	Big Integer	Nhân BigInt $\times$ int liên tiếp
Tính chính xác số Fibonacci F_{1000}	<i>209</i> chữ số	Big Integer	Cộng BigInt + BigInt quy hoạch động
Số có 10^5 chữ số nhưng chỉ cần $\% M$	$\leq M$	Modulo	Vòng lặp Horner: <code>cur = (cur * 10 + d) % M</code>

2. Mô phỏng từng bước (visual step-by-step simulation)

Ví dụ 1: Mô phỏng phép cộng số lớn $A = 9876$ và $B = 543$

- **Quy tắc:** Đảo ngược chuỗi để chữ số hàng đơn vị nằm ở chỉ số 0 .
- $A' = [6, 7, 8, 9]$, $B' = [3, 4, 5]$.

Vị trí i	Chữ số $A'[i]$	Chữ số $B'[i]$	Biến nhớ trước (carry)	Tổng $\text{sum} = A'[i] + B'[i] + \text{carry}$	Chữ số ghi nhận $(\text{sum} \bmod 10)$	Biến nhớ mới $(\lfloor \text{sum} / 10 \rfloor)$
0 (Hàng đv)	6	3	0	$6 + 3 + 0 = 9$	** 9 **	0
1 (Hàng chục)	7	4	0	$7 + 4 + 0 = 11$	** 1 **	1
2 (Hàng trăm)	8	5	1	$8 + 5 + 1 = 14$	** 4 **	1
3 (Hàng nghìn)	9	0	1	$9 + 0 + 1 = 10$	** 0 **	1
Dư cuối	—	—	1	$\text{carry} = 1$	** 1 **	0

- Kết quả đảo ngược: $[9, 1, 4, 0, 1] \Rightarrow 10419$.

Ví dụ 2: Mô phỏng phép nhân số lớn $A = 48$ với số nhỏ $b = 7$

- $A' = [8, 4]$.
- **Bước 0** ($i = 0$): $8 \times 7 + 0 = 56 \Rightarrow$ Ghi 6 , $\text{carry} = 5$.
- **Bước 1** ($i = 1$): $4 \times 7 + 5 = 33 \Rightarrow$ Ghi 3 , $\text{carry} = 3$.
- **Dư cuối:** Ghi $\text{carry} = 3$.
- Kết quả đảo ngược: $[6, 3, 3] \Rightarrow 336$.

3. Lý thuyết cốt lõi & bất biến thuật toán (core invariants)

3.1. Mô hình biểu diễn số lớn & little-endian

- **Biểu diễn Little-Endian:** Lưu các chữ số theo thứ tự từ hàng thấp đến hàng cao (chữ số hàng đơn vị nằm ở chỉ số 0).
- **Ưu điểm cốt lõi:** Hàng đơn vị nằm ở $\text{index} = 0$, nên khi cộng, trừ hoặc nhân ta có thể xử lý trực tiếp từ hàng thấp lên hàng cao và truyền biến nhớ carry/borrow sang phần tử kế tiếp ($a[0] \rightarrow a[1] \rightarrow a[2] \dots$). Ngoài ra, chữ số mới ở cuối có thể được thêm bằng $\text{push_back}()$ với chi phí amortized $O(1)$.

- ****Biểu diễn Base 10 vs Base 10^9 :****
- **Base 10 (`string` / `vector<int>`):** Mỗi phần tử lưu 1 chữ số thập phân (`0 ... 9`).
- ****Base 10^9 (`vector<int>` / `vector<long long>`):** Nhóm các cụm 9 chữ số từ phải sang trái.
- **Cấu trúc dữ liệu:** Mỗi chunk lưu kiểu `int` (`0 ... 999,999,999`); phép nhân giữa 2 chunks lưu kiểu `long long` (vì $(10^9 - 1) \times (10^9 - 1) \approx 10^{18} < 2^{63} - 1$).
- **Ví dụ:** Số `1234567890123456789` được tách thành:

$$chunks = [23456789, 123456789, 1]$$

$$Giá\ trị = 23456789 + 123456789 \times 10^9 + 1 \times (10^9)^2$$

3.2. Bảng tổng hợp các phép toán số nguyên lớn ($O(L^2)$)

Phép toán	Bản chất thuật toán	Độ phức tạp thời gian	Lưu ý quan trọng
So sánh (<code>A</code>, <code>B</code>)	So sánh độ dài trước, sau đó so sánh từ điển	$O(\max(L_A, L_B))$	Xóa sạch số 0 ở đầu trước khi so sánh
Cộng (<code>A + B</code>)	Mô phỏng cộng từng hàng kèm biến nhớ <code>carry</code>	$O(\max(L_A, L_B))$	Xử lý <code>carry</code> còn dư sau khi hết chữ số
Trừ (<code>A - B</code>)	Mô phỏng trừ có mượn <code>borrow</code> ($A \geq B$)	$O(L_A)$	Xóa sạch số 0 vô nghĩa ở đầu (<code>leading zeros</code>)
Nhân nhỏ (<code>A × b</code>)	Nhân từng chữ số của <code>A</code> với số nguyên <code>b</code>	$O(L_A)$	Biến <code>carry</code> có thể vượt quá 10, cần kiểu <code>long long</code>
Nhân lớn (<code>A × B</code>)	Tích lũy <code>C[i+j] += A[i] × B[j]</code> rồi normalize	$O(L_A \times L_B)$	Khởi tạo mảng <code>L_A + L_B</code> (áp dụng cho $L \leq 5000$)
Chia nhỏ (<code>A / b</code>, <code>A % b</code>)	Chia từ hàng cao nhất xuống hàng đơn vị	$O(L_A)$	Biến tích lũy <code>cur = cur * 10 + A[i]</code>

3.3. Thuật toán chia số lớn cho số nhỏ & bất biến horner

Khi chia số lớn `A` cho số nguyên `b` ($1 \leq b \leq 10^9$), ta duyệt từ chữ số hàng cao nhất xuống hàng đơn vị:


```

string divSmall(string a, long long b) {
    string res = "";
    long long cur = 0;
    for (char c : a) {
        cur = cur * 10 + (c - '0');
        int digit = cur / b;
        res.push_back(char('0' + digit));
        cur %= b; // cur luôn là số dư hiện tại
    }
    // Xóa số 0 vô nghĩa ở đầu
    int pos = 0;
    while (pos + 1 < (int)res.size() && res[pos] == '0') pos++;
    return res.substr(pos);
}

```

Ghi chú:

BẮT BIẾN TOÁN HỌC CỦA PHÉP CHIA TỪNG BƯỚC:

Vì trước mỗi bước lặp ta luôn duy trì số dư $0 \leq \text{cur} < b$, nên sau khi nhận thêm một chữ số mới $\text{cur} = \text{cur} \times 10 + \text{digit}$, giá trị luôn thỏa mãn $\text{cur} < 10b$. Do đó thương tại mỗi bước $\text{digit} = \text{cur} / b$ **chắc chắn luôn nằm trong khoảng $[0, 9]$ ** (là một chữ số thập phân hợp lệ duy nhất).

3.4. Tối ưu hóa base 10^9 (chunking optimization)

- Thay vì thực hiện phép nhân trên từng chữ số đơn lẻ (Base 10 có L chữ số), ta nén số lớn thành $\frac{L}{9}$ chunks trong Base 10^9 .
- Đánh giá hiệu năng:** Số lượng cặp chunk cần nhân giảm xấp xỉ $\frac{L}{9} \times \frac{L}{9} = \frac{L^2}{81}$ (giảm khoảng 81 lần về số lượng phép nhân chunk). Tốc độ thực tế tăng vọt từ hàng chục lần giúp vượt qua các bài toán $N \leq 10^5$.

4. Các bẫy lỗi lập trình kinh điển (bug traps)

1. Quên xóa số 0 vô nghĩa ở đầu (Leading Zeros):

- Sau phép trừ (ví dụ $1000 - 999 = 0001$), nếu không xóa số 0 thì chuỗi sẽ in ra `0001`.
- Cách xử lý:** `while (res.size() > 1 && res.back() == '0') res.pop_back();`

1. **Không xét trường hợp số 0:**

- Phép nhân $A \times 0$ phải trả về `"0"`, không được trả về rỗng `""`.

1. Biến `carry` trong phép nhân số nhỏ có thể rất lớn:

- Trong phép nhân $A \times b$ với $b = 10^9$, `carry` sau mỗi bước có thể lên tới 10^9 , do đó kiểu dữ liệu của `carry` bắt buộc phải là `long long`.

5. Mẫu cài đặt chuẩn thi đấu (competitive templates)

```

#include <bits/stdc++.h>
using namespace std;

// Hàm xóa số 0 vô nghĩa ở đầu chuỗi đảo ngược
void removeLeadingZeros(string &s) {
    while (s.size() > 1 && s.back() == '0') {
        s.pop_back();
    }
}

// Phép cộng 2 số nguyên lớn không âm (A + B)
string addBig(string a, string b) {
    reverse(a.begin(), a.end());
    reverse(b.begin(), b.end());

    string res = "";
    int carry = 0;
    int n = max(a.size(), b.size());

    for (int i = 0; i < n || carry; ++i) {
        int sum = carry;
        if (i < (int)a.size()) sum += a[i] - '0';
        if (i < (int)b.size()) sum += b[i] - '0';
        res.push_back((sum % 10) + '0');
        carry = sum / 10;
    }

    reverse(res.begin(), res.end());
    return res;
}

// Phép trừ 2 số nguyên lớn không âm (A - B với A ≥ B)
string subBig(string a, string b) {
    reverse(a.begin(), a.end());
    reverse(b.begin(), b.end());

    string res = "";
    int borrow = 0;

    for (int i = 0; i < (int)a.size(); ++i) {
        int diff = (a[i] - '0') - borrow;
        if (i < (int)b.size()) diff -= (b[i] - '0');
        if (diff < 0) {
            diff += 10;
            borrow = 1;
        } else {
            borrow = 0;
        }
        res.push_back(diff + '0');
    }
}

```

```

        removeLeadingZeros(res);
        reverse(res.begin(), res.end());
        return res;
    }

    // Phép nhân 2 số nguyên lớn chuẩn mực và an toàn (A * B)
    string mulBig(string a, string b) {
        if (a == "0" || b == "0") return "0";

        reverse(a.begin(), a.end());
        reverse(b.begin(), b.end());

        int n = a.size(), m = b.size();
        vector<int> c(n + m, 0);

        for (int i = 0; i < n; ++i) {
            for (int j = 0; j < m; ++j) {
                c[i + j] += (a[i] - '0') * (b[j] - '0');
            }
        }

        // Normalize: Đây biến nhớ carry sang các ô kế tiếp
        for (int i = 0; i + 1 < n + m; ++i) {
            c[i + 1] += c[i] / 10;
            c[i] %= 10;
        }

        while (c.size() > 1 && c.back() == 0) {
            c.pop_back();
        }

        string res = "";
        for (int i = (int)c.size() - 1; i ≥ 0; --i) {
            res.push_back(c[i] + '0');
        }

        return res;
    }

    int main() {
        ios::sync_with_stdio(false);
        cin.tie(nullptr);

        string a, b;
        if (!(cin >> a >> b)) return 0;

        cout << "A + B = " << addBig(a, b) << "\n";
        cout << "A * B = " << mulBig(a, b) << "\n";
        return 0;
    }

```

Câu hỏi trắc nghiệm củng cố khái niệm

Câu 1 (lưu trữ dữ liệu little-endian):

Tại sao khi cài đặt số nguyên lớn trong C++, ta thường đảo ngược chuỗi để chữ số hàng đơn vị nằm ở vị trí chỉ số 0 (Little-Endian)?

- A. Để tiết kiệm bộ nhớ RAM.
- B. **[Đáp án đúng]** Để thao tác thêm chữ số mới vào cuối mảng (`push_back`) đạt độ phức tạp amortized $O(1)$ thay vì phải dịch chuyển toàn bộ mảng trong $O(N)$.
- C. Để chuyển đổi sang kiểu `int` nhanh hơn.
- D. Bắt buộc theo chuẩn ngôn ngữ C++.

Giải thích: Trong `vector` hoặc `string`, thao tác `push_back()` vào cuối có chi phí trung bình amortized $O(1)$, trong khi chèn vào đầu tốn $O(N)$.

Câu 2 (độ phức tạp phép nhân):

Phép nhân hai số nguyên lớn có độ dài lần lượt là N chữ số và M chữ số theo thuật toán đặt tính cơ bản có độ phức tạp thời gian là:

- A. $O(N + M)$
- B. $O(\max(N, M))$
- C. **[Đáp án đúng]** $O(N \times M)$
- D. $O((N + M) \log(N + M))$

Giải thích: Mỗi chữ số của số thứ nhất phải nhân với từng chữ số của số thứ hai qua hai vòng lặp lồng nhau, tạo ra $N \times M$ phép nhân chữ số.

Câu 3 (độ dài tối đa kết quả phép nhân):

Tích của một số nguyên dương có N chữ số và một số nguyên dương có M chữ số có độ dài tối đa là bao nhiêu chữ số?

- A. $N \times M$
- B. $\max(N, M) + 1$
- C. **[Đáp án đúng]** $N + M$
- D. $N + M - 1$

Giải thích: Giá trị lớn nhất là $(10^N - 1)(10^M - 1) < 10^{N+M}$, do đó số chữ số tối đa luôn là $N + M$.

Câu 4 (xử lý số 0 vô nghĩa):

Sau khi thực hiện phép trừ số lớn $10005 - 10000$, chuỗi kết quả thu được là `"00005"`. Thao tác nào sau đây xử lý đúng để kết quả trở thành `"5"`?

- A. Gán chuỗi bằng `"5"`.
- B. **[Đáp án đúng]** Xóa các ký tự `'0'` ở đầu cho đến khi gặp ký tự khác `'0'` hoặc chuỗi chỉ còn đúng 1 ký tự `'0'`.
- C. Xóa toàn bộ ký tự `'0'` trong chuỗi.
- D. Đảo ngược chuỗi 2 lần.

Giải thích: Ta phải giữ lại ít nhất 1 chữ số trong trường hợp kết quả phép trừ bằng 0 (ví dụ $5 - 5 = 0$).

Câu 5 (phép chia số lớn cho số nhỏ):

Khi thực hiện phép chia một số lớn A (có N chữ số) cho một số nguyên b ($1 \leq b \leq 10^9$), ta duyệt các chữ số của A theo thứ tự nào?

- A. Từ hàng đơn vị lên hàng cao nhất (từ phải sang trái).
- B. **[Đáp án đúng]** Từ hàng cao nhất xuống hàng đơn vị (từ trái sang phải), duy trì số dư tích lũy $cur = cur * 10 + digit$.
- C. Duyệt từ giữa chuỗi sang hai bên.
- D. Thứ tự nào cũng cho kết quả như nhau.

Giải thích: Phép chia mô phỏng đúng quy tắc đặt tính chia của toán học: chia từ hàng cao nhất xuống hàng thấp nhất.

Câu 6 (trường hợp phép trừ số âm):

Nếu cần tính hiệu $A - B$ của hai số nguyên dương lớn nhưng chưa biết số nào lớn hơn, giải thuật chuẩn xác là gì?

- A. Vẫn thực hiện phép trừ bình thường $A - B$.
- B. **[Đáp án đúng]** So sánh A và B . Nếu $A \geq B$ thì tính $A - B$. Nếu $A < B$ thì tính $B - A$ rồi thêm dấu trừ "-" vào đầu kết quả.
- C. Báo lỗi không tính được.
- D. Lấy trị tuyệt đối của từng chữ số rồi trừ nhau.

Giải thích: Phép trừ số lớn trên mảng chỉ đúng khi số bị trừ lớn hơn hoặc bằng số trừ. Khi $A < B$, ta quy về $-(B - A)$.

Câu 7 (tối ưu base 10^9):

Thay vì lưu mỗi phần tử trong mảng là 1 chữ số thập phân (Base 10), việc gom 9 chữ số thập phân vào 1 số nguyên 32-bit (Base 10^9) mang lại lợi ích gì về mặt thuật toán?

- A. Giảm dung lượng bộ nhớ mảng đi khoảng 9 lần.
- B. Giảm số lượng phép tính của phép cộng/trừ đi khoảng 9 lần.
- C. Với phép nhân đặt tính, số cặp chunk cần xử lý giảm xấp xỉ $9^2 = 81$ lần.
- D. **[Đáp án đúng]** Cả A, B, C đều đúng.

Giải thích: Base 10^9 nén dữ liệu giúp giảm cả dung lượng bộ nhớ và số lượng phép toán chunk, giúp code BigInt chạy nhanh hơn rất nhiều trong các bài toán $N \leq 10^5$.

Câu 8 (giai thừa số lớn $1000!$):

Để tính chính xác $1000!$ mà không bị tràn số trong C++, ta áp dụng phương pháp nào?

- A. Dùng kiểu dữ liệu `double`.
- B. Dùng kiểu dữ liệu `__int128`.
- C. **[Đáp án đúng]** Khởi tạo `string ans = "1"`, sau đó thực hiện vòng lặp nhân lần lượt với các số từ 2 đến 1000 bằng hàm nhân số lớn với số nhỏ.
- D. Dùng công thức xấp xỉ Stirling.

Giải thích: $1000!$ có 2568 chữ số, vượt xa kiểu `__int128` (khoảng 38 chữ số), bắt buộc phải dùng phép nhân số lớn.

Câu 9 (lũy thừa số lớn A^B):

Khi cần tính A^B với $A = 2$ và $B = 10000$ (kết quả chính xác không lấy dư), phương pháp tối ưu là:

- A. Nhân 2 liên tiếp 10000 lần.
- B. **[Đáp án đúng]** Kết hợp thuật toán Lũy thừa nhị phân $O(\log B)$ với phép nhân 2 số nguyên lớn.
- C. Dùng hàm `pow(2, 10000)` trong thư viện `<cmath>`.
- D. Chuyển sang hệ nhị phân rồi in ra.

Giải thích: Lũy thừa nhị phân chỉ cần thực hiện ≈ 14 phép nhân số lớn thay vì 10000 phép nhân.

Câu 10 (so sánh hai số lớn dạng chuỗi):

Điều kiện nào sau đây quyết định chắc chắn số nguyên dương lớn A lớn hơn số nguyên dương lớn B (giả sử cả A và B không có số 0 vô nghĩa ở đầu)?

- A. Ký tự đầu tiên của A lớn hơn ký tự đầu tiên của B .
- B. **[Đáp án đúng]** Độ dài chuỗi $|A| > |B|$, hoặc nếu $|A| == |B|$ thì $A > B$ theo thứ tự từ điển.
- C. Tổng các chữ số của A lớn hơn tổng các chữ số của B .
- D. Chữ số tận cùng của A lớn hơn chữ số tận cùng của B .

Giải thích: Số có nhiều chữ số hơn luôn lớn hơn. Khi cùng số chữ số, so sánh từ điển từ trái sang phải phản ánh đúng thứ tự so sánh từ hàng cao nhất xuống hàng thấp nhất.

Ma trận bài tập thực hành (p0 → p5)

Ghi chú:

Phân tầng lộ trình học tập:

** **Nhóm Cốt Lõi (Core Foundations - Bắt buộc CPPB-BIG-01 → 12):** Mô hình biểu diễn, So sánh, 4 phép tính cơ bản (+, -, *, /), Giai thừa, Lũy thừa, Fibonacci và Tổng chữ số. **Nhóm Thử Thách Mở Rộng (Advanced / Challenge CPPB-BIG-13 → 16):** Chia hai số lớn, Căn bậc hai số lớn, Binary GCD và Tổ hợp chính xác kết hợp phân tích nguyên tố.*

STT	Mã Bài	Tên Bài Toán	Cấp Độ	Phân Loại	Dạng Thuật Toán & Kỹ Năng Cốt Lõi
01	CPPB-BIG-01	So Sánh Hai Số Nguyên Lớn	P0	Core	So sánh độ dài và so sánh từ điển chuỗi số
02	CPPB-BIG-02	**Cộng Hai Số Nguyên Lớn $(A + B)$ **	P0	Core	Mô phỏng phép cộng đặt tính và xử lý biến nhớ <code>carry</code>
03	CPPB-BIG-03	**Trừ Hai Số Nguyên Lớn $(A - B)$ **	P1	Core	Phép trừ có mượn $A \geq B$ và xóa số 0 vô nghĩa
04	CPPB-BIG-04	Trừ Hai Số Lớn Tổng Quát (Có Âm)	P1	Core	So sánh và gắn dấu <code>"-"</code> khi $A < B$
05	CPPB-BIG-05	**Nhân Số Lớn Với Số Nhỏ $(A \times b)$ **	P1	Core	Nhân từng chữ số với $b \leq 10^9$
06	CPPB-BIG-06	**Nhân Hai Số Nguyên Lớn $(A \times B)$ **	P2	Core	Thuật toán nhân chập và normalize $O(L_A \times L_B)$
07	CPPB-BIG-07	**Chia Số Lớn Cho Số Nhỏ (A / b) **	P2	Core	Chia từ hàng cao xuống thấp và lấy thương nguyên
08	CPPB-BIG-08	**Chia Lấy Dư Số Lớn Cho Số Nhỏ $(A \bmod b)$ **	P2	Core	Duy trì số dư <code>cur = (cur * 10 + digit) % b</code>
09	CPPB-BIG-09	**Tính Giai Thừa Số Lớn $(N!)$ **	P3	Core	Tính chính xác $N!$ với $N \leq 1000$
10	CPPB-BIG-10	**Lũy Thừa Số Lớn Chính Xác (A^B) **	P3	Core	Lũy thừa nhị phân kết hợp nhân số lớn
11	CPPB-BIG-11	**Số Fibonacci Lớn Thứ N **	P3	Core	Tính chính xác F_N với $N \leq 1000$ bằng cộng số lớn
12	CPPB-BIG-12	**Tổng Các Chữ Số Của $N!$ hoặc 2^N **	P3	Core	Tính số lớn và tính tổng chữ số
13	CPPB-BIG-13	**Chia Hai Số Nguyên Lớn (A / B) **	P4	Advanced	Tìm thương nguyên bằng tìm kiếm nhị phân hoặc Long Division
14	CPPB-BIG-14	**Căn Bậc Hai Số Nguyên Lớn $(\lfloor \sqrt{A} \rfloor)$ **	P4	Advanced	Tìm kiếm nhị phân trên không gian chuỗi kết hợp nhân BigInt
15				Challenge	

STT	Mã Bài	Tên Bài Toán	Cấp Độ	Phân Loại	Dạng Thuật Toán & Kỹ Năng Cốt Lõi
	CPPB- BIG-15	**Ước Chung Lớn Nhất Số Lớn ($gcd(A, B)$)**	P4		Thuật toán Stein's Binary GCD kết hợp phép chia 2 và trừ BigInt
16	CPPB- BIG-16	**Số Lớn Cực Hạn: Tổ Hợp $C(N, K)$ Chính Xác**	P5	Challenge	Tối ưu hóa: Phân tích thừa số nguyên tố kết hợp nhân lũy thừa số lớn (hoặc DP Pascal BigInt)

=====

Module 04: Độ quy, chia để trị & quay lui

<!-- Chuyên đề 10: Độ Quy Cơ Bản | 371 dòng | 22,301 bytes -->

Chuyên đề 10: Thuật toán đệ quy & cây gọi hàm

1. Bản chất vấn đề & trực giác thuật toán (the core problem & intuition)

Trong các bài toán lập trình cơ bản, chúng ta quen thuộc với tư duy lặp tuần tự (`for` , `while`): xử lý từng phần tử lần lượt từ đầu đến cuối. Tuy nhiên, trong thế giới cấu trúc dữ liệu và giải thuật nâng cao, rất nhiều bài toán mang bản chất **tự đồng dạng (Self-Similarity)**: Để giải một bài toán quy mô N , ta có thể giải bài toán tương tự nhưng ở quy mô nhỏ hơn $N-1$ hoặc $N/2$, sau đó kết hợp kết quả lại.

Khái niệm đệ quy (recursion):

Đệ quy là kỹ thuật lập trình trong đó **một hàm tự gọi lại chính nó** (trực tiếp hoặc gián tiếp) với các tham số đại diện cho bài toán con nhỏ hơn.

Mỗi hàm đệ quy chuẩn mực bắt buộc phải có đủ 2 thành phần cốt lõi:

1. **Điểm Dừng (Base Case / Anchor)**: Trường hợp bài toán đơn giản nhất đã biết trước đáp án mà không cần gọi tiếp đệ quy. Điểm dừng có nhiệm vụ **ngắt chuỗi lời gọi vô tận**.
2. **Bước Đệ Quy (Recursive Case / Reduction Step)**: Thu nhỏ quy mô bài toán bằng cách gọi lại chính hàm đó với tham số tiến dần về phía Base Case.

![[Cấu trúc điều hướng của hàm đệ quy: Base Case vs Recursive Case]](/Users/vu/Developer/ikhEdu_lessons/courses/cpp-bang-b/lessons/lesson-10-de-quy-co-ban/assets/recursion_structure_vi.svg)

2. Mô phỏng từng bước hoạt động của call stack (visual step-by-step simulation)

Để hiểu đệ quy, lập trình viên không được nhìn code như một vòng lặp phẳng, mà bắt buộc phải hình dung hoạt động của **Ngăn xếp cuộc gọi (Call Stack)** qua hai pha riêng biệt:

- **Pha Xuôi (Winding Phase)**: Các hàm được gọi liên tiếp và đẩy đè lên nhau trên đỉnh ngăn xếp (`Stack Frame Push`).
- **Pha Ngược (Unwinding Phase)**: Khi chạm Base Case, các hàm lần lượt tính xong kết quả, trả về (`Return`) và được giải phóng khỏi ngăn xếp (`Stack Frame Pop`).

Ví dụ 1: Mô phỏng hàm tính giai thừa `fact(4)`

```
long long fact(int n) {
    if (n ≤ 1) return 1;           // Base Case
    return n * fact(n - 1);        // Recursive Step
}
```

Bảng mô phỏng từng bước ngăn xếp (call stack trace):

Bước	Hành động	Trạng thái Call Stack (Đỉnh stack ở trên cùng)	Giá trị trả về tại bước đó
1	Gọi <code>fact(4)</code>	<code>[fact(4)]</code>	Đang đợi <code>fact(3)</code>
2	Gọi <code>fact(3)</code>	<code>[fact(3)] → [fact(4)]</code>	Đang đợi <code>fact(2)</code>
3	Gọi <code>fact(2)</code>	<code>[fact(2)] → [fact(3)] → [fact(4)]</code>	Đang đợi <code>fact(1)</code>
4	Gọi <code>fact(1)</code>	<code>[fact(1)] → [fact(2)] → [fact(3)] → [fact(4)]</code>	Chạm Base Case: Trả về 1
5	Unwind <code>fact(2)</code>	<code>[fact(2)] → [fact(3)] → [fact(4)]</code>	<code>fact(2) = 2 * 1 = 2</code>
6	Unwind <code>fact(3)</code>	<code>[fact(3)] → [fact(4)]</code>	<code>fact(3) = 3 * 2 = 6</code>
7	Unwind <code>fact(4)</code>	<code>[fact(4)]</code>	<code>fact(4) = 4 * 6 = 24</code>
8	Kết thúc	Stack rỗng	Đáp án: 24

Ví dụ 2: So sánh vị trí lệnh `in` (winding vs unwinding)

Quan sát sự khác biệt khi đặt lệnh `cout` trước vs sau lời gọi đệ quy:

```
// Dạng A: In trong Winding Phase (Trước khi gọi đệ quy)
void printBackward(int n) {
    if (n == 0) return;
    cout << n << " ";          // In ngay khi vào hàm
    printBackward(n - 1);
}
// Gọi printBackward(3) → Output: 3 2 1

// Dạng B: In trong Unwinding Phase (Sau khi gọi đệ quy)
void printForward(int n) {
    if (n == 0) return;
    printForward(n - 1);
    cout << n << " ";          // In khi hàm quay lui trở về
}
// Gọi printForward(3) → Output: 1 2 3
```

Quy luật vòng (winding vs unwinding):

- Các thao tác viết **trước lời gọi đệ quy** sẽ thực thi theo thứ tự từ ngoài vào trong ($N \rightarrow 1$).
- Các thao tác viết **sau lời gọi đệ quy** sẽ thực thi theo thứ tự từ trong ra ngoài ($1 \rightarrow N$), khi stack bắt đầu rút lui (Unwind).

3. Lý thuyết cốt lõi & bất biến thuật toán (core invariants)

3.1. Khái niệm stack frame & phân tích an toàn bộ nhớ (stack safety)

- Khi một hàm được gọi, mô hình thực thi của chương trình tạo ra một **Stack Frame (Activation Record)** lưu trữ trạng thái thực thi riêng biệt: tham số truyền vào, các biến cục bộ và địa chỉ trả về (Return Address) theo quy ước gọi (Calling Convention / ABI).
- Vùng nhớ ngăn xếp (Stack Memory) có kích thước hữu hạn và giới hạn cụ thể phụ thuộc vào môi trường thực thi, hệ điều hành và cấu hình của từng Online Judge.
- **Độ sâu đệ quy (Recursion Depth) vs Kích thước Stack Frame:**
- Để đánh giá an toàn bộ nhớ của hàm đệ quy, ta phải xem xét đồng thời **Độ sâu đệ quy tối đa (Maximum Depth)** và **Dung lượng bộ nhớ tiêu thụ trên mỗi Frame**. Nếu mỗi frame chứa mảng cục bộ lớn hoặc đệ quy vượt quá giới hạn bộ nhớ stack, chương trình sẽ gặp lỗi tràn ngăn xếp (**Stack Overflow / Segmentation Fault**).

Lưu ý kỹ thuật về tail recursion trong c++:

Trong lý thuyết ngôn ngữ, *Đệ quy đuôi (Tail Recursion)* là hàm đệ quy mà lời gọi hàm là câu lệnh cuối cùng. Tuy nhiên, **chuẩn ngôn ngữ C++ không bắt buộc trình biên dịch phải tối ưu hóa đệ quy đuôi (Tail-Call Optimization - TCO)** trong mọi cờ biên dịch thi đấu. Do đó, học sinh không được chủ quan giả định đệ quy đuôi sẽ luôn tự biến thành vòng lặp $O(1)$ bộ nhớ. Luôn phân tích độ sâu stack cẩn trọng!

3.2. Hệ thống phân loại thuật ngữ đệ quy (recursion taxonomy)

! [Hệ thống phân loại thuật toán đệ quy: Tuyến tính vs Phân nhánh] (/Users/vu/Developer/ikhEdu_lessons/courses/cpp-bang-b/lessons/lesson-10-de-quy-co-ban/assets/recursion_taxonomy_vi.svg)

1. Đệ quy Tuyến tính (Linear Recursion - 1 nhánh gọi / Frame):

- Trong mỗi Stack Frame chỉ thực hiện **đúng 1 lời gọi đệ quy con**. Cây gọi hàm là một đường thẳng đơn tuyến.
- Ví dụ:
- Giai thừa $N!$: Độ sâu N , thời gian $\Theta(N)$, Stack Space $\Theta(N)$.
- Thuật toán Euclid $\text{gcd}(A, B)$: Độ sâu $\Theta(\log(\min(A, B)))$, thời gian $\Theta(\log(\min(A, B)))$.

Lũy thừa nhị phân $\text{powerRec}(A, B/2)$ (khi lưu biến tạm `half`): Độ sâu $\Theta(\log B)$, thời gian $\Theta(\log B)$. Lưu ý: * Mặc dù quy mô bài toán giảm theo cấp số nhân ($B \rightarrow B/2$), cấu trúc cây gọi hàm vẫn là đường thẳng 1 nhánh đơn tuyến.

1. **Đệ quy Phân nhánh (Branching / Tree Recursion - ≥ 2 nhánh gọi / Frame):**

- Trong mỗi Stack Frame xuất hiện **từ 2 lời gọi đệ quy con trở lên**, làm bùng nổ không gian trạng thái tạo thành cây nhị phân hoặc cây đa phân.
- Ví dụ:
- Tháp Hà Nội: $T(N) = 2T(N-1) + 1 \Rightarrow \Theta(2^N)$ bước, Độ sâu N .
- Cây chia đôi tìm Min/Max: $T(N) = 2T(N/2) + O(1) \Rightarrow \Theta(N)$ thao tác, Độ sâu $\Theta(\log N)$.
- Fibonacci đệ quy thuần túy $F(N) = F(N-1) + F(N-2)$.

3.3. Độ phức tạp toán học của fibonacci đệ quy & cầu nối sang quy hoạch động

Xét cây gọi hàm khi tính $F(5)$ bằng đệ quy phân nhánh:

! [Cây đệ quy phân nhánh Fibonacci F(5) và hiện tượng bài toán con trùng lặp] (/Users/vu/Developer/ikhEdu_lessons/courses/cpp-bang-b/lessons/lesson-10-de-quy-co-ban/assets/fibonacci_recursion_tree_vi.svg)

• Phân tích độ phức tạp tiệm cận chính xác:

Số lời gọi hàm thỏa mãn hệ thức truy hồi $T(N) = T(N-1) + T(N-2) + 1$. Bằng phương trình đặc trưng $r^2 - r - 1 = 0$, ta chứng minh được số phép tính thực tế tăng theo **cấp số nhân chính xác**:

$$\Theta(\varphi^N) \text{ với } \varphi = \frac{1 + \sqrt{5}}{2} \approx 1.618 \text{ (Tỉ lệ vàng)}$$

Chặn trên $O(2^N)$ là một cận trên lỏng (Upper Bound).

• Hiện tượng Overlapping Subproblems:

Để tính $F(5)$, hàm $F(3)$ bị tính lại 2 lần, $F(2)$ bị tính lại 3 lần. Với $N = 40$, số lượng lời gọi đã lên tới hàng trăm triệu theo mô hình Fibonacci ($\Theta(\varphi^N)$), minh họa rõ hiện tượng bùng nổ thời gian.

- **Bài học sư phạm:** Đệ quy thuần túy rất đẹp nhưng sẽ bị tê liệt khi không gian trạng thái có các bài toán con trùng lặp. Việc **lưu lại kết quả đã tính vào bảng nhớ (Memoization)** sẽ được học bài bản ở **Module 05: Quy hoạch động (Dynamic Programming)**.

4. Các bẫy lỗi lập trình kinh điển (bug traps)

1. Thiếu Base Case hoặc Base Case không bao giờ chạm tới (Infinite Recursion):

- Viết `if (n == 0)` nhưng tham số truyền vào là số âm $\Rightarrow$ Gọi đệ quy vô tận cho tới khi sập ngăn xếp.
- **Cách phòng chống:** Luôn chặn cận bằng dấu $\leq$ (ví dụ `if (n ≤ 1) return 1;`).

1. Khai báo mảng lớn cục bộ bên trong hàm đệ quy:

- Viết `int temp[100000];` trong hàm đệ quy sẽ khiến mỗi Stack Frame tốn hàng trăm KB bộ nhớ $\Rightarrow$ Tràn stack chỉ sau vài chục lời gọi.
- **Cách phòng chống:** Dùng biến toàn cục hoặc truyền tham chiếu `const vector<int> &a`.

1. Bẫy Gọi Lặp Lại Đệ Quy (Recursive Call Duplication):

- Trong bài lũy thừa nhị phân, nếu viết `return power(a, b/2) * power(a, b/2);` thì từ đệ quy tuyến tính $O(\log B)$ sẽ bị nổ thành cây đệ quy phân nhánh $\Theta(B)$ thao tác.
- **Quy tắc vàng:** Không có Memoization, hai lời gọi hàm giống nhau là hai lần tính toán hoàn toàn độc lập. Tính 1 lần vào biến tạm: `long long half = power(a, b/2, m);`
`return (half * half) % m;`

5. Mẫu cài đặt chuẩn thi đấu (competitive templates)

```

#include <bits/stdc++.h>
using namespace std;

// 1. In dãy số 1..N và N..1 chuẩn Winding / Unwinding
void printForward(int n) {
    if (n ≤ 0) return;
    printForward(n - 1);
    cout << n << " ";
}

void printBackward(int n) {
    if (n ≤ 0) return;
    cout << n << " ";
    printBackward(n - 1);
}

// 2. Lũy thừa nhị phân đệ quy  $O(\log B)$  an toàn với  $M \leq 10^9$ 
long long powerRec(long long a, long long b, long long m) {
    if (b == 0) return 1 % m;
    long long half = powerRec(a, b / 2, m);
    long long res = (1LL * (half % m) * (half % m)) % m;
    if (b % 2 == 1) res = (1LL * res * (a % m)) % m;
    return res;
}

// 3. Tháp Hà Nội chuẩn Theta( $2^N$ )
void solveHanoi(int n, char from_rod, char to_rod, char aux_rod) {
    if (n == 0) return;
    solveHanoi(n - 1, from_rod, aux_rod, to_rod);
    cout << from_rod << " → " << to_rod << "\n";

    solveHanoi(n - 1, aux_rod, to_rod, from_rod);
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n = 4;
    cout << "Day so 1..N: ";
    printForward(n);
    cout << "\n";

    cout << "Day so N..1: ";
    printBackward(n);
    cout << "\n";

    cout << "2^10 mod 1000 = " << powerRec(2, 10, 1000) << "\n";
    return 0;
}

```

Câu hỏi trắc nghiệm củng cố khái niệm

Câu 1 (bản chất base case):

Thành phần nào trong một hàm đệ quy có vai trò quyết định giúp hàm không bị rơi vào vòng lặp vô tận và tránh lỗi tràn bộ nhớ ngăn xếp (Stack Overflow)?

- A. Khối lệnh gọi lại chính hàm đó (Recursive Step).
- B. **[Đáp án đúng]** Điều kiện dừng cơ sở (Base Case).
- C. Kiểu dữ liệu trả về của hàm.
- D. Danh sách các tham số truyền vào hàm.

Giải thích: Base Case là điều kiện chặn dưới, khi thỏa mãn điều kiện này hàm sẽ dừng gọi tiếp và bắt đầu quá trình trả lời lui về (Unwinding Phase).

Câu 2 (winding vs unwinding trace prediction):

Xét hàm đệ quy sau:

```
void trace(int n) {
    if (n == 0) return;
    cout << n << " ";
    trace(n - 1);
    cout << n << " ";
}
```

Khi gọi `trace(3)`, kết quả in ra màn hình chính xác là gì?

- A. 3 2 1
- B. 1 2 3 3 2 1
- C. **[Đáp án đúng]** 3 2 1 1 2 3
- D. 3 3 2 2 1 1

Giải thích: Lệnh `cout` đầu tiên in trong pha Winding (3 2 1), lệnh `cout` thứ hai in trong pha Unwinding (1 2 3), tạo chuỗi đối xứng 3 2 1 1 2 3 .

Câu 3 (cấu trúc bộ nhớ stack frame):

Mỗi lần một hàm đệ quy được gọi, thông tin nào sau đây được lưu vào một Stack Frame (Activation Record)?

- A. Toàn bộ mã nguồn C++ của chương trình.
- B. **[Đáp án đúng]** Các tham số của hàm, biến cục bộ và địa chỉ trả về (Return Address).
- C. Bảng mã ASCII của các ký tự.
- D. Dữ liệu của file đề bài.

Giải thích: Mỗi Stack Frame lưu trữ ngữ cảnh thực thi riêng biệt của lần gọi hàm đó (biến cục bộ, tham số và địa chỉ lệnh cần thực thi tiếp khi hàm con kết thúc).

Câu 4 (độ phức tạp chính xác của fibonacci đệ quy):

Hàm đệ quy tính số Fibonacci thuần túy:

```
int fib(int n) {
    if (n ≤ 1) return n;
    return fib(n - 1) + fib(n - 2);
}
```

Có độ phức tạp thời gian tiệm cận chính xác (Tight Bound) là bao nhiêu?

- A. $O(N)$
- B. $O(N^2)$
- C. $O(\log N)$
- D. **[Đáp án đúng]** $\Theta(\varphi^N)$ với $\varphi = (1 + \sqrt{5})/2 \approx 1.618$ (thường được chặn trên bởi $O(2^N)$).

Giải thích: Số lượng lời gọi hàm thỏa mãn hệ thức truy hồi Fibonacci, có nghiệm chính xác tỷ lệ với lũy thừa tỉ lệ vàng $\varphi^N \approx 1.618^N$.

Câu 5 (bẫy tràn stack overflow):

Yếu tố nào sau đây quyết định trực tiếp việc một hàm đệ quy có gây ra lỗi tràn bộ nhớ ngăn xếp (Stack Overflow) hay không?

- A. Hàm đệ quy có quá nhiều tham số kiểu `int`.
- B. **[Đáp án đúng]** Tích của độ sâu đệ quy tối đa và dung lượng bộ nhớ tiêu thụ trên mỗi Stack Frame vượt quá giới hạn stack của hệ thống.
- C. Hàm đệ quy có kiểu trả về là `void`.
- D. Hàm đệ quy chạy trên hệ điều hành 64-bit.

Giải thích: Stack có kích thước hữu hạn. An toàn stack đòi hỏi phải kiểm soát đồng thời cả chiều sâu đệ quy và kích thước biến cục bộ trong mỗi frame.

Câu 6 (bẫy gọi đệ quy lặp lại):

Trong thuật toán lũy thừa nhị phân A^B , nếu viết: `return power(a, b / 2) * power(a, b / 2);` thay vì lưu vào biến tạm `long long half = power(a, b / 2);`, độ phức tạp thời gian sẽ bị suy biến thành:

- A. Vẫn giữ nguyên $O(\log B)$.
- B. **[Đáp án đúng]** Bị suy biến thành $\Theta(B)$ (tương đương với vòng lặp nhân tuần tự).
- C. $O(1)$.
- D. $O(B^2)$.

Giải thích: Việc gọi lại 2 lần cùng một hàm con biến cây gọi hàm thành cây nhị phân đầy đủ có số lượng nút bằng $2\log_2 B = B$, làm mất hoàn toàn ưu thế của chia để trị.

Câu 7 (đặc điểm tail recursion trong c++):

Nhận định nào sau đây là chính xác nhất về Đệ quy đuôi (Tail Recursion) trong ngôn ngữ C++ chuẩn thi đấu?

- A. C++ luôn tự động tối ưu đệ quy đuôi thành vòng lặp với bộ nhớ $O(1)$ trong mọi trường hợp.
- B. **[Đáp án đúng]** C++ không đảm bảo luôn tối ưu đệ quy đuôi; mức độ tối ưu phụ thuộc vào trình biên dịch, cờ tối ưu và kiến trúc CPU, do đó vẫn có nguy cơ tràn stack.
- C. Đệ quy đuôi chạy chậm hơn đệ quy thông thường.
- D. Đệ quy đuôi chỉ áp dụng được cho hàm trả về `void`.

Giải thích: Chuẩn ngôn ngữ C++ không bắt buộc Tail Call Optimization (TCO), lập trình viên thi đấu không được phép giả định stack sẽ được giải phóng an toàn.

Câu 8 (số bước di chuyển tháp hà nội):

Với bài toán Tháp Hà Nội chuẩn gồm N đĩa, số bước di chuyển tối thiểu chính xác là:

- A. $2N$
- B. N^2
- C. **[Đáp án đúng]** $2^N - 1$ (đạt độ phức tạp thời gian $\Theta(2^N)$).
- D. $N!$

Giải thích: Hệ thức truy hồi số bước chuyển đĩa là $T(N) = 2T(N - 1) + 1$ với $T(1) = 1$, giải hệ thức thu được nghiệm tổng quát $T(N) = 2^N - 1$.

Câu 9 (bản chất đệ quy chia đôi tìm min/max):

Khi tìm Min/Max của mảng N phần tử bằng hàm đệ quy chia đôi $\text{getMin}(l, r) = \min(\text{getMin}(l, \text{mid}), \text{getMin}(\text{mid} + 1, r))$, độ phức tạp thời gian tiệm cận là:

- A. $O(\log N)$ vì mảng luôn được chia đôi ở mỗi bước.
- B. **[Đáp án đúng]** $\Theta(N)$ vì thuật toán bắt buộc phải thăm và so sánh toàn bộ N phần tử của cả hai nửa mảng.
- C. $O(N \log N)$.
- D. $O(1)$.

Giải thích: Hệ thức thời gian là $T(N) = 2T(N/2) + O(1)$. Theo định lý thợ (Master Theorem), độ phức tạp là $\Theta(N)$. "Chia đôi" không đồng nghĩa với $O(\log N)$ nếu phải duyệt cả hai nhánh.

Câu 10 (hiện tượng overlapping subproblems):

Hiện tượng nhiều hàm đệ quy con có cùng tham số đầu vào bị tính toán lặp đi lặp lại nhiều lần trên cây đệ quy là tiền đề trực tiếp để phát triển phương pháp tối ưu nào sau đây?

- A. Tìm kiếm nhị phân (Binary Search).
- B. Kỹ thuật hai con trỏ (Two Pointers).
- C. **[Đáp án đúng]** Quy hoạch động & Bảng nhớ (Dynamic Programming & Memoization).
- D. Sắp xếp trộn (Merge Sort).

Giải thích: Khi một bài toán có tính chất bài toán con trùng lặp (Overlapping Subproblems), ta có thể lưu kết quả tính được lần đầu vào bảng nhớ để tái sử dụng ngay trong $O(1)$ ở các lần gặp tiếp theo, chính là bản chất của Quy hoạch động.

Ma trận bài tập thực hành (p0 → p5)

Phân tầng lộ trình học tập lesson 10:

- ****Nhóm Cốt Lõi (Core Foundations - Bắt buộc CPPB-REC-01 → 12):**** Nắm vững Winding/Unwinding phase, Base case, Đề quy tuyến tính vs Đề quy nhị phân, Tháp Hà Nội, Khảo sát cây Fibonacci.
- ****Nhóm Thử Thách Mở Rộng (Advanced & Optional Extension CPPB-REC-13 → 16):**** Tháp Hà Nội ràng buộc nước đi ($\Theta(3^N)$), Sinh xâu không 2 số 1 liên kề, Đếm phân tích số thành tổng (Integer Partitioning không xét thứ tự), Đếm cấu hình cây nhị phân (Catalan Tree Recurrence).

STT	Mã Bài	Tên Bài Toán	Cấp Độ	Phân Loại	Time Complexity	Stack Space	Max Depth						
01	CPPB-REC-01	**In Dãy Số $I \dots N$ và $N \dots I$ **	P0	Core	$\Theta(N)$	$\Theta(N)$	N						
02	CPPB-REC-02	**Tính Tổng Dãy Số & Giai Thừa $N!$ **	P0	Core	$\Theta(N)$	$\Theta(N)$	N						
03	CPPB-REC-03	**Đếm & Tính Tổng Chữ Số Của N **	P1	Core	$\Theta(\log_{10} N)$	$\Theta(\log_{10} N)$	≤ 19						
04	CPPB-REC-04	Đảo Ngược Mảng Bằng Đệ Quy	P1	Core	$\Theta(N)$	$\Theta(N)$	$N/2$						
05	CPPB-REC-05	Kiểm Tra Chuỗi Palindrome	P1	Core	$\Theta(N)$	S		$\Theta(N)$	S	)	\$	S	/ 2\$
06	CPPB-REC-06	So Sánh Đệ Quy Tuyến Tính & Chia Đôi (Min/Max)	P2	Core	$\Theta(N)$	$\Theta(\log N)$	$\log^2 N$						
07	CPPB-REC-07	**Thuật Toán Euclid Tính $\gcd(A, B)$ **	P2	Core	$\Theta(\log(\min))$	$\Theta(\log(\min))$	≤ 90						
08	CPPB-REC-08	**Lũy Thừa Đệ Quy $A^B \bmod M$ **	P2	Core	$\Theta(\log B)$	$\Theta(\log B)$	$\log^2 B$						
09	CPPB-REC-09	Tháp Hà Nội (Tower of Hanoi)	P3	Core	$\Theta(2^N)$	$\Theta(N)$	N						
10	CPPB-REC-10	Dãy Fibonacci Đệ Quy &	P3	Core	$\Theta(\varphi^N)$	$\Theta(N)$	N						

STT	Mã Bài	Tên Bài Toán	Cấp Độ	Phân Loại	Time Complexity	Stack Space	Max Depth
		Cây Phân Nhánh					
11	CPPB- REC-11	**Chuyển Đổi Hệ Cơ Số 10 → 2 **	P3	Core	$\Theta(\log^2 N)$	$\Theta(\log^2 N)$	≤ 60
12	CPPB- REC-12	Xây Dựng Hệ Thức Truy Hồi Dây Số	P3	Core	$\Theta(N)$	$\Theta(N)$	N
13	CPPB- REC-13	Tháp Hà Nội Có Ràng Buộc Nước Đi	P4	Advanced	$\Theta(3^N)$	$\Theta(N)$	N
14	CPPB- REC-14	Sinh Xâu Nhị Phân Không 2 Số 1 Liền Kề	P4	Advanced	$\Theta(F_{N+2})$	$\Theta(N)$	N
15	CPPB- REC-15	Đếm Phân Hoạch Nguyên Của N Không Thứ Tự	P4	Extension	$\Theta(Exp)$	$\Theta(N)$	N
16	CPPB- REC-16	Đếm Cây Nhị Phân Có Thứ Tự (Catalan Rec)	P5	Extension	$\Theta(Catalan)$	$\Theta(N)$	N

<!-- Chuyên đề 11: Chia Để Trị | 383 dòng | 24,229 bytes -->

Chuyên đề 11: Kỹ thuật chia để trị

1. Cầu nối tư duy: Recurrence → recursion tree → complexity

Ở Chuyên đề 10, chúng ta đã làm chủ kỹ thuật Đệ quy: giải bài toán quy mô N bằng cách thu nhỏ dần bài toán. Từ cấu trúc code đệ quy, ta có thể thiết lập **Hệ thức truy hồi (Recurrence)** và phân tích qua **Cây đệ quy (Recursion Tree)** để tìm ra độ phức tạp chính xác:

Code Đệ Quy → Hệ Thức Truy Hồi (Recurrence) → Cây Đệ Quy (Recursion Tree) → Độ Phức Tạp (Complexity)

- Đệ quy tuyến tính (Chuyên đề 10):

$$T(N) = T(N-1) + O(1) \Rightarrow \text{Cây 1 nhánh thẳng, độ sâu } N \Rightarrow \Theta(N)$$

- Đệ quy phân nhánh chia đôi (Chuyên đề 11):

$$T(N) = 2T\left(\frac{N}{2}\right) + O(N) \Rightarrow \text{Cây nhị phân đầy đủ, chiều cao } \log_2 N \Rightarrow \Theta(N \log N)$$

- Đệ quy phân nhánh giảm 1 (Chuyên đề 10):

$$T(N) = 2T(N-1) + O(1) \Rightarrow \text{Cây nhị phân bùng nổ, } 2^N \text{ lá} \Rightarrow \Theta(2^N)$$

Quy luật cốt lõi: "Đệ quy" chỉ là cơ chế cài đặt; cấu trúc cây lời gọi và khối lượng công việc ở mỗi tầng mới là yếu tố quyết định độ phức tạp.

2. Phân biệt cốt lõi: Đệ quy (recursion) *ne* chia để trị (divide & conquer)

Học sinh rất dễ nhầm lẫn giữa hai khái niệm này:

Khái Niệm	Bản Chất	Ví dụ Điển Hình
Đệ Quy (Recursion)	Cơ chế thực thi: Kỹ thuật lập trình trong đó một hàm tự gọi lại chính nó thông qua Call Stack.	Tính $N!$, Fibonacci, Duyệt mảng tuần tự.
Chia Để Trị (D&C)	Chiến lược thiết kế thuật toán: Phân rã bài toán thành các bài toán con cùng bản chất nhưng nhỏ hơn, giải từng phần và gộp lại.	Merge Sort, Inversion Count, Closest Pair.

- Tính $N! = N \times (N-1)!$ là **Đệ quy** nhưng **không phải D&C** (vì chỉ thu nhỏ 1 phần tử mà không có bước chia và gộp cấu trúc).
- Merge Sort là **D&C hoàn chỉnh** sử dụng cơ chế Đệ quy để điều khiển.

3. Bản chất vấn đề & hai kiểu phân rã: Divide vs partition

Divide & Conquer là chiến lược phân rã một bài toán thành các bài toán con có cùng bản chất nhưng kích thước nhỏ hơn, giải các bài toán con, rồi kết hợp kết quả lại để thu được lời giải cho bài toán ban đầu.

Phân biệt hai kiểu phân rã dữ liệu:

1. Divide (Phân chia theo vị trí chỉ số):

- Chia cố định theo chỉ số mảng (thường là tại trung điểm $mid = l + (r - l) / 2$).
- Ví dụ: Merge Sort chia $[1\ 2\ 3\ 4\ 5\ 6\ 7\ 8]$ thành $[1\ 2\ 3\ 4]$ và $[5\ 6\ 7\ 8]$.

1. Partition (Phân hoạch theo quan hệ với Pivot):

- Chia động dựa trên việc so sánh các phần tử với một giá trị chốt ($pivot$), kích thước 2 nửa có thể không đều nhau.
- Ví dụ: QuickSelect phân hoạch $[7\ 2\ 9\ 1\ 5\ 3\ 8]$ với $pivot = 5$ thành $[2\ 1\ 3]$ (nhỏ hơn 5), $[5]$, và $[7\ 9\ 8]$ (lớn hơn 5).

![Mô hình Thuật toán Chia để trị (Divide & Conquer)](/Users/vu/Developer/ikhEdu_lessons/courses/cpp-bang-b/lessons/lesson-11-chia-de-tri/assets/dnc_model_vi.svg)

4. Khung tư duy d&c (the d&c mental model)

Trước bất kỳ bài toán nào nghi ngờ sử dụng Chia Để Trị, hãy luôn trả lời **4 câu hỏi định hướng**:

1. **Tôi chia bài toán ở đâu?** (Tại điểm giữa mid , theo trục tọa độ x , hay qua $pivot$?)
2. **Bài toán con có kích thước bao nhiêu?** ($N/2$, N_1 , N_2 ?)
3. **Tôi cần giải bao nhiêu bài toán con?** (Chỉ 1 nhánh như Binary Search/QuickSelect hay cả 2 nhánh như Merge Sort?)
4. **Tôi combine kết quả của các bài toán con như thế nào?** (Đây là bước quyết định độ phức tạp!)

![[Cây quyết định lựa chọn thuật toán Chia để trị]](/Users/vu/Developer/ikhEdu_lessons/courses/cpp-bang-b/lessons/lesson-11-chia-de-tri/assets/dnc_decision_tree_vi.svg)

5. Mô phỏng từng bước thuật toán sắp xếp trộn (merge sort simulation)

Xét mảng ban đầu: $A = [38, 27, 43, 3, 9, 82, 10]$.

Sơ đồ cây phân rã & gộp mảng (divide & merge tree):

![[Mô phỏng Cây phân rã và gộp Merge Sort]](/Users/vu/Developer/ikhEdu_lessons/courses/cpp-bang-b/lessons/lesson-11-chia-de-tri/assets/mergesort_tree_vi.svg)

6. Combine step & loop invariant — trái tim của divide & conquer

Chân lý sư phạm:** Sức mạnh và sự tinh tế của Chia Để Trị không nằm ở việc "bẻ nhỏ bài toán", mà nằm ở **cách ta xử lý và kết hợp (Combine) các kết quả ở pha Unwinding.

Bất biến vòng lặp (loop invariant) trong hàm `merge()` :

Khi gộp 2 mảng con đã sắp xếp $a[l..mid]$ và $a[mid+1..r]$ vào $temp$:

- **Bất biến:** Tại mỗi bước lặp, mảng đệm $temp[l..k-1]$ luôn chứa chính xác $(k-l)$ phần tử nhỏ nhất đã được chọn từ hai nửa $a[l..mid]$ và $a[mid+1..r]$ theo thứ tự không giảm.
- **Quy tắc chọn:**
 - Nếu $a[i] \leq a[j]$: Chọn $a[i]$ đưa vào $temp[k]$, tăng i và k (điều kiện $\leq$ bảo toàn tính **Stable Sort**).
 - Nếu $a[i] > a[j]$: Chọn $a[j]$ đưa vào $temp[k]$, tăng j và k .

7. Bảng trực giác recurrence & định lý thợ (master theorem)

Trước khi dùng công thức tổng quát, hãy nắm vững 4 hệ thức truy hồi kinh điển:

Hệ Thức Truy Hồi	Cấu Trúc Nhánh	Chi Phí Từng Tầng	Độ Phức Tạp	Thuật Toán Tiêu Biểu
$T(N) = T(N/2) + O(1)$	1 nhánh, giảm nửa	$O(1)$ mỗi tầng	$\Theta(\log N)$	Binary Search
$T(N) = T(N/2) + O(N)$	1 nhánh, quét N	$N + N/2 + N/4 + \dots$	$\Theta(N)$ (Expected)	QuickSelect
$T(N) = 2T(N/2) + O(1)$	2 nhánh, gộp $O(1)$	Số lá N thống trị	$\Theta(N)$	Tìm Min/Max chia đôi
$T(N) = 2T(N/2) + O(N)$	2 nhánh, gộp $O(N)$	Mỗi tầng đều tốn $O(N)$	$\Theta(N \log N)$	Merge Sort, Inversion

Công thức tổng quát (master theorem):

Với $T(N) = a \cdot T(N/b) + \Theta(N^d)$ ($a \geq 1, b > 1$):

- $a < b^d$ ($\log_b a < d$):** Chi phí ngoài đệ quy thống trị $\Rightarrow T(N) = \Theta(N^d)$.
- $a = b^d$ ($\log_b a = d$):** Chi phí phân bố đều trên $\log_b N$ tầng $\Rightarrow T(N) = \Theta(N^d \log N)$.
- $a > b^d$ ($\log_b a > d$):** Số nút lá bùng nổ thống trị $\Rightarrow T(N) = \Theta(N \log_b a)$ (ví dụ Karatsuba $a=3, b=2, d=1 \Rightarrow \Theta(N \log_2^3) \approx \Theta(N^{1.585})$).

8. Bảng nhận diện dấu hiệu thuật toán (d&c pattern recognition)

Dấu Hiệu Đặc Trưng	Mô Hình Thuật Toán D&C Phù Hợp
Mảng đã sắp xếp + cần tìm kiếm một giá trị	Binary Search đệ quy (D&C đơn nhánh)
Chia đôi mảng + cần sắp xếp cả 2 nửa	Merge Sort (D&C đa nhánh + Combine 2 con trỏ)
Cần đếm số cặp phần tử có quan hệ giữa 2 nửa	Inversion Counting (Đếm khi Merge)
Cần tìm phần tử nhỏ thứ K trên mảng chưa sắp xếp	QuickSelect (Partition chọn 1 nhánh)
Tìm max / min trong mô hình cây thi đấu đấu loại	Tournament Tree (Lưu lịch sử đối đầu)
Tìm đoạn con liên tiếp có tính chất tối ưu	Maximum Subarray D&C (Xét đoạn vắt qua mid)
Tập điểm trong không gian 2D cần tìm khoảng cách min	Closest Pair of Points (Chia hoành độ + Quét Strip)
Tìm trung vị của 2 dãy đã sắp xếp	Binary Partition D&C ($O(\log(\min))$)

9. Chuỗi chuyển giao tri thức: Merge → merge sort → inversion counting

Điểm đặc sắc nhất của chuyên đề là **chuỗi kế thừa thuật toán**:

Merge Step (2 con trỏ) → Merge Sort $\Theta(N \log N)$ → Đếm Nghịch Thế (Inversion Count)

Ứng dụng đỉnh cao: Đếm cặp nghịch thế (inversion counting) $O(N \log N)$

- **Khái niệm:** Cặp nghịch thế là cặp chỉ số (i, j) sao cho $i < j$ nhưng $A_i > A_j$.
- **Bất biến gộp kỳ diệu:** Khi chia mảng thành $Left[l..mid]$ và $Right[mid+1..r]$ đã sắp xếp:
- Khi duyệt con trỏ i trên $Left$ và j trên $Right$, nếu $L[i] > R[j]$, thì do L đã sắp xếp tăng dần, **toàn bộ các phần tử từ $L[i]$ đến $L[mid]$ đều lớn hơn $R[j]$ **!
- Ta cộng ngay một lượng bằng $(mid - i + 1)$ vào biến đếm nghịch thế trong $O(1)$ thao tác cộng dồn**.
- Bước *merge* vẫn tốn $O(N)$ thời gian, giúp tổng thời gian đếm toàn bộ mảng đạt $\Theta(N \log N)$ chuẩn thi đấu thay vì $O(N^2)$ vét cạn.

10. Các bẫy lỗi lập trình kinh điển (bug traps)

1. Bẫy cấp phát bộ nhớ đệm trong Merge Sort:

- Việc tạo các **vector** mới bên trong mỗi lần gọi **mergeSort()** gây thêm nhiều lần cấp phát/giải phóng động và sao chép dữ liệu, làm tăng constant factor.
- **Cách phòng chống:** Khai báo một mảng đệm toàn cục duy nhất **vector<long long> temp(N);** và truyền tham chiếu vào hàm đệ quy để tái sử dụng cho mọi bước gộp, giữ Auxiliary Memory ở mức $\Theta(N)$.

1. Tràn số nguyên 32-bit khi đếm số cặp nghịch thế:

- Số lượng cặp nghịch thế tối đa của mảng $N = 10^5$ là $N(N-1)/2 \approx 5 \times 10^9$ (vượt giới hạn của **int** 2×10^9).
- **Cách phòng chống:** Biến đếm nghịch thế bắt buộc dùng kiểu **long long**.

1. Bẫy tràn số khi tính Midpoint:

- Viết **mid = (l + r) / 2;** có thể tràn **int** khi $l + r > 2 \cdot 10^9$. Luôn viết:
mid = l + (r - l) / 2;

1. Bẫy bỏ sót đoạn crossing trong Maximum Subarray:

- Đoạn con lớn nhất có thể nằm trọn bên trái, trọn bên phải, hoặc vắt ngang qua tâm **mid**. Bắt buộc phải tính **maxCrossingSum** từ **mid** lan sang 2 phía.

11. Mẫu cài đặt chuẩn thi đấu (competitive templates)

```

#include <bits/stdc++.h>
using namespace std;

using ll = long long;

// 1. Thuật toán Merge Sort chuẩn  $O(N \log N)$  dùng buffer tái sử dụng
void merge(vector<ll> &a, vector<ll> &temp, int l, int mid, int r) {
    int i = l, j = mid + 1, k = l;
    while (i ≤ mid && j ≤ r) {
        // ≤ giữ tính stable: phần tử bên trái thắng khi bằng nhau
        if (a[i] ≤ a[j]) temp[k++] = a[i++];
        else temp[k++] = a[j++];
    }
    while (i ≤ mid) temp[k++] = a[i++];
    while (j ≤ r) temp[k++] = a[j++];
    for (int idx = l; idx ≤ r; ++idx) a[idx] = temp[idx];
}

void mergeSort(vector<ll> &a, vector<ll> &temp, int l, int r) {
    if (l ≥ r) return;
    int mid = l + (r - l) / 2;
    mergeSort(a, temp, l, mid);
    mergeSort(a, temp, mid + 1, r);
    merge(a, temp, l, mid, r);
}

// 2. Thuật toán Đếm Số Cặp Nghịch Thê  $O(N \log N)$ 
ll countInversions(vector<ll> &a, vector<ll> &temp, int l, int r) {
    if (l ≥ r) return 0;
    int mid = l + (r - l) / 2;
    ll inv = 0;
    inv += countInversions(a, temp, l, mid);
    inv += countInversions(a, temp, mid + 1, r);

    int i = l, j = mid + 1, k = l;
    while (i ≤ mid && j ≤ r) {
        if (a[i] ≤ a[j]) {
            temp[k++] = a[i++];
        } else {
            temp[k++] = a[j++];
            inv += (mid - i + 1); // Đếm  $O(1)$  nhờ cấu trúc đã sắp xếp
        }
    }
    while (i ≤ mid) temp[k++] = a[i++];
    while (j ≤ r) temp[k++] = a[j++];
    for (int idx = l; idx ≤ r; ++idx) a[idx] = temp[idx];
    return inv;
}

```



```
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    vector<ll> a = {38, 27, 43, 3, 9, 82, 10};

    int n = a.size();
    vector<ll> temp(n);

    cout << "Số cặp nghịch thế: " << countInversions(a, temp, 0, n - 1) << "\n";
    return 0;
}
```

Câu hỏi trắc nghiệm củng cố khái niệm

Câu 1 (bản chất 3 giai đoạn chia để trị):

Ba bước cơ bản trong một giải thuật Chia để trị (Divide and Conquer) diễn ra theo thứ tự nào sau đây?

- A. *Solve* → *Divide* → *Combine*.
- B. **[Đáp án đúng]** *Divide* (Chia bài toán) → *Solve* (Trị / Giải đệ quy các bài toán con) → *Combine* (Gộp kết quả).
- C. *Combine* → *Divide* → *Solve*.
- D. *Divide* → *Combine* → *Solve*.

Giải thích: Giải thuật D&C trước hết chia bài toán lớn thành các phần độc lập, giải đệ quy từng phần, sau đó gộp kết quả lại ở pha Unwinding.

Câu 2 (suy luận hệ thức truy hồi từ cấu trúc code):

Quan sát đoạn mã đệ quy sau:

```
void process(int n) {
    if (n ≤ 1) return;
    process(n / 2);
    process(n / 2);
    for (int i = 0; i < n; i++) {
        // Thao tác xử lý tốn O(1)
    }
}
```

Hệ thức truy hồi (Recurrence) mô tả chính xác thời gian thực thi $T(n)$ của hàm trên là:

- A. $T(n) = T(n/2) + O(n)$
- B. **[Đáp án đúng]** $T(n) = 2T(n/2) + O(n)$
- C. $T(n) = T(n-1) + O(n)$
- D. $T(n) = 2T(n-1) + O(1)$

*Giải thích: Hàm tạo ra 2 lời gọi đệ quy kích thước $n/2$ và một vòng lặp **for** chạy n lần tốn chi phí ngoài đệ quy $f(n) = O(n)$. Theo Master Theorem, $T(n) = \Theta(n \log n)$.*

Câu 3 (so sánh bản chất: Cùng chia đôi nhưng khác biệt độ phức tạp):

Hai thuật toán A ($T(N) = T(N/2) + O(1)$) và B ($T(N) = 2T(N/2) + O(N)$) đều chia đôi mảng ở mỗi bước. Lý do cốt lõi khiến thuật toán A đạt $O(\log N)$ trong khi B tốn $O(N \log N)$ là gì?

- A. Thuật toán A không dùng ngôn ngữ C++.
- B. **[Đáp án đúng]** Thuật toán A chỉ đi vào 1 nhánh duy nhất với chi phí mỗi tầng $O(1)$, trong khi thuật toán B bắt buộc phải giải cả 2 nhánh và tốn chi phí gộp $O(N)$ trên mỗi tầng trong tổng số $\log_2 N$ tầng.
- C. Thuật toán B tiêu tốn nhiều bộ nhớ RAM hơn.
- D. Thuật toán A chỉ chạy trên số nguyên dương.

Giải thích: Số lượng nhánh đệ quy được khám phá và chi phí gộp ngoài đệ quy quyết định toàn bộ sự khác biệt giữa $O(\log N)$ và $O(N \log N)$.

Câu 4 (kiểu dữ liệu cho đếm cặp nghịch thế):

Với mảng có $N = 10^5$ phần tử, biến lưu trữ tổng số cặp nghịch thế bắt buộc phải có kiểu dữ liệu nào để chống tràn số?

- A. `int`
- B. `float`
- C. **[Đáp án đúng]** `long long` (vì số cặp nghịch thế tối đa lên tới $N(N-1)/2 \approx 5 \times 10^9$, vượt quá giới hạn 32-bit).
- D. `bool`

Giải thích: Mảng giảm dần hoàn toàn có số cặp nghịch thế bằng $N(N-1)/2$, vượt ngưỡng 2×10^9 của `int` 32-bit.

Câu 5 (cơ chế đếm cặp nghịch thế khi merge):

Trong thuật toán đếm số cặp nghịch thế bằng Merge Sort, khi con trỏ i trở vào nửa trái $Left[l..mid]$ và con trỏ j trở vào nửa phải $Right[mid+1..r]$, nếu $Left[i] > Right[j]$, số lượng cặp nghịch thế được cộng thêm vào kết quả trong $O(1)$ là bao nhiêu?

- A. Đúng 1 cặp.
- B. **[Đáp án đúng]** $mid - i + 1$ cặp.
- C. $j - mid$ cặp.
- D. $r - l + 1$ cặp.

Giải thích: Vì mảng con $Left$ đã được sắp xếp tăng dần, nên nếu $Left[i] > Right[j]$ thì tất cả các phần tử từ chỉ số i đến mid trong mảng $Left$ đều lớn hơn $Right[j]$.

Câu 6 (bẫy maximum subarray d&c):

Khi tìm đoạn con có tổng lớn nhất bằng Chia để trị trên đoạn $[l, r]$, ngoài đoạn con lớn nhất nằm trọn ở nửa trái và trọn ở nửa phải, ta bắt buộc phải xem xét thêm trường hợp nào?

- A. Đoạn con rỗng.
- B. **[Đáp án đúng]** Đoạn con lớn nhất bắt đầu từ nửa trái kéo dài qua tâm mid sang nửa phải (Crossing Subarray).
- C. Toàn bộ mảng ban đầu.
- D. Phần tử nhỏ nhất trong mảng.

Giải thích: Đoạn con tối ưu có thể vắt ngang qua ranh giới phân chia giữa hai nửa mảng.

Câu 7 (tối ưu bộ nhớ trong merge sort):

Để tối ưu thời gian thực thi và tránh overhead cấp phát bộ nhớ động trong hàm `mergeSort()`, kỹ thuật cài đặt chuẩn thi đấu là gì?

- A. Khai báo `vector<long long>` mới trong mỗi lần gọi hàm `merge()`.
- B. **[Đáp án đúng]** Khai báo một mảng đệm tạm duy nhất `vector<long long> temp(N)` và truyền tham chiếu vào hàm đệ quy để tái sử dụng cho mọi bước gộp.
- C. Dùng vòng lặp `while(true)`.
- D. Ép kiểu toàn bộ mảng sang chuỗi ký tự.

Giải thích: Tái sử dụng một vùng nhớ đệm duy nhất giúp tránh việc cấp phát/giải phóng nhiều buffer trong quá trình đệ quy, giảm overhead và giữ auxiliary memory ở mức $\Theta(N)$.

Câu 8 (tournament tree tìm phần tử lớn thứ hai với $N = 2^k$):

Trên mảng có kích thước $N = 2^k$ (N là lũy thừa của 2), bằng kỹ thuật Tournament Tree (cây thi đấu), số phép so sánh tối thiểu để tìm ra phần tử lớn thứ hai là:

- A. $2N$
- B. **[Đáp án đúng]** $N + \log_2 N - 2$ phép so sánh.
- C. N^2
- D. $N \log N$

Giải thích: Tìm nhà vô địch tốn $N - 1$ phép so sánh. Phần tử lớn thứ hai bắt buộc phải là một trong những phần tử từng thua trực tiếp nhà vô địch trong cây thi đấu (đúng $\log_2 N$ phần tử). Tìm max trong nhóm này tốn thêm $\log_2 N - 1$ phép $\Rightarrow$ Tổng cộng đúng $(N - 1) + (\log_2 N - 1) = N + \log_2 N - 2$ phép.

Câu 9 (đặc tính stable sort của merge sort):

Merge Sort được gọi là thuật toán sắp xếp ổn định (Stable Sort) vì lý do nào sau đây?

- A. Thuật toán chạy không bao giờ bị lỗi bộ nhớ.
- B. **[Đáp án đúng]** Trong bước gộp `merge()`, khi hai phần tử có giá trị bằng nhau ($a[i] == a[j]$), thuật toán luôn ưu tiên chọn phần tử ở nửa trái (i) trước nhờ điều kiện $a[i] \leq a[j]$, giữ nguyên thứ tự xuất hiện ban đầu.
- C. Thuật toán có độ phức tạp như nhau trong mọi trường hợp.
- D. Thuật toán không sử dụng phép nhân.

Giải thích: Bất biến chọn phần tử bên trái khi bằng nhau bảo toàn tính thứ tự tương đối của các phần tử có khóa bằng nhau.

Câu 10 (lũy thừa ma trận chia để trị):

Tính lũy thừa ma trận vuông A^N cấp 2×2 modulo M bằng Chia để trị có độ phức tạp thời gian tiệm cận là:

- A. $\Theta(N)$
- B. $\Theta(N^2)$
- C. **[Đáp án đúng]** $\Theta(\log N)$ (mỗi phép nhân ma trận 2×2 tốn $O(1)$ với 8 phép nhân số học).
- D. $\Theta(1)$

Giải thích: Thuật toán chia đôi số mũ $N \rightarrow N/2$ sau mỗi bước tương tự như lũy thừa nhị phân số học, độ sâu đệ quy là $\log_2 N$.

Ma trận bài tập thực hành (p0 $\rightarrow$ p5)

Phân tầng lộ trình học tập lesson 11:

- ****Nhóm Cốt Lõi (Core Foundations - CPPB-DAC-01 $\rightarrow$ 12):**** Binary Search D&C, RMQ D&C, Tournament Tree, Cài đặt Merge Step, Merge Sort trọn vẹn, Đếm số cặp nghịch thế, Maximum Subarray D&C, Majority Element Voting, Lũy thừa ma trận 2×2 , Đỉnh mảng Unimodal, Tổng cấp số nhân D&C, Đếm cặp $A_i > 2A_j$.
- ****Nhóm Nâng Cao & Thử Thách (Advanced & Challenge CPPB-DAC-13 $\rightarrow$ 16):**** QuickSelect D&C ($O(N)$ expected), Đếm đoạn con tổng nằm trong $[L, R]$, Cặp điểm gần nhất trong mặt phẳng (Closest Pair $O(N \log N)$), Median của 2 mảng đã sắp xếp ($O(\log(\min(N, M)))$).

STT	Mã Bài	Tên Bài Toán	Cấp Độ	Phân Loại	Time Complexity	Call Stack	Aux Memory	Max Depth
01	CPPB-DAC-01	Tìm Kiếm Nhị Phân Bằng Độ Quy (Cầu Nối D&C)	P0	Core	$\Theta(\log N)$	$\Theta(\log N)$	$O(1)$	$\log_2 N$
02	CPPB-DAC-02	Tìm Min Trên Đoạn Bằng Chia Để Trị (RMQ D&C)	P0	Core	$\Theta(N)$	$\Theta(\log N)$	$O(1)$	$\log_2 N$
03	CPPB-DAC-03	Tìm Phần Tử Lớn Thứ Hai (Tournament Tree)	P1	Core	$\Theta(N)$	$\Theta(\log N)$	$\Theta(\log N)$	$\log_2 N$
04	CPPB-DAC-04	Gộp Hai Mảng Đã Sắp Xếp (Merge Step)	P1	Core	$\Theta(N + M)$	$O(1)$	$O(1)$	1
05	CPPB-DAC-05	Thuật Toán Sắp Xếp Trộn (Merge Sort)	P2	Core	$\Theta(N \log N)$	$\Theta(\log N)$	$\Theta(N)$	$\log_2 N$
06	CPPB-DAC-06	Đếm Số Cặp Nghịch Thế (Inversion Count)	P2	Core	$\Theta(N \log N)$	$\Theta(\log N)$	$\Theta(N)$	$\log_2 N$
07	CPPB-DAC-07	Đoạn Con Tổng Lớn Nhất (Maximum Subarray)	P2	Core	$\Theta(N \log N)$	$\Theta(\log N)$	$O(1)$	$\log_2 N$
08	CPPB-DAC-08	Tìm Phần Tử Đa Số (Majority Element) D&C	P2	Core	$\Theta(N \log N)$	$\Theta(\log N)$	$O(1)$	$\log_2 N$
09	CPPB-DAC-09	**Lũy Thừa Ma Trận Chia Để Trị 2×2 **	P3	Core	$\Theta(\log N)$	$\Theta(\log N)$	$O(1)$	$\log_2 N$
10	CPPB-DAC-10	Tìm Điểm Cực Đại Mảng Unimodal (Peak Index)	P3	Core	$\Theta(\log N)$	$\Theta(\log N)$	$O(1)$	$\log_2 N$
11	CPPB-DAC-11	Tính Tổng Cấp Số Nhân D&C	P3	Core	$\Theta(\log N)$	$\Theta(\log N)$	$O(1)$	$\log_2 N$
12	CPPB-DAC-12	**Đếm Số Cặp $A_i > A_j$ (Significant Inversions)**	P3	Core	$\Theta(N \log N)$	$\Theta(\log N)$	$\Theta(N)$	$\log_2 N$

13	CPPB- DAC-13	Thuật Toán QuickSelect Tìm K-th Element	P4	Advanced	$\Theta(N)$ exp / $\Theta(N^2)$ worst	$\Theta(\log N)$ exp / $\Theta(N)$ worst	$O(1)$	$\log^2 N$ exp / N worst
14	CPPB- DAC-14	**Đếm Số Đoạn Con Tổng Trong Đoạn $[L, R]$ **	P4	Advanced	$\Theta(N \log N)$	$\Theta(\log N)$	$\Theta(N)$	$\log^2 N$
15	CPPB- DAC-15	Cặp Điểm Gần Nhất (Closest Pair of Points)	P4	Challenge	$\Theta(N \log N)$	$\Theta(\log N)$	$\Theta(N)$	$\log^2 N$
16	CPPB- DAC-16	Median Của Hai Mảng Đã Sắp Xếp	P5	Challenge	$\Theta(\log(\min))$	$\Theta(\log(\min))$	$O(1)$	$\log^2(\min)$

<!-- Chuyên đề 12: Quay Lui & Nhánh Cận | 399 dòng | 27,304 bytes -->

Chuyên đề 12: Thuật toán quay lui & nhánh cận

1. Cầu nối kiến trúc: Recursion → divide & conquer → state-space search → dynamic programming

Để có cái nhìn toàn cảnh về các phương pháp giải thuật lớn trong Lập trình thi đấu:

! [Cầu nối kiến trúc các phương pháp thuật toán lớn: Đệ quy -> D&C / Quay lui / Nhánh cận -> Quy hoạch động] (/Users/vu/Developer/ikhEdu_lessons/courses/cpp-bang-b/lessons/lesson-12-quay-lui-nhanh-can/assets/search_paradigms_bridge_vi.svg)

- **Divide & Conquer:** Bài toán lớn → Các bài toán con riêng biệt .
- **Backtracking / State-Space Search:** Trạng thái hiện tại → Các nhánh quyết định thử nghiệm (Choices) .
- **Dynamic Programming:** Nhiều đường đi khác nhau → Cùng một State Identity (Overlapping States) ⇒ Memoization / Bảng DP .

2. Bản chất trạng thái (state definition & state identity)

Khái niệm state (trạng thái) & state identity:

Định nghĩa: State (Trạng thái) là tập thông tin tối thiểu cần thiết để xác định chính xác các lựa chọn tiếp theo và kết quả có thể đạt được từ trạng thái hiện tại. * **Không phải mọi biến xuất hiện trong hàm đệ quy đều là thành phần của State Identity; chỉ những thông tin có thể làm thay đổi các lựa chọn hoặc kết quả của phần còn lại mới cần thiết.**

Trong cài đặt DFS / Quay lui: State bao gồm cả dữ liệu cấu hình đang xây dựng và các đại lượng tích lũy (`current_value` , `current_cost`). **Khi chuyển sang Quy Hoạch Động (DP):** Ta chắt lọc những biến thực sự tạo nên "**State Identity**" (ví dụ: `dp[index][remaining_weight]` hoặc `dp[city][mask]`), còn giá trị mục tiêu trở thành giá trị lưu trong bảng DP thay vì là tham số đệ quy.

Bài Toán	State Trong Cài Đặt DFS	State Identity Khi Chuyển Sang DP (Nếu có Memoization/DP)
Sinh Hoán Vị	<code>(step, visited[], cur[])</code>	Thường không dùng DP kiểu thông thường (trừ Bitmask DP về sau)
N-Queens	<code>(row, col_used[], diag1[], diag2[])</code>	Không tự động trở thành DP chỉ vì có State (thiếu cấu trúc con tối ưu)
Subset Sum	<code>(index, current_sum, cur_set[])</code>	<code>dp[index][current_sum]</code>
Cái Túi 0/1 (Knapsack)	<code>(index, current_weight, current_value)</code>	<code>dp[index][remaining_weight]</code>
Người Du Lịch (TSP)	<code>(current_city, visited_mask, current_cost)</code>	<code>dp[current_city][visited_mask]</code>
Sudoku 9x9	<code>(board[9][9], empty_cells_list)</code>	Không phải ví dụ DP điển hình (CSP Backtracking)

Quy luật cốt lõi: Không phải cứ có State là có thể chuyển sang DP. Để chuyển sang DP, bài toán bắt buộc phải có **State Identity gọn gàng** + **Hiện tượng trùng lặp trạng thái (Overlapping Subproblems)** + **Cấu trúc con tối ưu (Optimal Substructure)**.

3. Khung phương pháp luận: Design-time framework vs runtime pattern

1. Khung thiết kế thuật toán (design-time framework):

- Define State:** Xác định các biến trạng thái tối thiểu cần thiết để mô tả bài toán.
- Generate Candidates:** Xác định danh sách các lựa chọn khả dĩ tại mỗi bước đi.
- Define Feasibility:** Thiết lập điều kiện ràng buộc hợp lệ (Feasibility Pruning).
- Define Bound:** Thiết lập hàm cận dưới `LB` hoặc cận trên `UB` nếu là bài toán tối ưu (Branch & Bound).
- Define Transition & Restoration:** Thiết lập quy tắc chuyển trạng thái (`Choose`), gọi đệ quy (`Explore`) và hoàn tác (`Unchoose`).

2. Khung thực thi mã nguồn (runtime pattern):

```
void search(State state) {
    if (isGoal(state)) {
        processSolution(state);
        return;
    }
    for (const auto &candidate : getCandidates(state)) {
        if (!isFeasible(state, candidate)) continue; // Feasibility Pruning

        if (boundSaysImpossible(state, candidate)) continue; // Optimality
        Pruning (B&B)

        choose(state, candidate); // 1. Chuyển sang State_new
        search(state);           // 2. Đi sâu vào nhánh con (Explore)
        unchoose(state, candidate); // 3. Hoàn tác về State_before (Restoration)
    }
}
```

4. Khung tư duy mental model: Hai sơ đồ cốt lõi của lesson 12

![[Cây tìm kiếm không gian trạng thái: Quay lui và Nhánh cận]](/Users/vu/Developer/ikhEdu_lessons/courses/cpp-bang-b/lessons/lesson-12-quay-lui-nhanh-can/assets/state_space_tree_vi.svg)

Quy trình 1: Luồng ra quyết định quay lui thuần túy (backtracking)

Quy trình 2: Luồng ra quyết định nhánh cận (branch & bound)

5. Bất biến trung tâm: State restoration invariant

Quy luật cốt lõi: *Choose-Explore-Unchoose* là một pattern cài đặt phổ biến. Bản chất kỹ thuật sâu sắc là **Bất biến Khôi phục Trạng Thái (State Restoration Invariant)**:

$State_{before} \xrightarrow{\text{Choose}} State_{new} \xrightarrow{\text{Explore}} Subtree \xrightarrow{\text{Unchoose}} State_{before}$
 Sau khi khám phá xong một nhánh con và hàm con return, trạng thái phải được trả về **nguyên vẹn 100%** như trước khi bước vào nhánh đó, đảm bảo nhánh kế tiếp bắt đầu từ cùng một trạng thái cha.

6. Phân biệt cắt tỉa ràng buộc (feasibility) vs cắt tỉa tối ưu (branch & bound)

- **BACKTRACKING:** "Xây dựng nghiệm từng bước + quay lui khi cần (có thể không cần pruning như sinh nhị phân)"
- **FEASIBILITY PRUNING:** "Cắt những trạng thái chắc chắn không thể dẫn tới nghiệm hợp lệ"
- **BRANCH AND BOUND:** "Framework tìm kiếm tối ưu trên không gian trạng thái, kết hợp hàm Cận (Bound) để cắt tỉa nhánh không thể tốt hơn best hiện tại"

Lưu ý mở rộng: Trong chuyên đề này, ta triển khai Branch & Bound trên nền DFS / Backtracking để nắm vững nguyên lý. Về tổng quát, Branch & Bound là một framework tìm kiếm tối ưu có thể triển khai bằng Best-First Search với hàng đợi ưu tiên `priority_queue` hoặc BFS.

Định nghĩa chuẩn xác: $OPT(state)$ Vs $best$ hiện tại:

- **$OPT(state)$:** Giá trị tốt nhất thực sự có thể đạt được khi hoàn thành nghiệm từ trạng thái hiện tại.
- **$best$ (hoặc $best_so_far$ / $incumbent$):** Nghiệm tốt nhất đã tìm thấy trên toàn bộ các nhánh đã khám phá tính đến thời điểm hiện tại (chưa chắc là nghiệm tối ưu toàn cục cho đến khi duyệt xong).

Nguyên tắc thiết lập hàm bound chuẩn xác:

1. Với bài toán Cực Tiểu Hóa (Minimization - ví dụ TSP, Đổi tiền ít xu nhất, Job Assignment):
 - Ta duy trì hàm Cận Dưới $LB(state) \leq OPT(state)$.
 - **Điều kiện cắt tỉa:** Nếu $LB(state) \geq best$, thì $OPT(state) \geq LB(state) \geq best \Rightarrow$ **Cắt tỉa ngay!**
 - Ví dụ: $best = 100$. Nếu tại một nhánh ta tính được $LB = 105 \Rightarrow$ Cắt tỉa ngay vì $OPT \geq 105 > 100$. Nếu $LB = 95 \Rightarrow$ **Không được cắt tỉa** vì OPT có thể là 95, 98 tốt hơn 100.
1. **Với bài toán Cực Đại Hóa (Maximization - ví dụ Cái túi Knapsack 0/1):**
 - Ta duy trì hàm Cận Trên $UB(state) \geq OPT(state)$.
 - **Điều kiện cắt tỉa:** Nếu $UB(state) \leq best$, thì $OPT(state) \leq UB(state) \leq best \Rightarrow$ **Cắt tỉa ngay!**

Mối liên hệ giữa Heuristic Ordering & Branch and Bound: Heuristic ordering giúp tìm ra nghiệm tốt sớm hơn $\Rightarrow best$ được cải thiện nhanh hơn $\Rightarrow$ Hàm Bound cắt tỉa được nhiều nhánh hơn $\Rightarrow$ Thuật toán B&B chạy nhanh hơn vượt trội!

7. Phân loại 4 cấp độ kỹ thuật trong tìm kiếm toàn vẹn

Kỹ Thuật	Ảnh Hưởng Đến Tính Đúng Đắn	Vai Trò Thuật Toán
Feasibility Pruning Hợp Lệ	Không mất nghiệm hợp lệ	Loại bỏ trạng thái chắc chắn vi phạm ràng buộc bài toán.
Valid Lower / Upper Bound	Không mất nghiệm tối ưu	Loại bỏ trạng thái đã chứng minh toán học không thể vượt qua best .
Heuristic Ordering	Không làm mất nghiệm	Sắp xếp thứ tự thử nhánh (như Warnsdorff) để tìm thấy nghiệm tốt sớm hơn; tính đầy đủ vẫn bảo toàn nếu duyệt hết.
Heuristic Pruning không chứng minh	Có nguy cơ mất nghiệm	Cắt nhánh theo cảm tính, có nguy cơ bỏ sót nghiệm tối ưu toàn cục.

8. Cầu nối sâu sang dp: Từ cây tìm kiếm (search tree) đến đồ thị trạng thái (state dag)

! [Từ Cây tìm kiếm Search Tree đến Đồ thị trạng thái State DAG] (/Users/vu/Developer/ikhEdu_lessons/courses/cpp-bang-b/lessons/lesson-12-quay-lui-nhanh-can/assets/state_dag_overlapping_vi.svg)

- **Duyệt cây thuần túy (Tree Search):** Phải tính toán lại trạng thái **E** nhiều lần ở các nhánh con khác nhau.
- **Quan điểm Đồ thị (State DAG View):** **E** chỉ là một đỉnh duy nhất trong không gian trạng thái.
- **Quy Hoạch Động (Dynamic Programming / Memoization):** Trong những bài toán mà State Identity có số lượng trạng thái đa thức theo kích thước input, Memoization/DP có thể giảm một cây tìm kiếm hàm mũ xuống *Số trạng thái* $\times$ *Chi phí chuyển trạng thái*; ví dụ Knapsack đạt $O(N \cdot W)$ khi W là tham số giới hạn. Với các bài như TSP, Bitmask DP đạt $O(N^2 \cdot 2^N)$ nhanh hơn rất nhiều so với vét cạn $N!$.

9. Các bẫy lỗi lập trình kinh điển (bug traps)

1. Quên hoàn tác trạng thái (Missing Unchoose Step):

- Sau khi gọi đệ quy `backtrack(i + 1)`, quên viết `visited[val] = false;` hoặc `cur_sum -= val;` $\Rightarrow$ Trạng thái của nhánh trước bị rò rỉ sang nhánh sau, làm mất toàn bộ các nghiệm tiếp theo.

1. Bẫy cắt tỉa `break` trong Subset Sum có số âm:

- Lệnh `if (current_sum + A[i] > S) break;` chỉ an toàn khi **mọi phần tử $A_i > 0$ ** và mảng đã sort tăng dần. Tuyệt đối không áp dụng nguyên trạng cho mảng có phần tử âm!

1. **Đánh dấu sai mảng các họ đường chéo trong bài N -Queens:**

- Với chỉ số 1-based:
- Họ đường chéo xuôi $\backslash$: $row - col \in [-(N-1), N-1] \Rightarrow row - col + N \in [1, 2N-1]$.
- Họ đường chéo ngược $/$: $row + col \in [2, 2N]$.
- Khai báo `diag1` và `diag2` tối thiểu kích thước $2N + 1$.

1. Giả thiết về mệnh giá xu trong Coin Change:

- Mọi mệnh giá xu $C_i \geq 1$ để đảm bảo độ sâu tối đa bị chặn trên bởi $\lfloor \text{floor } S / C_{\min} \rfloor$.

10. Mẫu cài đặt chuẩn thi đấu (competitive templates)

```

#include <bits/stdc++.h>
using namespace std;

using ll = long long;

// 1. Sinh Hoán Vị 1..N chuẩn State Restoration Invariant
int n = 3;
vector<int> cur;

vector<bool> visited;

void genPermutations(int step) {
    if (step > n) {
        for (int i = 0; i < n; ++i) cout << cur[i] << (i + 1 == n ? "" : " ");
        cout << "\n";
        return;
    }
    for (int val = 1; val ≤ n; ++val) {
        if (!visited[val]) {
            visited[val] = true;      // 1. CHOOSE
            cur.push_back(val);
            genPermutations(step + 1); // 2. EXPLORE
            cur.pop_back();           // 3. UNCHOOSE (Khôi phục)
            visited[val] = false;
        }
    }
}

// 2. Bài Toán N-Queens (Đếm số cách đặt N quân hậu)
int n_queens = 4;
ll queen_ways = 0;
vector<bool> col_used, diag1_used, diag2_used;

void solveNQueens(int row) {
    if (row > n_queens) {
        queen_ways++;
        return;
    }
    for (int col = 1; col ≤ n_queens; ++col) {
        if (!col_used[col] && !diag1_used[row - col + n_queens] && !
diag2_used[row + col]) {
            col_used[col] = diag1_used[row - col + n_queens] = diag2_used[row +
col] = true; // CHOOSE
            solveNQueens(row + 1); // EXPLORE
            col_used[col] = diag1_used[row - col + n_queens] = diag2_used[row +
col] = false; // UNCHOOSE
        }
    }
}

```

```

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    visited.assign(n + 1, false);
    genPermutations(1);

    col_used.assign(n_queens + 1, false);
    diag1_used.assign(2 * n_queens + 1, false);
    diag2_used.assign(2 * n_queens + 1, false);
    solveNQueens(1);
    cout << "Số cách đặt " << n_queens << " quân hậu: " << queen_ways << "\n";
    return 0;
}

```

Câu hỏi trắc nghiệm củng cố khái niệm

Câu 1 (bản chất 3 bước quay lui):

Thứ tự thực hiện chuẩn mực trong thân vòng lặp của một hàm quay lui (Backtracking) là:

- A. *Explore* → *Choose* → *Unchoose*.
- B. **[Đáp án đúng]** *Choose* (Thử và đánh dấu trạng thái) → *Explore* (Gọi đệ quy đi sâu) → *Unchoose* (Hoàn tác trạng thái sau khi đệ quy return).
- C. *Unchoose* → *Choose* → *Explore*.
- D. *Choose* → *Unchoose* → *Explore*.

Giải thích: Quy trình chuẩn là chọn thử một ứng viên hợp lệ, đệ quy khám phá cây con, sau đó bắt buộc phải hoàn tác khi hàm đệ quy return để thử ứng viên tiếp theo.

Câu 2 (phân biệt *feasibility* vs *optimality pruning*):

Sự khác biệt cốt lõi giữa Cắt tỉa tính khả thi (Feasibility Pruning) và Cắt tỉa tính tối ưu (Optimality Pruning - Branch & Bound) là gì?

- A. Feasibility Pruning chỉ dùng cho bài toán tìm đường đi.
- B. **[Đáp án đúng]** Feasibility Pruning cắt nhánh vì vi phạm ràng buộc không thể tạo nghiệm hợp lệ; Optimality Pruning cắt nhánh vì hàm cận chứng minh nhánh này không thể tạo ra nghiệm tốt hơn *best* hiện có.
- C. Optimality Pruning chạy chậm hơn.
- D. Hai khái niệm hoàn toàn giống hệt nhau.

Giải thích: Feasibility loại bỏ nghiệm sai; Optimality loại bỏ nghiệm đúng nhưng kém tối ưu so với best hiện tại.

Câu 3 (hậu quả của việc quên unchoose):

Điều gì sẽ xảy ra nếu lập trình viên quên câu lệnh hoàn tác `visited[i] = false` sau lời gọi đệ quy trong bài toán sinh hoán vị?

- A. Chương trình vẫn chạy đúng nhưng tốn nhiều bộ nhớ hơn.
- B. **[Đáp án đúng]** Trạng thái bị rò rỉ, các nhánh duyệt tiếp theo coi phần tử `i` đã được dùng và bỏ qua, dẫn đến thiếu sót nghiêm trọng các nghiệm hợp lệ.
- C. Chương trình bị tràn số `int`.
- D. Mảng tự động sắp xếp lại.

Giải thích: Quên hoàn tác vi phạm State Restoration Invariant, làm đóng băng trạng thái của các nhánh sau.

Câu 4 (đánh dấu các họ đường chéo n-queens):

Trong bài toán xếp `N` quân hậu trên bàn cờ $N \times N$ (1-based indexing), để tránh chỉ số mảng bị âm khi đánh dấu một họ đường chéo (hướng `\`) đi qua ô `(row, col)`, công thức chỉ số chuẩn xác là:

- A. `row - col`
- B. **[Đáp án đúng]** `row - col + N` (với `N` là kích thước bàn cờ, chỉ số thuộc `[1, 2N-1]`).
- C. `row × col`
- D. `(row + col) bmod N`

Giải thích: Vì `row - col` có thể nhận giá trị âm từ $-(N-1)$ đến $N-1$, cộng thêm `N` đảm bảo chỉ số luôn nằm trong khoảng an toàn `[1, 2N-1]`.

Câu 5 (bản chất quy tắc Warnsdorff trong mã đi tuần):

Trong bài toán Mã đi tuần (Knight's Tour), quy tắc Heuristic Warnsdorff (ưu tiên nhảy vào ô có ít nước đi tiếp theo nhất) có vai trò chuẩn xác là gì?

- A. Đảm bảo chắc chắn tìm thấy nghiệm trong $O(1)$ bước mà không cần quay lui.
- B. **[Đáp án đúng]** Trong framework bài học này, Warnsdorff được xem là Heuristic Ordering: nó thay đổi thứ tự ưu tiên thử nước đi để tìm nghiệm sớm hơn, không tự động loại bỏ các nhánh còn lại.
- C. Dừng để cắt bỏ hoàn toàn các nhánh khác.
- D. Là một thuật toán Quy hoạch động.

Giải thích: Heuristic chỉ đóng vai trò sắp xếp thứ tự thử nước đi (ordering), không thay thế cho toàn bộ cây tìm kiếm.

Câu 6 (tình huống thực tế đánh giá hàm bound trong bài toán cực tiểu):

Trong bài toán tìm hành trình TSP ngắn nhất, giả sử nghiệm tốt nhất tìm được tính tới thời điểm hiện tại là $best = 100$. Tại một trạng thái nhánh X , hàm Cận Dưới tính ra $LB(X) = 105$. Quyết định chuẩn xác của thuật toán là gì?

- A. **[Đáp án đúng]** Cắt tỉa (Prune) ngay lập tức nhánh X , vì chi phí thực tế $OPT(X) \geq LB(X) = 105 > 100 = best$, nhánh này chắc chắn không thể cải thiện nghiệm.
- B. Đi sâu tiếp vào nhánh X vì có thể chi phí thực tế sẽ giảm xuống dưới 100.
- C. Đặt lại giá trị $best = 105$.
- D. Dừng toàn bộ chương trình.

Giải thích: Vì $LB(X) \leq OPT(X)$, nếu $LB(X) \geq best$ thì chi phí thực tế chắc chắn không thể tốt hơn $best$.

Câu 7 (độ phức tạp không gian: Exponential tree vs exponential stack):

Thuật toán quay lui sinh tất cả $N!$ hoán vị của tập hợp $\{1, \dots, N\}$ tiêu tốn bộ nhớ ngăn xếp (Call Stack Space) tối đa là bao nhiêu?

- A. $\Theta(N!)$
- B. $\Theta(N^2)$
- C. **[Đáp án đúng]** $\Theta(N)$ (Search Space đo tổng số trạng thái lá $N!$, nhưng Call Stack chỉ đo độ sâu của một đường đi đang khám phá là N).
- D. $\Theta(1)$

Giải thích: Cây tìm kiếm khổng lồ không đồng nghĩa với Call Stack khổng lồ; độ sâu ngăn xếp chỉ tỷ lệ thuận với chiều dài nghiệm đang xây dựng.

Câu 8 (cắt tỉa kết hợp sắp xếp trong subset sum):

Khi tìm các tập con của mảng các số nguyên dương ($A_i > 0$) có tổng bằng S , nếu mảng đã được sắp xếp tăng dần, điều kiện cắt tỉa tính khả thi hiệu quả nhất tại vòng lặp duyệt phần tử A_i là gì?

- A. Dừng lại khi mảng còn hơn 10 phần tử.
- B. **[Đáp án đúng]** Dừng lệnh `break` dừng duyệt toàn bộ các phần tử còn lại ngay khi `current\_sum + A[i] > S` (dựa trên tính đơn điệu Monotonicity: các phần tử sau $A_{i+1} \geq A_i$ chắc chắn cũng vượt S).
- C. Dừng lại khi gặp số chẵn.
- D. Dừng lại khi `current\_sum == 0`.

Giải thích: Sắp xếp mảng trước kết hợp giả thiết $A_i > 0$ giúp chuyển điều kiện từ `continue` ở từng nhánh thành `break` triệt tiêu toàn bộ cây con phía sau.

Câu 9 (độ phức tạp tổng thể khi in toàn bộ xâu nhị phân):

Chương trình sinh và in toàn bộ các xâu nhị phân độ dài N ra màn hình có tổng thời gian thực thi (Time Complexity) là:

- A. $\Theta(2^N)$
- B. **[Đáp án đúng]** $\Theta(N \cdot 2^N)$ (có đúng 2^N xâu nghiệm, và mỗi xâu tốn $O(N)$ thời gian để xuất ra màn hình).
- C. $\Theta(N!)$
- D. $\Theta(N)$

Giải thích: Cần phân biệt rõ giữa số lượng nghiệm lá ($\Theta(2^N)$) và tổng thời gian thực thi khi phải xuất toàn bộ nội dung từng nghiệm ($\Theta(N \cdot 2^N)$).

Câu 10 (câu nối từ backtracking sang dynamic programming):

Khi một bài toán quay lui có hiện tượng nhiều nhánh trạng thái khác nhau gặp lại cùng một trạng thái con (Overlapping States trong đồ thị State DAG), dấu hiệu này gợi ý điều gì?

- A. Thuật toán quay lui đã bị lỗi bộ nhớ.
- B. **[Đáp án đúng]** Trùng lặp trạng thái là dấu hiệu quan trọng để xem xét Memoization / Dynamic Programming, lưu kết quả mỗi trạng thái 1 lần duy nhất thay vì tính lại trên cây.
- C. Bỏ qua hoàn toàn bài toán.
- D. Tăng kích thước mảng lên gấp đôi.

Giải thích: Chuyển từ duyệt cây tìm kiếm (Tree Search) sang đồ thị trạng thái (State DAG) có lưu vết chính là bản chất của Quy Hoạch Động.

Câu 11 (bản chất state identity trong bài toán subset sum):

Trong bài toán Subset Sum, giả sử hai lời gọi đệ quy khác nhau đều đang đứng tại chỉ số `index = 5`, nhưng một nhánh có `current_sum = 12` và nhánh kia có `current_sum = 18`. Hai lời gọi này có được xem là cùng một State Identity trong DP không?

- A. Có, vì chúng có cùng chỉ số `index = 5`.
- B. **[Đáp án đúng]** Không, vì `current_sum` quyết định trực tiếp đến các lựa chọn và khả năng đạt tổng mục tiêu còn lại, nên `(index, current_sum)` mới là State Identity hoàn chỉnh.
- C. Có, vì chỉ số mảng quan trọng hơn tổng.
- D. Tùy thuộc vào việc mảng có số âm hay không.

Giải thích: State Identity phải bao hàm đủ thông tin để xác định không gian nghiệm phía sau; khác `current_sum` dẫn đến các bài toán con phía sau hoàn toàn khác nhau.

Câu 12 (nguyên tắc an toàn của hàm bound trong branch & bound):

Nếu một lập trình viên thiết kế một hàm Cận Dưới `LB(state)` cho bài toán tìm chi phí nhỏ nhất, nhưng trong một số trường hợp hiếm gặp `LB(state) > OPT(state)` (ước lượng quá cao so với thực tế), hậu quả là gì?

- A. Thuật toán chạy nhanh hơn và luôn cho kết quả đúng.
- B. **[Đáp án đúng]** Thuật toán có thể vô tình cắt tia nhánh chứa nghiệm tối ưu thực sự và đưa ra kết quả sai (Invalid Bound).
- C. Bộ nhớ bị tràn.
- D. Không có ảnh hưởng gì vì hiếm khi xảy ra.

Giải thích: Bất biến sống còn của Branch & Bound là $LB \leq OPT$; chỉ cần vi phạm một lần, nghiệm tối ưu có thể bị xóa sổ khỏi không gian tìm kiếm.

Ma trận bài tập thực hành (p0 → p5)

Lộ trình phân tầng học tập chuẩn mực:

- ****LEVEL 1: Pattern Sinh Cấu Hình Cơ Bản (CPPB-BKT-01 → 04):**** Xâu nhị phân, Tập con, Hoán vị, Tổ hợp chập K .
- ****LEVEL 2: Constraint Backtracking / Feasibility Pruning (CPPB-BKT-05 → 09):**** Dãy ngoặc đúng, N -Queens, Mê cung, Subset Sum, Chia tập bằng nhau.
- **LEVEL 3: Optimization Search & Branch and Bound (CPPB-BKT-10 , 13 , 14 , 16):** Đổi tiền xu ít nhất (B&B), Cái túi 0/1 B&B, TSP B&B, Phân công công việc B&B.
- **LEVEL 4: Advanced CSP & Heuristic Search (CPPB-BKT-11 , 12 , 15):** Mã đi tuần Warnsdorff, Sudoku 9×9 , Tô màu đồ thị (K -Coloring).

STT	Mã Bài	Tên Bài Toán	Difficulty	Concept Group	Output / Time Complexity	Search Space / Number of Solutions	Call Stack	Max Depth
01	CPPB-BKT-01	**Sinh Xâu Nhị Phân Độ Dài N **	P0	Level 1	$\Theta(N \cdot 2^N)$ (Output-sensitive)	2^N nghiệm lá	$\Theta(N)$	N
02	CPPB-BKT-02	**Sinh Tập Con Của Tập N Phần Tử**	P0	Level 1	$\Theta(N \cdot 2^N)$ (Output-sensitive)	2^N tập con	$\Theta(N)$	N
03	CPPB-BKT-03	**Sinh Hoán Vị $1 \dots N$ **	P1	Level 1	$\Theta(N \cdot N!)$ (Output-sensitive)	$N!$ hoán vị lá	$\Theta(N)$	N
04	CPPB-BKT-04	**Sinh Tổ Hợp Chập K Của N **	P1	Level 1	$\Theta(K \cdot C_N^K)$ (Output-sensitive)	C_N^K tổ hợp	$\Theta(K)$	K
05	CPPB-BKT-05	**Sinh Dây Ngoặc Hợp Lệ Độ Dài $2N$ **	P1	Level 2	$\Theta(N \cdot Catalan(N))$	$Catalan(N)$ nghiệm	$\Theta(N)$	$2N$
06	CPPB-BKT-06	**Bài Toán N -Queens (Đếm Số Cách)**	P2	Level 2	$O(N!)$ (Worst-case bound)	$\leq N!$ gán thô (pruned mạnh)	$\Theta(N)$	N
07	CPPB-BKT-07	Mê Cung (Rat in a Maze)	P2	Level 2	$O(4N^2)$ (Loose bound)	Cây đường đi $\leq 4N^2$	$\Theta(N^2)$	N^2
08	CPPB-BKT-08	**Tập Con Có Tổng Bằng S (Subset Sum)**	P2	Level 2	$O(2^N)$ (Pruned)	$\leq 2^N$ tập con ($A_i > 0$)	$\Theta(N)$	N

09	CPPB- BKT-09	Chia Tập Thành 2 Phần Bằng Nhau	P3	Level 2	$O(2^N)$ (Pruned)	$\leq 2^N$ phân hoạch	$\theta(N)$	N
10	CPPB- BKT-10	Đổi Tiền Xu Ít Nhất (B&B Coin Change)	P3	Level 3	Exponential worst-case	Phụ thuộc chất lượng Bound	$O(S / C_{m_i} \lfloor \frac{S}{C_{m_i}} \rfloor)$	$\leq \lfloor \frac{S}{C_{m_i}} \rfloor$ $S / C_{m_i} \lfloor \frac{S}{C_{m_i}} \rfloor$
11	CPPB- BKT-11	Mã Đi Tuần (Knight's Tour)	P3	Level 4	$O(8N^2)$ (Loose bound)	Search tree with heuristic order	$\theta(N^2)$	N^2
12	CPPB- BKT-12	**Trò Chơi Sudoku 9×9 **	P3	Level 4	$O(9^E)$ ($E \leq 81$ ô trống)	Không gian gán thô $\leq 9^E$	$O(E)$	$E \leq 81$
13	CPPB- BKT-13	**Bài Toán Cái Túi 0/1 Nhánh Cận (B&B)**	P4	Level 3	Exponential worst-case	Fractional Bound Pruned	$\theta(N)$	N
14	CPPB- BKT-14	Người Du Lịch (TSP) Nhánh Cận	P4	Level 3	$O(N!)$ worst- case	Min-edge Bound Pruned	$\theta(N)$	N
15	CPPB- BKT-15	**Tô Màu Đồ Thị (Graph K -Coloring)**	P4	Level 4	$O(K^V)$ (Pruned)	$\leq K^V$ trạng thái màu	$\theta(V)$	V
16	CPPB- BKT-16	Phân Công Công Việc Tối Ưu (Job Assign)	P5	Level 3	$O(N!)$ worst- case	Min-row Bound Pruned	$\theta(N)$	N

Module 05: Quy hoạch động (dynamic programming)

<!-- Chuyên đề 13: Quy Hoạch Động 1D & LIS | 467 dòng | 28,715 bytes -->

Chuyên đề 13: Quy hoạch động 1d & dãy con tăng dài nhất

1. Bản chất vấn đề & cầu nối từ đệ quy sang quy hoạch động

Trong Chuyên đề 10 và 12, ta đã chứng kiến hiện tượng **Bùng nổ Không gian Trạng thái (Combinatorial Explosion)** khi duyệt đệ quy phân nhánh:

- Khi tính số Fibonacci $F(N) = F(N-1) + F(N-2)$, trạng thái $F(3)$ bị tính lại 2 lần, $F(2)$ bị tính lại 3 lần. Độ phức tạp thời gian tăng vọt lên cấp số nhân $\Theta(\varphi^N)$ *approx* $\Theta(1.618^N)$.
- **Nguyên nhân gốc rễ:** Hàm đệ quy thuần túy không có cơ chế "ghi nhớ" (Memory). Mỗi lần bước vào một nhánh mới, nó xem bài toán con đó như một thực thể hoàn toàn xa lạ và tính toán lại từ đầu.

Quy Hoạch Động (Dynamic Programming - DP) giải quyết vấn đề này bằng nguyên lý cốt lõi:

DP loại bỏ việc tính toán lại các bài toán con trùng lặp bằng cách lưu trữ kết quả vào Bảng phương án (DP Table) và tái sử dụng ngay lập tức trong $O(1)$.

![[Mô hình Đồ thị trạng thái DAG Quy hoạch động 1D]](/Users/vu/Developer/ikhEdu_lessons/courses/cpp-bang-b/lessons/lesson-13-quy-hoach-dong-1d/assets/dp_1d_state_dag_vi.svg)

2. Khung phương pháp luận: 7 Bước dp state invariant

Để giải quyết chính xác bất kỳ bài toán Quy hoạch động nào, ta áp dụng khung 7 thành phần logic chuẩn mực:

1. **State Definition & Invariant (Định nghĩa Trạng thái & Bất biến):** $dp[i]$ đại diện chính xác 100% cho đại lượng nào? (Là giá trị tối ưu, số cách thực hiện, hay trạng thái logic?).
2. **Base Cases (Trường hợp cơ sở):** Các trạng thái biên nhỏ nhất không thể phân rã thêm ($i = 0, 1$) có giá trị bằng bao nhiêu?
3. **State Transition (Hệ thức chuyển trạng thái):** Trạng thái $dp[i]$ phụ thuộc vào các trạng thái con $dp[j]$ ($j < i$) nào trước đó qua công thức toán học nào?
4. **Evaluation Order (Thứ tự tính toán):** Chiều duyệt vòng lặp ($i = 1 \rightarrow N$ xuôi hay ngược) theo thứ tự Topo DAG để đảm bảo mọi bài toán con phụ thuộc đều đã được tính xong trước khi dùng.
5. **Answer Extraction (Trích xuất kết quả):** Đáp án của bài toán gốc nằm ở đâu? (Tại $dp[N]$, hay $\max_{i=1}^N dp[i]$?).

6. **Space & Time Optimization (Tối ưu hóa):** Phân tích độ phức tạp thời gian $O(Time)$, bộ nhớ $O(Space)$ và khả năng nén mảng.
7. **Reconstruction (Khôi phục nghiệm):** Dùng mảng truy vết `trace[i]` hoặc duyệt ngược trên bảng `dp` để tái tạo lại cấu hình nghiệm tối ưu (Ghi *N/A* nếu bài toán chỉ yêu cầu giá trị).

3. Các mô hình quy hoạch động 1d cốt lõi (core patterns)

3.1. Mô hình bậc thang & bước nhảy (staircase / frog jump)

- **Bối cảnh:** Một chú ếch đứng ở bậc 0 , muốn nhảy lên bậc N . Tại mỗi bậc, ếch có thể nhảy 1 bước hoặc 2 bước.
- **State Definition:** $dp[i]$ là số cách khác nhau để ếch nhảy từ bậc 0 đến bậc i .
- **Base Cases:** $dp[0] = 1$ (có đúng một cách để hoàn thành hành trình từ bậc 0 đến bậc 0 — không thực hiện bước nhảy nào), $dp[1] = 1$.
- **State Transition:** Để đến bậc i , bước nhảy cuối cùng bắt buộc phải xuất phát từ bậc $i-1$ (nhảy 1 bước) hoặc bậc $i-2$ (nhảy 2 bước):

$$dp[i] = dp[i-1] + dp[i-2] \text{ pmod}\{10^9+7\}$$

- **Evaluation Order:** Duyệt xuôi từ $i = 2 \rightarrow N$.

3.2. Mô hình đổi tiền ít xu nhất (coin change 1d)

- **Bối cảnh:** Cho hệ thống gồm K đồng xu có mệnh giá $C = \{c_1, c_2, \dots, c_K\}$. Cần đổi số tiền S sao cho tổng số đồng xu là ít nhất.
- **State Definition:** $dp[i]$ là số lượng đồng xu **ít nhất** để tạo ra đúng tổng giá trị i .
- **Base Cases:** $dp[0] = 0$ (Tổng tiền bằng 0 cần đúng 0 đồng xu). Khởi tạo mọi $dp[i] = \infty$ với $i \geq 1$.
- **State Transition:** Thử chọn đồng xu cuối cùng là mệnh giá $c \in C$:

$$dp[i] = 1 + \min\{c \in C \text{ s.t. } i \geq c\} dp[i - c]$$

- **Evaluation Order:** Duyệt xuôi $i = 1 \rightarrow S$. Nếu $dp[S] = \infty \Rightarrow$ Không thể đổi được.

Lưu ý quan trọng: Quy tắc thứ tự vòng lặp phân biệt Hoán vị / Tổ hợp dưới đây áp dụng cho bài toán đếm số cách. Với bài toán tối ưu số đồng xu ít nhất $dp[i] = 1 + \min(dp[i-c])$, do phép toán $\min$ có tính chất giao hoán và kết hợp nên ta luôn duyệt i từ $1 \rightarrow S$ mà không làm thay đổi giá trị tối ưu.

- **Phân biệt Sự phạm Cốt lõi Trong Bài Toán Đếm Số Cách (Counting Coin Change):**

- **Bài toán Hoán vị (Permutation):** Thứ tự các đồng xu có phân biệt (ví dụ $1+2 \neq 2+1$). Vòng lặp ngoài duyệt Tiền $i = 1 \rightarrow S$, vòng lặp trong thử từng đồng xu $c \in C$.
- **Bài toán Tổ hợp (Combination):** Thứ tự các đồng xu không phân biệt (ví dụ $1+2$ và $2+1$ là một cách). Vòng lặp ngoài duyệt từng đồng xu $c \in C$, vòng lặp trong duyệt Tiền $i = c \rightarrow S$.

![[Bài toán Đổi tiền Coin Change và DAG trạng thái]](/Users/vu/Developer/ikhEdu_lessons/courses/cpp-bang-b/lessons/lesson-13-quy-hoach-dong-1d/assets/coin_change_dag_vi.svg)

3.3. Dãy con tăng dài nhất (longest increasing subsequence — lis $O(N^2)$)

- **Bối cảnh:** Cho dãy số $A = [a_1, a_2, \dots, a_N]$. Tìm độ dài dãy con tăng nghiêm ngặt dài nhất.
- **State Definition (Từ huyệt định nghĩa):** $dp[i]$ là độ dài của dãy con tăng dài nhất **kết thúc bắt buộc tại phần tử $A[i]$ **.
- **Base Cases:** $dp[i] = 1$ với mọi $1 \leq i \leq N$ (bản thân mỗi phần tử đơn lẻ là dãy con độ dài 1).
- **State Transition:** Duyệt qua mọi phần tử $A[j]$ đứng trước $A[i]$ ($1 \leq j < i$):

$$dp[i] = 1 + \max\{1 \leq j < i \text{ mà } A[j] < A[i]\} dp[j]$$

- **Answer Extraction:** Kết quả toàn cục là $\max_{i=1}^N dp[i]$.
- **Độ phức tạp:** $O(N^2)$ thời gian, $O(N)$ bộ nhớ. Thường phù hợp với N cỡ vài nghìn, tùy thuộc vào time limit và hệ số hằng số.

![[Mô hình Dãy con tăng dài nhất LIS $O(N^2)$ và Truy vết]](/Users/vu/Developer/ikhEdu_lessons/courses/cpp-bang-b/lessons/lesson-13-quy-hoach-dong-1d/assets/lis_quadratic_model_vi.svg)

3.4. Kỹ thuật khôi phục vết nghiệm 1d (traceback / reconstruction)

Để in ra chính xác cấu hình dãy phần tử tạo nên nghiệm tối ưu:

1. Duy trì mảng $trace[i] = j$ ghi nhận chỉ số phần tử đứng ngay trước $A[i]$ trong cấu hình tối ưu.
2. Tìm vị trí $best_idx$ có $dp[best_idx]$ đạt cực trị.
3. Lăn ngược mảng $trace$ từ $best_idx$ về điểm xuất phát, lưu các phần tử vào một vector rồi đảo ngược ($reverse$).

3.5. Mở rộng nâng cao: Lis $O(N \log N)$ bằng binary search

Khi $N \leq 10^5$, thuật toán $O(N^2)$ sẽ bị Quá thời gian (TLE).

- **Ý tưởng:** Duy trì mảng phụ $tails$ trong C++ (chỉ số 0-based), trong đó phần tử $tails[len - 1]$ lưu giá trị phần tử kết thúc nhỏ nhất của một dãy con tăng có độ dài đúng bằng len .

- Mảng `tails` luôn có tính chất **đơn điệu tăng dần** $\Rightarrow$ Dùng Tìm kiếm nhị phân (`lower_bound`) để tìm và cập nhật vị trí thích hợp cho mỗi `A[i]` trong $O(\log N)$.
- Tổng thời gian:** $O(N \log N)$. Đây là kỹ thuật mở rộng tối ưu hóa nâng cao (Challenge Extension).

4. Các biến thể thiết kế trạng thái từ mô hình 1d (state design variations)

Mục tiêu lớn nhất của Module 05 không phải là học thuộc các công thức, mà là rèn luyện khả năng **Thiết Kế Trạng Thái (State Design)** trước các biến thể bài toán mới:

4.1. Pattern a: Quyết định nhị phân (binary choice — chọn / bỏ qua)

- Bối cảnh (House Robber):** Không được chọn hai phần tử liền kề nhau.
- State Invariant:** `dp[i]` là tổng giá trị lớn nhất khi chỉ xét tiền tố từ `1 ... i`.
- Transition:** Tại vị trí `i`, có 2 lựa chọn loại trừ lẫn nhau:

$$dp[i] = \max(\underbrace{dp[i-1]}_{\text{Không chọn } i}, \underbrace{dp[i-2] + A[i]}_{\text{Chọn } i})$$

- Chuyển đổi bài toán (Delete and Earn):** Khi chọn giá trị `v`, ta nhận toàn bộ tổng điểm `points[v] = v × count(v)` nhưng bị cấm chọn `v-1` và `v+1`. Bằng cách gom nhóm dữ liệu theo trục giá trị `v`, bài toán được quy đổi hoàn toàn về mô hình House Robber trên mảng `points`.

4.2. Pattern b: Mở rộng trạng thái hữu hạn (state dimension expansion)

- Bối cảnh (Alternating Subsequence):** Dãy con đan dấu (tăng $\rightarrow$ giảm $\rightarrow$ tăng $\rightarrow$ giảm).
- Vấn đề:** Nếu chỉ dùng `dp[i]`, ta không biết phần tử `A[i]` đang đóng vai trò là "đỉnh tăng" hay "đáy giảm".
- Thiết kế Trạng thái:** Bổ sung thêm một chiều trạng thái hữu hạn `state ∈ {0, 1}`:
- `dp[i][0]`: Độ dài dãy đan dấu kết thúc tại `A[i]` với bước nhảy cuối cùng là **GIẢM** (`A[j] > A[i]`).
- `dp[i][1]`: Độ dài dãy đan dấu kết thúc tại `A[i]` với bước nhảy cuối cùng là **TĂNG** (`A[j] < A[i]`).
- Lưu ý:** Chiều bắt đầu của dãy con có thể linh hoạt bắt đầu bằng tăng hoặc giảm tùy theo yêu cầu đề bài; kết quả toàn cục thường là $\max(\max_i dp[i][0], \max_i dp[i][1])$.
- > Lưu ý:** Với dãy có độ dài 1, chưa tồn tại bước nhảy tăng/giảm; giá trị khởi tạo cụ thể của `dp[i][0]`, `dp[i][1]` phụ thuộc vào định nghĩa bài toán và cách triển khai. Phần này được xem như mô hình mở rộng, không phải template cài đặt đầy đủ trong chuyên đề này.

4.3. Pattern c: Thay đổi đại lượng tối ưu (maximum sum increasing subsequence — msis)

- **Bối cảnh:** Thay vì tìm dãy con tăng có độ dài lớn nhất, bài toán yêu cầu tìm dãy con tăng có tổng giá trị các phần tử lớn nhất.
- **Điều chỉnh Invariant:**
- LIS: $dp[i]$ = độ dài LIS $\Rightarrow dp[i] = 1 + \max(dp[j])$.
- MSIS: $dp[i]$ = Tổng lớn nhất của dãy con tăng kết thúc tại $A[i]$:

$$dp[i] = A[i] + \max_{j < i, A[j] < A[i]} dp[j]$$

4.4. Pattern d: Phân hoạch đoạn tối ưu (pattern mở rộng / preview — optimal array partitioning & rod cutting)

- **Bối cảnh:** Cắt một thanh gỗ độ dài N (hoặc phân chia dãy số $A[1 \dots N]$ thành các đoạn con liên tiếp) sao cho tổng giá trị/chi phí là tối ưu.
- **State Invariant:** $dp[i]$ là chi phí/giá trị tối ưu khi phân hoạch tiền tố $A[1 \dots i]$.
- **Transition:** Thử mọi điểm cắt cuối cùng $j \in [0, i-1]$:

$$dp[i] = \min_{0 \leq j < i} (dp[j] + cost(j+1, i))$$

- **Áp dụng cho bài Mastery CPPB-DP1-15:** Tìm cách phân chia dãy số thành các khối đoạn con thỏa mãn điều kiện ràng buộc với chi phí nhỏ nhất.

5. Các bẫy lỗi lập trình kinh điển (bug traps)

1. Bẫy khởi tạo giá trị vô cùng (Infinity Overflow):

- Khi tìm giá trị nhỏ nhất, nếu dùng `INT_MAX` cho kiểu `int`, phép tính `dp[i-c] + 1` sẽ gây tràn số thành số âm cực lớn.
- **Quy tắc an toàn:** Dùng hằng số `const long long INF = 1e18;` (hoặc `1LL << 60`) và luôn kiểm tra `if (dp[i - c] != INF)` trước khi chuyển trạng thái.

1. Bẫy định nghĩa sai Trạng thái trong LIS:

- Ngộ nhận: " $dp[i]$ là độ dài LIS trong đoạn từ 1 đến i ". Nếu định nghĩa như vậy, ta không thể biết phần tử kết thúc là bao nhiêu để so sánh với $A[i+1]$.
- **Bất biến đúng:** Bắt buộc $dp[i]$ phải là độ dài LIS kết thúc tại chính $A[i]$.

1. Bẫy nhầm lẫn thứ tự vòng lặp trong Coin Change Đếm Số Cách:

- Duyệt Tiền trước, Coin sau $\Rightarrow$ Tạo ra bài toán Hoán vị (đếm lặp thứ tự).
- Duyệt Coin trước, Tiền sau $\Rightarrow$ Tạo ra bài toán Tổ hợp (đếm không trùng lặp).

6. Mẫu cài đặt chuẩn thi đấu (competitive templates)

Mẫu 1: Dãy con tăng dài nhất lis $O(N^2)$ kèm truy vết nghiệm

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;

    if (n ≤ 0) return 0;

    vector<long long> a(n);

    for (int i = 0; i < n; ++i) {
        cin >> a[i];
    }

    vector<int> dp(n, 1);
    vector<int> trace(n, -1);

    int max_len = 1;
    int best_end = 0;

    // Quy hoạch động  $O(N^2)$ 
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < i; ++j) {
            if (a[j] < a[i] && dp[j] + 1 > dp[i]) {

                dp[i] = dp[j] + 1;
                trace[i] = j;
            }
        }
        if (dp[i] > max_len) {

            max_len = dp[i];
            best_end = i;
        }
    }

    cout << max_len << "\n";

    // Khôi phục vết nghiệm
    vector<long long> lis_elements;

    int curr = best_end;
    while (curr ≠ -1) {
        lis_elements.push_back(a[curr]);
        curr = trace[curr];
    }
}

```

```
reverse(lis_elements.begin(), lis_elements.end());

for (int i = 0; i < (int)lis_elements.size(); ++i) {
    cout << lis_elements[i] << (i + 1 == (int)lis_elements.size() ? "" : "
");
}
cout << "\n";

return 0;
}
```


Mẫu 2: Đổi tiền ít xu nhất (coin change 1d)

```

#include <bits/stdc++.h>
using namespace std;

const long long INF = 1e18;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int k;
    long long s;
    if (!(cin >> k >> s)) return 0;

    vector<long long> c(k);

    for (int i = 0; i < k; ++i) {
        cin >> c[i];
    }

    vector<long long> dp(s + 1, INF);

    dp[0] = 0;

    for (int i = 1; i ≤ s; ++i) {
        for (long long coin : c) {
            if (i ≥ coin && dp[i - coin] ≠ INF) {
                dp[i] = min(dp[i], dp[i - coin] + 1);
            }
        }
    }

    if (dp[s] = INF) {
        cout << -1 << "\n";
    } else {
        cout << dp[s] << "\n";
    }

    return 0;
}

```

7. Hệ thống câu hỏi kiểm tra khái niệm (concept quiz)

Câu 1 (bản chất quy hoạch động):

Điểm khác biệt cốt lõi nhất giữa Thuật toán Đệ quy thuần túy và Quy hoạch động là gì?

- A. Đệ quy chạy nhanh hơn Quy hoạch động.
- B. **[Đáp án đúng]** Quy hoạch động loại bỏ việc tính lại bài toán con trùng lặp bằng cách ghi nhớ kết quả trong bảng phương án.
- C. Quy hoạch động không sử dụng mảng nhớ.
- D. Đệ quy không thể chuyển đổi thành Quy hoạch động.

Giải thích: Quy hoạch động tận dụng tính chất bài toán con trùng lặp (Overlapping Subproblems) và cấu trúc con tối ưu (Optimal Substructure) để lưu kết quả vào bảng dp , giảm thời gian từ hàm mũ xuống đa thức.

Câu 2 (bất biến trạng thái lis):

Trong thuật toán tìm Dãy con tăng dài nhất LIS $O(N^2)$, $dp[i]$ đại diện chính xác cho điều gì?

- A. Độ dài LIS của đoạn con từ $A[1]$ đến $A[i]$.
- B. **[Đáp án đúng]** Độ dài LIS kết thúc bắt buộc tại phần tử $A[i]$.
- C. Số lượng dãy con tăng có trong mảng.
- D. Giá trị nhỏ nhất của dãy con tăng độ dài i .

Giải thích: Bắt buộc $dp[i]$ phải kết thúc tại chính $A[i]$ để khi xét phần tử $A[k]$ đứng sau, ta chỉ cần so sánh điều kiện $A[i] < A[k]$ để quyết định nối dài dãy.

Câu 3 (trường hợp cơ sở bài toán bậc thang):

Trong bài toán ếch nhảy bậc thang $dp[i] = dp[i-1] + dp[i-2]$, tại sao giá trị cơ sở $dp[0] = 1$?

- A. Vì ếch bắt buộc phải nhảy 1 bước đầu tiên.
- B. **[Đáp án đúng]** Vì có đúng một cách để hoàn thành hành trình từ bậc 0 đến bậc 0 — không thực hiện bước nhảy nào.
- C. Vì $dp[0]$ không có ý nghĩa toán học nên gán tạm bằng 1.
- D. Vì số bậc thang luôn lớn hơn 0.

Giải thích: $dp[0] = 1$ là empty path hợp lệ, đảm bảo khi tính $dp[2] = dp[1] + dp[0] = 1 + 1 = 2$ (gồm bước 1+1 và bước nhảy thẳng 2).

Câu 4 (thứ tự tính toán evaluation order):

Trong bài toán **Đổi tiền ít đồng xu nhất với hệ thức truy hồi $dp[i] = 1 + \min(dp[i-c])$ **, tại sao vòng lặp tính i phải duyệt xuôi từ $1 \rightarrow S$?

- A. Để in ra các đồng xu theo thứ tự tăng dần.
- B. **[Đáp án đúng]** Để đảm bảo mọi giá trị $dp[i-c]$ (với $i-c < i$) đều đã được tính toán tối ưu trước khi dùng.
- C. Để tránh tràn bộ nhớ mảng.
- D. Vì duyệt ngược sẽ làm tăng độ phức tạp thời gian.

Giải thích: Cấu trúc đồ thị trạng thái DAG quy định trạng thái i phụ thuộc vào các trạng thái nhỏ hơn $i-c$. Do đó các trạng thái nhỏ hơn phải được hoàn tất trước theo thứ tự Topo.

Câu 5 (bẫy số nguyên vô cùng):

Khi khởi tạo mảng dp tìm giá trị nhỏ nhất, giá trị nào sau đây an toàn nhất để tránh tràn số khi cộng thêm 1?

- A. `INT_MAX` (khoảng $2 \cdot 10^9$) với kiểu `int`.
- B. **[Đáp án đúng]** `1e18` với kiểu `long long` kèm điều kiện kiểm tra khác vô cùng trước khi cộng.
- C. `-1`.
- D. `0`.

Giải thích: `INT_MAX + 1` sẽ bị tràn số nguyên thành số âm cực lớn. Dùng `const long long INF = 1e18;` và luôn kiểm tra `if (dp[v] ne INF)` là chuẩn mực an toàn.

Câu 6 (khôi phục vết nghiệm traceback):

Để khôi phục lại các phần tử thuộc dãy con tăng dài nhất LIS, kỹ thuật nào sau đây là chuẩn mực nhất?

- A. Chạy lại thuật toán LIS lần thứ hai.
- B. **[Đáp án đúng]** Lưu chỉ số phần tử đứng trước vào mảng `trace[i]`, sau đó lần ngược từ phần tử kết thúc tối ưu về đầu và đảo ngược vector.
- C. In trực tiếp mảng `dp`.
- D. Dùng thuật toán quay lui vết cạn lại từ đầu.

Giải thích: Mảng $trace[i] = j$ lưu vết trực tiếp trong $O(1)$ tại thời điểm cập nhật $dp[i]$, cho phép truy vết nghiệm trong $O(N)$.

Câu 7 (độ phức tạp bài toán đổi tiền):

Cho K loại đồng xu và số tiền cần đổi S . Độ phức tạp thời gian và không gian của thuật toán DP 1D là bao nhiêu?

- A. [Đáp án đúng] Thời gian $O(K \cdot S)$, Không gian $O(S)$.
- B. Thời gian $O(S^2)$, Không gian $O(K)$.
- C. Thời gian $O(2^K)$, Không gian $O(S)$.
- D. Thời gian $O(K \log S)$, Không gian $O(1)$.

Giải thích: Vòng lặp ngoài chạy S bước, vòng lặp trong thử K đồng xu $\Rightarrow$ Tổng số phép tính là $K \cdot S$, mảng dp có kích thước $S + 1$.

Câu 8 (bài toán house robber 1d):

Một tên trộm không được trộm hai ngôi nhà liền kề. Gọi $A[i]$ là số tiền ở nhà i . Hệ thức chuyển trạng thái nào sau đây là chính xác cho $dp[i]$ (tiền nhiều nhất trộm được từ $1 \rightarrow i$)?

- A. $dp[i] = dp[i-1] + A[i]$
- B. [Đáp án đúng] $dp[i] = \max(dp[i-1], dp[i-2] + A[i])$
- C. $dp[i] = \max(dp[i-1], dp[i-2])$
- D. $dp[i] = dp[i-2] + A[i]$

Giải thích: Tại nhà i , tên trộm có 2 lựa chọn: (1) Không trộm nhà $i \Rightarrow$ Nhận $dp[i-1]$; (2) Trộm nhà $i \Rightarrow$ Không được trộm nhà $i-1$, nhận $dp[i-2] + A[i]$.

Câu 9 (mô hình cắt thanh gỗ rod cutting):

Cho thanh gỗ độ dài N và bảng giá $P[len]$ cho từng đoạn gỗ độ dài len . Công thức tính giá trị lớn nhất $dp[i]$ khi cắt thanh gỗ độ dài i là:

- A. [Đáp án đúng] $dp[i] = \max(\{P[j] + dp[i-j] \mid 1 \leq j \leq i\})$
- B. $dp[i] = P[i]$
- C. $dp[i] = dp[i-1] + P[1]$
- D. $dp[i] = \sum P[j]$

Giải thích: Thử nhất cắt đầu tiên có độ dài $j \in [1, i]$, giá trị thu được là giá trị đoạn j ($P[j]$) cộng với giá trị tối ưu của phần còn lại độ dài $i-j$ ($dp[i-j]$).

Câu 10 (ranh giới lis $O(N^2)$ vs $O(N \log n)$):

Khi $N = 10^5$, tại sao thuật toán LIS $O(N^2)$ không thể vượt qua giới hạn thời gian 1.0 giây?

- **A. [Đáp án đúng]** Vì với $N = 10^5$, thuật toán $O(N^2)$ phải xét khoảng $N^2/2 \approx 5 \times 10^9$ cặp, thường vượt xa giới hạn thời gian thông thường của CPU trong 1.0 giây. Trong khi $O(N \log N)$ chỉ cần khoảng $N \log_2 N \approx 1.7 \times 10^6$ bước ở quy mô này.
- **B.** Vì mảng `dp` chiếm quá nhiều bộ nhớ RAM.
- **C.** Vì hàm `max` trong C++ chạy chậm.
- **D.** Vì số nguyên 64-bit bị tràn.

Giải thích: Thuật toán $O(N^2)$ thường không phù hợp khi N đạt cỡ 10^4 trở lên trong các giới hạn thời gian thi đấu thông thường. Cần chuyển sang $O(N \log N)$ bằng mảng đơn điệu và tìm kiếm nhị phân.

Câu 11 (state transformation — lis sang msis):

Khi chuyển từ bài toán LIS (Độ dài lớn nhất) sang MSIS (Tổng giá trị lớn nhất), thành phần nào trong hệ thức quy hoạch động thay đổi?

- **A.** Điều kiện $A[j] < A[i]$ bị bỏ đi.
- **B. [Đáp án đúng]** Giá trị khởi tạo và phép cộng dồn chuyển từ `+1` (đếm số lượng) sang `+A[i]` (cộng dồn giá trị phần tử).
- **C.** Mảng `dp` phải tăng lên 2 chiều.
- **D.** Thứ tự duyệt i phải đảo ngược.

Giải thích: Bất biến chuyển từ đếm độ dài sang cực đại hóa tổng: $dp[i] = A[i] + \max(dp[j])$ với $j < i$ và $A[j] < A[i]$.

Câu 12 (state dimension — alternating subsequence):

Tại sao trong bài toán Dãy con đan dấu, ta cần mở rộng trạng thái thành $dp[i][2]$ thay vì chỉ dùng $dp[i]$?

- A. Để lưu thêm vị trí của phần tử đứng trước.
- B. **[Đáp án đúng]** Vì cần phân biệt trạng thái bước nhảy cuối cùng đang là TĂNG hay GIẢM để so sánh điều kiện kế tiếp.
- C. Để giảm độ phức tạp bộ nhớ.
- D. Vì mảng có 2 nửa chẵn và lẻ.

Giải thích: Chiều thứ hai mang thông tin ngữ nghĩa: 0 nghĩa là bước cuối đi xuống, 1 nghĩa là bước cuối đi lên.

Câu 13 (segmentation dp — phân hoạch đoạn):

Trong bài toán Phân đoạn dãy số tối ưu $A[1 \dots N]$, hệ thức chuyển trạng thái tổng quát để tính $dp[i]$ (chi phí tối ưu cho tiền tố $1 \dots i$) là gì?

- A. $dp[i] = dp[i-1] + cost(i, i)$
- B. **[Đáp án đúng]** $dp[i] = \min(\{dp[j] + cost(j+1, i) \mid 0 \leq j < i\})$
- C. $dp[i] = dp[i/2]$
- D. $dp[i] = \min(dp[i-1], dp[i-2])$

Giải thích: Thử mọi điểm cắt j để tách tiền tố $1 \dots i$ thành phần đã tối ưu $1 \dots j$ và đoạn con mới nhất $j+1 \dots i$.

Câu 14 (coin change — phân biệt thứ tự duyệt):

Để đếm số cách đổi tiền không phân biệt thứ tự (tổ hợp: $1+2$ và $2+1$ là một cách), thứ tự duyệt 2 vòng lặp phải như thế nào?

- A. Vòng ngoài duyệt Tiền $1 \rightarrow S$, vòng trong duyệt từng Đồng xu.
- B. **[Đáp án đúng]** Vòng ngoài duyệt từng Đồng xu, vòng trong duyệt Tiền từ mệnh giá xu đến S .
- C. Duyệt ngẫu nhiên.
- D. Duyệt tiền giảm dần.

Giải thích: Duyệt từng đồng xu ở vòng ngoài đảm bảo các đồng xu mệnh giá nhỏ được đưa vào trước, đồng xu lớn đưa vào sau $\Rightarrow$ Không bao giờ sinh ra hoán vị lặp lại.

Câu 15 (ý nghĩa mảng *tails* trong lis $O(N \log n)$):

Trong thuật toán LIS $O(N \log N)$ (chỉ số 0-based), phần tử `tails[len - 1]` lưu trữ giá trị gì?

- **A.** Độ dài lớn nhất của dãy con tăng.
- **B. [Đáp án đúng]** Giá trị phần tử kết thúc nhỏ nhất của một dãy con tăng có độ dài đúng bằng `len`.
- **C.** Tổng giá trị của dãy con tăng độ dài `len`.
- **D.** Vị trí ban đầu của phần tử trong mảng gốc.

Giải thích: Lưu phần tử kết thúc nhỏ nhất tạo điều kiện thuận lợi nhất để các phần tử đứng sau ghép nối vào tạo thành dãy con dài hơn.

8. Ma trận 15 bài tập thực hành theo mức độ (p0 → p5)

Mã Bài Tập	Tên Bài Toán	Mức Độ	Trọng Tâm Kiến Thức & DP Invariant				
CPPB-DP1-01	Bước Nhảy Bậc Thang Cơ Bản	P0	Nhận diện Base Case $dp[0]=1$, $dp[1]=1$ & Công thức truy hồi ID .				
CPPB-DP1-02	Chú Ếch Nhảy Cốc Chi Phí Min	P1	$dp[i] = \min(dp[i-1] +$	h_{i-1}	$dp[i-2]$	h_{i-2}	$)$
CPPB-DP1-03	Trộm Nhà Không Liên Kề (House Robber)	P1	Quyết định nhị phân: $dp[i] = \max(dp[i-1], dp[i-2] + A[i])$.				
CPPB-DP1-04	Delete and Earn Tối Đa Điểm	P2	Quy đổi bài toán trên mảng giá trị về mô hình House Robber.				
CPPB-DP1-05	Đổi Tiền Ít Đồng Xu Nhất	P2	$dp[S] = 1 + \min(dp[S-c])$, xử lý khởi tạo giá trị vô cùng ∞ .				
CPPB-DP1-06	Đếm Số Cách Đổi Tiền (Tổ Hợp)	P2	Vòng lặp Coin ngoài, Tiền trong để đếm không trùng lặp.				
CPPB-DP1-07	Dãy Con Tăng Dài Nhất Cơ Bản (LIS)	P2	Cài đặt chuẩn $O(N^2)$ và trích xuất $\max(dp[i])$.				
CPPB-DP1-08	Dãy Con Tăng Có Tổng Lớn Nhất (MSIS)	P3	$dp[i] = A[i] + \max(dp[j])$, biến thể cực đại hóa tổng giá trị.				
CPPB-DP1-09	Mua Bán Cổ Phiếu Tối Ưu 1 Lần	P3	Duy trì giá trị nhỏ nhất tiền tố kết hợp DP $O(N)$ thời gian, $O(1)$ bộ nhớ.				
CPPB-DP1-10	Dãy Con Đan Dấu Dài Nhất	P3	Mở rộng trạng thái $dp[i][0]$ (bước giảm) và $dp[i][1]$ (bước tăng).				
CPPB-DP1-11	Cắt Thanh Gỗ Tối Ưu (Rod Cutting)	P3	Thử mọi nhát cắt $j \in [1..i]$, tối ưu hóa doanh thu.				
CPPB-DP1-12	Khôi Phục Dãy LIS Cụ Thể	P4	Cài đặt mảng <code>trace[i]</code> và lần ngược tái tạo dãy phần tử tối ưu.				
CPPB-DP1-13		P4					

Mã Bài Tập	Tên Bài Toán	Mức Độ	Trọng Tâm Kiến Thức & DP Invariant
	Khôi Phục Danh Sách Đồng Xu Đổi Tiền		Truy vết các mệnh giá xu đã được lựa chọn để tạo nên tổng <code>S</code> .
<code>CPPB-DP1-14</code>	LIS Tối Ưu <code>N log N</code> (Challenge)	P4	Kỹ thuật mảng <code>tails</code> kết hợp Tìm kiếm nhị phân <code>lower_bound</code> .
<code>CPPB-DP1-15</code>	Phân Đoạn Dãy Số Tối Ưu (Mastery)	P5	<code>dp[i] = min(dp[j] + cost(j+1, i))</code> , phân hoạch tiền tố tối ưu.

<!-- Chuyên đề 14: Quy Hoạch Động 2D & Knapsack | 400 dòng | 21,360 bytes -->

Chuyên đề 14: Quy hoạch động 2d & bài toán cái túi (knapsack)

1. Bản chất không gian trạng thái 2d

Trong Chuyên đề 13, trạng thái $dp[i]$ chỉ phụ thuộc vào một tham số đơn lẻ (vị trí trên dãy số hoặc giá trị tổng tiền). Tuy nhiên, trong thực tế thi đấu, bài toán thường yêu cầu thỏa mãn đồng thời **hai điều kiện độc lập**:

- Quy hoạch động trên Lưới tọa độ (Grid DP):** Trạng thái được định vị bởi cặp tọa độ (i, j) trên ma trận $N \times M$.
- Quy hoạch động Bài toán Cái túi (Knapsack DP):** Trạng thái cần theo dõi đồng thời **Chỉ số món đồ đang xét i và Sức chứa còn lại của cái túi w** .

Bản chất Không gian Trạng thái 2D: Mỗi ô $dp[i][j]$ là một đỉnh trong Đồ thị trạng thái DAG 2 chiều. Thứ tự tính toán phải quét qua toàn bộ các hàng và cột theo chiều tăng dần (hoặc giảm dần có kiểm soát) để đảm bảo tính đúng đắn của mọi quan hệ phụ thuộc.

! [Ma trận Quy hoạch động trên Lưới 2D] (/Users/vu/Developer/ikhEdu_lessons/courses/cpp-bang-b/lessons/lesson-14-quy-hoach-dong-2d-knapsack/assets/grid_dp_matrix_vi.svg)

2. Quy hoạch động trên lưới tọa độ (grid dp)

2.1. Đếm số đường đi trên lưới & xử lý vật cản

- Bối cảnh:** Bắt đầu từ ô $(1, 1)$, cần đi đến ô (N, M) . Tại mỗi ô (i, j) , chỉ được phép di chuyển **Sang phải** $(i, j+1)$ hoặc **Xuống dưới** $(i+1, j)$. Trên lưới có một số ô là vật cản không thể đi vào.
- State Definition:** $dp[i][j]$ là số đường đi hợp lệ từ $(1, 1)$ đến (i, j) .
- Base Case:** $dp[1][1] = (\text{grid}[1][1] == 0 ? 1 : 0)$.
- State Transition:** Nếu ô (i, j) là vật cản $\Rightarrow dp[i][j] = 0$. Ngược lại:

$$dp[i][j] = (dp[i-1][j] + dp[i][j-1]) \bmod \{10^9 + 7\}$$

- Evaluation Order:** Duyệt lồng 2 vòng lặp: Hàng $i = 1 \rightarrow N$, Cột $j = 1 \rightarrow M$.

2.2. Tìm đường đi có tổng giá trị lớn nhất / nhỏ nhất

- Mỗi ô (i, j) chứa một số nguyên $A[i][j]$. Cần tìm đường đi từ $(1, 1)$ đến (N, M) có tổng giá trị lớn nhất:

$$dp[i][j] = A[i][j] + \max(dp[i-1][j], dp[i][j-1])$$

3. Bài toán cái túi 0/1 (0/1 knapsack problem)

3.1. Bảng phương án 2d chuẩn mực ($DP[i][w]$)

- Bối cảnh:** Cho N món đồ, món thứ i có khối lượng wt_i và giá trị val_i . Cái túi có sức chứa tối đa W . Mỗi món đồ được chọn **tối đa 1 lần** (0 hoặc 1).
- State Definition:** $dp[i][w]$ là tổng giá trị lớn nhất có thể đạt được khi **chỉ xét trong i món đồ đầu tiên** với tổng khối lượng không vượt quá w .
- Base Cases:** $dp[0][w] = 0$ với mọi $0 \leq w \leq W$ (Không có đồ thì giá trị bằng 0).
- State Transition:** Tại món đồ thứ i , ta có 2 quyết định:
 - Nếu $w < wt_i \Rightarrow dp[i][w] = dp[i-1][w]$ (Không đủ sức chứa để chọn món i).
 - Nếu $w \geq wt_i \Rightarrow dp[i][w] = \max(dp[i-1][w], val_i + dp[i-1][w - wt_i])$.
- Độ phức tạp:** Thời gian $O(N \cdot W)$, Bộ nhớ $O(N \cdot W)$.

3.2. Tuyệt kỹ nén mảng 1d (space optimization & backward traversal)

Nhận xét rằng dòng $dp[i][...]$ **chỉ phụ thuộc duy nhất vào dòng ngay trước nó** là $dp[i-1][...]$. Ta có thể nén bảng 2D thành một mảng 1D $dp[w]$ kích thước $W + 1$.

- Từ huyệt bắt buộc:** Vòng lặp sức chứa w bắt buộc phải **duyệt ngược** từ W giảm dần về wt_i :

```
for (int w = W; w ≥ wt[i]; --w) {
    dp[w] = max(dp[w], val[i] + dp[w - wt[i]]);
}
```

- Tại sao phải duyệt ngược?** Khi tính $dp[w]$, ô $dp[w - wt[i]]$ vẫn giữ nguyên giá trị của tầng $i-1$ (chưa bị đồ thứ i ghi đè), đảm bảo mỗi món đồ chỉ được dùng tối đa 1 lần duy nhất!

![[Kỹ thuật Nén mảng 1D trong 0/1 Knapsack](/Users/vu/Developer/ikhEdu_lessons/courses/cpp-bang-b/lessons/lesson-14-quy-hoach-dong-2d-knapsack/assets/knapsack_01_compression_vi.svg)]

4. Bài toán cái túi không giới hạn (unbounded knapsack)

Khi mỗi món đồ được phép chọn **vô số lần** không giới hạn:

- **Hệ thức 2D:** $dp[i][w] = \max(dp[i-1][w], val_i + dp[i][w - wt_i])$.
- **Kỹ thuật mảng 1D:** Vòng lặp `w` duyệt ****XUÔI** từ `wti` tăng dần lên `W`:

```
for (int w = wt[i]; w ≤ W; ++w) {
    dp[w] = max(dp[w], val[i] + dp[w - wt[i]]);
}
```

- Duyệt xuôi cho phép trạng thái `dp[w]` kế thừa ngay lập tức kết quả của chính món đồ `i` vừa được thêm vào ở `dp[w - wt[i]]`.

! [So sánh 0/1 Knapsack vs Unbounded Knapsack] (/Users/vu/Developer/ikhEdu_lessons/courses/cpp-bang-b/lessons/lesson-14-quy-hoach-dong-2d-knapsack/assets/unbounded_vs_01_knapsack_vi.svg)

5. Kỹ thuật đổi trục dp khi sức chứa W quá lớn ($DP[v] = \text{Min Weight}$)

- **Bối cảnh:** $N \leq 100$, nhưng sức chứa $W \leq 10^9$ (không thể tạo mảng kích thước 10^9), trong khi tổng giá trị tối đa $V_{sum} = \sum val_i \leq 10^5$.
- **Đổi Trục Trạng Thái (State Redesign):**
 - Đặt `dp[v]` là **Khối lượng nhỏ nhất** để đạt được đúng tổng giá trị `v`.
 - Base case: `dp[0] = 0`, mọi `dp[v] = INF` ($v \geq 1$).
 - Chuyển trạng thái: Duyệt ngược `v` từ `Vsum` về `vali`:

$$dp[v] = \min(dp[v], wt_i + dp[v - val_i])$$

- Đáp án: $\max \{v \mid dp[v] \leq W\}$.
- **Độ phức tạp:** $O(N \cdot V_{sum})$ — Chạy mượt mà dưới `0.05` giây!

6. Các bẫy lỗi lập trình kinh điển (bug traps)

1. Bẫy duyệt xuôi trong 0/1 Knapsack mảng 1D:

- Viết `for (int w = wt[i]; w ≤ W; ++w)` cho bài `0/1` Knapsack sẽ biến thuật toán thành Unbounded Knapsack $\Rightarrow$ Món đồ bị lấy nhiều lần, sai hoàn toàn kết quả!

1. Bẫy tràn chỉ số biên âm trên Lưới 2D:

- Tại hàng 1 và cột 1, `dp[i-1][j]` hoặc `dp[i][j-1]` sẽ truy cập vào chỉ số 0.

- **Quy tắc an toàn:** Khai báo mảng 1-based kích thước $(N+2) \times (M+2)$ và khởi tạo viền bằng 0 (cho bài đếm cách) hoặc `-inf` (cho bài tìm Max).

1. Bẫy mảng 2D quá lớn gây tràn bộ nhớ (Memory Limit Exceeded - MLE):

- Khai báo `long long dp[2000][2000]` tốn $2000 \times 2000 \times 8 \text{ bytes approx } 32 \text{ MB}$ (an toàn).
Nhưng `long long dp[10000][10000]` tốn $800 \text{ MB} \Rightarrow$ Sập bộ nhớ 256 MB . Bắt buộc phải nén thành mảng 1D.

7. Mẫu cài đặt chuẩn thi đấu (competitive templates)

Mẫu 1: Bài toán cái túi 0/1 nén mảng 1d

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    long long W;
    if (!(cin >> n >> W)) return 0;

    if (n <= 0 || W < 0) return 0;

    vector<long long> wt(n), val(n);

    for (int i = 0; i < n; ++i) {
        cin >> wt[i] >> val[i];
    }

    vector<long long> dp(W + 1, 0);

    for (int i = 0; i < n; ++i) {
        // Duyệt ngược w giảm dần từ W về wt[i]
        for (long long w = W; w >= wt[i]; --w) {
            dp[w] = max(dp[w], val[i] + dp[w - wt[i]]);
        }
    }

    cout << dp[W] << "\n";
    return 0;
}
```

Mẫu 2: Đường đi có tổng lớn nhất trên lưới 2d

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    if (!(cin >> n >> m)) return 0;

    if (n ≤ 0 || m ≤ 0) return 0;

    vector<vector<long long>> a(n + 1, vector<long long>(m + 1));

    for (int i = 1; i ≤ n; ++i) {
        for (int j = 1; j ≤ m; ++j) {
            cin >> a[i][j];
        }
    }

    vector<vector<long long>> dp(n + 1, vector<long long>(m + 1, 0));

    for (int i = 1; i ≤ n; ++i) {
        for (int j = 1; j ≤ m; ++j) {
            if (i == 1 && j == 1) {
                dp[i][j] = a[1][1];
            } else if (i == 1) {
                dp[i][j] = dp[1][j - 1] + a[i][j];
            } else if (j == 1) {
                dp[i][j] = dp[i - 1][1] + a[i][j];
            } else {
                dp[i][j] = a[i][j] + max(dp[i - 1][j], dp[i][j - 1]);
            }
        }
    }

    cout << dp[n][m] << "\n";
    return 0;
}

```

8. Hệ thống câu hỏi kiểm tra khái niệm (concept quiz)

Câu 1 (bản chất không gian trạng thái 2d):

Trong bài toán Cái túi 0/1 Knapsack, tại sao cần đến hai tham số trạng thái $dp[i][w]$?

- A. Vì mỗi món đồ có hai thuộc tính là tên gọi và mã số.
- B. **[Đáp án đúng]** Vì cần theo dõi đồng thời tập món đồ đã xét ($1 \rightarrow i$) và lượng sức chứa đã tiêu tốn (w).
- C. Để tăng độ phức tạp thời gian thuật toán.
- D. Vì ma trận 2D luôn chạy nhanh hơn mảng 1D.

Giải thích: Một chiều đại diện cho tiến trình duyệt các lựa chọn (i), chiều còn lại đại diện cho giới hạn tài nguyên bị ràng buộc (w).

Câu 2 (chiều duyệt vòng lặp nén mảng 0/1 knapsack):

Khi nén bảng $DP[i][w]$ thành mảng 1D $DP[w]$ trong bài toán 0/1 Knapsack, chiều duyệt của biến w bắt buộc phải như thế nào?

- A. Duyệt xuôi từ $wt_i \rightarrow W$.
- B. **[Đáp án đúng]** Duyệt ngược từ $W \rightarrow wt_i$.
- C. Duyệt ngẫu nhiên không theo thứ tự.
- D. Duyệt nhảy cóc 2 bước.

Giải thích: Duyệt ngược đảm bảo khi tính $dp[w]$, giá trị $dp[w - wt_i]$ chưa bị món đồ thứ i cập nhật đè lên, giữ trọn vẹn bất biến mỗi đồ dùng tối đa 1 lần.

Câu 3 (unbounded knapsack vs 0/1 knapsack):

Điểm khác biệt duy nhất trong mã nguồn C++ giữa 0/1 Knapsack mảng 1D và Unbounded Knapsack mảng 1D là gì?

- A. Thay hàm `max` bằng hàm `min`.
- B. **[Đáp án đúng]** 0/1 Knapsack duyệt ngược w giảm dần, còn Unbounded Knapsack duyệt xuôi w tăng dần.
- C. Khởi tạo mảng bằng `-1`.
- D. Dùng thêm một mảng phụ thứ hai.

Giải thích: Duyệt xuôi cho phép trạng thái $dp[w]$ tận dụng ngay kết quả vừa cập nhật của chính món đồ i tại $dp[w - wt_i]$, tương đương việc lấy thêm món đồ i lần thứ 2, 3...

Câu 4 (xử lý vật cản trên lưới 2d):

Khi đếm số đường đi trên lưới $N \times M$, nếu ô (i, j) có chứa vật cản, giá trị $dp[i][j]$ phải được xử lý như thế nào?

- A. $dp[i][j] = -1$
- B. **[Đáp án đúng]** $dp[i][j] = 0$ (Gán bằng 0 và không nhận luồng đi qua).
- C. $dp[i][j] = dp[i-1][j] + dp[i][j-1]$
- D. Bỏ qua không khởi tạo.

Giải thích: Ô có vật cản không thể bước vào $\Rightarrow$ Số cách đi đến ô này bằng 0. Khi các ô phía sau cộng dồn sẽ không nhận thêm cách nào từ ô cản này.

Câu 5 (kỹ thuật đổi trục dp theo value):

Khi $N = 100$ và $W = 10^9$ nhưng $\sum val_i \leq 10^5$, tại sao ta đổi trạng thái thành $dp[v]$ = Khối lượng nhỏ nhất để đạt giá trị v ?

- A. Vì thuật toán cũ bị sai đáp án.
- B. **[Đáp án đúng]** Vì không thể tạo mảng kích thước 10^9 ($O(W)$ gây MLE/TLE), trong khi mảng kích thước 10^5 chạy cực nhanh và tốn rất ít bộ nhớ.
- C. Để làm cho code ngắn hơn.
- D. Vì khối lượng luôn nhỏ hơn giá trị.

Giải thích: Đây là bài học kinh điển về Thiết kế Trạng thái: Chọn trục DP dựa trên đại lượng có miền giá trị khả thi trong giới hạn tài nguyên.

Câu 6 (bài toán subset sum):

Cho mảng N số nguyên dương và số S . Bài toán kiểm tra tồn tại tập con có tổng bằng S thực chất là trường hợp đặc biệt của bài toán nào?

- A. Dãy con tăng dài nhất (LIS).
- B. **[Đáp án đúng]** Bài toán Cái túi 0/1 Knapsack với $wt_i = val_i = A[i]$ và kiểu dữ liệu boolean.
- C. Bài toán Hai con trò.
- D. Thuật toán Euclid.

Giải thích: $dp[w] = true$ nếu có thể tạo ra tổng khối lượng đúng bằng w . Chuyển trạng thái:
 $dp[w] = dp[w] \text{ lor } dp[w - A[i]]$.

Câu 7 (khôi phục danh sách đồ trong cái túi 0/1):

Để in ra danh sách các món đồ được chọn trong bài toán Cái túi 0/1, ta cần lưu trữ bảng phương án ở dạng nào?

- A. Mảng 1D $dp[w]$.
- B. [Đáp án đúng] Bảng phương án 2D đầy đủ $dp[i][w]$, sau đó lần ngược từ (N, W) .
- C. Không thể khôi phục được.
- D. Chỉ cần lưu mảng ban đầu.

Giải thích: Truy vết 2D so sánh nếu $dp[i][w] \neq dp[i-1][w] \Rightarrow$ Món i đã được chọn, ghi nhận món i và lùi về $w \geq w - wt_i$, $i \geq i - 1$.

Câu 8 (độ phức tạp lưới tam giác triangle dp):

Cho tam giác số gồm N hàng, hàng thứ i có i số. Độ phức tạp thời gian để tìm đường đi từ đỉnh xuống đáy có tổng lớn nhất là bao nhiêu?

- A. $O(2^N)$
- B. [Đáp án đúng] $O(N^2)$
- C. $O(N)$
- D. $O(N \log N)$

Giải thích: Tổng số ô trong tam giác là $1 + 2 + \dots + N = N(N+1)/2 \approx N^2/2$. Mỗi ô tính trong $O(1) \Rightarrow O(N^2)$.

Câu 9 (bẫy bộ nhớ 2d mle):

Khai báo mảng `long long dp[5000][5000]` trong C++ sẽ tiêu tốn xấp xỉ bao nhiêu bộ nhớ RAM?

- A. 25 MB
- B. [Đáp án đúng] *approx 200 MB* ($5000 \times 5000 \times 8 \text{ bytes} = 200,000,000 \text{ bytes}$).
- C. 2 GB
- D. 10 MB

Giải thích: Cần tính toán kích thước bộ nhớ trước khi khai báo: 200MB nằm sát giới hạn 256MB của nhiều kỳ thi, rất dễ bị MLE nếu có thêm các mảng phụ khác.

Câu 10 (bài toán phân chia tập con bằng nhau):

Để kiểm tra xem mảng A có thể chia thành 2 tập con có tổng bằng nhau hay không, điều kiện cần đầu tiên là gì?

- **A.** Số phần tử N phải là số chẵn.
- **B. [Đáp án đúng]** Tổng toàn bộ mảng $S = \sum A_i$ phải là số chẵn, sau đó quy về bài toán tìm tập con tổng $S/2$.
- **C.** Mảng phải được sắp xếp tăng dần.
- **D.** Mọi phần tử phải dương.

Giải thích: Nếu S lẻ thì không thể chia đôi thành 2 số nguyên bằng nhau. Nếu S chẵn, bài toán trở thành Subset Sum với mục tiêu $target = S/2$.

Câu 11 (quy hoạch động từ dưới lên trên triangle dp):

Tại sao khi giải bài toán Tam giác số, duyệt từ hàng đáy $N-1$ ngược lên đỉnh 0 lại tiện lợi hơn duyệt từ đỉnh xuống?

- **A.** Vì chạy nhanh hơn gấp đôi.
- **B. [Đáp án đúng]** Vì đáp án cuối cùng gom lại đúng 1 ô duy nhất tại đỉnh $dp[0][0]$, không cần tìm max trên toàn bộ hàng đáy.
- **C.** Vì không cần dùng mảng.
- **D.** Vì tránh được tràn số.

Giải thích: Chuyển trạng thái từ dưới lên: $dp[i][j] = A[i][j] + \max(dp[i+1][j], dp[i+1][j+1]) \Rightarrow$ Kết quả hội tụ về $dp[0][0]$.

Câu 12 (knapsack 2 chiều ràng buộc):

Nếu cái túi vừa có giới hạn khối lượng W , vừa có giới hạn thể tích V , mảng phương án nén 1D cần mở rộng thành mảng mấy chiều?

- A. Vẫn là mảng 1D.
- B. **[Đáp án đúng]** Mảng 2D $dp[w][v]$ với 2 vòng lặp duyệt ngược w từ $W \rightarrow w_i$ và v từ $V \rightarrow vol_i$.
- C. Mảng 4D.
- D. Không thể giải bằng DP.

Giải thích: Nén chiều món đồ i , giữ lại 2 chiều tài nguyên ràng buộc (w, v) , cả 2 vòng lặp đều duyệt ngược để đảm bảo mỗi đồ dùng tối đa 1 lần.

Câu 13 (tối ưu hóa bộ nhớ lưới 2d bằng 2 dòng):

Khi tính $dp[i][j] = A[i][j] + \max(dp[i-1][j], dp[i][j-1])$ trên lưới $N \times M$, nếu $N, M \leq 10^4$ nhưng bộ nhớ giới hạn $16MB$, ta có thể tối ưu không gian như thế nào?

- A. Dùng thuật toán đệ quy.
- B. **[Đáp án đúng]** Chỉ lưu 2 dòng phương án `prev_row` và `curr_row` kích thước $O(M)$, giảm bộ nhớ từ $O(N \times M)$ về $O(M)$.
- C. Bỏ qua không dùng DP.
- D. Ép kiểu dữ liệu về `char`.

Giải thích: Ô (i, j) chỉ phụ thuộc ô cùng cột của hàng trên $(i-1, j)$ và ô bên trái $(i, j-1)$, do đó chỉ cần duy trì 2 dòng liên tiếp.

Câu 14 (knapsack phân chia chênh lệch nhỏ nhất):

Cho mảng N phần tử tổng S . Để chia thành 2 nhóm có tổng s_1, s_2 sao cho $|s_1 - s_2|$ nhỏ nhất, ta tìm giá trị s_1 như thế nào?

- A. $s_1 = S / 2$.
- B. **[Đáp án đúng]** Chạy Subset Sum tìm tổng $s_1 \leq S / 2$ lớn nhất có thể đạt được, sau đó độ chênh lệch là $S - 2 \cdot s_1$.
- C. Sắp xếp mảng rồi chia đôi.
- D. Lấy phần tử lớn nhất trừ phần tử nhỏ nhất.

Giải thích: $s_1 + s_2 = S \Rightarrow |s_1 - s_2| = |S - 2s_1|$. Cực tiểu hóa đại lượng này tương đương tìm $s_1 \leq \lfloor S/2 \rfloor$ lớn nhất có $dp[s_1] = true$.

Câu 15 (truy vết đường đi trên lưới 2d):

Khi lần ngược từ ô (N, M) về ô $(1, 1)$ để in ra các bước đi D (Down) và R (Right), thứ tự các bước đi được ghi nhận như thế nào?

- **A.** In trực tiếp không cần đảo ngược.
- **B. [Đáp án đúng]** Lưu các ký tự vào chuỗi rồi đảo ngược lại chuỗi trước khi in ra.
- **C.** Chạy lại thuật toán từ đầu.
- **D.** Dùng đệ quy in xuôi.

Giải thích: Vì lần ngược từ đích về xuất phát nên chuỗi thu được bị ngược chiều, cần **reverse** để có lộ trình chuẩn từ $(1, 1) \rightarrow (N, M)$.

9. Ma trận 15 bài tập thực hành theo mức độ (p0 → p5)

Mã Bài Tập	Tên Bài Toán	Mức Độ	Trọng Tâm Kiến Thức & DP Invariant		
CPPB-DP2-01	Đường Đi Trên Lưới Chi Phí Min	P0	$dp[i][j] = A[i][j] + \min(dp[i-1][j], dp[i][j-1])$ cơ bản.		
CPPB-DP2-02	Đếm Số Đường Đi Trên Lưới	P1	$dp[i][j] = (dp[i-1][j] + dp[i][j-1]) \bmod \{10^9+7\}$.		
CPPB-DP2-03	Đường Đi Lưới Có Vật Cản	P1	Xử lý ô cấm gán $dp[i][j] = 0$, xử lý Base Case xuất phát.		
CPPB-DP2-04	Tam Giác Số Tối Ưu (Triangle DP)	P2	Quy hoạch động từ đáy lên đỉnh hội tụ tại $dp[0][0]$.		
CPPB-DP2-05	Cái Túi 0/1 Knapsack Cơ Bản	P2	Cài đặt chuẩn mảng nén 1D $dp[w]$ duyệt ngược w giảm.		
CPPB-DP2-06	Kiểm Tra Tập Con Có Tổng Bằng S	P2	Subset Sum boolean $dp[w] = dp[w] \vee dp[w - A[i]]$.	$dp[w - A[i]]$.	
CPPB-DP2-07	Chia Mảng Thành 2 Phần Bằng Nhau	P2	Kiểm tra tổng chẵn và quy về Subset Sum với mục tiêu $S/2$.		
CPPB-DP2-08	Phân Chia Tập Hợp Chênh Lệch Min	P3	Tìm tổng tập con gần $S/2$ nhất, cực tiểu hóa \$	$S - 2 \cdot s_1$	\$.
CPPB-DP2-09	Unbounded Knapsack (Đồ Vô Hạn)	P3	Vòng lặp w duyệt xuôi tăng dần từ $w_i \rightarrow W$.		
CPPB-DP2-10	Đổi Tiền 2D Số Cách Tổ Hợp	P3	Đếm số cách đổi tiền không phân biệt thứ tự (Unbounded Ways).		
CPPB-DP2-11	Knapsack Theo Tổng Giá Trị (Value DP)	P3	Đổi trực $dp[v] = \text{Min Weight}$ khi $W \leq 10^9, V \leq 10^5$.		
CPPB-DP2-12	Khôi Phục Đường Đi Trên Lưới 2D	P4	Lần ngược từ (N, M) về $(1, 1)$ in ra chuỗi bước đi D và R .		
CPPB-DP2-13	Khôi Phục Danh Sách Món Đồ Cái Túi	P4	Lần ngược trên bảng $dp[i][w]$ tái tạo các món đồ được chọn.		
CPPB-DP2-14	Knapsack 2 Chiều Khối Lượng & Thể Tích	P4	Nén 2 chiều $dp[w][v]$ duyệt ngược 2 biến độc lập.		
CPPB-DP2-15	Quy Hoạch Động Lưới Thi Đấu (Mastery)	P5	Lưới ma trận với quy tắc di chuyển mở rộng chuẩn Olympic.		

<!-- Chuyên đề 15: Quy Hoạch Động Chuỗi & LCS | 410 dòng | 20,461 bytes -->

Chuyên đề 15: Quy hoạch động chuỗi: Lcs & edit distance

1. Không gian trạng thái hai tiền tố (2-prefix state)

Trong xử lý chuỗi ký tự thi đấu, các bài toán so khớp, tìm chuỗi tương đồng hay biến đổi xâu thường thao tác trên hai chuỗi A (độ dài N) và B (độ dài M).

- **Nguyên lý Thiết kế Trạng thái:** Ta định nghĩa trạng thái dựa trên **Cặp tiền tố** của hai chuỗi:

**** $dp[i][j]$ đại diện cho kết quả tối ưu khi xét tiền tố $A[1 \dots i]$ (gồm i ký tự đầu của A) và tiền tố $B[1 \dots j]$ (gồm j ký tự đầu của B). ****

- **Trường hợp cơ sở (Base Cases):** Khi một trong hai tiền tố có độ dài bằng 0 ($i = 0$ hoặc $j = 0$), tương đương với chuỗi rỗng varepsilon .

2. Dãy con chung dài nhất (longest common subsequence — lcs)

2.1. Bản chất toán học & hệ thức truy hồi

- **Định nghĩa:** Dãy con là dãy thu được bằng cách xóa đi một số ký tự mà **không làm thay đổi thứ tự** của các ký tự còn lại.
- **State Definition:** $dp[i][j]$ là độ dài của dãy con chung dài nhất giữa $A[1 \dots i]$ và $B[1 \dots j]$.
- **Base Cases:** $dp[0][j] = 0$ và $dp[i][0] = 0$ với mọi i, j .
- **State Transition:** So sánh ký tự đuôi $A[i]$ và $B[j]$:

1. Nếu $A[i] == B[j]$: Ký tự này chắc chắn thuộc LCS chung:

$$dp[i][j] = 1 + dp[i-1][j-1]$$

1. Nếu $A[i] \neq B[j]$: Bỏ qua $A[i]$ hoặc bỏ qua $B[j]$ để lấy phương án tốt hơn:

$$dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$$

- **Độ phức tạp:** Thời gian $O(N \cdot M)$, Bộ nhớ $O(N \cdot M)$.

![Bảng phương án LCS và Đường truy vết](/Users/vu/Developer/ikhEdu_lessons/courses/cpp-bang-b/lessons/lesson-15-quy-hoach-dong-chuoi-lcs/assets/lcs_table_traceback_vi.svg)

2.2. Kỹ thuật khôi phục chuỗi lcs tối ưu (traceback)

Từ ô kết quả (N, M) trên bảng phương án 2D:

1. Nếu $A[i] == B[j] \Rightarrow$ Thêm $A[i]$ vào chuỗi kết quả, lùi chéo về $(i-1, j-1)$.
2. Nếu $A[i] \neq B[j] \Rightarrow$ Đi về ô có giá trị lớn hơn: lên trên $(i-1, j)$ nếu $dp[i-1][j] \geq dp[i][j-1]$, ngược lại sang trái $(i, j-1)$.
3. Dừng lại khi $i = 0$ hoặc $j = 0$. Đảo ngược chuỗi kết quả thu được.

3. Khoảng cách biến đổi chuỗi (edit distance / levenshtein distance)

3.1. Bản chất 3 phép biến đổi

Cần tìm số phép biến đổi ít nhất để biến chuỗi A thành chuỗi B . Các phép thao tác hợp lệ gồm:

1. **Chèn (Insert):** Thêm 1 ký tự vào chuỗi A .
2. **Xóa (Delete):** Xóa 1 ký tự khỏi chuỗi A .
3. **Thay thế (Replace):** Đổi 1 ký tự của A thành ký tự khác.

3.2. Hệ thức chuyển trạng thái

- **State Definition:** $dp[i][j]$ là số thao tác ít nhất biến $A[1 \dots i]$ thành $B[1 \dots j]$.
- **Base Cases:**
 - $dp[i][0] = i$ (Biến chuỗi độ dài i thành chuỗi rỗng cần i phép xóa).
 - $dp[0][j] = j$ (Biến chuỗi rỗng thành chuỗi độ dài j cần j phép chèn).
- **State Transition:**
 - Nếu $A[i] == B[j] \Rightarrow dp[i][j] = dp[i-1][j-1]$ (Không tốn chi phí).
 - Nếu $A[i] \neq B[j]$:

$$dp[i][j] = 1 + \min(\underbrace{dp[i-1][j-1]}_{\text{Thay thế}}, \underbrace{dp[i-1][j]}_{\text{Xóa}}, \underbrace{dp[i][j-1]}_{\text{Chèn}})$$

![Khoảng cách biến đổi chuỗi Edit Distance](/Users/vu/Developer/ikhEdu_lessons/courses/cpp-bang-b/lessons/lesson-15-quy-hoach-dong-chuoi-lcs/assets/edit_distance_transitions_vi.svg)

4. Phân biệt rạch ròi: Chuỗi con đối xứng (substring) vs dãy con đối xứng (subsequence)

Đây là từ huyệt thuật ngữ cực kỳ quan trọng trong lập trình thi đấu:

![Phân biệt Xâu con liên tiếp vs Dãy con đối xứng](/Users/vu/Developer/ikhEdu_lessons/courses/cpp-bang-b/lessons/lesson-15-quy-hoach-dong-chuoi-lcs/assets/palindrome_substring_vs_subsequence_vi.svg)

4.1. Xâu con liên tiếp đối xứng dài nhất (longest palindromic substring)

- **Đặc tính:** Các ký tự phải **liên tiếp liền kề** nhau.
- **State Definition:** $dp[i][j]$ kiểu boolean, nhận giá trị **true** nếu đoạn con liên tiếp $S[i \dots j]$ là một xâu đối xứng.
- **Transition:** $dp[i][j] = (S[i] == S[j]) \text{ and } dp[i+1][j-1]$.
- **Duyệt:** Theo độ dài xâu con $len = 1 \rightarrow N$.

4.2. Dãy con không liên tiếp đối xứng dài nhất (longest palindromic subsequence)

- **Đặc tính:** Các ký tự **không cần liên tiếp**.
- **State Definition:** $dp[i][j]$ là độ dài lớn nhất của dãy con đối xứng trích xuất từ đoạn $S[i \dots j]$.
- **Transition:**
 - Nếu $S[i] == S[j] \Rightarrow dp[i][j] = 2 + dp[i+1][j-1]$.
 - Nếu $S[i] \neq S[j] \Rightarrow dp[i][j] = \max(dp[i+1][j], dp[i][j-1])$.
- **Cách giải tương đương:** Tính **LCS** giữa xâu S và xâu đảo ngược S^R !

5. Các bẫy lỗi lập trình kinh điển (bug traps)

1. Bẫy lệch chỉ số 0-based của Xâu ký tự với 1-based của Bảng DP:

- Trong C++, **string** có chỉ số từ **0** đến $N-1$. Khi truy cập ký tự thứ i trong bảng DP 1-based, phải viết **$A[i - 1]$** thay vì **$A[i]$** .

1. Bẫy thứ tự duyệt trong Quy hoạch động trên Đoạn con Palindrome:

- Trạng thái $dp[i][j]$ phụ thuộc vào $dp[i+1][j-1]$ (đoạn ngắn hơn). Nếu duyệt i từ $1 \rightarrow N$ xuôi thì ô $dp[i+1][j-1]$ chưa được tính $\Rightarrow$ Kết quả sai!
- **Quy tắc đúng:** Luôn duyệt theo độ dài $len = 1 \rightarrow N$, sau đó duyệt điểm đầu $i = 1 \rightarrow N - len + 1$ và $j = i + len - 1$.

1. Bẫy khởi tạo Base Case của Edit Distance:

- Quên khởi tạo cột **0** ($dp[i][0] = i$) và hàng **0** ($dp[0][j] = j$) sẽ dẫn đến toàn bộ bảng nhận giá trị rác.

6. Mẫu cài đặt chuẩn thi đấu (competitive templates)

Mẫu 1: Dãy con chung dài nhất (lcs) kèm khôi phục sâu

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string a, b;
    if (!(cin >> a >> b)) return 0;

    int n = a.size();
    int m = b.size();
    if (n == 0 || m == 0) {
        cout << 0 << "\n\n";
        return 0;
    }

    vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));

    for (int i = 1; i ≤ n; ++i) {
        for (int j = 1; j ≤ m; ++j) {
            if (a[i - 1] == b[j - 1]) {
                dp[i][j] = 1 + dp[i - 1][j - 1];
            } else {
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
            }
        }
    }

    cout << dp[n][m] << "\n";

    // Khôi phục chuỗi LCS
    string lcs_str = "";
    int i = n, j = m;
    while (i > 0 && j > 0) {
        if (a[i - 1] == b[j - 1]) {
            lcs_str.push_back(a[i - 1]);
            --i;
            --j;
        } else if (dp[i - 1][j] ≥ dp[i][j - 1]) {
            --i;
        } else {
            --j;
        }
    }
    reverse(lcs_str.begin(), lcs_str.end());
    cout << lcs_str << "\n";

    return 0;
}

```

Mẫu 2: Khoảng cách biến đổi xâu (edit distance)

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string a, b;
    if (!(cin >> a >> b)) return 0;

    int n = a.size();
    int m = b.size();

    vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));

    for (int i = 0; i ≤ n; ++i) dp[i][0] = i;
    for (int j = 0; j ≤ m; ++j) dp[0][j] = j;

    for (int i = 1; i ≤ n; ++i) {
        for (int j = 1; j ≤ m; ++j) {
            if (a[i - 1] == b[j - 1]) {
                dp[i][j] = dp[i - 1][j - 1];
            } else {
                dp[i][j] = 1 + min({dp[i - 1][j - 1], // Replace
                                   dp[i - 1][j],      // Delete
                                   dp[i][j - 1]});    // Insert
            }
        }
    }

    cout << dp[n][m] << "\n";
    return 0;
}
```

7. Hệ thống câu hỏi kiểm tra khái niệm (concept quiz)

Câu 1 (không gian trạng thái 2-prefix):

Trong bài toán LCS trên hai chuỗi A và B , trạng thái $dp[i][j]$ đại diện chính xác cho điều gì?

- A. Độ dài xâu A cộng với độ dài xâu B .
- B. **[Đáp án đúng]** Độ dài dãy con chung dài nhất của tiền tố $A[1 \dots i]$ và tiền tố $B[1 \dots j]$.
- C. Số lượng ký tự giống nhau ở vị trí i và j .
- D. Vị trí đầu tiên mà hai xâu khác nhau.

Giải thích: Bất biến trạng thái 2 tiền tố: Xét độc lập bài toán con trên tiền tố độ dài i của A và tiền tố độ dài j của B .

Câu 2 (trường hợp ký tự trùng khớp trong lcs):

Khi $A[i] == B[j]$, tại sao ta chuyển trạng thái $dp[i][j] = 1 + dp[i-1][j-1]$ mà không cần xét $\max(dp[i-1][j], dp[i][j-1])$?

- **A.** Vì phép gán này chạy nhanh hơn.
- **B. [Đáp án đúng]** Vì ghép cặp $A[i]$ với $B[j]$ luôn tối ưu tối đa và đảm bảo không làm mất bất kỳ nghiệm tối ưu nào.
- **C.** Vì $dp[i-1][j]$ luôn bằng 0.
- **D.** Vì hai chuỗi có độ dài bằng nhau.

Giải thích: Bằng chứng minh Tham lam / Cấu trúc con tối ưu, khi 2 ký tự cuối trùng nhau, việc đưa cặp ký tự này vào LCS luôn đạt kết quả tốt nhất.

Câu 3 (cơ sở của edit distance):

Tại sao trong bài toán Edit Distance, $dp[i][0] = i$ và $dp[0][j] = j$?

- **A.** Vì các ô này không dùng đến nên khởi tạo đại diện.
- **B. [Đáp án đúng]** Vì để biến chuỗi i ký tự thành chuỗi rỗng cần đúng i phép xóa, và từ chuỗi rỗng tạo chuỗi j ký tự cần đúng j phép chèn.
- **C.** Vì $i + j$ luôn dương.
- **D.** Để tránh mảng bị tràn số âm.

Giải thích: Đây là các trường hợp cơ sở tự nhiên khi một trong hai chuỗi là chuỗi rỗng $varepsilon$.

Câu 4 (phân biệt substring vs subsequence):

Cho xâu $S = "ABBA"$. Khẳng định nào sau đây là đúng về bản chất Substring và Subsequence?

- **A.** Mọi Subsequence đều là Substring.
- **B. [Đáp án đúng]** Mọi Substring đều là Subsequence, nhưng Subsequence không bắt buộc phải liên tiếp.
- **C.** Substring và Subsequence là hai khái niệm hoàn toàn tương đương.
- **D.** Substring dài hơn Subsequence.

Giải thích: Substring là chuỗi con liên tiếp (khối liền kề). Subsequence là dãy con được tạo bằng cách giữ nguyên thứ tự nhưng có thể bỏ qua một số ký tự trung gian.

Câu 5 (độ phức tạp của lcs và edit distance):

Cho hai xâu có độ dài lần lượt là N và M . Thuật toán DP chuẩn mực chạy trong thời gian và bộ nhớ là bao nhiêu?

- A. Thời gian $O(N + M)$, Bộ nhớ $O(1)$.
- B. **[Đáp án đúng]** Thời gian $O(N \cdot M)$, Bộ nhớ $O(N \cdot M)$ (hoặc $O(\min(N, M))$ nếu nén mảng).
- C. Thời gian $O(2N + M)$, Bộ nhớ $O(N)$.
- D. Thời gian $O(N^2 \cdot M^2)$, Bộ nhớ $O(N \cdot M)$.

Giải thích: Bảng quy hoạch động có kích thước $(N+1) \times (M+1)$, mỗi ô tính trong $O(1) \Rightarrow O(N \cdot M)$.

Câu 6 (mối liên hệ giữa lcs và xâu con chung ngắn nhất scs):

Độ dài xâu ngắn nhất chứa cả hai xâu A (độ dài N) và B (độ dài M) dưới dạng dãy con (Shortest Common Supersequence) được tính bằng công thức nào?

- A. $N + M$
- B. **[Đáp án đúng]** $N + M - LCS(A, B)$
- C. $LCS(A, B)$
- D. $2 \cdot LCS(A, B)$

Giải thích: Tổng độ dài 2 xâu là $N + M$. Các ký tự thuộc phần chung $LCS(A, B)$ chỉ cần xuất hiện đúng 1 lần trong xâu siêu chuỗi $\Rightarrow$ Trừ bớt $LCS(A, B)$.

Câu 7 (tìm dãy con đối xứng dài nhất bằng lcs):

Để tìm độ dài Dãy con không liên tiếp đối xứng dài nhất (Longest Palindromic Subsequence) của xâu S , ta có thể quy về bài toán nào?

- A. Tìm LIS trên xâu S .
- B. **[Đáp án đúng]** Tìm $LCS(S, S^R)$ với S^R là xâu đảo ngược của S .
- C. Tìm Edit Distance giữa S và chuỗi rỗng.
- D. Tìm kiểm nhị phân trên S .

Giải thích: Dãy con đối xứng là dãy con xuất hiện giống nhau theo cả chiều xuôi và chiều ngược $\Rightarrow$ Chính là dãy con chung dài nhất giữa S và S^R .

Câu 8 (thứ tự duyệt trong dp đoạn con palindrome substring):

Khi tính $dp[i][j] = (S[i] == S[j]) \text{ and } dp[i+1][j-1]$, thứ tự duyệt vòng lặp nào sau đây là đúng?

- A. Duyệt $i = 1 \rightarrow N$, $j = 1 \rightarrow N$.
- B. **[Đáp án đúng]** Duyệt độ dài $len = 1 \rightarrow N$, sau đó duyệt điểm đầu $i = 1 \rightarrow N - len + 1$ và đặt $j = i + len - 1$.
- C. Duyệt ngẫu nhiên.
- D. Duyệt $j = N \rightarrow 1$.

Giải thích: Trạng thái đoạn $[i \dots j]$ độ dài len phụ thuộc vào đoạn con bên trong $[i+1 \dots j-1]$ có độ dài $len - 2$. Do đó các đoạn ngắn hơn phải được tính xong trước.

Câu 9 (bảng truy cập ký tự 0-based trong c++):

Trong C++, nếu xâu `string s = "CODE"` và bảng DP khai báo 1-based từ $1 \rightarrow 4$, ký tự tương ứng với chỉ số $i = 3$ trong bảng DP được truy cập là gì?

- A. `s[3]`
- B. **[Đáp án đúng]** `s[2]` (Tức `s[i - 1]`).
- C. `s[4]`
- D. `s[i + 1]`

Giải thích: Chỉ số xâu trong C++ là 0-based (0, 1, 2, 3). Ký tự thứ i (1-based) tương ứng với ô `s[i - 1]`.

Câu 10 (số phép chèn tối thiểu để tạo xâu đối xứng):

Cho xâu S độ dài N . Số ký tự ít nhất cần chèn thêm vào S để biến nó thành một xâu đối xứng là:

- A. N
- B. **[Đáp án đúng]** $N - LPS(S)$ (với $LPS(S)$ là độ dài dãy con đối xứng dài nhất).
- C. $LPS(S) / 2$
- D. $N / 2$

Giải thích: Giữ nguyên $LPS(S)$ ký tự đối xứng có sẵn, chỉ cần chèn thêm đối xứng cho $N - LPS(S)$ ký tự còn lại.

Câu 11 (truy vết chuỗi thao tác edit distance):

Khi truy vết từ ô (i, j) trong bảng Edit Distance, nếu $dp[i][j] == dp[i-1][j] + 1$, thao tác đã được thực hiện là gì?

- A. Chèn ký tự $B[j]$.
- B. **[Đáp án đúng]** Xóa ký tự $A[i]$ khỏi xâu A .
- C. Thay thế ký tự $A[i]$ bằng $B[j]$.
- D. Không làm gì cả.

Giải thích: Ô $(i-1, j)$ tương ứng với việc tiền tố A bớt đi 1 ký tự trong khi tiền tố B giữ nguyên $\Rightarrow$ Thao tác xóa ký tự $A[i]$.

Câu 12 (so khớp chuỗi mẫu đại diện wildcard dp):

Ký tự đại diện $*$ trong so khớp mẫu (khớp với chuỗi ký tự bất kỳ có độ dài ≥ 0) có hệ thức chuyển trạng thái là:

- A. $dp[i][j] = dp[i-1][j-1]$
- B. **[Đáp án đúng]** $dp[i][j] = dp[i-1][j] \text{ lor } dp[i][j-1]$ (Khớp với 1/nhiều ký tự hoặc khớp với chuỗi rỗng).
- C. $dp[i][j] = dp[i][j]$
- D. $dp[i][j] = \text{false}$

Giải thích: $dp[i-1][j]$ đại diện cho việc $\$$ tiếp tục khớp thêm ký tự $A[i]$, còn $dp[i][j-1]$ đại diện cho việc $\$$ đại diện cho chuỗi rỗng không lấy ký tự nào.

Câu 13 (nén bộ nhớ lcs còn 2 dòng):

Nếu chỉ cần tìm độ dài của LCS giữa 2 chuỗi độ dài N và M (không yêu cầu truy vết xâu), ta có thể nén bộ nhớ về mức nào?

- A. $O(1)$
- B. **[Đáp án đúng]** $O(\min(N, M))$ bằng cách chỉ lưu 2 dòng phương án $prev$ và $curr$.
- C. $O(N \cdot M)$
- D. $O(\log(N+M))$

Giải thích: Dòng i chỉ phụ thuộc dòng $i-1$, do đó có thể nén bộ nhớ về 2 dòng của chuỗi ngắn hơn, đạt $O(\min(N, M))$.

Câu 14 (bài toán xâu đan xen interleaving string):

Cho ba xâu A (dài N), B (dài M) và C (dài $N+M$). Trạng thái $dp[i][j]$ là boolean kiểm tra xem tiền tố $C[1 \dots i+j]$ có thể tạo từ $A[1 \dots i]$ và $B[1 \dots j]$. Hệ thức chuyển trạng thái là:

- A. $dp[i][j] = dp[i-1][j-1]$
- B. **[Đáp án đúng]** $dp[i][j] = (dp[i-1][j] \text{ and } A[i] == C[i+j]) \text{ or } (dp[i][j-1] \text{ and } B[j] == C[i+j])$
- C. $dp[i][j] = dp[i-1][j] + dp[i][j-1]$
- D. $dp[i][j] = \text{true}$

Giải thích: Ký tự cuối cùng của tiền tố C phải khớp với $A[i]$ (và phần trước tạo từ $A[1 \dots i-1]$, $B[1 \dots j]$) hoặc khớp với $B[j]$.

Câu 15 (đếm số lần xuất hiện xâu con distinct subsequences):

Cho xâu S và xâu T . Gọi $dp[i][j]$ là số lần xâu $T[1 \dots j]$ xuất hiện dưới dạng dãy con trong $S[1 \dots i]$. Khi $S[i] == T[j]$, công thức chuyển trạng thái là:

- A. $dp[i][j] = dp[i-1][j-1]$
- B. **[Đáp án đúng]** $dp[i][j] = (dp[i-1][j] + dp[i-1][j-1]) \bmod MOD$
- C. $dp[i][j] = dp[i-1][j] \times dp[i-1][j-1]$
- D. $dp[i][j] = dp[i][j-1]$

Giải thích: Có 2 lựa chọn: Không dùng $S[i]$ để khớp ($dp[i-1][j]$ cách) hoặc dùng $S[i]$ để khớp với $T[j]$ ($dp[i-1][j-1]$ cách).

8. Ma trận 15 bài tập thực hành theo mức độ (p0 → p5)

Mã Bài Tập	Tên Bài Toán	Mức Độ	Trọng Tâm Kiến Thức & DP Invariant
CPPB-DPS-01	Dãy con chung dài nhất Cơ Bản (LCS)	P0	$dp[i][j]$ trên 2 tiền tố, so khớp $A[i]=B[j]$.
CPPB-DPS-02	Độ Dài LCS Của 3 Xâu Ngắn	P1	Quy hoạch động 3 chiều $dp[i][j][k]$ kích thước nhỏ.
CPPB-DPS-03	Xâu Con Chung Ngắn Nhất (SCS)	P1	Áp dụng công thức $N + M - LCS(A, B)$.
CPPB-DPS-04	Chèn Ít Ký Tự Nhất Tạo Xâu Đối Xứng	P1	Quy đổi về bài toán $N - LPS(S)$.
CPPB-DPS-05	Khoảng cách biến đổi xâu (Edit Distance)	P2	3 thao tác Chèn, Xóa, Thay thế kèm khởi tạo biên $O(N \cdot M)$.
CPPB-DPS-06	Edit Distance Chi Phí Thao Tác Bất Đối Xứng	P2	Chi phí chèn c_I , xóa c_D , thay thế c_R khác nhau.
CPPB-DPS-07	Xâu Con Liên Tiếp Đối Xứng Dài Nhất	P2	Longest Palindromic Substring boolean $dp[i][j]$ duyệt theo độ dài len .
CPPB-DPS-08	Dãy Con Không Liên Tiếp Đối Xứng Dài Nhất	P2	Longest Palindromic Subsequence $dp[i][j]$ trên 2 đầu mút.
CPPB-DPS-09	Đếm Số Lần Xuất Hiện Dây Con (Distinct Subseq)	P3	Đếm số lần xâu T xuất hiện dưới dạng dây con của S modulo 10^9+7 .
CPPB-DPS-10	Xâu Đan Xen (Interleaving String)	P3	Kiểm tra xâu C có được tạo bởi việc đan xen 2 xâu A và B .
CPPB-DPS-11	Xóa Ít Ký Tự Nhất Để Hai Xâu Bằng Nhau	P3	Cực tiểu hóa chi phí xóa ký tự mã ASCII.
CPPB-DPS-12	Khôi Phục Chuỗi LCS Cụ Thể	P4	Lần ngược từ (N, M) tái tạo chính xác xâu ký tự con chung.
CPPB-DPS-13	Khôi Phục Lộ Trình Biến Đổi Edit Distance	P4	In ra từng bước thao tác Insert, Delete, Replace cụ thể.
CPPB-DPS-14	So Khớp Ký Tự Đại Diện (Wildcard Matching)	P4	Xử lý ký tự $?$ (1 ký tự) và $*$ (chuỗi bất kỳ ≥ 0).
CPPB-DPS-15	Quy Hoạch Động Chuỗi Olympic (Mastery)	P5	Bài toán tối ưu hóa xâu kết hợp điều kiện từ vựng chuẩn thi đấu.



Module 06: Cấu trúc dữ liệu nâng cao

<!-- Chuyên đề 16: STL: set, map & Nén Tọa Độ | 380 dòng | 18,338 bytes -->

Chuyên đề 16: Cấu trúc dữ liệu stl: Set, map & heap

1. Bản chất các cấu trúc dữ liệu nâng cao trong thư viện chuẩn stl

Trong lập trình thi đấu hiện đại, việc tự cài đặt lại cây nhị phân cân bằng hay bảng băm từ đầu cho mọi bài toán là không khả thi. C++ Standard Template Library (STL) cung cấp các cấu trúc dữ liệu tối ưu hóa cực mạnh:

- `std::set` / `std::map` : Cây đỏ-đen (Red-Black Tree) tự cân bằng, luôn duy trì các phần tử theo thứ tự tăng dần. Các thao tác tìm kiếm, chèn, xóa đều có độ phức tạp đảm bảo $O(\log N)$.
- `std::unordered_map` / `std::unordered_set` : Bảng băm trực tiếp (Hash Table), đạt thời gian trung bình $O(1)$ cho các truy vấn.
- `std::priority_queue` : Cấu trúc Heap nhị phân hoàn chỉnh, cho phép truy xuất phần tử lớn nhất (hoặc nhỏ nhất) trong $O(1)$ và thêm/bớt trong $O(\log N)$.

! [So sánh Set Map vs Unordered Map] (/Users/vu/Developer/ikhEdu_lessons/courses/cpp-bang-b/lessons/lesson-16-cau-truc-du-lieu-stl-set-map/assets/stl_set_map_rb_tree_vi.svg)

2. Kỹ thuật nén tọa độ (coordinate compression)

2.1. Bản chất bài toán & khi nào cần nén tọa độ?

- **Vấn đề:** Các giá trị trong mảng A có thể rất lớn ($A[i] \leq 10^9$ hoặc 10^{18}), ta không thể dùng giá trị này làm chỉ số mảng đếm tần suất hoặc dựng cây Segment Tree / Fenwick Tree. Tuy nhiên, số lượng phần tử N lại rất nhỏ ($N \leq 10^5$).
- **Nguyên lý Nén Tọa Độ:** Ánh xạ tập giá trị rời rạc ban đầu về tập số nguyên liên tiếp $\{0, 1, 2, \dots, K-1\}$ ($K \leq N$) sao cho **giữ nguyên thứ tự tương quan lớn bé** giữa các phần tử:

$$A[i] < A[j] \Leftrightarrow \text{rank}(A[i]) < \text{rank}(A[j])$$

! [Mô hình Nén Tọa Độ] (/Users/vu/Developer/ikhEdu_lessons/courses/cpp-bang-b/lessons/lesson-16-cau-truc-du-lieu-stl-set-map/assets/coordinate_compression_model_vi.svg)

2.2. Quy trình 4 bước chuẩn mực trong c++

1. Sao chép mảng: `vector<long long> vals = a;`
1. Sắp xếp tăng dần: `sort(vals.begin(), vals.end());`
2. Loại bỏ trùng lặp: `vals.erase(unique(vals.begin(), vals.end()), vals.end());`
3. Ánh xạ bằng Tìm kiếm nhị phân:


```
int compressed_val = lower_bound(vals.begin(), vals.end(), a[i]) -
vals.begin();
```

3. Hàng đợi ưu tiên (priority queue / heap)

![Hàng đợi ưu tiên Max-Heap vs Min-Heap](/Users/vu/Developer/ikhEdu_lessons/courses/cpp-bang-b/lessons/lesson-16-cau-truc-du-lieu-stl-set-map/assets/priority_queue_heap_vi.svg)

- **Max-Heap (Mặc định):** `priority_queue<long long> max_pq;` $\Rightarrow$ `top()` trả về giá trị lớn nhất.
- **Min-Heap (Đảo thứ tự):** `priority_queue<long long, vector<long long>, greater<long long>> min_pq;` $\Rightarrow$ `top()` trả về giá trị nhỏ nhất.
- **Ứng dụng kinh điển:** Tìm K phần tử lớn nhất/nhỏ nhất trong luồng dữ liệu online, thuật toán Dijkstra, thuật toán Prim, duy trì Trung vị động (Median) bằng 2 Heap.

4. Các bẫy lỗi lập trình kinh điển (bug traps)

1. ****Bẫy Worst-case $O(N)$ của `unordered_map` do Anti-Hash Test:****

- Trong các kỳ thi competitive, hàm băm mặc định `std::hash` của `unordered_map` rất dễ bị các test đối kháng (Anti-hash tests) làm tràn bucket $\Rightarrow$ Độ phức tạp tụt xuống $O(N^2)$ và bị TLE.
- **Quy tắc an toàn:** Dùng `std::map` khi $N \leq 2 \cdot 10^5$ (đảm bảo $O(N \log N)$), hoặc dùng Custom Hash an toàn với hằng số thời gian ngẫu nhiên `chrono`.

1. **Bẫy xóa phần tử trong `std::multiset`:**

- Lệnh `ms.erase(x)` sẽ xóa **TOÀN BỘ** các phần tử có giá trị bằng x trong multiset!
- **Cú pháp chuẩn khi chỉ muốn xóa 1 bản sao:** `ms.erase(ms.find(x));`

1. **Bẫy truy cập `map[key]` tự động chèn phần tử mới:**

- Khi gọi `if (mp[x] > 0)`, nếu x chưa tồn tại trong map, C++ sẽ tự động chèn cặp $(x, 0)$ vào map làm tăng kích thước bộ nhớ.
- **Cú pháp an toàn:** Dùng `if (mp.count(x))` hoặc `if (mp.find(x) != mp.end())`.

5. Mẫu cài đặt chuẩn thi đấu (competitive templates)

Mẫu 1: Kỹ thuật nén tọa độ chuẩn mực

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;

    if (n ≤ 0) return 0;

    vector<long long> a(n);

    for (int i = 0; i < n; ++i) {
        cin >> a[i];
    }

    // 1. Tạo mảng nén
    vector<long long> vals = a;

    sort(vals.begin(), vals.end());
    vals.erase(unique(vals.begin(), vals.end()), vals.end());

    // 2. Ánh xạ từng phần tử
    vector<int> compressed(n);

    for (int i = 0; i < n; ++i) {
        compressed[i] = lower_bound(vals.begin(), vals.end(), a[i]) -
vals.begin();
    }

    for (int i = 0; i < n; ++i) {
        cout << compressed[i] << (i + 1 == n ? "" : " ");
    }
    cout << "\n";

    return 0;
}
```

Mẫu 2: Duy trì trung vị động bằng 2 heap (median of stream)

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;

    if (n ≤ 0) return 0;

    priority_queue<long long> left_max; // Nửa nhỏ hơn (Max-Heap)

    priority_queue<long long, vector<long long>, greater<long long>>
right_min; // Nửa lớn hơn (Min-Heap)

    for (int i = 0; i < n; ++i) {
        long long x;
        cin >> x;

        if (left_max.empty() || x ≤ left_max.top()) {
            left_max.push(x);
        } else {
            right_min.push(x);
        }

        // Tự cân bằng kích thước: left_max luôn có size = right_min hoặc size
= right_min + 1
        if (left_max.size() > right_min.size() + 1) {
            right_min.push(left_max.top());
            left_max.pop();
        } else if (right_min.size() > left_max.size()) {
            left_max.push(right_min.top());
            right_min.pop();
        }

        // In trung vị hiện tại
        cout << left_max.top() << (i + 1 == n ? "" : " ");
    }
    cout << "\n";

    return 0;
}

```

Câu hỏi trắc nghiệm củng cố khái niệm

Câu 1 (bản chất `std::set`):

Cấu trúc dữ liệu `std::set` trong C++ được cài đặt dựa trên cấu trúc cây nào?

- A. Cây nhị phân tìm kiếm thông thường (BST).
- B. **[Đáp án đúng]** Cây Đỏ-Đen (Red-Black Tree) tự cân bằng.
- C. Cây phân đoạn (Segment Tree).
- D. Bảng băm (Hash Table).

Giải thích: Red-Black Tree đảm bảo độ cao của cây luôn là $O(\log N)$, giữ cho mọi thao tác tìm kiếm, chèn, xóa đạt $O(\log N)$ trong mọi trường hợp.

Câu 2 (xóa 1 phần tử trong `std::multiset`):

Để xóa đúng một phần tử có giá trị `x` trong `std::multiset<int> ms`, cú pháp nào sau đây là chính xác?

- A. `ms.erase(x);`
- B. **[Đáp án đúng]** `auto it = ms.find(x); if (it != ms.end()) ms.erase(it);`
- C. `ms.pop(x);`
- D. `ms.remove(x);`

Giải thích: `ms.erase(x)` sẽ xóa sạch toàn bộ các phần tử có giá trị `x`. Để xóa 1 phần tử, phải truyền iterator thông qua `ms.find(x)`.

Câu 3 (độ phức tạp nén tọa độ):

Cho mảng `N` phần tử. Quy trình nén tọa độ gồm sao chép, sắp xếp `sort`, lọc `unique` và ánh xạ `lower_bound` có tổng độ phức tạp thời gian là bao nhiêu?

- A. $O(N^2)$
- B. **[Đáp án đúng]** $O(N \log N)$
- C. $O(N)$
- D. $O(\log N)$

Giải thích: Bước `sort` tốn $O(N \log N)$, bước `unique` tốn $O(N)$, N lần gọi `lower_bound` tốn $N \log N \Rightarrow$ Tổng thời gian $O(N \log N)$.

Câu 4 (mục đích cốt lõi của nén tọa độ):

Tại sao ta cần nén tọa độ khi giá trị các phần tử lên tới 10^9 ?

- A. Để mảng có thứ tự tăng dần.
- B. **[Đáp án đúng]** Để chuyển miền giá trị cực lớn về đoạn $[0, K-1]$ ($K \leq N$) giúp sử dụng được mảng đếm tần suất hoặc cây Fenwick/Segment Tree.
- C. Để xóa các phần tử âm.
- D. Để giảm thời gian đọc dữ liệu.

Giải thích: Thu nhỏ miền giá trị mà không làm thay đổi thứ tự quan hệ lớn bé giữa các phần tử.

Câu 5 (cấu trúc min-heap trong c++):

Khai báo nào sau đây tạo ra một Hàng đợi ưu tiên Min-Heap trong C++?

- A. `priority_queue<int> pq;`
- B. **[Đáp án đúng]** `priority_queue<int, vector<int>, greater<int>> pq;`
- C. `priority_queue<int, less<int>> pq;`
- D. `min_heap<int> pq;`

Giải thích: Mặc định `priority_queue` dùng functor `less<T>` tạo Max-Heap. Dùng `greater<T>` để đảo chiều so sánh thành Min-Heap.

Câu 6 (độ phức tạp các thao tác priority_queue):

Trong `std::priority_queue`, độ phức tạp thời gian của các hàm `top()`, `push()`, `pop()` lần lượt là:

- A. $O(1), O(1), O(1)$
- B. **[Đáp án đúng]** $O(1), O(\log N), O(\log N)$
- C. $O(\log N), O(\log N), O(\log N)$
- D. $O(N), O(\log N), O(1)$

Giải thích: `top()` chỉ đọc đỉnh heap trong $O(1)$. `push()` và `pop()` thực hiện vun đống (heapify up/down) theo chiều cao cây $O(\log N)$.

Câu 7 (bẫy truy cập std::map):

Khi thực hiện kiểm tra `if (mp[key] = 5)` mà `key` chưa từng xuất hiện trong `map`, điều gì sẽ xảy ra?

- A. Chương trình báo lỗi biên dịch.
- B. **[Đáp án đúng]** C++ tự động chèn `key` vào `map` với giá trị mặc định là 0, làm tăng kích thước của map.
- C. Hàm trả về `false` và map không thay đổi.
- D. Chương trình bị Runtime Error.

Giải thích: Toán tử `[]` của `std::map` có side-effect tự động khởi tạo phần tử mới nếu chưa tồn tại. Cần dùng `mp.find(key)` để kiểm tra an toàn.

Câu 8 (khác biệt giữa set và unordered_set):

Ưu điểm lớn nhất của `std::set` so với `std::unordered_set` là gì?

- A. Chạy nhanh hơn trong mọi trường hợp.
- B. **[Đáp án đúng]** Luôn duy trì thứ tự tăng dần và hỗ trợ các hàm tìm kiếm cận `lower_bound`, `upper_bound` trong $O(\log N)$.
- C. Chiếm ít bộ nhớ hơn.
- D. Cho phép chứa các phần tử trùng lặp.

Giải thích: `std::unordered_set` không có thứ tự và không hỗ trợ tìm kiếm cận trên/dưới.

Câu 9 (tìm kiếm lower_bound trên std::Set):

Để tìm phần tử nhỏ nhất $\geq x$ trong `std::set<int> st`, cú pháp nào có hiệu năng tối ưu $O(\log N)$?

- A. `lower_bound(st.begin(), st.end(), x);`
- B. **[Đáp án đúng]** `st.lower_bound(x);`
- C. `find(st.begin(), st.end(), x);`
- D. `binary_search(st.begin(), st.end(), x);`

Giải thích: Phải dùng hàm thành viên `st.lower_bound(x)` chạy trên cây trong $O(\log N)$. Hàm toàn cục `std::lower_bound` duyệt trên iterator của set sẽ mất $O(N)$.

Câu 10 (duy trì trung vị động):

Trong bài toán tìm trung vị động của luồng dữ liệu, ta sử dụng cặp cấu trúc dữ liệu nào tối ưu nhất?

- A. Hai mảng `vector`.
- B. **[Đáp án đúng]** Một Max-Heap chứa nửa phần tử nhỏ và một Min-Heap chứa nửa phần tử lớn.
- C. Một `std::queue` và một `std::stack`.
- D. Hai cây phân đoạn.

Giải thích: Hai Heap giữ cân bằng kích thước cho phép truy xuất phần tử trung vị ở đỉnh heap trong $O(1)$ và thêm phần tử mới trong $O(\log N)$.

Câu 11 (cấu trúc std:: : Map lồng nhau):

Muốn lưu tần suất xuất hiện của các cặp tọa độ (x, y) , kiểu dữ liệu nào sau đây là chuẩn mực?

- A. `vector<int> mp;`
- B. **[Đáp án đúng]** `map<vector<long long>, int> mp;` hoặc `map<pair<long long, long long>, int> mp;`
- C. `set<int> mp;`
- D. `unordered_map<pair<int, int>, int> mp;` (không có custom hash).

Giải thích: `std::pair` hoặc `vector` có sẵn toán tử so sánh `<` nên dùng trực tiếp làm key trong `std::map` cực kỳ an toàn.

Câu 12 (đếm số phần tử phân biệt trong cửa sổ):

Để đếm số phần tử phân biệt trong cửa sổ trượt độ dài K một cách hiệu quả, cấu trúc nào sau đây là tối ưu nhất?

- A. Mỗi bước tạo một `set` mới $O(K \log K)$.
- B. **[Đáp án đúng]** Duy trì một mảng đếm tần suất hoặc `map` kết hợp biến đếm `distinct_count` cập nhật trong $O(1)$.
- C. Dùng `stack`.
- D. Sắp xếp lại cửa sổ mỗi bước.

Giải thích: Khi cửa sổ trượt, chỉ có 1 phần tử thêm vào và 1 phần tử bị loại ra, cập nhật biến đếm trong $O(1)$.

Câu 13 (hàm `std::unique` trong C++):

Hàm `std::unique(v.begin(), v.end())` chỉ hoạt động chính xác khi nào?

- A. Khi vector có kích thước chẵn.
- B. **[Đáp án đúng]** Khi vector đã được sắp xếp trước đó.
- C. Khi vector chỉ chứa số dương.
- D. Luôn hoạt động chính xác với mọi mảng chưa sắp xếp.

Giải thích: `std::unique` chỉ loại bỏ các phần tử trùng lặp đứng LIỀN KÈ nhau. Do đó mảng bắt buộc phải được `sort` trước.

Câu 14 (hàng đợi ưu tiên lưu `struct` / `comparator`):

Muốn `priority_queue` ưu tiên phần tử có giá trị nhỏ nhất, nếu dùng `Struct` thì toán tử `operator<` phải định nghĩa như thế nào?

- A. `bool operator<(const Node& other) const { return val < other.val; }`
- B. **[Đáp án đúng]** `bool operator<(const Node& other) const { return val > other.val; }` (Đảo dấu so sánh).
- C. `bool operator<(const Node& other) const { return val == other.val; }`
- D. Không thể dùng `struct` trong `priority_queue`.

Giải thích: `priority_queue` mặc định đưa phần tử lớn nhất theo quan hệ `<` lên đỉnh. Để phần tử nhỏ nhất lên đỉnh, ta đảo ngược định nghĩa thành `val > other.val`.

Câu 15 (duyệt toàn bộ phần tử trong `std::map`):

Cách duyệt in toàn bộ các cặp `(key, value)` trong `std::map<string, int> mp` theo thứ tự từ điển chuẩn C++11 là:

- A. `for (int i = 0; i < mp.size(); ++i) cout << mp[i];`
- B. **[Đáp án đúng]**
`for (const auto& p : mp) cout << p.first << " " << p.second << "\n";`
- C. `for (auto it = mp.end(); it != mp.begin(); ++it)`
- D. Dùng vòng lặp `while`.

Giải thích: Dùng range-based for loop duyệt tuần tự các pair `(first, second)` theo đúng thứ tự sắp xếp của cây đồ đen.

Ma trận bài tập thực hành (p0 → p5)

Mã Bài Tập	Tên Bài Toán	Mức Độ	Trọng Tâm Kiến Thức & Kỹ Năng STL
CPPB-STL-01	Đếm Số Phần Tử Phân Biệt	P0	Dùng <code>std::set</code> hoặc sort + unique cơ bản $O(N \log N)$.
CPPB-STL-02	Bảng Tra Cứu Tần Suất Từ Khóa	P1	Sử dụng <code>std::map<string, int></code> đếm số lần xuất hiện.
CPPB-STL-03	Nén Tọa Độ Mảng Số Lớn	P1	Áp dụng quy trình 4 bước nén giá trị về $[0, K-1]$.
CPPB-STL-04	Tìm Phần Tử Nhỏ Nhất Lớn Hơn X	P2	Sử dụng <code>st.lower_bound(x)</code> trên <code>std::set</code> .
CPPB-STL-05	Hàng đợi ưu tiên K Phần Tử Lớn Nhất	P2	Dùng Min-Heap kích thước K duy trì top K phần tử.
CPPB-STL-06	Quản Lý Tập Hợp Đa Trùng Lặp (Multiset)	P2	Thao tác chèn, tìm kiếm và xóa đúng 1 bản sao với <code>ms.find()</code> .
CPPB-STL-07	Hợp Nhất Các Đoạn Số (Merge Intervals)	P2	Sắp xếp các đoạn theo đầu mút kết hợp cấu trúc dữ liệu.
CPPB-STL-08	Tìm Trung Vị Động Trong Luồng Dữ Liệu	P3	Cặp Max-Heap / Min-Heap tự cân bằng $O(\log N)$ mỗi truy vấn.
CPPB-STL-09	Đếm Số Phần Tử Phân Biệt Trong Cửa Sổ K	P3	Cửa sổ trượt kết hợp <code>map</code> tần suất duy trì <code>distinct_count</code> .
CPPB-STL-10	Nối Dây Chi Phí Nhỏ Nhất (Huffman Greedy)	P3	Min-Heap liên tục lấy 2 phần tử nhỏ nhất và đẩy tổng vào lại.
CPPB-STL-11	Lập Lịch Công Việc Tối Ưu Máy Chủ	P3	<code>priority_queue</code> quản lý thời điểm máy chủ rảnh rỗi.
CPPB-STL-12	Đếm Cặp Số Có Hiệu Bằng K Số Lớn	P4	Nén tọa độ kết hợp mảng đếm tần suất hoặc tìm kiếm nhị phân.
CPPB-STL-13	Truy Vấn Phần Tử Xuất Hiện Nhiều Nhất	P4	Cấu trúc dữ liệu kết hợp duy trì tần suất cực đại online.
CPPB-STL-14	Tìm Cặp Điểm Gần Nhất (Closest Pair)	P4	Đường quét (Sweep-line) kết hợp <code>std::set</code> $O(N \log N)$.
CPPB-STL-15	Hệ Thống Xếp Hạng Thi Đấu Dynamic (Mastery)	P5	Cấu trúc dữ liệu STL đa tiêu chí hỗ trợ cập nhật điểm và xếp hạng.

<!-- Chuyên đề 17: Stack & Monotonic Stack | 365 dòng | 17,937 bytes -->

Chuyên đề 17: Cấu trúc dữ liệu ngăn xếp (stack) & monotonic stack

1. Bản chất cấu trúc dữ liệu ngăn xếp (stack)

Ngăn xếp (Stack) là cấu trúc dữ liệu hoạt động theo nguyên lý **LIFO (Last In, First Out — Vào sau, Ra trước)**:

- Phần tử được thêm vào cuối cùng sẽ là phần tử đầu tiên được lấy ra.
- Các thao tác cơ bản trong C++ `std::stack`: `push(x)` (thêm vào đỉnh), `pop()` (xóa đỉnh), `top()` (truy cập đỉnh), `empty()`, `size()`. Toàn bộ thao tác đều đạt thời gian tối ưu tuyệt đối $O(1)$.

![Cơ chế LIFO của Stack và Khớp Dấu Ngoặc](/Users/vu/Developer/ikhEdu_lessons/courses/cpp-bang-b/lessons/lesson-17-ngan-xep-stack-monotonic/assets/stack_lifo_operation_vi.svg)

2. Kỹ thuật ngăn xếp đơn điệu (monotonic stack)

2.1. Bản chất bài toán & khi nào cần monotonic stack?

- **Vấn đề:** Cho mảng A gồm N phần tử. Với mỗi vị trí i , cần tìm vị trí phần tử **đầu tiên bên phải (hoặc bên trái)** có giá trị lớn hơn (hoặc nhỏ hơn) $A[i]$.
- **Cách ngây thơ:** Duyệt 2 vòng lặp lồng nhau $\Rightarrow O(N^2)$ (bị TLE khi $N = 10^5$).
- **Nguyên lý Monotonic Stack:** Duy trì một ngăn xếp chứa các chỉ số mà giá trị tương ứng trong mảng luôn tuân theo tính chất **đơn điệu** (tăng dần hoặc giảm dần). Khi gặp phần tử mới vi phạm tính đơn điệu, ta liên tục `pop()` các phần tử ở đỉnh ngăn xếp và ghi nhận đáp án cho chúng.

![Mô hình Monotonic Stack NGE](/Users/vu/Developer/ikhEdu_lessons/courses/cpp-bang-b/lessons/lesson-17-ngan-xep-stack-monotonic/assets/monotonic_stack_nge_vi.svg)

2.2. Phân tích độ phức tạp khấu hao (amortized analysis $O(N)$)

Mỗi phần tử của mảng được `push()` vào ngăn xếp đúng 1 lần và bị `pop()` ra khỏi ngăn xếp tối đa 1 lần trong toàn bộ quá trình chạy.

Tổng số thao tác trên Stack $\leq 2N \Rightarrow$ Thời gian trung bình $O(N)$!

3. Bài toán kinh điển: Hình chữ nhật lớn nhất trên biểu đồ cột (largest rectangle in histogram)

! [Hình chữ nhật lớn nhất trên Histogram] (/Users/vu/Developer/ikhEdu_lessons/courses/cpp-bang-b/lessons/lesson-17-ngan-xep-stack-monotonic/assets/histogram_max_rectangle_vi.svg)

- **Bản chất:** Với mỗi cột i có chiều cao $H[i]$, ta cần tìm:
 1. $L[i]$: Vị trí cột đầu tiên bên trái có chiều cao $< H[i]$.
 2. $R[i]$: Vị trí cột đầu tiên bên phải có chiều cao $< H[i]$.
- Khi đó, hình chữ nhật lớn nhất nhận $H[i]$ làm chiều cao tối đa sẽ có chiều rộng $W = R[i] - L[i] - 1$, diện tích là $S[i] = H[i] \times (R[i] - L[i] - 1)$.
- Sử dụng 2 lượt Monotonic Stack (hoặc 1 lượt thông minh), ta tính toàn bộ mảng L và R trong $O(N)$.

4. Các bẫy lỗi lập trình kinh điển (bug traps)

1. **Bẫy gọi `st.top()` hoặc `st.pop()` khi Stack rỗng:**
 - Truy cập đỉnh ngăn xếp khi `st.empty() == true` sẽ dẫn đến lỗi bộ nhớ nghiêm trọng (Segmentation Fault / Runtime Error).
 - **Quy tắc an toàn:** Luôn kiểm tra `while (!st.empty() && ...)` trước khi gọi `st.top()` hay `st.pop()`.
1. **Bẫy quên kiểm tra `st.empty()` ở cuối bài toán Dãy ngoặc đúng:**
 - Sau khi duyệt hết chuỗi, nếu không còn ngoặc đóng nào nhưng trong stack vẫn còn ngoặc mở dư thừa (ví dụ chuỗi "`((("`), dãy ngoặc vẫn là **KHÔNG HỢP LỆ**.
 - **Điều kiện đủ:** Dãy hợp lệ khi và chỉ khi không bị lỗi giữa chừng VÀ `st.empty() == true` ở cuối.
1. **Bẫy tràn số khi tính diện tích hình chữ nhật lớn nhất:**
 - Chiều cao $H[i] \leq 10^9$ và chiều rộng $W \leq 10^5 \Rightarrow$ Diện tích có thể lên tới 10^{14} , vượt quá giới hạn 32-bit `int`. Bắt buộc phải ép kiểu sang `long long`.

5. Mẫu cài đặt chuẩn thi đấu (competitive templates)

Mẫu 1: Tìm phần tử lớn hơn tiếp theo (next greater element)

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;

    if (n ≤ 0) return 0;

    vector<long long> a(n);

    for (int i = 0; i < n; ++i) {
        cin >> a[i];
    }

    vector<long long> nge(n, -1);

    stack<int> st; // Lưu chỉ số

    for (int i = 0; i < n; ++i) {
        while (!st.empty() && a[i] > a[st.top()]) {
            nge[st.top()] = a[i];
            st.pop();
        }
        st.push(i);
    }

    for (int i = 0; i < n; ++i) {
        cout << nge[i] << (i + 1 == n ? "" : " ");
    }
    cout << "\n";

    return 0;
}
```

Mẫu 2: Hình chữ nhật lớn nhất trên biểu đồ cột (histogram)

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;

    if (n ≤ 0) return 0;

    vector<long long> h(n);

    for (int i = 0; i < n; ++i) {
        cin >> h[i];
    }

    // Thêm phần tử lính canh 0 ở cuối để đẩy toàn bộ stack ra
    h.push_back(0);
    stack<int> st;

    long long max_area = 0;

    for (int i = 0; i ≤ n; ++i) {
        while (!st.empty() && h[i] < h[st.top()]) {
            long long height = h[st.top()];
            st.pop();
            long long width = st.empty() ? i : (i - st.top() - 1);
            max_area = max(max_area, height * width);
        }
        st.push(i);
    }

    cout << max_area << "\n";
    return 0;
}

```

Câu hỏi trắc nghiệm củng cố khái niệm

Câu 1 (bản chất lifo của stack):

Nguyên lý hoạt động cơ bản của cấu trúc dữ liệu Ngăn xếp (Stack) là gì?

- A. FIFO (Vào trước, Ra trước).
- B. **[Đáp án đúng]** LIFO (Vào sau, Ra trước).
- C. Truy cập ngẫu nhiên phần tử bất kỳ trong $O(1)$.
- D. Tự động sắp xếp các phần tử tăng dần.

Giải thích: Stack chỉ cho phép thêm và lấy phần tử ở một đầu duy nhất gọi là đỉnh (top), tuân theo nguyên lý LIFO.

Câu 2 (độ phức tạp monotonic stack):

Tại sao thuật toán tìm phần tử lớn hơn tiếp theo dùng Monotonic Stack chỉ mất tổng thời gian $O(N)$ dù có vòng lặp `while` lồng bên trong vòng `for` ?

- A. Vì số vòng lặp `while` luôn nhỏ hơn 3.
- B. **[Đáp án đúng]** Vì mỗi phần tử trong mảng chỉ được `push` vào stack đúng 1 lần và bị `pop` ra tối đa 1 lần trong toàn bộ chương trình (phân tích khấu hao).
- C. Vì stack tự động bỏ qua các phần tử trùng nhau.
- D. Vì mảng đã được sắp xếp trước.

Giải thích: Phân tích khấu hao (Amortized Analysis): Tổng số thao tác `push` và `pop` trên toàn mảng không vượt quá $2N \Rightarrow O(N)$.

Câu 3 (điều kiện dãy ngoặc đúng):

Một chuỗi ngoặc chỉ gồm (và) là hợp lệ khi và chỉ khi thỏa mãn điều kiện nào?

- A. Số lượng ngoặc mở bằng số lượng ngoặc đóng.
- B. **[Đáp án đúng]** Tại mọi vị trí tiền tố, số ngoặc mở luôn $\geq$ số ngoặc đóng, và khi kết thúc chuỗi số ngoặc mở bằng đúng số ngoặc đóng.
- C. Ký tự đầu tiên là ngoặc mở.
- D. Độ dài chuỗi là số chẵn.

Giải thích: Dùng Stack: Không bao giờ bị `pop` khi rỗng (tiền tố hợp lệ) và stack phải rỗng hoàn toàn ở cuối chuỗi.

Câu 4 (lưu trữ trong monotonic stack):

Trong thuật toán tìm Next Greater Element hay Histogram, thông thường ta nên lưu giá trị gì vào trong **stack** ?

- A. Lưu giá trị của phần tử $A[i]$.
- B. **[Đáp án đúng]** Lưu chỉ số vị trí i của phần tử trong mảng.
- C. Lưu số lượng phần tử nhỏ hơn.
- D. Lưu địa chỉ con trỏ.

Giải thích: Lưu chỉ số i cho phép truy cập đồng thời cả giá trị $A[i]$ lẫn tính toán khoảng cách/chiều rộng $j - i$.

Câu 5 (hình chữ nhật lớn nhất trong ma trận 0-1):

Bài toán tìm hình chữ nhật toàn số 1 có diện tích lớn nhất trong ma trận nhị phân $N \times M$ có thể quy về bài toán nào?

- A. Quy hoạch động trên cây.
- B. **[Đáp án đúng]** Với mỗi hàng, tính chiều cao các cột 1 liên tiếp rồi áp dụng bài toán Hình chữ nhật lớn nhất trên biểu đồ cột (Histogram).
- C. Thuật toán Dijkstra.
- D. Tìm kiếm nhị phân trên lưới.

Giải thích: Duyệt từng hàng $1 \rightarrow N$, duy trì chiều cao cột $h[j] = (matrix[i][j] == 1 ? h[j] + 1 : 0)$, sau đó chạy Monotonic Stack Histogram trong $O(M) \Rightarrow$ Tổng thời gian $O(N \times M)$.

Câu 6 (biểu thức hậu tố rpn):

Để tính giá trị của một biểu thức toán học dạng Hậu tố (Reverse Polish Notation — RPN, ví dụ $3\ 4\ +\ 2\ *$), ta sử dụng cấu trúc dữ liệu nào?

- A. Hàng đợi Queue.
- B. **[Đáp án đúng]** Ngăn xếp Stack (gặp số thì push, gặp toán tử thì pop 2 số tính rồi push kết quả vào lại).
- C. Cây nhị phân.
- D. Bảng băm.

Giải thích: Stack là công cụ kinh điển để xử lý cú pháp và đánh giá biểu thức toán học.

Câu 7 (bẫy runtime error với stack):

Đoạn mã C++ nào sau đây có nguy cơ gây lỗi sập chương trình (Crash / Runtime Error)?

- A. `if (!st.empty()) st.pop();`
- B. **[Đáp án đúng]** `if (st.top() == '(') st.pop();` khi chưa kiểm tra `st.empty()`.
- C. `st.push(5);`
- D. `int sz = st.size();`

Giải thích: Gọi `st.top()` khi `st.empty() = true` là hành vi không xác định (Undefined Behavior) dẫn đến Segmentation Fault.

Câu 8 (phần tử nhỏ hơn gần nhất bên trái):

Để tìm phần tử đầu tiên bên trái nhỏ hơn `A[i]` (Previous Smaller Element), ta duy trì Monotonic Stack theo tính chất nào?

- A. Đơn điệu giảm dần.
- B. **[Đáp án đúng]** Đơn điệu tăng dần từ đáy lên đỉnh.
- C. Không cần đơn điệu.
- D. Sắp xếp ngẫu nhiên.

Giải thích: Stack đơn điệu tăng dần đảm bảo phần tử ở đỉnh ngay dưới sẽ là phần tử nhỏ hơn gần nhất.

Câu 9 (mục đích của phần tử lính canh trong histogram):

Tại sao khi cài đặt bài toán Histogram, ta thường thêm một cột chiều cao `0` vào cuối mảng (`h.push_back(0)`)?

- A. Để tăng kích thước mảng cho đẹp.
- B. **[Đáp án đúng]** Để đảm bảo mọi phần tử còn sót lại trong stack đều được kích hoạt `pop()` và tính diện tích khi kết thúc vòng lặp.
- C. Để tránh tràn số nguyên.
- D. Vì cột cuối cùng luôn có chiều cao bằng 0.

Giải thích: Cột chiều cao 0 nhỏ hơn mọi chiều cao dương, đóng vai trò lính canh xả cạn toàn bộ stack.

Câu 10 (dãy con có tổng nhỏ nhất / min subarray):

Để tìm tổng giá trị nhỏ nhất của mọi đoạn con trong mảng, kỹ thuật nào sau đây kết hợp Monotonic Stack là tối ưu nhất?

- A. Thử mọi cặp (i, j) trong $O(N^2)$.
- B. **[Đáp án đúng]** Dùng Monotonic Stack tìm phạm vi $[L[i], R[i]]$ mà $A[i]$ là phần tử nhỏ nhất, đóng góp $A[i] \times (i - L[i]) \times (R[i] - i)$ vào tổng toàn cục trong $O(N)$.
- C. Dùng thuật toán tham lam.
- D. Dùng đệ quy quay lui.

Giải thích: Kỹ thuật đếm số đoạn nhận $A[i]$ làm cực trị trong $O(N)$ là bài toán kinh điển trong các kỳ thi học sinh giỏi.

Câu 11 (xóa k chữ số để được số nhỏ nhất):

Cho chuỗi số S và số K . Để xóa K chữ số sao cho số thu được là nhỏ nhất, cấu trúc dữ liệu nào được sử dụng?

- A. Hàng đợi hai đầu Deque.
- B. **[Đáp án đúng]** Monotonic Stack (khi gặp chữ số nhỏ hơn đỉnh stack và còn lượt xóa $K > 0$, ta *pop* đỉnh stack).
- C. Sắp xếp chuỗi.
- D. Chia để trị.

Giải thích: Giữ các chữ số có thứ tự tăng dần từ trái sang phải để cực tiểu hóa các chữ số ở hàng cao nhất.

Câu 12 (kiểm tra dãy ngoặc nhiều loại):

Khi kiểm tra chuỗi có cả ngoặc tròn $()$, ngoặc vuông $[]$, ngoặc nhọn $\{\}$:

- A. Đếm số lượng từng loại độc lập bằng 3 biến đếm.
- B. **[Đáp án đúng]** Bắt buộc phải dùng Stack để kiểm tra thứ tự lồng nhau hợp lệ giữa các loại ngoặc.
- C. Chỉ cần kiểm tra ký tự đầu và cuối.
- D. Dùng mảng tiền tố.

Giải thích: 3 biến đếm không thể phát hiện lỗi giao nhau sai quy tắc như $([])$. Bắt buộc phải dùng Stack.

Câu 13 (thuật toán shunting-yard):

Thuật toán Shunting-Yard của Edsger Dijkstra sử dụng Stack để làm gì?

- A. Tìm đường đi ngắn nhất.
- B. **[Đáp án đúng]** Chuyển đổi biểu thức toán học từ dạng Trung tố (Infix: $a + b * c$) sang Hậu tố (Postfix: $a b c * +$).
- C. Sắp xếp mảng số nguyên.
- D. Tìm cây khung nhỏ nhất.

Giải thích: Stack toán tử duy trì độ ưu tiên của các phép toán nhân/chia trước, cộng/trừ sau.

Câu 14 (hứng nước mưa — trapping rain water):

Bài toán tính lượng nước mưa đọng lại giữa các cột có thể giải bằng Monotonic Stack trong thời gian bao nhiêu?

- A. $O(N^2)$
- B. **[Đáp án đúng]** $O(N)$ thời gian và $O(N)$ bộ nhớ.
- C. $O(N \log N)$
- D. $O(2^N)$

Giải thích: Duy trì stack giảm dần, khi gặp cột cao hơn sẽ hình thành "vũng nước" giữa cột hiện tại, đáy trũng (đỉnh stack vừa pop) và biên trái (đỉnh stack mới).

Câu 15 (stack dùng mảng tự tạo vs std::stack):

Trong C++, việc tự tạo stack bằng một mảng `int st[N]` và biến con trỏ `top_idx = 0` so với dùng `std::stack` có ưu điểm gì?

- A. Giúp code chạy chính xác hơn.
- B. **[Đáp án đúng]** Tốc độ thực thi nhanh hơn do giảm bớt overhead của class và hỗ trợ truy cập ngẫu nhiên các phần tử bên dưới nếu cần.
- C. Tự động kiểm tra tràn mảng.
- D. Không cần khai báo kích thước.

Giải thích: Mảng tĩnh tự cài có hằng số thời gian cực nhỏ, rất được ưa chuộng trong Competitive Programming đỉnh cao.

Ma trận bài tập thực hành (p0 → p5)

Mã Bài Tập	Tên Bài Toán	Mức Độ	Trọng Tâm Kiến Thức & Kỹ Năng Stack
CPPB-STK-01	Kiểm Tra Dãy Ngoặc Đúng Cơ Bản	P0	Cài đặt <code>std::stack</code> kiểm tra chuỗi ngoặc đơn loại <code>()</code> .
CPPB-STK-02	Dãy Ngoặc Hỗn Hợp Nhiều Loại	P1	Xử lý ghép cặp đồng thời <code>()</code> , <code>[]</code> , <code>{}</code> và bẫy stack rỗng.
CPPB-STK-03	Đánh Giá Biểu Thức Hậu Tố (RPN)	P1	Đọc chuỗi token, thực hiện phép toán số học trên Stack.
CPPB-STK-04	Xóa Các Ký Tự Trùng Lặp Liên Kề	P2	Duyệt chuỗi kết hợp Stack khử các cặp ký tự giống nhau.
CPPB-STK-05	Phần Tử Lớn Hơn Tiếp Theo (NGE)	P2	Monotonic Stack cơ bản $O(N)$ tìm vị trí đầu tiên bên phải.
CPPB-STK-06	Phần Tử Nhỏ Hơn Gần Nhất Bên Trái	P2	Monotonic Stack tìm biên trái nhỏ hơn cho từng phần tử.
CPPB-STK-07	Độ Dài Dãy Ngoặc Đúng Dài Nhất	P2	Stack lưu chỉ số vị trí tính khoảng cách đoạn ngoặc hợp lệ.
CPPB-STK-08	Xóa K Chữ Số Để Được Số Nhỏ Nhất	P3	Monotonic Stack tham lam giữ các chữ số nhỏ ở hàng cao.
CPPB-STK-09	Hình Chữ Nhật Lớn Nhất Trên Histogram	P3	Tìm biên trái và biên phải nhỏ hơn trong $O(N)$.
CPPB-STK-10	Hứng Nước Mưa (Trapping Rain Water)	P3	Monotonic Stack tính diện tích nước đọng theo từng lớp ngang.
CPPB-STK-11	Hình Chữ Nhật Toàn 1 Lớn Nhất Ma Trận	P3	Quy đổi ma trận 2D về N bài toán Histogram 1D.
CPPB-STK-12	Tổng Giá Trị Nhỏ Nhất Mọi Đoạn Con	P4	Đếm số đoạn con nhận <code>A[i]</code> làm min, tối ưu hóa tổng $O(N)$.
CPPB-STK-13	NGE Trên Mảng Xoay Vòng (Circular Array)	P4	Kỹ thuật nhân đôi mảng $2N$ kết hợp Monotonic Stack.
CPPB-STK-14	Tòa Tháp Tầm Nhìn (Visible Towers)	P4	Monotonic Stack đếm số lượng cặp đỉnh có thể nhìn thấy nhau.
CPPB-STK-15	Đánh Giá Biểu Thức Đại Số Đầy Đủ (Mastery)	P5	Thuật toán Shunting-Yard xử lý ngoặc và thứ tự ưu tiên toán tử.

<!-- Chuyên đề 18: Queue & Deque | 389 dòng | 17,816 bytes -->

Chuyên đề 18: Cấu trúc dữ liệu hàng đợi (queue, deque) & monotonic deque

1. Bản chất cấu trúc dữ liệu hàng đợi (queue & deque)

1.1. Hàng đợi chuẩn (queue — fifo)

Hàng đợi hoạt động theo nguyên lý **FIFO (First In, First Out — Vào trước, Ra trước)**:

- Phần tử được thêm vào ở đuôi (`push`), và được lấy ra ở đầu (`pop`).
- Đây là cấu trúc dữ liệu nền tảng của thuật toán Tìm kiếm theo chiều rộng (BFS).

! [Cơ chế FIFO của Queue và Lan tỏa BFS] (/Users/vu/Developer/ikhEdu_lessons/courses/cpp-bang-b/lessons/lesson-18-hang-doi-queue-deque/assets/queue_fifo_operation_vi.svg)

1.2. Hàng đợi hai đầu (double-ended queue — deque)

`std::deque` cho phép thực hiện thêm và xóa phần tử ở **CẢ HAI ĐẦU** với độ phức tạp tối ưu $O(1)$:

- `push_front()` , `pop_front()` : Thao tác ở đầu hàng đợi.
- `push_back()` , `pop_back()` : Thao tác ở đuôi hàng đợi.

2. Kỹ thuật deque cửa sổ trượt min/max $O(N)$ (sliding window monotonic deque)

2.1. Bản chất bài toán

- Cho mảng A gồm N phần tử và số K . Cần tìm giá trị nhỏ nhất (hoặc lớn nhất) trong mọi cửa sổ trượt độ dài K : $[i-K+1 \dots i]$ ($K \leq i \leq N$).
- **Cách dùng Multiset / Priority Queue:** Mất $O(N \log K)$.
- **Cách dùng Monotonic Deque:** Đạt thời gian tối ưu tuyệt đối $O(N)$ tuyến tính**!

! [Monotonic Deque Cửa Sổ Trượt] (/Users/vu/Developer/ikhEdu_lessons/courses/cpp-bang-b/lessons/lesson-18-hang-doi-queue-deque/assets/deque_sliding_window_minmax_vi.svg)

2.2. Bất biến 3 bước duy trì min cửa sổ

Tại mỗi vị trí i khi phần tử $A[i]$ bước vào:

1. **Loại bỏ phần tử hết hạn (Out of Window):** Nếu phần tử ở đầu `dq.front() < i - K + 1` $\Rightarrow$ `dq.pop_front()`.
2. **Duy trì tính đơn điệu tăng:** Trong khi `!dq.empty()` và `A[dq.back()] >= A[i]` $\Rightarrow$ `dq.pop_back()` (vì $A[i]$ vừa nhỏ hơn vừa tồn tại lâu hơn các phần tử ở đuôi).

3. Thêm phần tử mới và lấy đáp án: `dq.push_back(i)`. Khi $i \geq K-1$, giá trị nhỏ nhất của cửa sổ hiện tại chính là `A[dq.front()]`.

3. Ứng dụng nền tảng: Tìm đường đi ngắn nhất bằng queue (bfs nhập môn)

!Đường đi ngắn nhất bằng BFS[/Users/vu/Developer/ikhEdu_lessons/courses/cpp-bang-b/lessons/lesson-18-hang-doi-queue-deque/assets/bfs_shortest_path_unweighted_vi.svg]

- Trên đồ thị không có trọng số (hoặc đồ thị lưới di chuyển 4 hướng có chi phí mỗi bước bằng 1), thuật toán BFS sử dụng Queue luôn đảm bảo:

Lần đầu tiên một đỉnh v được lấy ra khỏi Queue, khoảng cách $dist[v]$ chắc chắn là khoảng cách ngắn nhất từ đỉnh nguồn S .

4. Các bẫy lỗi lập trình kinh điển (bug traps)

1. Bẫy gọi `q.front()` khi Queue rỗng:

- Tương tự Stack, gọi `q.front()` hoặc `q.pop()` khi `q.empty() == true` gây Segmentation Fault.

1. Bẫy lưu giá trị thay vì lưu chỉ số trong Monotonic Deque:

- Nếu chỉ lưu giá trị `A[i]`, ta không thể kiểm tra xem phần tử ở đầu `dq.front()` đã vượt ra khỏi phạm vi cửa sổ $i - K + 1$ hay chưa.
- **Quy tắc bắt buộc:** Luôn lưu chỉ số i vào trong Deque!

1. Bẫy quên đánh dấu `visited` ngay khi `push` vào Queue trong BFS:

- Nếu chờ đến khi `pop` mới đánh dấu `visited[u] = true`, một đỉnh có thể bị đẩy vào Queue hàng chục lần từ các đỉnh lân cận $\Rightarrow$ Bùng nổ bộ nhớ và thời gian (TLE/MLE).
- **Quy tắc sống còn:** Bắt buộc gán `visited[v] = true` ngay tại thời điểm `q.push(v)`.

5. Mẫu cài đặt chuẩn thi đấu (competitive templates)

Mẫu 1: Min trên mọi cửa sổ trượt độ dài k bằng monotonic deque $O(N)$

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, k;
    if (!(cin >> n >> k)) return 0;

    if (n ≤ 0 || k ≤ 0 || k > n) return 0;

    vector<long long> a(n);

    for (int i = 0; i < n; ++i) {
        cin >> a[i];
    }

    deque<int> dq; // Lưu chỉ số, duy trì A[dq[i]] tăng dần

    vector<long long> result;

    for (int i = 0; i < n; ++i) {
        // 1. Xóa phần tử quá hạn của số
        while (!dq.empty() && dq.front() < i - k + 1) {
            dq.pop_front();
        }

        // 2. Duy trì tính đơn điệu tăng
        while (!dq.empty() && a[dq.back()] ≥ a[i]) {
            dq.pop_back();
        }

        // 3. Thêm phần tử hiện tại
        dq.push_back(i);

        // 4. Ghi nhận kết quả khi cửa sổ đủ kích thước k
        if (i ≥ k - 1) {
            result.push_back(a[dq.front()]);
        }
    }

    for (int i = 0; i < (int)result.size(); ++i) {
        cout << result[i] << (i + 1 == (int)result.size() ? "" : " ");
    }
    cout << "\n";

    return 0;
}

```

Mẫu 2: Bfs tìm bước đi ngắn nhất từ 1 đến n

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    if (!(cin >> n >> m)) return 0;

    if (n ≤ 0) return 0;

    vector<vector<int>> adj(n + 1);

    for (int i = 0; i < m; ++i) {
        int u, v;
        cin >> u >> v;

        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    vector<int> dist(n + 1, -1);

    queue<int> q;

    // Khởi tạo gốc 1
    dist[1] = 0;
    q.push(1);

    while (!q.empty()) {
        int u = q.front();
        q.pop();

        for (int v : adj[u]) {
            if (dist[v] == -1) { // Chưa thăm
                dist[v] = dist[u] + 1;
                q.push(v); // Đánh dấu ngay khi push
            }
        }
    }

    cout << dist[n] << "\n";
    return 0;
}

```

Câu hỏi trắc nghiệm củng cố khái niệm

Câu 1 (bản chất fifo của queue):

Điểm khác biệt cốt lõi giữa `std::queue` và `std::stack` là gì?

- A. Queue cho phép truy cập ngẫu nhiên theo chỉ số.
- B. **[Đáp án đúng]** Queue lấy phần tử vào trước ra trước (FIFO), còn Stack lấy phần tử vào sau ra trước (LIFO).
- C. Queue có dung lượng giới hạn còn Stack thì không.
- D. Queue tự động sắp xếp dữ liệu.

Giải thích: Queue đẩy ở đuôi và lấy ở đầu, phục vụ mô hình hàng đợi thực tế và thuật toán loang BFS.

Câu 2 (độ phức tạp monotonic deque):

Thuật toán tìm Min trên cửa sổ trượt độ dài K bằng `std::deque` có độ phức tạp thời gian là bao nhiêu?

- A. $O(N \log K)$
- B. **[Đáp án đúng]** $O(N)$ tuyến tính (mỗi phần tử vào và ra Deque tối đa 1 lần).
- C. $O(N \cdot K)$
- D. $O(N^2)$

Giải thích: Phân tích khấu hao: N phần tử được thêm ở đuôi 1 lần và bị xóa tối đa 1 lần $\Rightarrow O(N)$.

Câu 3 (khi nào cần deque thay vì queue):

Cấu trúc `std::deque` vượt trội hơn `std::queue` ở điểm nào?

- A. Chiếm ít bộ nhớ hơn.
- B. **[Đáp án đúng]** Cho phép thêm và xóa phần tử ở cả 2 đầu (front và back) trong $O(1)$ và hỗ trợ toán tử truy cập `[]`.
- C. Chạy nhanh hơn `vector`.
- D. Tự động loại bỏ phần tử trùng nhau.

Giải thích: `std::deque` là double-ended queue cực kỳ linh hoạt cho các kỹ thuật nâng cao.

Câu 4 (bắt đánh dấu visited trong bfs):

Tại sao trong thuật toán BFS, ta bắt buộc phải đánh dấu `visited[v] = true` ngay khi gọi `q.push(v)` thay vì khi `q.pop()` ?

- A. Để in ra thứ tự duyệt đẹp hơn.
- B. **[Đáp án đúng]** Để ngăn không cho đỉnh `v` bị các đỉnh lân cận khác tiếp tục đẩy vào Queue nhiều lần gây tràn bộ nhớ và TLE.
- C. Vì hàm `push` yêu cầu mảng visited.
- D. Để tính khoảng cách chính xác hơn.

Giải thích: Nếu chỉ đánh dấu khi pop, trong khoảng thời gian đỉnh `v` nằm trong queue, các đỉnh kề khác duyệt tới sẽ lại đẩy thêm `v` vào hàng đợi nhiều lần.

Câu 5 (duy trì max cửa sổ bằng deque):

Để tìm GIÁ TRỊ LỚN NHẤT (Max) trên cửa sổ trượt, ta duy trì Deque theo thứ tự nào?

- A. Đơn điệu tăng dần.
- B. **[Đáp án đúng]** Đơn điệu giảm dần từ đầu đến đuôi (loại bỏ mọi phần tử ở đuôi $\leq A[i]$).
- C. Giữ nguyên thứ tự ban đầu.
- D. Đảo ngược mảng.

Giải thích: Khi duy trì giảm dần, phần tử lớn nhất của cửa sổ hiện tại luôn nằm tại `dq.front()`.

Câu 6 (thuật toán bfs 0-1):

Trên đồ thị mà trọng số các cạnh chỉ có thể là `0` hoặc `1`, ta có thể tìm đường đi ngắn nhất trong $O(V + E)$ bằng cấu trúc nào?

- A. Dùng Dijkstra với `priority_queue` $O(E \log V)$.
- B. **[Đáp án đúng]** Dùng `std::deque`: Đi qua cạnh 0 thì `push_front()`, đi qua cạnh 1 thì `push_back()`.
- C. Dùng `std::stack`.
- D. Dùng đệ quy DFS.

Giải thích: Kỹ thuật 0-1 BFS duy trì tính đơn điệu khoảng cách trong Deque mà không cần cấu trúc Heap phức tạp.

Câu 7 (bài toán đổi tiền ít xu nhất bằng bfs):

Bài toán đổi số tiền S với ít đồng xu nhất có thể giải bằng BFS trên đồ thị trạng thái khi nào?

- A. Khi số lượng đồng xu lớn hơn 100.
- B. **[Đáp án đúng]** Luôn luôn giải được vì mỗi bước chuyển từ $x \rightarrow x + c$ tương đương cạnh có trọng số bằng 1, đỉnh đầu tiên đạt tới S là nghiệm tối ưu.
- C. Không thể giải bằng BFS.
- D. Chỉ giải được khi các đồng xu là số chẵn.

Giải thích: BFS trên không gian trạng thái $0 \rightarrow S$ tìm số bước nhảy ít nhất cực kỳ trực quan.

Câu 8 (đoạn con có tổng lớn nhất độ dài tối đa k):

Để tìm đoạn con có tổng lớn nhất có độ dài không vượt quá K , ta kết hợp Mảng tiền tố $pref[i]$ với cấu trúc dữ liệu nào?

- A. Monotonic Stack.
- B. **[Đáp án đúng]** Monotonic Deque duy trì giá trị $pref[j]$ nhỏ nhất trong cửa sổ $j \in [i-K, i-1]$.
- C. Bảng băm `unordered_map`.
- D. Sắp xếp mảng.

Giải thích: Tổng đoạn con là $pref[i] - pref[j]$. Để cực đại hóa hiệu này với $i - j \leq K$, ta cần cực tiểu hóa $pref[j]$ trong cửa sổ trượt độ dài K .

Câu 9 (trạng thái rỗng của deque):

Lệnh nào sau đây xóa sạch toàn bộ các phần tử trong `std::deque<int> dq` ?

- A. `dq.erase();`
- B. **[Đáp án đúng]** `dq.clear();`
- C. `dq.reset();`
- D. `dq.empty();`

Giải thích: `dq.clear()` hủy toàn bộ phần tử và đưa kích thước về 0 trong $O(N)$.

Câu 10 (sinh các số nhị phân từ 1 đến n):

Để sinh danh sách N số nhị phân đầu tiên ("1", "10", "11", "100" ...) theo thứ tự tăng dần, ta sử dụng Queue như thế nào?

- A. Chuyển đổi từng số nguyên sang nhị phân.
- B. **[Đáp án đúng]** Khởi tạo `q.push("1")`, mỗi bước lấy xâu `s = q.front()`, in ra, rồi đẩy `s + "0"` và `s + "1"` vào đuôi Queue.
- C. Dùng Stack đảo ngược.
- D. Dùng thuật toán đệ quy quay lui.

Giải thích: Cây nhị phân sinh số được duyệt theo từng tầng chuẩn mực bằng Queue BFS.

Câu 11 (truy vết đường đi trong bfs):

Để in ra chính xác các đỉnh trên đường đi ngắn nhất từ S đến T trong BFS, ta duy trì mảng phụ nào?

- A. Mảng `visited`.
- B. **[Đáp án đúng]** Mảng `parent[v] = u` ghi nhận đỉnh cha đã dẫn tới v , sau đó lần ngược từ T về S .
- C. Mảng đếm bậc của đỉnh.
- D. Mảng tính tổng trọng số.

Giải thích: Mỗi khi cập nhật $dist[v] = dist[u] + 1$, ta lưu `parent[v] = u` để khôi phục lộ trình trong $O(V)$.

Câu 12 (queue vòng tròn circular queue):

Khi tự cài đặt Queue bằng mảng cố định `a[MAXN]`, công thức tăng con trỏ đuôi `rear` khi thêm phần tử là:

- A. `rear = rear + 1;`
- B. **[Đáp án đúng]** `rear = (rear + 1) % MAXN;`
- C. `rear = rear * 2;`
- D. `rear = 0;`

Giải thích: Phép toán modulo giúp mảng quay vòng tận dụng lại các ô nhớ ở đầu đã bị pop ra.

Câu 13 (kiểm tra đồ thị hai phía bipartite graph):

Thuật toán BFS kiểm tra đồ thị hai phía (2-coloring) bằng cách tô màu như thế nào?

- A. Tô mọi đỉnh cùng một màu.
- B. **[Đáp án đúng]** Đỉnh gốc tô màu 1, các đỉnh kề tô màu $3 - color[u]$. Nếu gặp đỉnh kề đã tô cùng màu $\Rightarrow$ Không phải đồ thị hai phía.
- C. Tô màu ngẫu nhiên.
- D. Đếm số cạnh của đồ thị.

Giải thích: BFS lan tỏa theo từng tầng, các tầng chẵn và lẻ nhận 2 màu xen kẽ nhau.

Câu 14 (hàng đợi hai đầu trong sliding window median):

Tại sao `std::deque` không thể dùng trực tiếp để tìm Trung vị (Median) trong cửa sổ trượt?

- A. Vì Deque chạy chậm.
- B. **[Đáp án đúng]** Vì Monotonic Deque loại bỏ các phần tử bị vi phạm tính đơn điệu nên không còn lưu đủ toàn bộ K phần tử để xác định vị trí trung vị.
- C. Vì Deque chỉ chứa số nguyên.
- D. Vì trung vị bắt buộc phải dùng mảng tĩnh.

Giải thích: Monotonic Deque chỉ giữ lại các ứng viên cực trị (Min/Max), không lưu đầy đủ tập hợp phần tử. Tìm Median cần dùng 2 Multiset hoặc PBDS Tree.

Câu 15 (số bước biến đổi từ a sang b nhỏ nhất):

Cho số nguyên A , mỗi bước có thể nhân 2 ($A \times 2$) hoặc trừ 1 ($A - 1$). Để tìm số bước ít nhất biến A thành B , phương pháp tối ưu là:

- A. Thuật toán Tham lam trừ dần.
- B. **[Đáp án đúng]** Tìm kiếm theo chiều rộng (BFS) trên đồ thị trạng thái với Queue.
- C. Thuật toán Quay lui vét cạn.
- D. Quy hoạch động 2 chiều.

Giải thích: Mỗi thao tác tốn 1 bước $\Rightarrow$ BFS tìm đường ngắn nhất trên đồ thị không trọng số tìm ra đáp án nhanh nhất.

Ma trận bài tập thực hành (p0 → p5)

Mã Bài Tập	Tên Bài Toán	Mức Độ	Trọng Tâm Kiến Thức & Kỹ Năng Queue/Deque
CPPB-QUE-01	Cài Đặt Hàng Đợi Cơ Bản	P0	Thao tác <code>push</code> , <code>pop</code> , <code>front</code> và kiểm tra rỗng với <code>std::queue</code> .
CPPB-QUE-02	Sinh Chuỗi Số Nhị Phân Bằng Queue	P1	Hàng đợi sinh tuần tự N chuỗi nhị phân tăng dần.
CPPB-QUE-03	BFS Tìm Đường Đi Ngắn Nhất Đồ Thị	P1	Cài đặt BFS chuẩn mực trên danh sách kề không trọng số.
CPPB-QUE-04	Truy Vết Lộ Trình Ngắn Nhất BFS	P2	Sử dụng mảng <code>parent</code> khôi phục chính xác các đỉnh đi qua.
CPPB-QUE-05	Min Mọi Cửa Sổ Trượt Độ Dài K	P2	Monotonic Deque cơ bản $O(N)$ duy trì giá trị nhỏ nhất.
CPPB-QUE-06	Max Mọi Cửa Sổ Trượt Độ Dài K	P2	Monotonic Deque duy trì giá trị lớn nhất trên cửa sổ trượt.
CPPB-QUE-07	Kiểm Tra Đồ Thị Hai Phía (2-Coloring)	P2	BFS tô màu luân phiên <code>1</code> và <code>2</code> phát hiện chu trình lẻ.
CPPB-QUE-08	Biến Đổi Số Bước Nhỏ Nhất (A sang B)	P3	BFS trên không gian số nguyên với mảng đánh dấu <code>visited</code> .
CPPB-QUE-09	0-1 BFS Tìm Đường Ngắn Nhất Trọng Số 0/1	P3	Dùng <code>std::deque</code> tối ưu hóa đường đi trong $O(V + E)$.
CPPB-QUE-10	Đoạn Con Tổng Lớn Nhất Độ Dài Tối Đa K	P3	Kết hợp Prefix Sum và Monotonic Deque cực tiểu hóa <code>pref[j]</code> .
CPPB-QUE-11	Trò Chơi Vòng Tròn Josephus Bằng Queue	P3	Mô phỏng loại trừ vòng tròn bằng Queue quay vòng $O(N \cdot K)$.
CPPB-QUE-12	Khoảng Cách Đến Trạm Cứu Hòa Gần Nhất	P4	Multi-source BFS (BFS đa nguồn) đẩy toàn bộ trạm vào Queue ban đầu.
CPPB-QUE-13	Cửa Sổ Trượt Chênh Lệch $\text{Max-Min} \leq C$	P4	Duy trì đồng thời 2 Monotonic Deque (1 Min, 1 Max) trong $O(N)$.
CPPB-QUE-14	Cắt Băng Rôn Quảng Cáo Tối Ưu	P4	Deque tối ưu hóa quy hoạch động 1D trên mảng.
CPPB-QUE-15	Đua Xe Mê Cung Đổi Hướng (Mastery)	P5	0-1 BFS / BFS nhiều chiều trạng thái (r, c, dir) chuẩn Olympic.



Module 07: Đồ thị & cây truy vấn đoạn

<!-- Chuyên đề 19: Lý Thuyết Đồ Thị, BFS & DFS | 405 dòng | 18,374 bytes -->

Chuyên đề 19: Lý thuyết đồ thị cơ bản: Duyệt bfs & dfs

1. Bản chất đồ thị & các phương pháp biểu diễn

Đồ thị $G = (V, E)$ là cấu trúc toán học biểu diễn tập hợp các đỉnh (Vertices — V) và các cạnh nối giữa chúng (Edges — E). Đồ thị có thể là vô hướng (Undirected) hoặc có hướng (Directed), có trọng số hoặc không có trọng số.

! [Biểu diễn Đồ thị: Ma trận kề vs Danh sách kề] (/Users/vu/Developer/ikhEdu_lessons/courses/cpp-bang-b/lessons/lesson-19-ly-thuyet-do-thi-bfs-dfs/assets/graph_representations_vi.svg)

1.1. Ma trận kề (adjacency matrix)

- Mảng 2 chiều `int adj[N][N]` : `adj[u][v] = 1` nếu có cạnh nối giữa u và v .
- Ưu điểm:** Kiểm tra cạnh (u, v) trong $O(1)$.
- Nhược điểm:** Tốn $O(N^2)$ bộ nhớ. Khi $N = 10^5$, ma trận cần **40GB** RAM $\Rightarrow$ Sập bộ nhớ ngay lập tức (MLE). Chỉ dùng khi $N \leq 1000$.

1.2. Danh sách kề (adjacency list — chuẩn mực thi đấu)

- Sử dụng mảng các vector `vector<int> adj[N + 1]` : `adj[u]` chứa toàn bộ các đỉnh kề trực tiếp với u .
- Bộ nhớ:** Đúng $O(V + E)$, cực kỳ tiết kiệm và tối ưu cho đồ thị thưa trong lập trình thi đấu ($N, M \leq 2 \cdot 10^5$).
- Duyệt đỉnh kề:** `for (int v : adj[u])` tốn thời gian tỷ lệ thuận với bậc của đỉnh $O(deg(u))$.

2. Hai chiến lược duyệt đồ thị kinh điển: Bfs vs dfs

! [So sánh BFS vs DFS] (/Users/vu/Developer/ikhEdu_lessons/courses/cpp-bang-b/lessons/lesson-19-ly-thuyet-do-thi-bfs-dfs/assets/bfs_vs_dfs_traversal_vi.svg)

2.1. Tìm kiếm theo chiều rộng (breadth-first search — bfs)

- Sử dụng **Hàng đợi (Queue)**, lan tỏa theo từng tầng bán kính $d = 0, 1, 2, \dots$ tính từ đỉnh nguồn S .
- Đặc tính vàng:** Tìm đường đi có ít cạnh nhất (ngắn nhất) trên đồ thị không trọng số.

2.2. Tìm kiếm theo chiều sâu (depth-first search — dfs)

- Sử dụng **Đệ quy (hoặc Stack)**, đi sâu hết mức có thể trên một nhánh cho đến khi gặp ngõ cụt thì quay lui (Backtracking).
- Đặc tính vàng:** Cực kỳ hiệu quả để đếm thành phần liên thông, phát hiện chu trình, kiểm tra tính liên thông, định hướng cây DFS.

3. Ứng dụng: Đếm số thành phần liên thông & kiểm tra chu trình

![Đếm số thành phần liên thông](/Users/vu/Developer/ikhEdu_lessons/courses/cpp-bang-b/lessons/lesson-19-ly-thuyet-do-thi-bfs-dfs/assets/connected_components_vi.svg)

- Thuật toán đếm thành phần liên thông:** Duyệt qua mọi đỉnh $i \in [1, N]$. Nếu đỉnh i chưa được thăm (`!visited[i]`), tăng biến đếm số thành phần liên thông `components++` và gọi `DFS(i)` để loang thăm toàn bộ các đỉnh thuộc cùng thành phần.
- Phát hiện chu trình trên đồ thị vô hướng bằng DFS:** Khi duyệt từ u sang đỉnh kề v , nếu v đã được thăm (`visited[v] == true`) và $v \neq \text{parent}[u]$, ta khẳng định đồ thị **CÓ CHU TRÌNH!**

4. Các bẫy lỗi lập trình kinh điển (bug traps)

1. Bẫy thêm cạnh đồ thị vô hướng chỉ thêm 1 chiều:

- Với đồ thị vô hướng, cạnh giữa u và v phải thêm cả 2 chiều: `adj[u].push_back(v); adj[v].push_back(u);`. Quên thêm chiều thứ hai làm đồ thị biến thành đồ thị có hướng sai hoàn toàn.

1. Bẫy tràn ngăn xếp đệ quy (Stack Overflow) khi DFS đồ thị sâu:

- Nếu đồ thị là một đường thẳng $N = 2 \cdot 10^5$ đỉnh, hàm đệ quy `DFS` sẽ gọi sâu $2 \cdot 10^5$ tầng, vượt quá giới hạn ngăn xếp (Call Stack) của một số hệ điều hành và gây Runtime Error.

1. Bẫy cạnh lặp và khuyên (Multiple Edges & Self-loops):

- Đề bài có thể cho nhiều cạnh nối giữa cùng một cặp đỉnh (u, v) hoặc cạnh tự nối $u \rightarrow u$. Cần kiểm tra hoặc xử lý cẩn thận nếu thuật toán yêu cầu tính toán bậc hoặc trọng số tối thiểu.

5. Mẫu cài đặt chuẩn thi đấu (competitive templates)

Mẫu 1: Dfs đếm số thành phần liên thông và tìm kích thước từng thành phần



Lưu ý về Stack Overflow: Hàm DFS đệ quy dưới đây có thể gây tràn ngăn xếp hệ thống (Segmentation Fault) khi đồ thị có dạng đường thẳng $N = 2 \times 10^5$ đỉnh (độ sâu đệ quy lên tới N tầng). Trong thi đấu thực tế, nên dùng **DFS bằng `std::stack` tường minh** hoặc thiết lập `ulimit -s unlimited` (Linux) trước khi chạy. Mẫu đệ quy được giữ lại ở đây vì tính trực quan sự phạm.


```

#include <bits/stdc++.h>
using namespace std;

int n, m;
vector<vector<int>>> adj;

vector<bool> visited;

int dfs(int u) {
    visited[u] = true;
    int comp_size = 1;
    for (int v : adj[u]) {
        if (!visited[v]) {
            comp_size += dfs(v);
        }
    }
    return comp_size;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    if (!(cin >> n >> m)) return 0;

    if (n ≤ 0) return 0;

    adj.assign(n + 1, vector<int>());
    visited.assign(n + 1, false);

    for (int i = 0; i < m; ++i) {
        int u, v;
        cin >> u >> v;

        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    int component_count = 0;
    vector<int> component_sizes;

    for (int i = 1; i ≤ n; ++i) {
        if (!visited[i]) {
            component_count++;
            int sz = dfs(i);
            component_sizes.push_back(sz);
        }
    }

    cout << component_count << "\n";
    for (int i = 0; i < (int)component_sizes.size(); ++i) {
        cout << component_sizes[i] << (i + 1 == (int)component_sizes.size() ?

```

```
"" : " ");  
}  
cout << "\n";  
  
return 0;  
}
```

Mẫu 2: Phát hiện chu trình trên đồ thị vô hướng bằng dfs

```

#include <bits/stdc++.h>
using namespace std;

int n, m;
vector<vector<int>> adj;

vector<bool> visited;

bool has_cycle = false;

void dfs_cycle(int u, int p) {
    visited[u] = true;
    for (int v : adj[u]) {
        if (!visited[v]) {
            dfs_cycle(v, u);
        } else if (v != p) {
            has_cycle = true; // Gặp lại đỉnh đã thăm khác cha → Chu trình!
        }
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    if (!(cin >> n >> m)) return 0;

    if (n <= 0) return 0;

    adj.assign(n + 1, vector<int>());
    visited.assign(n + 1, false);

    for (int i = 0; i < m; ++i) {
        int u, v;
        cin >> u >> v;

        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    for (int i = 1; i <= n; ++i) {
        if (!visited[i]) {
            dfs_cycle(i, 0);
        }
    }

    if (has_cycle) {
        cout << "YES\n";
    } else {
        cout << "NO\n";
    }
}

```

```
return 0;
}
```

Câu hỏi trắc nghiệm củng cố khái niệm

Câu 1 (bộ nhớ của danh sách kề):

Với đồ thị gồm V đỉnh và E cạnh, danh sách kề `vector<int> adj[V + 1]` chiếm dung lượng bộ nhớ là bao nhiêu?

- A. $O(V^2)$
- B. [Đáp án đúng] $O(V + E)$
- C. $O(E^2)$
- D. $O(V \cdot E)$

Giải thích: Mỗi đỉnh lưu một vector chứa các cạnh kề. Tổng số phần tử trong toàn bộ các vector đúng bằng $2E$ (vô hướng) hoặc E (có hướng) $\Rightarrow O(V + E)$.

Câu 2 (độ phức tạp thời gian duyệt toàn bộ đồ thị):

Thuật toán BFS và DFS khi duyệt qua toàn bộ đồ thị biểu diễn bằng danh sách kề có độ phức tạp thời gian là:

- A. $O(V \cdot E)$
- B. [Đáp án đúng] $O(V + E)$
- C. $O(V^2)$
- D. $O(E \log V)$

Giải thích: Mỗi đỉnh được thăm đúng 1 lần ($O(V)$) và mỗi cạnh được duyệt qua tối đa 2 lần ($O(E)$) $\Rightarrow$ Tổng thời gian $O(V + E)$.

Câu 3 (điều kiện phát hiện chu trình đồ thị vô hướng):

Trong hàm `DFS(u, p)` với p là đỉnh cha trực tiếp của u , dấu hiệu nào khẳng định có chu trình?

- A. Gặp một đỉnh kề v chưa được thăm.
- B. [Đáp án đúng] Gặp đỉnh kề v đã được thăm ($visited[v] = true$) và $v \neq p$.
- C. Đỉnh u có bậc lớn hơn 2.
- D. Khi DFS kết thúc mà còn đỉnh chưa thăm.

Giải thích: Nếu đi tới một đỉnh đã thăm mà không phải quay ngược lại đỉnh vừa sinh ra mình, ta vừa đi vòng qua một chu trình khép kín.

Câu 4 (tìm đường đi ngắn nhất không trọng số):

Để tìm đường đi qua ít cạnh nhất từ đỉnh S đến đỉnh T , thuật toán nào luôn đảm bảo tìm ra kết quả tối ưu đầu tiên?

- A. DFS.
- B. **[Đáp án đúng]** BFS (Duyệt theo chiều rộng).
- C. Tìm kiếm nhị phân.
- D. Quy hoạch động 1D.

Giải thích: BFS lan tỏa theo từng tầng khoảng cách tăng dần (0, 1, 2...), nên lần đầu tiên chạm tới T chắc chắn là đường đi ngắn nhất.

Câu 5 (bậc của đỉnh trong đồ thị vô hướng):

Tổng bậc của toàn bộ các đỉnh trong đồ thị vô hướng $G = (V, E)$ luôn bằng:

- A. E
- B. **[Đáp án đúng]** $2E$ (Định lý Bắt tay — Handshaking Lemma).
- C. V
- D. $V + E$

Giải thích: Mỗi cạnh nối giữa 2 đỉnh đóng góp đúng 1 bậc vào đỉnh u và 1 bậc vào đỉnh $v \Rightarrow \sum \deg(v) = 2E$.

Câu 6 (kiểm tra đồ thị liên thông):

Đồ thị vô hướng G gồm N đỉnh là liên thông khi và chỉ khi:

- A. Số cạnh $M \geq N$.
- B. **[Đáp án đúng]** Sau khi gọi `DFS(1)` (hoặc `BFS(1)`), toàn bộ N đỉnh đều có `visited[i] = true` (số thành phần liên thông đúng bằng 1).
- C. Đồ thị không có chu trình.
- D. Mọi đỉnh đều có bậc chẵn.

Giải thích: Liên thông nghĩa là từ một đỉnh bất kỳ có thể đi tới mọi đỉnh còn lại trong đồ thị.

Câu 7 (thứ tự duyệt topo trên dag):

Thuật toán Sắp xếp Tô-pô (Topological Sort) chỉ áp dụng được trên loại đồ thị nào?

- A. Đồ thị vô hướng bất kỳ.
- B. **[Đáp án đúng]** Đồ thị có hướng không có chu trình (Directed Acyclic Graph — DAG).
- C. Đồ thị có chu trình âm.
- D. Cây nhị phân hoàn hảo.

Giải thích: Tô-pô sắp xếp các đỉnh theo thứ tự tiên quyết, nếu có chu trình thì sẽ xảy ra mâu thuẫn phụ thuộc vòng tròn.

Câu 8 (đồ thị đầy đủ K_N):

Đồ thị đơn vô hướng đầy đủ gồm N đỉnh có chính xác bao nhiêu cạnh?

- A. $N(N - 1)$
- B. **[Đáp án đúng]** $\frac{N(N - 1)}{2}$
- C. N^2
- D. $2N - 1$

Giải thích: Mỗi cặp 2 đỉnh bất kỳ đều có 1 cạnh nối $\Rightarrow \text{binom}\{N\}\{2\} = \frac{N(N-1)}{2}$.

Câu 9 (độ sâu của cây dfs):

Khi DFS trên một đồ thị hình đường thẳng (Line Graph) gồm $N = 10^5$ đỉnh, ngăn xếp đệ quy sẽ đạt độ sâu tối đa là:

- A. $O(1)$
- B. **[Đáp án đúng]** $O(N) = 10^5$ tầng đệ quy.
- C. $O(\log N)$
- D. $O(\sqrt{N})$

Giải thích: Nhánh đệ quy đi thẳng từ đầu nút này sang đầu nút kia mà không rẽ nhánh $\Rightarrow$ Độ sâu bằng N .

Câu 10 (cạnh cầu trong đồ thị bridge):

Một cạnh trong đồ thị vô hướng được gọi là Cạnh Cầu (Bridge) khi nào?

- A. Khi nó thuộc một chu trình.
- B. **[Đáp án đúng]** Khi xóa cạnh đó đi, số thành phần liên thông của đồ thị sẽ tăng lên.
- C. Khi trọng số của nó lớn nhất.
- D. Khi nó nối với đỉnh cô lập.

Giải thích: Cạnh cầu là nút thắt duy nhất kết nối 2 phần của đồ thị, loại bỏ nó sẽ làm đồ thị bị chia cắt.

Câu 11 (đỉnh khớp trong đồ thị articulation point):

Một đỉnh u được gọi là Đỉnh Khớp (Cut Vertex) khi nào?

- A. Khi u có bậc lớn hơn 3.
- B. **[Đáp án đúng]** Khi xóa đỉnh u cùng toàn bộ các cạnh kề với nó, số thành phần liên thông của đồ thị tăng lên.
- C. Khi u là đỉnh gốc của DFS.
- D. Khi u không có cạnh nối.

Giải thích: Đỉnh khớp là điểm chốt liên kết, loại bỏ nó sẽ làm mất tính liên thông giữa các vùng khác.

Câu 12 (cây khung của đồ thị spanning tree):

Cây khung của một đồ thị vô hướng liên thông gồm N đỉnh là:

- A. Một đồ thị con chứa N đỉnh và N cạnh.
- B. **[Đáp án đúng]** Một đồ thị con chứa toàn bộ N đỉnh, đúng $N-1$ cạnh và không chứa chu trình.
- C. Đường đi ngắn nhất giữa 2 đỉnh.
- D. Đồ thị hai phía.

Giải thích: Cây khung là cây tối thiểu kết nối toàn bộ N đỉnh của đồ thị ban đầu.

Câu 13 (đồ thị euler):

Một đồ thị vô hướng liên thông có chu trình Euler (đi qua mỗi cạnh đúng 1 lần và quay về điểm xuất phát) khi và chỉ khi:

- A. Mọi đỉnh đều có bậc lẻ.
- B. **[Đáp án đúng]** Mọi đỉnh của đồ thị đều có bậc chẵn.
- C. Số cạnh bằng số đỉnh trừ 1.
- D. Có đúng 2 đỉnh bậc lẻ.

Giải thích: Định lý Euler: Bậc chẵn đảm bảo mỗi khi đi vào một đỉnh bằng 1 cạnh thì luôn có 1 cạnh khác chưa dùng để đi ra.

Câu 14 (bfs đa nguồn multi-source bfs):

Khi cần tìm khoảng cách ngắn nhất từ một tập hợp gồm K đỉnh nguồn đến mọi đỉnh còn lại trên đồ thị, kỹ thuật chuẩn mực là:

- A. Chạy K lần BFS độc lập tốn $O(K \cdot (V + E))$.
- B. **[Đáp án đúng]** Đẩy toàn bộ K đỉnh nguồn vào Queue ban đầu với khoảng cách bằng 0 rồi chạy đúng 1 lần BFS trong $O(V + E)$.
- C. Dùng thuật toán Floyd-Warshall $O(V^3)$.
- D. Sắp xếp các đỉnh.

Giải thích: Multi-source BFS lan tỏa đồng thời từ toàn bộ các nguồn, đỉnh nào gần nguồn bất kỳ nhất sẽ được thăm trước.

Câu 15 (đồ thị có hướng liên thông mạnh scc):

Một đồ thị có hướng được gọi là Liên thông mạnh (Strongly Connected) khi:

- A. Có ít nhất một đỉnh đi được tới mọi đỉnh.
- B. **[Đáp án đúng]** Giữa hai đỉnh bất kỳ u và v , luôn tồn tại đường đi từ $u \rightarrow v$ và đường đi từ $v \rightarrow u$.
- C. Không có chu trình.
- D. Số đỉnh bằng số cạnh.

Giải thích: Liên thông mạnh đòi hỏi sự thông suốt hai chiều giữa mọi cặp đỉnh trong đồ thị có hướng (thuật toán Tarjan / Kosaraju).

Ma trận bài tập thực hành (p0 → p5)

Mã Bài Tập	Tên Bài Toán	Mức Độ	Trọng Tâm Kiến Thức & Kỹ Năng Đồ Thị
CPPB- GRA-01	Chuyển Đổi Danh Sách Cạnh Sang Danh Sách Kề	P0	Xây dựng <code>adj[u]</code> và tính bậc của từng đỉnh trong $O(V+E)$.
CPPB- GRA-02	Duyệt Đồ Thị Theo Chiều Sâu (DFS Traversal)	P1	In thứ tự các đỉnh được thăm bằng hàm đệ quy <code>DFS</code> .
CPPB- GRA-03	Duyệt Đồ Thị Theo Chiều Rộng (BFS Traversal)	P1	In thứ tự các đỉnh được thăm bằng hàng đợi <code>std::queue</code> .
CPPB- GRA-04	Đếm Số Thành Phần Liên Thông	P2	Đếm số lần gọi <code>DFS</code> trên các đỉnh chưa thăm.
CPPB- GRA-05	Tìm Kích Thước Thành Phần Liên Thông Lớn Nhất	P2	DFS tích lũy số lượng đỉnh trong từng vùng liên thông.
CPPB- GRA-06	Kiểm Tra Đường Đi Giữa Hai Đỉnh (Path Finding)	P2	DFS / BFS kiểm tra xem T có đến được từ S .
CPPB- GRA-07	Phát Hiện Chu Trình Trên Đồ Thị Vô Hướng	P2	DFS phát hiện cạnh ngược nối về đỉnh đã thăm khác cha.
CPPB- GRA-08	Tìm Đường Đi Ngắn Nhất Bằng BFS	P3	Tính mảng <code>dist</code> và dùng mảng <code>parent</code> truy vết đường đi.
CPPB- GRA-09	Kiểm Tra Đồ Thị Cây (Tree Verification)	P3	Kiểm tra đồ thị liên thông và có đúng $N-1$ cạnh.
CPPB- GRA-10	Sắp Xếp Tô-pô (Topological Sort) Trên DAG	P3	Thuật toán Kahn dùng Bán bậc vào (In-degree) hoặc DFS hậu thứ tự.
CPPB- GRA-11	Tìm Chu Trình Độ Dài Nhỏ Nhất (Shortest Cycle)	P3	BFS từ từng đỉnh tìm cạnh ngược ngắn nhất.
CPPB- GRA-12	Đếm Cặp Đỉnh Không Thể Đi Tới Nhau	P4	Tổ hợp trên kích thước các thành phần liên thông: $\sum s_{z_i} \times (N - s_{z_i}) / 2$.
CPPB- GRA-13	Multi-Source BFS Lan Tỏa Virus	P4	Khởi tạo Queue với toàn bộ các ổ dịch, lan tỏa theo thời gian.
CPPB- GRA-14	Tìm Cạnh Cầu Trên Đồ Thị (Bridge Finding)	P4	Thuật toán Tarjan cơ bản với mảng <code>num</code> và <code>low</code> .
CPPB- GRA-15	Mạng Lưới Giao Thông Tối Ưu Olympic (Mastery)	P5	Bài toán đồ thị tổng hợp kết hợp liên thông và đường đi tối ưu.

<!-- Chuyên đề 20: Đồ Thị Lưới 2D & Flood Fill | 407 dòng | 18,604 bytes -->

Chuyên đề 20: Đồ thị lưới 2d, kỹ thuật flood fill & tính chất cây

1. Bản chất mô hình hóa lưới 2d thành đồ thị

Trong lập trình thi đấu, ma trận bảng vuông $N \times M$ có thể được xem là một đồ thị đặc biệt:

- Mỗi ô (r, c) là một **Đỉnh** của đồ thị ($1 \leq r \leq N, 1 \leq c \leq M$). Tổng số đỉnh $|V| = N \times M$.
- Mỗi bước di chuyển sang các ô kề cạnh (4 hướng: Trên, Dưới, Trái, Phải) tương đương với một **Cạnh** vô hướng có trọng số bằng 1. Tổng số cạnh $|E| \leq 4NM$.
- Ưu điểm vượt trội:** Không cần dựng danh sách kề `vector<int> adj[]`, ta duyệt trực tiếp trên ma trận bằng **Mảng Hướng Dịch Chuyển (dr, dc)**.

![Mô hình hóa Đồ thị Lưới 2D và Mảng Hướng] (/Users/vu/Developer/ikhEdu_lessons/courses/cpp-bang-b/lessons/lesson-20-do-thi-luoi-2d-flood-fill/assets/grid_2d_graph_modeling_vi.svg)

2. Thuật toán loang (flood fill)

![Thuật toán Loang Flood Fill] (/Users/vu/Developer/ikhEdu_lessons/courses/cpp-bang-b/lessons/lesson-20-do-thi-luoi-2d-flood-fill/assets/flood_fill_maze_vi.svg)

- Bản chất:** Từ một ô xuất phát (r_0, c_0) , thuật toán lan tỏa (bằng DFS hoặc BFS) sang tất cả các ô lân cận có cùng tính chất (cùng màu, ô đất liền không phải nước biển, ô đường đi không có vật cản).
- Điều kiện biên hợp lệ (Boundary Invariant):**

```
bool isValid(int r, int c) {
    return (r >= 1 && r <= n && c >= 1 && c <= m && grid[r][c] != '#' && !
visited[r][c]);
}
```

- Ứng dụng kinh điển:** Đếm số lượng hòn đảo (Number of Islands), tính diện tích vùng lớn nhất, tô màu sơn vùng kín, tìm đường thoát khỏi mê cung.

3. Lý thuyết cây trên đồ thị (tree properties & invariants)

![Đặc tính Bất biến của Cây] (/Users/vu/Developer/ikhEdu_lessons/courses/cpp-bang-b/lessons/lesson-20-do-thi-luoi-2d-flood-fill/assets/tree_properties_and_cycles_vi.svg)

Cây (Tree) là một dạng đồ thị vô hướng đặc biệt có cấu trúc phân cấp chặt chẽ:

- Đồ thị liên thông gồm N đỉnh và có **đúng $N - 1$ cạnh**.

- Giữa 2 đỉnh bất kỳ trong cây có **duy nhất một đường đi đơn**.
- Không chứa bất kỳ chu trình nào.
- Duyệt cây bằng DFS**: Bắt đầu từ gốc `root`, khi duyệt từ `u` sang `v` chỉ cần điều kiện `if (v != parent)` mà không cần dùng mảng `visited` !

4. Các bẫy lỗi lập trình kinh điển (bug traps)

1. Bẫy tràn chỉ số biên ma trận (Index Out of Bounds):

- Truy cập `grid[r + dr[d]][c + dc[d]]` trước khi kiểm tra `1 <= r+dr[d] <= N` sẽ gây lỗi Segmentation Fault.
- Quy tắc an toàn**: Luôn kiểm tra tọa độ trong phạm vi $[1, N] \times [1, M]$ trước tiên!

1. Bẫy nhầm lẫn thứ tự tọa độ Hàng và Cột (`r` vs `c` , `x` vs `y`):

- Trong toán học, trục `x` là ngang, `y` là dọc. Nhưng trong ma trận máy tính, chỉ số thứ nhất là **Hàng** (chiều dọc, `N`), chỉ số thứ hai là **Cột** (chiều ngang, `M`).
- Chuẩn hóa đặt tên**: Dùng `r` (row) và `c` (col) hoặc `dr` và `dc` để triệt tiêu hoàn toàn sự nhầm lẫn.

1. **Bẫy kích thước ma trận hình chữ nhật (`N ne M`):**

- Viết nhầm `c <= n` thay vì `c <= m` khi ma trận có số hàng khác số cột sẽ dẫn đến truy cập sai vùng nhớ hoặc đọc thiếu dữ liệu.

5. Mẫu cài đặt chuẩn thi đấu (competitive templates)

Mẫu 1: Đếm số lượng hòn đảo và diện tích lớn nhất (flood fill dfs)

⚠ **Lưu ý về Stack Overflow**: Hàm DFS đệ quy trên lưới 2D có thể gây tràn ngăn xếp hệ thống khi hòn đảo có kích thước lớn (ví dụ lưới 500×500 toàn ô đất tạo ra độ sâu đệ quy $250,000$ tầng). Trong thi đấu thực tế với lưới lớn ($N \times M \geq 10^5$), **nên dùng BFS bằng `std::queue` (xem Mẫu 2 bên dưới)** để tránh hoàn toàn rủi ro này. Mẫu DFS đệ quy được giữ lại vì tính trực quan sư phạm.

```

#include <bits/stdc++.h>
using namespace std;

int n, m;
vector<string> grid;

vector<vector<bool>> visited;

const int dr[] = {-1, 1, 0, 0};
const int dc[] = {0, 0, -1, 1};

bool isValid(int r, int c) {
    return (r ≥ 0 && r < n && c ≥ 0 && c < m && grid[r][c] == '1' && !
visited[r][c]);
}

int dfs(int r, int c) {
    visited[r][c] = true;
    int area = 1;

    for (int d = 0; d < 4; ++d) {
        int nr = r + dr[d];
        int nc = c + dc[d];
        if (isValid(nr, nc)) {
            area += dfs(nr, nc);
        }
    }
    return area;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    if (!(cin >> n >> m)) return 0;

    if (n ≤ 0 || m ≤ 0) return 0;

    grid.resize(n);
    for (int i = 0; i < n; ++i) {
        cin >> grid[i];
    }

    visited.assign(n, vector<bool>(m, false));
    int island_count = 0;
    int max_area = 0;

    for (int r = 0; r < n; ++r) {
        for (int c = 0; c < m; ++c) {
            if (grid[r][c] == '1' && !visited[r][c]) {
                island_count++;
            }
        }
    }
}

```

```
        max_area = max(max_area, dfs(r, c));  
    }  
    }  
}  
  
cout << island_count << " " << max_area << "\n";  
return 0;  
}
```


Mẫu 2: Tìm đường đi ngắn nhất trong mê cung (grid bfs)

```

#include <bits/stdc++.h>
using namespace std;

int n, m;
vector<string> grid;

vector<vector<int>> dist;

const int dr[] = {-1, 1, 0, 0};
const int dc[] = {0, 0, -1, 1};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    if (!(cin >> n >> m)) return 0;

    if (n ≤ 0 || m ≤ 0) return 0;

    grid.resize(n);
    int sr = -1, sc = -1, er = -1, ec = -1;

    for (int r = 0; r < n; ++r) {
        cin >> grid[r];

        for (int c = 0; c < m; ++c) {
            if (grid[r][c] == 'S') { sr = r; sc = c; }
            if (grid[r][c] == 'E') { er = r; ec = c; }
        }
    }

    dist.assign(n, vector<int>(m, -1));
    queue<pair<int, int>> q;

    dist[sr][sc] = 0;
    q.push({sr, sc});

    while (!q.empty()) {
        auto [r, c] = q.front();
        q.pop();

        if (r == er && c == ec) break;

        for (int d = 0; d < 4; ++d) {
            int nr = r + dr[d];
            int nc = c + dc[d];

            if (nr ≥ 0 && nr < n && nc ≥ 0 && nc < m && grid[nr][nc] ≠ '#' &&
                dist[nr][nc] == -1) {
                dist[nr][nc] = dist[r][c] + 1;
                q.push({nr, nc});
            }
        }
    }
}

```

```

    }
}

cout << dist[er][ec] << "\n";
return 0;
}

```

Câu hỏi trắc nghiệm củng cố khái niệm

Câu 1 (số cạnh tối đa trên lưới 2d 4 hướng):

Lưới ma trận $N \times M$ với quy tắc di chuyển 4 hướng có số đỉnh và số cạnh tối đa là:

- A. $|V| = N + M, |E| = NM$
- B. **[Đáp án đúng]** $|V| = N \cdot M, |E| \leq 4NM$ (chính xác là $2NM - N - M$ cạnh vô hướng).
- C. $|V| = N^2, |E| = M^2$
- D. $|V| = 2NM, |E| = N \cdot M$

Giải thích: Mỗi ô là 1 đỉnh (NM đỉnh), mỗi đỉnh có tối đa 4 liên kết với các ô lân cận.

Câu 2 (mảng hướng 8 hướng bao gồm cả đường chéo):

Để di chuyển 8 hướng (kể cả 4 hướng chéo) trên lưới 2D, mảng dịch chuyển dr và dc cần có bao nhiêu phần tử?

- A. 4 phần tử.
- B. **[Đáp án đúng]** 8 phần tử: $dr = \{-1, -1, -1, 0, 0, 1, 1, 1\}, dc = \{-1, 0, 1, -1, 1, -1, 0, 1\}$.
- C. 6 phần tử.
- D. 16 phần tử.

Giải thích: 8 hướng gồm 4 hướng chính (trên, dưới, trái, phải) và 4 hướng chéo góc.

Câu 3 (độ phức tạp thuật toán loang flood fill):

Thuật toán Flood Fill duyệt qua toàn bộ ma trận $N \times M$ có độ phức tạp thời gian và bộ nhớ là:

- A. Thời gian $O(N^2 \cdot M^2)$, Bộ nhớ $O(1)$.
- B. **[Đáp án đúng]** Thời gian $O(N \cdot M)$, Bộ nhớ $O(N \cdot M)$.
- C. Thời gian $O(2N^+ M)$, Bộ nhớ $O(N)$.
- D. Thời gian $O(N \log M)$, Bộ nhớ $O(M)$.

Giải thích: Mỗi ô (r, c) được thăm đúng 1 lần và kiểm tra 4 hướng lân cận trong $O(1) \Rightarrow O(N \cdot M)$.

Câu 4 (đường đi của quân mã trên bàn cờ knight moves):

Quân mã trong cờ vua có bao nhiêu bước nhảy hợp lệ và biểu diễn mảng hướng như thế nào?

- A. 4 bước nhảy dạng chữ thập.
- B. **[Đáp án đúng]** 8 bước nhảy hình chữ L: $dr = \{-2, -2, -1, -1, 1, 1, 2, 2\}$, $dc = \{-1, 1, -2, 2, -2, 2, -1, 1\}$.
- C. 8 bước nhảy đường chéo.
- D. 2 bước nhảy.

Giải thích: Quân mã di chuyển 2 ô theo một trục và 1 ô theo trục vuông góc, tạo thành 8 vị trí có thể đến.

Câu 5 (đặc tính bất biến của cây n đỉnh):

Một đồ thị vô hướng gồm N đỉnh là một Cây khi thỏa mãn đồng thời hai điều kiện nào sau đây?

- A. Có N cạnh và liên thông.
- B. **[Đáp án đúng]** Liên thông và có đúng $N - 1$ cạnh (hoặc không có chu trình và có đúng $N - 1$ cạnh).
- C. Mọi đỉnh đều có bậc ≥ 2 .
- D. Có chu trình Euler.

Giải thích: Định lý cơ bản của lý thuyết cây: Đồ thị liên thông có $N - 1$ cạnh tương đương với đồ thị phi chu trình có $N - 1$ cạnh.

Câu 6 (đường kính của cây — tree diameter):

Đường kính của cây (khoảng cách lớn nhất giữa hai đỉnh bất kỳ trên cây) có thể tìm bằng mấy lần BFS/DFS?

- A. 1 lần duy nhất.
- B. **[Đáp án đúng]** 2 lần BFS/DFS: Lần 1 từ đỉnh bất kỳ tìm đỉnh xa nhất u ; Lần 2 từ u tìm đỉnh xa nhất v , khoảng cách $dist(u, v)$ chính là đường kính cây.
- C. N lần BFS từ mọi đỉnh $O(N^2)$.
- D. Bắt buộc dùng thuật toán Dijkstra.

Giải thích: Đây là thuật toán 2 lượt BFS kinh điển chạy trong $O(N)$ cực kỳ đẹp mắt trên cây.

Câu 7 (độ cao của cây khi chọn gốc):

Khi chọn đỉnh R làm gốc (Root) của cây, chiều cao của cây được định nghĩa là:

- A. Tổng số đỉnh trong cây.
- B. **[Đáp án đúng]** Khoảng cách lớn nhất từ gốc R đến một đỉnh lá bất kỳ ($\max_v \text{dist}(R, v)$).
- C. Bậc lớn nhất của một đỉnh.
- D. Số lượng cạnh của cây.

Giải thích: Chiều cao là độ sâu lớn nhất của một nút lá tính từ gốc R .

Câu 8 (tô màu vùng kín enclosed regions):

Để tìm các vùng nước biển bị bao bọc hoàn toàn bên trong đất liền (không thông ra biên ma trận), chiến lược tối ưu là:

- A. Flood fill từ từng ô bên trong.
- B. **[Đáp án đúng]** Chạy Flood Fill từ toàn bộ các ô biên ngoài cùng để đánh dấu các ô "thông ra ngoài", các ô còn lại chưa thăm chính là vùng kín bên trong.
- C. Dùng thuật toán quay lui.
- D. Sắp xếp các ô.

Giải thích: Đảo ngược bài toán: Loang từ biên ngoài vào trong giúp loại bỏ toàn bộ phần không bị bao bọc trong $O(NM)$.

Câu 9 (số lượng lá tối thiểu của một cây $N \geq 2$):

Mọi cây có $N \geq 2$ đỉnh luôn có ít nhất bao nhiêu đỉnh lá (đỉnh có bậc bằng 1)?

- A. 0 lá.
- B. **[Đáp án đúng]** Ít nhất 2 đỉnh lá.
- C. Đúng 1 lá.
- D. $N/2$ lá.

Giải thích: Bằng chứng minh phản chứng qua định lý bắt tay, một cây luôn có tối thiểu 2 đỉnh lá ở 2 đầu mút của đường đi dài nhất.

Câu 10 (truy vết đường đi trong mê cung 2d):

Để in ra chuỗi ký tự các bước đi 'U', 'D', 'L', 'R' từ S đến E trong mê cung, ta lưu thông tin gì trong BFS?

- A. Lưu mảng boolean `visited`.
- B. **[Đáp án đúng]** Mảng `parent[r][c] = {pr, pc}` và `move_dir[r][c] = 'D'`, sau đó lần ngược từ E về S rồi đảo ngược chuỗi.
- C. In trực tiếp khi đang duyệt.
- D. Dùng hàm đệ quy in xuôi.

Giải thích: Lưu vết hướng đi và tọa độ cha cho phép tái tạo chính xác lộ trình từng bước.

Câu 11 (thuật toán loang đa nguồn trên lưới):

Trong bài toán "Cháy rừng" (nhiều điểm cháy cùng lúc lan sang các cây xung quanh mỗi giây), ta giải bằng cấu trúc nào?

- A. Chạy DFS độc lập từ từng đám cháy.
- B. **[Đáp án đúng]** Multi-source BFS: Đẩy toàn bộ tọa độ các đám cháy ban đầu vào Queue với `dist = 0`, sau đó loang từng lớp theo thời gian.
- C. Dùng quy hoạch động 2 chiều.
- D. Dùng thuật toán Dijkstra.

Giải thích: BFS đa nguồn mô phỏng chính xác sự lan tỏa đồng thời của các đám cháy theo từng đơn vị thời gian.

Câu 12 (bậc của đỉnh trong cây):

Trên một Cây có gốc, một đỉnh u có K nút con trực tiếp. Bậc của đỉnh u (vô hướng) bằng bao nhiêu?

- A. Luôn bằng K .
- B. **[Đáp án đúng]** Bằng K (nếu u là gốc) hoặc $K + 1$ (nếu u không phải gốc, gồm K con và 1 cha).
- C. Bằng $K - 1$.
- D. Bằng $2K$.

Giải thích: Nút không phải gốc có 1 cạnh nối lên nút cha và K cạnh nối xuống các nút con.

Câu 13 (cây con subtree size):

Để tính kích thước của mọi cây con $sz[u]$ (số lượng đỉnh thuộc cây con gốc u), ta sử dụng hàm đệ quy DFS theo thứ tự nào?

- A. Tiền thứ tự (Pre-order, tính trước khi duyệt con).
- B. **[Đáp án đúng]** Hậu thứ tự (Post-order: $sz[u] = 1 + \sum_{v \in \text{children}[u]} sz[v]$ sau khi đã tính xong mọi con).
- C. Duyệt ngẫu nhiên.
- D. Không thể tính bằng DFS.

Giải thích: Kích thước cây con được gom dồn từ dưới đáy lá ngược lên gốc (Post-order DP on Trees).

Câu 14 (chu trình trong đồ thị lưới):

Một đồ thị lưới 2D kích thước 2×2 có chứa chu trình hay không?

- A. Không có chu trình vì lưới là cây.
- B. **[Đáp án đúng]** Có chứa đúng 1 chu trình độ dài 4:
 $(1,1) \rightarrow (1,2) \rightarrow (2,2) \rightarrow (2,1) \rightarrow (1,1)$.
- C. Có chứa 2 chu trình.
- D. Tùy thuộc vào hướng di chuyển.

Giải thích: 4 ô tạo thành một vòng khép kín độ dài 4 $\Rightarrow$ Lưới 2D không phải là cây mà là đồ thị tổng quát có nhiều chu trình.

Câu 15 (số thành phần liên thông của tập ô đất liền):

Cho ma trận biển đảo, sau khi biến một ô nước '0' thành ô đất '1', số thành phần liên thông đảo sẽ thay đổi tối đa như thế nào?

- A. Luôn tăng thêm 1.
- B. **[Đáp án đúng]** Có thể tăng 1, giữ nguyên, hoặc giảm tối đa 3 (khi ô mới đóng vai trò cầu nối hợp nhất 4 hòn đảo xung quanh lại thành 1).
- C. Luôn giảm đi 1.
- D. Không bao giờ thay đổi.

Giải thích: Ô đất mới kết nối tối đa 4 đảo kề cạnh thành 1 hòn đảo duy nhất $\Rightarrow$ Giảm tối đa 4 - 1 = 3 thành phần.

Ma trận bài tập thực hành (p0 → p5)

Mã Bài Tập	Tên Bài Toán	Mức Độ	Trọng Tâm Kiến Thức & Kỹ Năng Lưới 2D
CPPB-GRD-01	Duyệt 4 Hướng Trên Ma Trận Cơ Bản	P0	Cài đặt mảng hướng <code>dr</code> , <code>dc</code> và hàm kiểm tra <code>isValid</code> .
CPPB-GRD-02	Đếm Số Lượng Hòn Đảo (Count Islands)	P1	Flood Fill DFS đếm số thành phần liên thông các ô đất <code>'1'</code> .
CPPB-GRD-03	Diện Tích Hòn Đảo Lớn Nhất	P1	DFS gom dồn số ô đất trong từng thành phần liên thông.
CPPB-GRD-04	Tìm Đường Thoát Khỏi Mê Cung BFS	P2	BFS tìm số bước ngắn nhất từ vị trí <code>S</code> đến <code>E</code> .
CPPB-GRD-05	Truy Vết Đường Đi Mê Cung (L, R, U, D)	P2	Lưu mảng <code>parent</code> và in ra chuỗi ký tự hướng đi cụ thể.
CPPB-GRD-06	Đếm Vùng Kín Không Thông Ra Biên	P2	Loang từ toàn bộ các ô viền biên để khử các vùng mở.
CPPB-GRD-07	Chu Vi Hòn Đảo (Island Perimeter)	P2	Đếm số cạnh tiếp xúc với nước hoặc tiếp xúc với biên ma trận.
CPPB-GRD-08	Nước Tràn Mê Cung (Multi-Source BFS)	P3	Đẩy đồng thời nhiều nguồn nước vào Queue ban đầu.
CPPB-GRD-09	Bước Nhảy Quân Mã Ngắn Nhất (Knight Moves)	P3	BFS với mảng 8 hướng di chuyển hình chữ L trên bàn cờ $N \times M$.
CPPB-GRD-10	Đường Kính Của Cây (Tree Diameter)	P3	Thuật toán 2 lần BFS/DFS tìm khoảng cách lớn nhất giữa 2 đỉnh cây.
CPPB-GRD-11	Kích Thước Cây Con & Trọng Tâm Của Cây	P3	DFS hậu thứ tự tính <code>sz[u]</code> và tìm nút trọng tâm (Centroid).
CPPB-GRD-12	Mê Cung Có Cửa Dịch Chuyển Tức Thời (Teleport)	P4	Mô hình hóa các ô cùng màu kết nối nhau trong $O(1)$.
CPPB-GRD-13	Làm Đầy Hồ Chứa Nước (Rotting Oranges)	P4	BFS tính thời gian tối thiểu để toàn bộ cam bị hỏng.
CPPB-GRD-14	Hòn Đảo Nhân Tạo Lớn Nhất (Making A Large Island)	P4	Đánh số ID từng đảo rồi thử lật từng ô nước thành đất trong $O(NM)$.
CPPB-GRD-15	Thoát Khỏi Mê Cung Quái Vật Olympic (Mastery)	P5	2 lượt BFS đồng thời: Quái vật lan tỏa trước, Người đi sau chuẩn thi đấu.

<!-- Chuyên đề 21: Cây Phân Đoạn & Fenwick Tree | 468 dòng | 20,303 bytes -->

Chuyên đề 21: Cấu trúc cây phân đoạn (segment tree) & fenwick tree (bit)

1. Bản chất bài toán truy vấn đoạn động (dynamic range queries)

Cho mảng A gồm N phần tử. Cần thực hiện liên tiếp Q thao tác thuộc 2 loại:

- Cập nhật điểm (Point Update):** Thay đổi giá trị $A[i] \geq v$ (hoặc $A[i] \geq A[i] + v$).
- Truy vấn đoạn (Range Query):** Tính tổng $\sum_{k=L}^R A[k]$ hoặc tìm $\min_{k=L}^R A[k]$ / $\max_{k=L}^R A[k]$.

Cấu Trúc	Khởi Tạo (Build)	Cập Nhật Điểm (Update)	Truy Vấn Đoạn (Query)	Bộ Nhớ
Mảng Tiền Tố (Prefix Sum)	$O(N)$	$O(N)$ (Quá chậm khi có update)	$O(1)$	$O(N)$
Cây Fenwick (BIT)	$O(N)$	$O(\log N)$ (Cực nhanh)	$O(\log N)$	$O(N)$
Cây Phân Đoạn (Segment Tree)	$O(N)$	$O(\log N)$ (Cực nhanh)	$O(\log N)$	$O(4N)$

! [So sánh các cấu trúc Range Query] (/Users/vu/Developer/ikhEdu_lessons/courses/cpp-bang-b/lessons/lesson-21-cay-phan-doan-segment-tree-fenwick/assets/point_update_range_query_vi.svg)

2. Cây fenwick (binary indexed tree — bit)

! [Cây Fenwick và Phép toán Lowbit] (/Users/vu/Developer/ikhEdu_lessons/courses/cpp-bang-b/lessons/lesson-21-cay-phan-doan-segment-tree-fenwick/assets/fenwick_tree_lowbit_vi.svg)

2.1. Phép toán ma thuật: $\text{lowbit}(x) = x \& (-x)$

Phép toán $x \& (-x)$ trích xuất bit 1 thấp nhất (trọng số nhỏ nhất) của số nguyên x .

- Mỗi vị trí x trong mảng $\text{bit}[x]$ quản lý tổng của một đoạn con có độ dài đúng bằng $\text{lowbit}(x)$ kết thúc tại x :

$$\text{Đoạn quản lý của } x = (x - \text{lowbit}(x), x]$$

2.2. Hai thao tác cốt lõi siêu tinh gọn (chỉ 5 dòng code)

```
void update(int x, long long val) {
    for (; x ≤ n; x += x & -x) bit[x] += val;
}

long long query(int x) { // Tính tổng tiền tố A[1..x]
    long long sum = 0;
    for (; x > 0; x -= x & -x) sum += bit[x];

    return sum;
}

long long range_query(int L, int R) {
    return query(R) - query(L - 1);
}
```

3. Cây phân đoạn (segment tree)

![[Cây Phân Đoạn Segment Tree]](/Users/vu/Developer/ikhEdu_lessons/courses/cpp-bang-b/lessons/lesson-21-cay-phan-doan-segment-tree-fenwick/assets/segment_tree_binary_tree_vi.svg)

3.1. Cấu trúc cây nhị phân hoàn hảo

- Cây phân đoạn biểu diễn mảng quản lý theo cây nhị phân: Nút gốc $id = 1$ quản lý toàn đoạn $[1, N]$.
- Nút con trái quản lý nửa trái $[L, mid]$ tại vị trí $2 \cdot id$.
- Nút con phải quản lý nửa phải $[mid + 1, R]$ tại vị trí $2 \cdot id + 1$.
- **Quy tắc bộ nhớ:** Mảng cây Segment Tree cần khai báo **$4N$ phần tử** để đảm bảo không bị tràn chỉ số khi N không phải là lũy thừa của 2.

3.2. Ưu thế vượt trội của segment tree

Khác với Fenwick Tree chủ yếu hỗ trợ phép toán có tính nghịch đảo (như phép cộng tổng), Segment Tree hỗ trợ **MỌI PHÉP TOÁN KẾT HỢP (Associative Operations)**:

- Tìm giá trị nhỏ nhất / lớn nhất trên đoạn (Range Minimum / Maximum Query — RMQ).
- Tìm ước chung lớn nhất trên đoạn ($GCD(A[L \dots R])$).
- Đếm số lượng phần tử đạt cực đại trên đoạn.

4. Các bẫy lỗi lập trình kinh điển (bug traps)

1. **Bẫy quên khai báo mảng Segment Tree kích thước $4N$:**

- Khai báo `tree[2 N]` hoặc `tree[N]` sẽ bị tràn mảng (Out of Bounds) khi $N = 10^5$. Bắt buộc phải khai báo kích thước tối thiểu $4N$ (`vector<long long> tree(4 n + 5)`).

1. **Bẫy chỉ số 0-based của Fenwick Tree (Vòng lặp vô tận):**

- Trong Fenwick Tree, `lowbit(0) = 0 & -0 = 0`. Nếu gọi `update(0, val)` hoặc `query(0)`, vòng lặp `x ≥ ts x + (x | -x)` sẽ biến thành `x += 0` và chạy vô tận ⇒ Time Limit Exceeded!
- **Bất biến sống còn:** Fenwick Tree **BẮT BUỘC DÙNG CHỈ SỐ 1-BASED** ($x \geq 1$).

1. **Bẫy tràn số khi cộng dồn tổng trên cây:**

- Mảng $N = 10^5$ phần tử có giá trị $10^9 \Rightarrow$ Tổng đoạn có thể lên tới 10^{14} . Mảng `tree` và `bit` bắt buộc phải dùng kiểu `long long`.

5. Mẫu cài đặt chuẩn thi đấu (competitive templates)

Mẫu 1: Cây fenwick (bit) point update & range sum query

```

#include <bits/stdc++.h>
using namespace std;

struct FenwickTree {
    int n;
    vector<long long> bit;

    FenwickTree(int n) : n(n), bit(n + 1, 0) {}

    void update(int x, long long val) {
        for (; x ≤ n; x += x & -x) {
            bit[x] += val;
        }
    }

    long long query(int x) {
        long long sum = 0;
        for (; x > 0; x -= x & -x) {
            sum += bit[x];
        }
        return sum;
    }

    long long queryRange(int l, int r) {
        if (l > r) return 0;

        return query(r) - query(l - 1);
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q;
    if (!(cin >> n >> q)) return 0;

    if (n ≤ 0) return 0;

    FenwickTree ft(n);

    for (int i = 1; i ≤ n; ++i) {
        long long x;
        cin >> x;

        ft.update(i, x);
    }

    while (q--) {
        int type;
        cin >> type;
    }
}

```

```
        if (type == 1) { // Update: A[pos] += val
            int pos;
            long long val;
            cin >> pos >> val;

            ft.update(pos, val);
        } else { // Query: Sum(L..R)
            int l, r;
            cin >> l >> r;

            cout << ft.queryRange(l, r) << "\n";
        }
    }

    return 0;
}
```


Mẫu 2: Cây phân đoạn (segment tree) range minimum query (rmq)

```

#include <bits/stdc++.h>
using namespace std;

const long long INF = 1e18;

struct SegmentTree {
    int n;
    vector<long long> tree;

    SegmentTree(int n) : n(n), tree(4 * n + 5, INF) {}

    void build(const vector<long long>& a, int id, int l, int r) {
        if (l == r) {
            tree[id] = a[l];
            return;
        }
        int mid = (l + r) / 2;
        build(a, 2 * id, l, mid);
        build(a, 2 * id + 1, mid + 1, r);
        tree[id] = min(tree[2 * id], tree[2 * id + 1]);
    }

    void update(int id, int l, int r, int pos, long long val) {
        if (l == r) {
            tree[id] = val;
            return;
        }
        int mid = (l + r) / 2;
        if (pos <= mid) {
            update(2 * id, l, mid, pos, val);
        } else {
            update(2 * id + 1, mid + 1, r, pos, val);
        }
        tree[id] = min(tree[2 * id], tree[2 * id + 1]);
    }

    long long query(int id, int l, int r, int u, int v) {
        if (v < l || r < u) return INF; // Ngoài đoạn
        if (u <= l && r <= v) return tree[id]; // Nằm trọn trong đoạn

        int mid = (l + r) / 2;
        return min(query(2 * id, l, mid, u, v),
                   query(2 * id + 1, mid + 1, r, u, v));
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q;
    if (!(cin >> n >> q)) return 0;

```

```

    if (n ≤ 0) return 0;

    vector<long long> a(n + 1);

    for (int i = 1; i ≤ n; ++i) {
        cin >> a[i];
    }

    SegmentTree st(n);
    st.build(a, 1, 1, n);

    while (q--) {
        int type;
        cin >> type;

        if (type == 1) { // Gán A[pos] = val
            int pos;
            long long val;
            cin >> pos >> val;

            st.update(1, 1, n, pos, val);
        } else { // Tìm Min trong đoạn [L, R]
            int l, r;
            cin >> l >> r;

            cout << st.query(1, 1, n, l, r) << "\n";
        }
    }

    return 0;
}

```

Câu hỏi trắc nghiệm củng cố khái niệm

Câu 1 (khi nào dùng segment tree thay vì fenwick tree):

Ưu điểm quan trọng nhất của Segment Tree so với Fenwick Tree (BIT) cơ bản là gì?

- **A.** Segment Tree chạy nhanh hơn và tốn ít bộ nhớ hơn.
- **B. [Đáp án đúng]** Segment Tree hỗ trợ mọi phép toán kết hợp như Range Minimum/Maximum (RMQ), GCD, trong khi Fenwick Tree cơ bản chỉ hỗ trợ các phép toán có tính nghịch đảo (như phép cộng tổng).
- **C.** Segment Tree code ngắn hơn.
- **D.** Segment Tree không dùng đệ quy.

Giải thích: Phép toán $\min(x, y)$ không thể làm ngược lại (không có phép trừ min), do đó RMQ cần dùng Segment Tree.

Câu 2 (kích thước mảng segment tree):

Với mảng N phần tử, tại sao mảng cây Segment Tree dạng mảng 1D phẳng cần khai báo kích thước tối thiểu là $4N$?

- A. Vì mỗi nút có 4 nút con.
- B. **[Đáp án đúng]** Vì khi N không phải là lũy thừa của 2, cây nhị phân mở rộng xuống tầng lá tiếp theo có thể đạt tới chỉ số $4N - 1$.
- C. Để tránh tràn số nguyên.
- D. Vì cây có 4 tầng đệ quy.

Giải thích: Chỉ số lớn nhất của nút lá trong cây nhị phân phân đoạn có thể đạt tới xấp xỉ $4N$. Khai báo $4N$ đảm bảo an toàn tuyệt đối 100%.

Câu 3 (phép toán bit lowbit trong fenwick tree):

Giá trị của $\text{lowbit}(12)$ (tức $12 \& -12$) bằng bao nhiêu?

- A. 1
- B. 2
- C. **[Đáp án đúng]** 4 (Số 12 có dạng nhị phân là 1100 , bit 1 thấp nhất có trọng số là $2^2 = 4$).
- D. 8

Giải thích: $12 = 8 + 4 = 1100_2 \Rightarrow$ Bit 1 tận cùng bên phải có giá trị là 4.

Câu 4 (bẫy số 0 trong fenwick tree):

Điều gì sẽ xảy ra nếu ta gọi hàm `update(0, val)` trên cây Fenwick Tree chuẩn?

- A. Hàm cập nhật thành công ô số 0.
- B. **[Đáp án đúng]** Vòng lặp vô tận (Infinite Loop) vì $\text{lowbit}(0) == 0$, lệnh $x \geq x + (x \& -x)$ giữ nguyên $x = 0$ mãi mãi $\Rightarrow$ TLE.
- C. Báo lỗi biên dịch.
- D. Không có chuyện gì xảy ra.

Giải thích: $0 \mid \& \mid -0 = 0$, biến lặp không bao giờ tăng $\Rightarrow$ Cây Fenwick bắt buộc phải 1-based indexing.

Câu 5 (độ phức tạp khởi tạo build segment tree):

Hàm `build` dựng toàn bộ cây Segment Tree N phần tử từ mảng ban đầu có độ phức tạp thời gian là:

- A. $O(N \log N)$
- B. **[Đáp án đúng]** $O(N)$ tuyến tính (cây có tổng cộng $2N-1$ nút, mỗi nút tính trong $O(1)$).
- C. $O(N^2)$
- D. $O(\log N)$

Giải thích: Tổng số nút trên cây phân đoạn là $2N - 1$. Duyệt qua mỗi nút 1 lần duy nhất tốn $O(N)$.

Câu 6 (đếm số cặp nghịch thế bằng fenwick tree):

Để đếm số lượng cặp nghịch thế ($i < j$ và $A[i] > A[j]$) trong mảng N phần tử, ta kết hợp Fenwick Tree với kỹ thuật nào?

- A. Thuật toán hai con trỏ.
- B. **[Đáp án đúng]** Nén tọa độ về $[1, N]$, duyệt từ phải sang trái: cộng dồn `query(A[i] - 1)` vào đáp án rồi gọi `update(A[i], 1)` trong tổng thời gian $O(N \log N)$.
- C. Tìm kiếm nhị phân.
- D. Quy hoạch động 2 chiều.

Giải thích: Fenwick Tree đóng vai trò mảng đếm tần suất động, `query(A[i] - 1)` đếm số phần tử nhỏ hơn $A[i]$ đã xuất hiện phía sau.

Câu 7 (độ phức tạp mỗi truy vấn segment tree):

Tại sao hàm `query(L, R)` trên Segment Tree chỉ mất tối đa $O(\log N)$ dù phải rẽ nhánh đệ quy?

- A. Vì cây chỉ có 2 nút lá.
- B. **[Đáp án đúng]** Vì tại mỗi tầng của cây, truy vấn chỉ ghé thăm tối đa 4 nút (các đoạn nằm trọn bên trong trả về kết quả ngay lập tức).
- C. Vì hàm query không dùng đệ quy.
- D. Vì mảng đã được sắp xếp.

Giải thích: Bằng chứng minh toán học, số lượng nút được duyệt tại mỗi độ sâu của cây luôn $\leq 4 \Rightarrow 4 \log_2 N = O(\log N)$.

Câu 8 (cập nhật đoạn range update & điểm point query bằng fenwick):

Để hỗ trợ thao tác "Cộng thêm v vào toàn bộ đoạn $[L, R]$ " và "Hỏi giá trị tại vị trí i ", ta cài đặt Fenwick Tree trên mảng nào?

- A. Trên mảng gốc A .
- B. **[Đáp án đúng]** Trên Mảng hiệu (Difference Array $D[i] = A[i] - A[i-1]$): Update đoạn bằng `update(L, v)` và `update(R + 1, -v)`; Query điểm bằng `query(i)`.
- C. Trên mảng tiền tố.
- D. Trên mảng đảo ngược.

Giải thích: Sự kết hợp hoàn hảo giữa Mảng hiệu (Chuyên đề 04) và Cây Fenwick cho phép biến Range Update thành 2 phép Point Update.

Câu 9 (tìm kiếm k-th phần tử trên fenwick tree):

Để tìm phần tử thứ K nhỏ nhất còn tồn tại trong tập hợp bằng Fenwick Tree, ta áp dụng kỹ thuật nào trong $O(\log N)$?

- A. Tìm kiếm nhị phân thông thường $O(\log^2 N)$.
- B. **[Đáp án đúng]** Nhảy nhị phân trực tiếp trên cây Fenwick (Binary Lifting on BIT / BIT Walking) trong $O(\log N)$.
- C. Duyệt tuần tự $O(N)$.
- D. Sắp xếp lại cây.

Giải thích: Tương tự tìm kiếm nhị phân các lũy thừa của 2, tận dụng cấu trúc các bước nhảy 2^p của Fenwick Tree.

Câu 10 (nút cha và nút con trong segment tree dạng mảng):

Nút hiện tại có chỉ số id . Chỉ số của nút con trái và nút con phải lần lượt là:

- A. $id + 1$ và $id + 2$
- B. **[Đáp án đúng]** $2 \cdot id$ (hoặc `$id \ll 1$`) và $2 \cdot id + 1$ (hoặc `$(id \ll 1) | 1$`).
- C. $id/2$ và $id/2 + 1$
- D. $id \cdot 4$ và $id \cdot 4 + 1$

Giải thích: Đây là cách định vị nút chuẩn mực trong biểu diễn cây nhị phân hoàn hảo trên mảng 1 chiều.

Câu 11 (truy vấn ước chung lớn nhất range gcd):

Khi dùng Segment Tree để tính $GCD(A[L \dots R])$, công thức kết hợp tại mỗi nút cha là:

- A. $tree[id] = tree[2 \cdot id] + tree[2 \cdot id + 1]$
- B. **[Đáp án đúng]** $tree[id] = gcd(tree[2 \cdot id], tree[2 \cdot id + 1])$
- C. $tree[id] = \min(tree[2 \cdot id], tree[2 \cdot id + 1])$
- D. $tree[id] = tree[2 \cdot id] \times tree[2 \cdot id + 1]$

Giải thích: Phép tính GCD có tính kết hợp: $GCD(a, b, c) = GCD(GCD(a, b), c)$, áp dụng tự nhiên trên Segment Tree.

Câu 12 (điều kiện dừng đệ quy trong query segment tree):

Trong hàm `query(id, l, r, u, v)`, khi đoạn hiện tại $[l, r]$ nằm hoàn toàn bên ngoài đoạn truy vấn $[u, v]$ ($r < u$ hoặc $v < l$), ta trả về giá trị gì cho bài toán Range Sum?

- A. Trả về `1`.
- B. **[Đáp án đúng]** Trả về `0` (Phần tử trung hòa của phép cộng).
- C. Trả về vô cùng `∞`.
- D. Báo lỗi và dừng chương trình.

Giải thích: Giá trị trung hòa của phép cộng là 0 (đối với phép tìm Min là $+\infty$, phép tìm Max là $-\infty$, phép nhân là 1).

Câu 13 (segment tree không đệ quy — iterative segment tree):

Ưu điểm lớn nhất của Cây phân đoạn không đệ quy (Iterative / Bottom-Up Segment Tree) là:

- A. Hỗ trợ nhiều phép toán hơn.
- B. **[Đáp án đúng]** Tốc độ chạy nhanh hơn gấp 2–3 lần do loại bỏ hoàn toàn chi phí gọi hàm đệ quy và chỉ tốn đúng $2N$ bộ nhớ.
- C. Tự động nén tọa độ.
- D. Không cần truyền chỉ số.

Giải thích: Cây phân đoạn duyệt từ lá lên gốc (Bottom-Up) có hằng số thời gian cực nhỏ và code rất ngắn gọn.

Câu 14 (đếm số lượng số 0 trong đoạn):

Để đếm số lượng số 0 trong đoạn $[L, R]$ hỗ trợ cập nhật điểm, cấu hình Segment Tree lưu trữ gì?

- A. Lưu tổng các số.
- B. **[Đáp án đúng]** Mỗi lá lưu 1 nếu $A[i] == 0$ (ngược lại lưu 0), các nút cha lưu tổng số lượng số 0 của hai con: $tree[id] = tree[2 \cdot id] + tree[2 \cdot id + 1]$.
- C. Lưu giá trị nhỏ nhất.
- D. Dùng mảng hiệu.

Giải thích: Quy đổi bài toán đếm sang tính tổng các cờ boolean trên cây phân đoạn.

Câu 15 (kỹ thuật lazy propagation — giới thiệu mở rộng):

Khi cần thực hiện **Cập nhật cả đoạn (Range Update)** và **Truy vấn cả đoạn (Range Query)** trên Segment Tree trong $O(\log N)$, kỹ thuật nào được áp dụng?

- A. Dùng đệ quy quay lui.
- B. **[Đáp án đúng]** Kỹ thuật Lan truyền lười (Lazy Propagation): Chỉ cập nhật nút hiện tại và lưu giá trị chờ (lazy tag), chỉ đẩy xuống con khi có truy vấn cần thiết.
- C. Dùng 2 cây Fenwick.
- D. Chia mảng thành $\sqrt{N}$ khối (Mo's Algorithm).

Giải thích: Lazy Propagation là kỹ thuật đỉnh cao trì hoãn cập nhật giúp thực hiện Range Update trong $O(\log N)$.

Ma trận bài tập thực hành (p0 → p5)

Mã Bài Tập	Tên Bài Toán	Mức Độ	Trọng Tâm Kiến Thức & Kỹ Năng Range Query
CPPB-RNG-01	Cài Đặt Fenwick Tree Tính Tổng Đoạn	P0	Cài đặt chuẩn <code>update</code> và <code>query</code> trên cây BIT 1-based.
CPPB-RNG-02	Cài Đặt Segment Tree Tìm Min Đoạn (RMQ)	P1	Hàm <code>build</code> , <code>update</code> điểm và <code>query</code> trên cây nhị phân $4N$.
CPPB-RNG-03	Cập Nhật Đoạn & Truy Vấn Điểm Bằng BIT	P1	Fenwick Tree trên Mảng hiệu <code>D[i]</code> hỗ trợ Range Add.
CPPB-RNG-04	Tìm Giá Trị Lớn Nhất & Đếm Số Lần Xuất Hiện	P2	Segment Tree lưu cặp <code>(max_val, count)</code> kết hợp tại nút cha.
CPPB-RNG-05	Đếm Số Cặp Nghịch Thế (Inversion Count)	P2	Nén tọa độ kết hợp Fenwick Tree đếm số phần tử nhỏ hơn phía sau.
CPPB-RNG-06	Truy Vấn Ước Chung Lớn Nhất Đoạn (Range GCD)	P2	Segment Tree với hàm kết hợp <code>std::gcd</code> $O(\log N)$.
CPPB-RNG-07	Tìm Phần Tử Thứ K Nhỏ Nhất (K-th Element)	P2	Nhảy nhị phân trên Fenwick Tree (Binary Lifting on BIT).
CPPB-RNG-08	Tìm Vị Trí Đầu Tiên Có Giá Trị $\geq X$	P3	Tìm kiếm nhị phân trực tiếp trên Segment Tree trong $O(\log N)$.
CPPB-RNG-09	Dãy Con Tăng Dài Nhất LIS $O(N \log N)$ Bằng BIT	P3	Nén tọa độ, dùng BIT lưu <code>max(dp)</code> theo tiền tố giá trị.
CPPB-RNG-10	Đoạn Con Có Tổng Lớn Nhất (Maximum Subarray Sum)	P3	Segment Tree lưu 4 giá trị: <code>sum</code> , <code>pref_max</code> , <code>suff_max</code> , <code>max_sub</code> .
CPPB-RNG-11	Đếm Số Điểm Nằm Trong Hình Chữ Nhật	P3	Đường quét (Sweep-line) theo trục <code>X</code> kết hợp Fenwick Tree trên trục <code>Y</code> .
CPPB-RNG-12	Đổi Dấu Đoạn & Tìm Tổng Lớn Nhất	P4	Segment Tree lưu trạng thái đa trường hỗ trợ truy vấn phức tạp.
CPPB-RNG-13	Cây Phân Đoạn 2D Độc Lập (2D Fenwick Tree)	P4	BIT lồng nhau 2 chiều <code>bit[x][y]</code> tính tổng hình chữ nhật con.
CPPB-RNG-14	Giới Thiệu Lazy Propagation Cộng Đoạn	P4	Kỹ thuật gắn nhãn <code>lazy</code> hỗ trợ Range Add & Range Sum $O(\log N)$.
CPPB-RNG-15	Hệ Thống Quản Lý Dữ Liệu Olympic (Mastery)	P5	Bài toán cấu trúc dữ liệu tổng hợp kết hợp Segment Tree và hình học/đồ thị.

```
=====
<!--      KẾT      THÚC      NỘI      DUNG      21      CHUYÊN      ĐỀ      -->
=====
```

Thống kê: 21 chuyên đề đã nối | 0 thiếu | 8,251 dòng | 450,254 bytes tổng cộng